

Comprehensive Technical Documentation: Porting CLI Agent Features into HelixCode

Table of Contents

- 1. [Part 1: Claude Code Feature Porting Guides](#)
- 2. [Part 2: Aider Feature Porting Guides](#)
- 3. [Part 3: Cline Feature Porting Guides](#)
- 4. [Part 4: Codex Feature Porting Guides](#)
- 5. [Part 5: Architecture Diagrams & Schemas](#)
- 6. [Part 6: Submodule Integration Wiring](#)

PART 1: CLAUDE CODE FEATURE PORTING GUIDES

Context & Foundation

HelixCode is a Go-based platform (`dev.helix.code`) built on: - **CLI Framework:** Cobra + Viper - **Web Framework:** Gin - **Persistence:** PostgreSQL + Redis - **Concurrency Model:** Actor Model (8 agent types) - **LLM Integration:** 29+ providers - **Code Analysis:** Tree-sitter code mapping - **UI:** 6-platform support - **API:** OpenAPI 3.0 REST API

Present submodules: `LLMsVerifier`, `HelixQA`, `Challenges`, `Containers Missing`
submodules: `HelixAgent`, `HelixLLM`, `HelixMemory`, `HelixSpecifier`

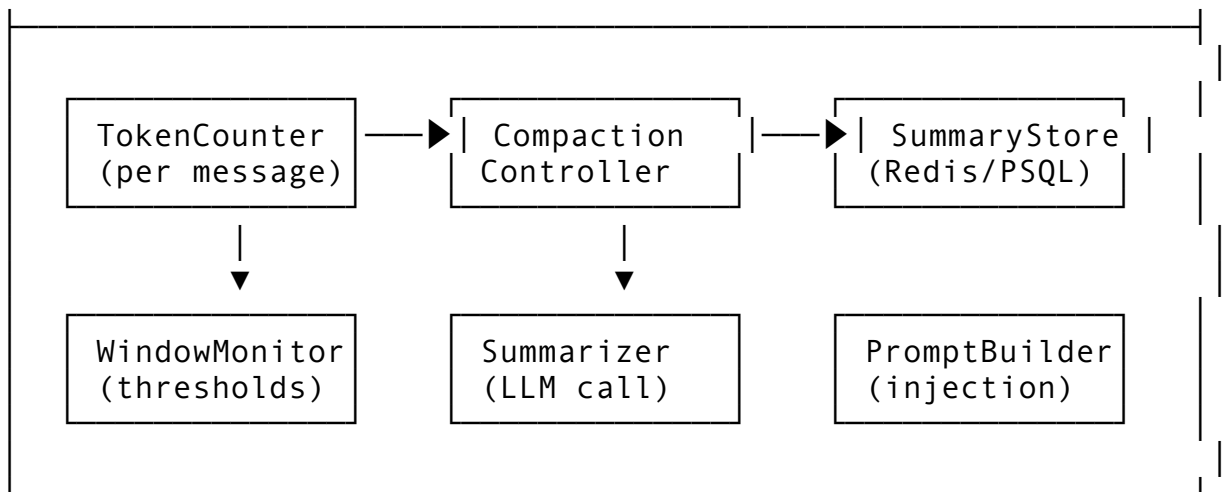
Feature 1: Auto-Compaction System

1.1 Feature Description

The Auto-Compaction System automatically summarizes and compresses conversation history when context window limits are approached. This prevents token overflow errors while preserving critical semantic information from earlier parts of the conversation.

Why it matters: Without compaction, long-running sessions hit LLM context limits (typically 128K-200K tokens), causing failures. Manual context management is error-prone and interrupts workflow.

1.2 Architecture Design



HelixCode Integration: - HelixMemory submodule (new): Stores summaries and compaction metadata - HelixLLM submodule (new): Provides LLM calls for summarization - Actor system: Compaction runs as a background actor message

1.3 API Design

```

// Package: internal/compact
package compact

import (
    "context"
    "time"
)

// CompactionLevel defines how aggressive compaction should be
type CompactionLevel int

const (
    CompactionLight  CompactionLevel = iota // Keep key details,
        full reasoning
    CompactionMedium // Summarize
        reasoning, keep conclusions
    CompactionHeavy // Aggressive
        summarization
    CompactionCritical // Minimal viable
        context only
)

// MessageSummary represents a compacted conversation segment
type MessageSummary struct {
    ID          string    `json:"id" db:"id"`
    SessionID   string    `json:"session_id"
        db:"session_id"`
    OriginalRange [2]int    `json:"original_range"
        db:"original_range" // [start_msg_idx, end_msg_idx]

```



```

    EnableIncremental    bool
        `mapstructure:"enable_incremental"`    // Summarize as we
        go vs batch
}

// CompactionEngine is the main interface
type CompactionEngine interface {
    // CheckAndCompact evaluates if compaction is needed and
    // executes it
    CheckAndCompact(ctx context.Context, sessionID string,
        messages []Message) (*CompactionResult, error)

    // CompactMessages explicitly compacts a range of messages
    CompactMessages(ctx context.Context, sessionID string,
        startIdx, endIdx int, level CompactionLevel)
        (*MessageSummary, error)

    // GetCompactedSession rebuilds a session with summaries
    // injected
    GetCompactedSession(ctx context.Context, sessionID string)
        ([]Message, error)

    // RegisterTokenCounter sets the token counting
    // implementation
    RegisterTokenCounter(counter TokenCounter)

    // ForceCompaction triggers compaction regardless of
    // thresholds
    ForceCompaction(ctx context.Context, sessionID string, level
        CompactionLevel) error
}

// TokenCounter estimates token usage
type TokenCounter interface {
    CountMessages(messages []Message) (int, error)
    CountText(text string) (int, error)
    GetWindowLimit() int
}

// CompactionResult reports what was compacted
type CompactionResult struct {
    WasNeeded    bool    `json:"was_needed"`
    Summary      *MessageSummary `json:"summary,omitempty"`
    TokensBefore int    `json:"tokens_before"`
    TokensAfter  int    `json:"tokens_after"`
    ReductionRatio float64 `json:"reduction_ratio"`
    MessagesRemoved int    `json:"messages_removed"`
}

```

```
// Message represents a conversation message (existing HelixCode
    type)
type Message struct {
    Role    string `json:"role"`
    Content string `json:"content"`
    // ... other fields
}
```

1.4 Integration Points

Package	Modification	Description
cmd/helix/main.go	Add --compaction flag	CLI flag for compaction level
internal/compact/	NEW PACKAGE	Core compaction engine
internal/actor/session.go	Inject compaction check	Before sending to LLM, check window
internal/llm/client.go	Add summary model config	Configure which model handles compaction
internal/memory/	NEW (HelixMemory)	Store summaries persistently
internal/db/migrations/	Add compaction_summaries table	PostgreSQL schema
internal/redis/	Add compaction cache layer	Fast summary retrieval

1.5 Implementation Steps

Step 1: Create database migration

```
-- migrations/004_add_compaction_summaries.sql
CREATE TABLE compaction_summaries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id UUID NOT NULL REFERENCES sessions(id) ON DELETE
        CASCADE,
    original_range INT[2] NOT NULL,
    summary TEXT NOT NULL,
    key_decisions TEXT[] DEFAULT '{}',
    code_changes JSONB DEFAULT '[]',
    token_count INT NOT NULL,
    original_tokens INT NOT NULL,
    compaction_level INT NOT NULL DEFAULT 1,
    model_used VARCHAR(64),
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
```

```

    INDEX idx_session_id (session_id),
    INDEX idx_created_at (created_at)
);

```

Step 2: Implement TokenCounter using tiktoken-style counting

```

// internal/compact/token_counter.go
package compact

import (
    "github.com/pkoukk/tiktoken-go"
)

type TiktokenCounter struct {
    model      string
    windowLimit int
    encoding    *tiktoken.Tiktoken
}

func NewTiktokenCounter(model string, windowLimit int)
    (*TiktokenCounter, error) {
    encoding, err := tiktoken.EncodingForModel(model)
    if err != nil {
        return nil, err
    }
    return &TiktokenCounter{
        model:      model,
        windowLimit: windowLimit,
        encoding:    encoding,
    }, nil
}

func (t *TiktokenCounter) CountMessages(messages []Message)
    (int, error) {
    total := 0
    for _, msg := range messages {
        // Per-message overhead + content tokens
        total += 3 + len(t.encoding.Encode(msg.Content, nil,
            nil))
    }
    total += 3 // Every reply is primed with <|start|>assistant<|
        message|>
    return total, nil
}

func (t *TiktokenCounter) CountText(text string) (int, error) {
    return len(t.encoding.Encode(text, nil, nil)), nil
}

```

```
func (t *TiktokenCounter) GetWindowLimit() int {
    return t.windowLimit
}
```

Step 3: Implement CompactionEngine

```
// internal/compact/engine.go
package compact

import (
    "context"
    "fmt"
    "math"
    "strings"
    "time"

    "dev.helix.code/internal/llm"
    "dev.helix.code/internal/memory"
    "dev.helix.code/pkg/logger"
)

type DefaultCompactionEngine struct {
    config      *CompactionConfig
    counter     TokenCounter
    llmClient   llm.Client
    memory      memory.Store
    log         logger.Logger
}

func NewDefaultCompactionEngine(
    config *CompactionConfig,
    llmClient llm.Client,
    memory memory.Store,
    log logger.Logger,
) (*DefaultCompactionEngine, error) {
    return &DefaultCompactionEngine{
        config:      config,
        llmClient:   llmClient,
        memory:      memory,
        log:         log,
    }, nil
}

func (e *DefaultCompactionEngine) RegisterTokenCounter(counter
    TokenCounter) {
    e.counter = counter
}

func (e *DefaultCompactionEngine) CheckAndCompact(
```

```

ctx context.Context,
sessionID string,
messages []Message,
) (*CompactionResult, error) {
    if !e.config.Enabled {
        return &CompactionResult{WasNeeded: false}, nil
    }

    tokenCount, err := e.counter.CountMessages(messages)
    if err != nil {
        return nil, fmt.Errorf("counting tokens: %w", err)
    }

    limit := e.counter.GetWindowLimit()
    ratio := float64(tokenCount) / float64(limit)

    // Determine if compaction is needed
    if ratio < e.config.WindowThreshold {
        return &CompactionResult{
            WasNeeded: false,
            TokensBefore: tokenCount,
        }, nil
    }

    // Determine compaction level based on urgency
    level := CompactionMedium
    if ratio >= e.config.CriticalThreshold {
        level = CompactionCritical
    }

    // Calculate range to compact (skip last N messages)
    totalMsgs := len(messages)
    preserveCount := e.config.PreserveLastN
    if preserveCount >= totalMsgs {
        preserveCount = totalMsgs / 2 // Always compact at least
        half
    }

    compactEnd := totalMsgs - preserveCount
    compactStart := 0

    // Ensure minimum messages to compact
    if compactEnd < e.config.MinMessagesToCompact {
        compactEnd = e.config.MinMessagesToCompact
    }

    summary, err := e.CompactMessages(ctx, sessionID,
        compactStart, compactEnd, level)

```



```

    if err != nil {
        return nil, fmt.Errorf("compacting messages: %w", err)
    }

    tokensAfter := summary.TokenCount +
        e.estimateTokenCount(messages[compactEnd:])

    return &CompactionResult{
        WasNeeded:      true,
        Summary:        summary,
        TokensBefore:   tokenCount,
        TokensAfter:    tokensAfter,
        ReductionRatio: float64(tokenCount-tokensAfter) /
            float64(tokenCount),
        MessagesRemoved: compactEnd - compactStart,
    }, nil
}

func (e *DefaultCompactionEngine) CompactMessages(
    ctx context.Context,
    sessionID string,
    startIdx, endIdx int,
    level CompactionLevel,
) (*MessageSummary, error) {
    if startIdx >= endIdx {
        return nil, fmt.Errorf("invalid range: start %d >= end %d", startIdx, endIdx)
    }

    // Fetch messages from memory
    messages, err := e.memory.GetMessages(ctx, sessionID,
        startIdx, endIdx)
    if err != nil {
        return nil, fmt.Errorf("fetching messages: %w", err)
    }

    originalTokens, _ := e.counter.CountMessages(messages)

    // Build summarization prompt based on level
    prompt := e.buildSummarizationPrompt(messages, level)

    // Call LLM to generate summary
    response, err := e.llmClient.Complete(ctx,
        llm.CompletionRequest{
            Model:    e.config.SummaryModel,
            Prompt:  prompt,
            MaxTokens: int(float64(originalTokens) *
                e.config.TargetReductionRatio),
        },

```

```

        Temperature: 0.1, // Low temperature for factual
        summarization
    })
    if err != nil {
        return nil, fmt.Errorf("summarization LLM call: %w", err)
    }

    // Parse summary response
    summary := e.parseSummaryResponse(response.Text, messages)
    summary.SessionID = sessionID
    summary.OriginalRange = [2]int{startIdx, endIdx}
    summary.OriginalTokens = originalTokens
    summary.CompactionLevel = level
    summary.ModelUsed = e.config.SummaryModel
    summary.CreatedAt = time.Now()

    // Store summary
    if err := e.memory.StoreSummary(ctx, summary); err != nil {
        return nil, fmt.Errorf("storing summary: %w", err)
    }

    // Log compaction
    e.log.Info("compaction completed",
        "session_id", sessionID,
        "original_tokens", originalTokens,
        "summary_tokens", summary.TokenCount,
        "level", level,
    )

    return summary, nil
}

func (e *DefaultCompactionEngine)
    buildSummarizationPrompt(messages []Message, level
    CompactionLevel) string {
    var b strings.Builder
    b.WriteString("Summarize the following conversation
        concisely. ")

    switch level {
    case CompactionLight:
        b.WriteString("Preserve key reasoning steps, technical
            details, and all code changes. ")
    case CompactionMedium:
        b.WriteString("Summarize reasoning but preserve all
            conclusions, decisions, and code changes. ")
    case CompactionHeavy:

```

```

        b.WriteString("Be aggressive: only preserve critical
            decisions and file change summaries. ")
    case CompactionCritical:
        b.WriteString("Minimal summary: only what is needed to
            continue the task. ")
    }

    b.WriteString("Format your response as:\n")
    b.WriteString("SUMMARY: <brief overview>\n")
    b.WriteString("KEY_DECISIONS: <list of decisions made>\n")
    b.WriteString("CODE_CHANGES: <list of files changed and
        how>\n\n")
    b.WriteString("Conversation:\n")

    for _, msg := range messages {
        b.WriteString(fmt.Sprintf("%s: %s\n", msg.Role,
            msg.Content))
    }

    return b.String()
}

func (e *DefaultCompactionEngine) parseSummaryResponse(text
    string, original []Message) *MessageSummary {
    summary := &MessageSummary{TokenCount:
        e.estimateTokenCountString(text)}

    lines := strings.Split(text, "\n")
    section := ""
    for _, line := range lines {
        if strings.HasPrefix(line, "SUMMARY:") {
            section = "summary"
            summary.Summary = strings.TrimPrefix(line,
                "SUMMARY:")
        } else if strings.HasPrefix(line, "KEY_DECISIONS:") {
            section = "decisions"
        } else if strings.HasPrefix(line, "CODE_CHANGES:") {
            section = "code"
        } else if section == "decisions" &&
            strings.TrimSpace(line) != "" {
            summary.KeyDecisions = append(summary.KeyDecisions,
                strings.TrimSpace(line))
        } else if section == "code" && strings.TrimSpace(line) !=
            "" {
            // Parse code change references
            summary.CodeChanges = append(summary.CodeChanges,
                e.parseCodeChange(line))
        }
    }
}

```

```

    }

    // If parsing failed, use entire text as summary
    if summary.Summary == "" {
        summary.Summary = text
    }

    return summary
}

func (e *DefaultCompactionEngine) parseCodeChange(line string)
    CodeChangeRef {
    // Simple parsing: "file.go - modified: description"
    parts := strings.SplitN(line, " - ", 2)
    if len(parts) == 2 {
        return CodeChangeRef{
            FilePath:    strings.TrimSpace(parts[0]),
            ChangeType:  "modified", // Default
            Description: strings.TrimSpace(parts[1]),
        }
    }
    return CodeChangeRef{Description: line}
}

func (e *DefaultCompactionEngine) GetCompactedSession(
    ctx context.Context,
    sessionID string,
) ([]Message, error) {
    // Get all summaries for this session
    summaries, err := e.memory.GetSummaries(ctx, sessionID)
    if err != nil {
        return nil, err
    }

    // Get remaining uncompressed messages
    allMessages, err := e.memory.GetAllMessages(ctx, sessionID)
    if err != nil {
        return nil, err
    }

    // Rebuild message list with summaries injected
    var result []Message
    coveredRange := 0

    for _, sum := range summaries {
        // Add uncompressed messages before this summary
        if sum.OriginalRange[0] > coveredRange {

```

```

        result = append(result,
            allMessages[coveredRange:sum.OriginalRange[0]]...)
    }

    // Inject summary as a system message
    summaryMsg := Message{
        Role:    "system",
        Content: e.formatSummaryAsMessage(sum),
    }
    result = append(result, summaryMsg)
    coveredRange = sum.OriginalRange[1]
}

// Add remaining uncompressed messages
if coveredRange < len(allMessages) {
    result = append(result, allMessages[coveredRange:]...)
}

return result, nil
}

func (e *DefaultCompactionEngine) formatSummaryAsMessage(sum
    *MessageSummary) string {
    var b strings.Builder
    b.WriteString("[COMPACTED HISTORY]\n")
    b.WriteString(sum.Summary + "\n")
    if len(sum.KeyDecisions) > 0 {
        b.WriteString("\nKey Decisions:\n")
        for _, d := range sum.KeyDecisions {
            b.WriteString("- " + d + "\n")
        }
    }
    if len(sum.CodeChanges) > 0 {
        b.WriteString("\nCode Changes:\n")
        for _, c := range sum.CodeChanges {
            b.WriteString(fmt.Sprintf("- %s (%s): %s\n",
                c.FilePath, c.ChangeType, c.Description))
        }
    }
    return b.String()
}

func (e *DefaultCompactionEngine) ForceCompaction(
    ctx context.Context,
    sessionID string,
    level CompactionLevel,
) error {
    messages, err := e.memory.GetAllMessages(ctx, sessionID)

```

```

    if err != nil {
        return err
    }
    _, err = e.CompactMessages(ctx, sessionID, 0,
        len(messages)-e.config.PreserveLastN, level)
    return err
}

func (e *DefaultCompactionEngine) estimateTokenCount(msgs
    []Message) int {
    count, _ := e.counter.CountMessages(msgs)
    return count
}

func (e *DefaultCompactionEngine) estimateTokenCountString(text
    string) int {
    count, _ := e.counter.CountText(text)
    return count
}

```

Step 4: Wire into session actor

```

// internal/actor/session.go - Add to SessionActor
func (a *SessionActor) BeforeLLMCall(ctx context.Context, msgs
    []Message) ([]Message, error) {
    if a.compactionEngine == nil {
        return msgs, nil
    }

    result, err := a.compactionEngine.CheckAndCompact(ctx,
        a.sessionID, msgs)
    if err != nil {
        a.log.Warn("compaction check failed", "error", err)
        return msgs, nil // Fail open - don't break the session
    }

    if result.WasNeeded {
        a.log.Info("compaction applied",
            "reduction_ratio", result.ReductionRatio,
            "messages_removed", result.MessagesRemoved,
        )
        return a.compactionEngine.GetCompactedSession(ctx,
            a.sessionID)
    }

    return msgs, nil
}

```

1.6 Testing Approach

```
// internal/compact/engine_test.go
package compact

import (
    "context"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/mock"
)

// MockTokenCounter implements TokenCounter for testing
type MockTokenCounter struct {
    mock.Mock
}

func (m *MockTokenCounter) CountMessages(messages []Message)
    (int, error) {
    args := m.Called(messages)
    return args.Int(0), args.Error(1)
}

func (m *MockTokenCounter) CountText(text string) (int, error) {
    args := m.Called(text)
    return args.Int(0), args.Error(1)
}

func (m *MockTokenCounter) GetWindowLimit() int {
    args := m.Called()
    return args.Int(0)
}

func TestCompactionEngine_CheckAndCompact_BelowThreshold(t
    *testing.T) {
    counter := new(MockTokenCounter)
    counter.On("CountMessages", mock.Anything).Return(5000, nil)
    counter.On("GetWindowLimit").Return(128000)

    engine := &DefaultCompactionEngine{
        config: &CompactionConfig{
            Enabled:         true,
            WindowThreshold: 0.75,
        },
        counter: counter,
    }
}
```

```

    result, err := engine.CheckAndCompact(context.Background(),
        "session-1", []Message{
            {Role: "user", Content: "Hello"},
        })

    assert.NoError(t, err)
    assert.False(t, result.WasNeeded)
    assert.Equal(t, 5000, result.TokensBefore)
}

func TestCompactionEngine_CheckAndCompact_AboveThreshold(t
    *testing.T) {
    counter := new(MockTokenCounter)
    counter.On("CountMessages", mock.Anything).Return(100000,
        nil)
    counter.On("CountText", mock.Anything).Return(5000, nil)
    counter.On("GetWindowLimit").Return(128000)

    // Mock LLM client and memory store would be set up here
    // ... test implementation
}

func TestCompactionEngine_GetCompactedSession(t *testing.T) {
    // Test that compacted session rebuilds correctly with
    // summaries injected
}

func TestCompactionEngine_ForceCompaction(t *testing.T) {
    // Test manual compaction trigger
}

// Benchmark tests
func BenchmarkCompactionEngine_CheckAndCompact(b *testing.B) {
    // Benchmark the decision-making path
}

```

Integration Tests: 1. Create a 150K token session, verify automatic compaction triggers 2. Verify summary is semantically useful by continuing the conversation 3. Test concurrent compaction on multiple sessions 4. Verify database persistence and retrieval of summaries 5. Test compaction with various LLM providers

1.7 Edge Cases

Edge Case	Handling
Compaction LLM fails	Fail open - return original messages, log warning
Summary exceeds target tokens	Apply recursive compaction or truncate
All messages are “preserve” range	

Edge Case	Handling
	Skip compaction, log that session is too short
Nested compaction (summary of summary)	Track compaction depth, max depth = 3
Code changes lost in summary	Parse and store CodeChangeRef separately
Race condition during compaction	Use session-level mutex, compaction is single-threaded per session
Non-English content	Include language hint in summarization prompt
Binary/file attachment in history	Extract text description, skip binary content

Feature 2: Permission Rule System (5 Modes + Wildcard Rules)

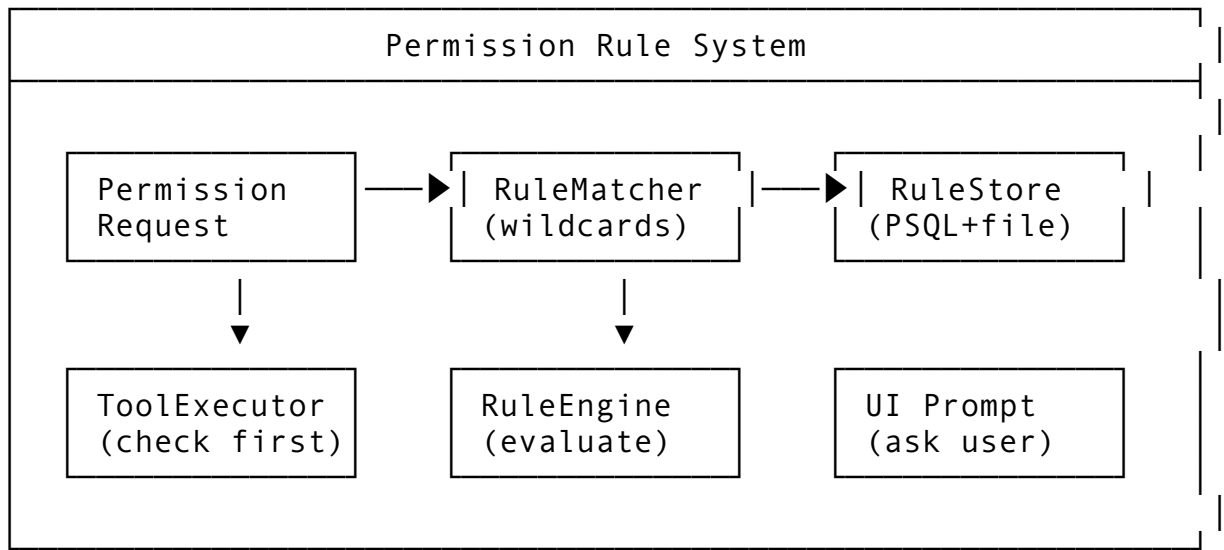
2.1 Feature Description

The Permission Rule System controls which tools/commands the agent can execute without human confirmation. It supports 5 permission modes plus wildcard-based rules for fine-grained control over potentially destructive operations.

Why it matters: Users need confidence that the agent won’t accidentally delete production databases or push code without review. Fine-grained rules balance autonomy with safety.

The 5 Modes: 1. `always_ask` - Every tool requires confirmation 2. `allow_once` - Allow this specific invocation, ask next time 3. `allow_session` - Allow for current session only 4. `allow_workspace` - Allow for this project/workspace 5. `allow_forever` - Persist rule permanently

2.2 Architecture Design



2.3 API Design

```
// Package: internal/permissions
package permissions

import (
    "context"
    "fmt"
    "regexp"
    "strings"
    "time"
)

// PermissionMode defines the lifetime scope of a permission
type PermissionMode int

const (
    AlwaysAsk PermissionMode = iota    // 0: Always require
        confirmation

    AllowOnce // 1: Allow this
        single invocation

    AllowSession // 2: Allow for
        current session

    AllowWorkspace // 3: Allow for this
        workspace

    AllowForever // 4: Persist across
        all sessions
)

func (m PermissionMode) String() string {
    switch m {
    case AlwaysAsk:    return "always_ask"
    case AllowOnce:    return "allow_once"
    case AllowSession: return "allow_session"
    case AllowWorkspace: return "allow_workspace"
    case AllowForever: return "allow_forever"
    default:           return "unknown"
    }
}

func ParsePermissionMode(s string) (PermissionMode, error) {
    switch strings.ToLower(s) {
    case "always_ask": return AlwaysAsk, nil
    case "allow_once": return AllowOnce, nil
    case "allow_session": return AllowSession, nil
    case "allow_workspace": return AllowWorkspace, nil
    }
```

```

    case "allow_forever": return AllowForever, nil
    default: return AlwaysAsk, fmt.Errorf("unknown permission
        mode: %s", s)
}
}

// ToolCategory groups related tools for blanket permissions
type ToolCategory string

const (
    CategoryFileRead      ToolCategory = "file_read"
    CategoryFileWrite     ToolCategory = "file_write"
    CategoryFileDelete    ToolCategory = "file_delete"
    CategoryShellExec     ToolCategory = "shell_exec"
    CategoryGitOp         ToolCategory = "git_operation"
    CategoryDBQuery       ToolCategory = "database_query"
    CategoryNetworkCall   ToolCategory = "network_call"
    CategoryLSPCall       ToolCategory = "lsp_call"
    CategoryMCPCall       ToolCategory = "mcp_call"
)

// PermissionRule defines a single permission rule
type PermissionRule struct {
    ID          string          `json:"id" db:"id"`
    Name        string          `json:"name" db:"name"` //
        Human-readable name
    Category    ToolCategory `json:"category" db:"category"`
    ToolName    string       `json:"tool_name"
        db:"tool_name"` // "*" for wildcard
    Pattern     string       `json:"pattern"
        db:"pattern"` // Wildcard pattern for matching
    CompiledPattern *regexp.Regexp `json:"- " db:"- "` // Compiled
        at load time
    Mode        PermissionMode `json:"mode" db:"mode"`

    // Scope fields (for non-forever modes)
    SessionID *string `json:"session_id,omitempty"
        db:"session_id"`
    Workspace *string `json:"workspace,omitempty"
        db:"workspace"`

    // Conditions
    Conditions RuleConditions `json:"conditions"
        db:"conditions"`

    // Metadata
    CreatedAt time.Time `json:"created_at"
        db:"created_at"`

```

```

ExpiresAt    *time.Time    `json:"expires_at,omitempty"
    db:"expires_at"`
UseCount     int           `json:"use_count" db:"use_count"`
LastUsedAt   *time.Time    `json:"last_used_at,omitempty"
    db:"last_used_at"`
}

// RuleConditions adds fine-grained conditions
type RuleConditions struct {
    AllowedPaths    []string
        `json:"allowed_paths,omitempty"`    // Glob patterns for
        allowed paths
    BlockedPaths    []string
        `json:"blocked_paths,omitempty"`    // Glob patterns for
        blocked paths
    AllowedCommands []string
        `json:"allowed_commands,omitempty"` // Shell command
        whitelist
    BlockedCommands []string
        `json:"blocked_commands,omitempty"` // Shell command
        blacklist
    MaxArgs         *int
        `json:"max_args,omitempty"`         // Max number of
        arguments
    RequireReview   bool
        `json:"require_review"`             // Even with allow,
        show what was done
    DryRunOnly      bool
        `json:"dry_run_only"`               // Only allow dry-run
        variants
}

// PermissionRequest represents a tool asking for permission
type PermissionRequest struct {
    ID            string    `json:"id"`
    SessionID     string    `json:"session_id"`
    Workspace     string    `json:"workspace"`
    Category      ToolCategory `json:"category"`
    ToolName      string    `json:"tool_name"`
    Arguments     map[string]any `json:"arguments"`
    Description    string    `json:"description"` // Human-
        readable
    RiskLevel     RiskLevel `json:"risk_level"`
    Timestamp     time.Time `json:"timestamp"`
}

// RiskLevel categorizes operation danger
type RiskLevel int

```

```

const (
    RiskLow RiskLevel = iota    // Reading, listing
    RiskMedium                // Writing files, git status
    RiskHigh                  // Deleting, git push, shell
                             exec
    RiskCritical              // DB mutations, prod
                             deployments
)

// PermissionResult is the outcome of a permission check
type PermissionResult struct {
    Allowed      bool        `json:"allowed"`
    Mode         PermissionMode `json:"mode,omitempty"`
    RuleID       string       `json:"rule_id,omitempty"`
    Message      string       `json:"message,omitempty"`
    RequiresConfirmation bool    `json:"requires_confirmation"`
    RiskLevel    RiskLevel   `json:"risk_level"`
    Preview      string       `json:"preview,omitempty"` // What will happen
}

// PermissionEngine is the main interface
type PermissionEngine interface {
    // CheckPermission evaluates if a tool invocation is
    // permitted
    CheckPermission(ctx context.Context, req PermissionRequest)
        (*PermissionResult, error)

    // GrantPermission creates a new permission rule
    GrantPermission(ctx context.Context, rule PermissionRule)
        error

    // RevokePermission removes a rule
    RevokePermission(ctx context.Context, ruleID string) error

    // ListRules returns rules, optionally filtered
    ListRules(ctx context.Context, filter RuleFilter)
        ([]PermissionRule, error)

    // UpdateRule modifies an existing rule
    UpdateRule(ctx context.Context, ruleID string, updates
        RuleUpdates) error

    // CleanupExpired removes expired session/temporary rules
    CleanupExpired(ctx context.Context) error

    // ExportRules serializes rules for backup/sharing

```

```

    ExportRules(ctx context.Context, scope string) ([]byte,
        error)

    // ImportRules loads rules from serialized form
    ImportRules(ctx context.Context, data []byte) error
}

// RuleFilter filters rule queries
type RuleFilter struct {
    Category    *ToolCategory `json:"category,omitempty"`
    ToolName    *string       `json:"tool_name,omitempty"`
    Mode        *PermissionMode `json:"mode,omitempty"`
    SessionID   *string       `json:"session_id,omitempty"`
    Workspace   *string       `json:"workspace,omitempty"`
    ActiveOnly  bool          `json:"active_only"`
}

// RuleUpdates partial update struct
type RuleUpdates struct {
    Mode        *PermissionMode `json:"mode,omitempty"`
    Pattern     *string         `json:"pattern,omitempty"`
    ExpiresAt   *time.Time      `json:"expires_at,omitempty"`
    Conditions  *RuleConditions `json:"conditions,omitempty"`
}

```

2.4 Integration Points

Package	Modification	Description
internal/ permissions/	NEW PACKAGE	Core permission engine
internal/tool/	Add pre-execution hook	Every tool checks permissions before running
cmd/helix/ main.go	Add --permission-mode flag	Default permission mode
internal/ config/	Add permission config section	Viper config for rules
internal/db/ migrations/	Add permission_rules table	Persist forever/workspace rules
internal/ui/	Add permission prompt UI	Interactive confirmation dialogs
internal/ actor/ session.go	Session-scoped rules	Track session rules lifecycle

2.5 Implementation Steps

Step 1: Database schema

```
-- migrations/005_add_permission_rules.sql
CREATE TYPE permission_mode AS ENUM ('always_ask', 'allow_once',
    'allow_session', 'allow_workspace', 'allow_forever');
CREATE TYPE risk_level AS ENUM ('low', 'medium', 'high',
    'critical');
CREATE TYPE tool_category AS ENUM (
    'file_read', 'file_write', 'file_delete',
    'shell_exec', 'git_operation', 'database_query',
    'network_call', 'lsp_call', 'mcp_call'
);

CREATE TABLE permission_rules (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(256) NOT NULL,
    category tool_category NOT NULL,
    tool_name VARCHAR(128) NOT NULL,
    pattern VARCHAR(512) NOT NULL DEFAULT '*',
    mode permission_mode NOT NULL DEFAULT 'always_ask',
    session_id UUID REFERENCES sessions(id) ON DELETE CASCADE,
    workspace VARCHAR(512),
    conditions JSONB DEFAULT '{}',
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    expires_at TIMESTAMP,
    use_count INT NOT NULL DEFAULT 0,
    last_used_at TIMESTAMP,
    INDEX idx_category (category),
    INDEX idx_tool_name (tool_name),
    INDEX idx_session (session_id),
    INDEX idx_workspace (workspace)
);
```

Step 2: Wildcard matcher implementation

```
// internal/permissions/matcher.go
package permissions

import (
    "path/filepath"
    "regexp"
    "strings"
)

// WildcardMatcher handles pattern matching for permission rules
type WildcardMatcher struct {
    pattern string
    regex   *regexp.Regexp
}

}
```

```

func NewWildcardMatcher(pattern string) (*WildcardMatcher,
    error) {
    // Convert glob-style patterns to regex
    regexPattern := globToRegex(pattern)
    re, err := regexp.Compile("^" + regexPattern + "$")
    if err != nil {
        return nil, err
    }
    return &WildcardMatcher{pattern: pattern, regex: re}, nil
}

func (m *WildcardMatcher) Match(input string) bool {
    return m.regex.MatchString(input)
}

func globToRegex(pattern string) string {
    var result strings.Builder
    for i := 0; i < len(pattern); i++ {
        c := pattern[i]
        switch c {
            case '*':
                if i+1 < len(pattern) && pattern[i+1] == '*' {
                    // ** matches across path separators
                    result.WriteString(".*")
                    i++ // Skip second *
                } else {
                    // * matches within path segment
                    result.WriteString("[^/]*")
                }
            case '?':
                result.WriteString(".")
            case '[':
                // Character class - pass through
                j := i
                for j < len(pattern) && pattern[j] != ']' {
                    j++
                }
                result.WriteString(pattern[i:j+1])
                i = j
            default:
                // Escape special regex characters
                if strings.ContainsRune("\\.^\$+|(){}", rune(c)) {
                    result.WriteString("\\")
                }
                result.WriteByte(c)
            }
        }
    }
    return result.String()
}

```



```

}

func matchGlob(pattern, path string) bool {
    matched, _ := filepath.Match(pattern, path)
    return matched
}

```

Step 3: Permission engine core

```

// internal/permissions/engine.go
package permissions

import (
    "context"
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "time"

    "dev.helix.code/internal/db"
    "dev.helix.code/pkg/logger"
)

type DefaultPermissionEngine struct {
    store      db.Store
    log        logger.Logger
    workspace  string
    sessionID  string
    rules      []PermissionRule // Cached rules
    matcherCache map[string]*WildcardMatcher
}

func NewPermissionEngine(store db.Store, log logger.Logger,
    workspace, sessionID string) (*DefaultPermissionEngine,
    error) {
    engine := &DefaultPermissionEngine{
        store:      store,
        log:        log,
        workspace:  workspace,
        sessionID:  sessionID,
        matcherCache: make(map[string]*WildcardMatcher),
    }

    // Load persisted rules
    if err := engine.reloadRules(context.Background()); err !=
        nil {
        return nil, fmt.Errorf("loading rules: %w", err)
    }
}

```

```

    return engine, nil
}

func (e *DefaultPermissionEngine) CheckPermission(
    ctx context.Context,
    req PermissionRequest,
) (*PermissionResult, error) {
    // Determine risk level if not set
    risk := req.RiskLevel
    if risk == 0 && req.Category != "" {
        risk = e.calculateRiskLevel(req)
    }

    // 1. Check for matching rules (most specific first)
    matchingRules := e.findMatchingRules(req)

    for _, rule := range matchingRules {
        // Check if rule is active
        if rule.ExpiresAt != nil &&
            rule.ExpiresAt.Before(time.Now()) {
            continue
        }

        // Check conditions
        if !e.checkConditions(rule, req) {
            continue
        }

        // Rule matches - apply it
        result := &PermissionResult{
            Allowed:      rule.Mode != AlwaysAsk,
            Mode:          rule.Mode,
            RuleID:         rule.ID,
            RiskLevel:     risk,
            RequiresConfirmation: rule.Mode == AlwaysAsk ||
                rule.Conditions.RequireReview,
        }

        // Update use count
        e.incrementUseCount(ctx, rule.ID)

        // Handle AllowOnce (delete after use)
        if rule.Mode == AllowOnce {
            e.RevokePermission(ctx, rule.ID)
        }

        return result, nil
    }
}

```

```

    }

    // No matching rule - default to asking
    return &PermissionResult{
        Allowed:                false,
        RequiresConfirmation:    true,
        RiskLevel:               risk,
        Message:                 "No matching permission rule
                                found",
    }, nil
}

func (e *DefaultPermissionEngine) findMatchingRules(req
    PermissionRequest) []PermissionRule {
    var matching []PermissionRule

    for _, rule := range e.rules {
        // Check category match (exact or wildcard)
        if rule.Category != req.Category && rule.Category != "" {
            continue
        }

        // Check tool name match
        if rule.ToolName != req.ToolName && rule.ToolName != "*"
        {
            continue
        }

        // Check scope
        if rule.SessionID != nil && *rule.SessionID !=
            req.SessionID {
            continue
        }
        if rule.Workspace != nil && *rule.Workspace !=
            req.Workspace {
            continue
        }

        // Check pattern match against arguments
        if rule.Pattern != "" && rule.Pattern != "*" {
            argStr := fmt.Sprintf("%v", req.Arguments)
            matcher, ok := e.matcherCache[rule.Pattern]
            if !ok {
                var err error
                matcher, err = NewWildcardMatcher(rule.Pattern)
                if err != nil {
                    continue
                }
            }
        }
    }
}

```

```

        e.matcherCache[rule.Pattern] = matcher
    }
    if !matcher.Match(argStr) {
        continue
    }
}

matching = append(matching, rule)
}

// Sort by specificity (tool name > category > pattern)
e.SortBySpecificity(matching)

return matching
}

func (e *DefaultPermissionEngine) sortBySpecificity(rules
    []PermissionRule) {
    // Sort: specific tool > specific category > wildcards
    // Implementation using sort.Slice
}

func (e *DefaultPermissionEngine) checkConditions(rule
    PermissionRule, req PermissionRequest) bool {
    cond := rule.Conditions

    // Check allowed paths
    if len(cond.AllowedPaths) > 0 {
        path, ok := req.Arguments["path"].(string)
        if ok {
            found := false
            for _, pattern := range cond.AllowedPaths {
                if matchGlob(pattern, path) {
                    found = true
                    break
                }
            }
            if !found {
                return false
            }
        }
    }

    // Check blocked paths
    if len(cond.BlockedPaths) > 0 {
        path, ok := req.Arguments["path"].(string)
        if ok {
            for _, pattern := range cond.BlockedPaths {

```

```

        if matchGlob(pattern, path) {
            return false
        }
    }
}

// Check shell command restrictions
if rule.Category == CategoryShellExec {
    cmd, ok := req.Arguments["command"].(string)
    if ok {
        cmdName := strings.Fields(cmd)[0]

        if len(cond.BlockedCommands) > 0 {
            for _, blocked := range cond.BlockedCommands {
                if cmdName == blocked {
                    return false
                }
            }
        }

        if len(cond.AllowedCommands) > 0 {
            found := false
            for _, allowed := range cond.AllowedCommands {
                if cmdName == allowed {
                    found = true
                    break
                }
            }
            if !found {
                return false
            }
        }
    }
}

// Check max args
if cond.MaxArgs != nil {
    if args, ok := req.Arguments["args"].([]string); ok {
        if len(args) > *cond.MaxArgs {
            return false
        }
    }
}

return true
}

```

```

func (e *DefaultPermissionEngine) calculateRiskLevel(req
    PermissionRequest) RiskLevel {
    switch req.Category {
    case CategoryFileRead, CategoryLSPCall:
        return RiskLow
    case CategoryFileWrite, CategoryGitOp, CategoryNetworkCall:
        return RiskMedium
    case CategoryFileDelete, CategoryShellExec, CategoryMCPCall:
        return RiskHigh
    case CategoryDBQuery:
        // DB queries could be either read or write
        query, ok := req.Arguments["query"].(string)
        if ok {
            upper := strings.ToUpper(query)
            if strings.Contains(upper, "INSERT") ||
                strings.Contains(upper, "UPDATE") ||
                strings.Contains(upper, "DELETE") ||
                strings.Contains(upper, "DROP") {
                return RiskCritical
            }
        }
        return RiskMedium
    default:
        return RiskMedium
    }
}

```

```

func (e *DefaultPermissionEngine) GrantPermission(ctx
    context.Context, rule PermissionRule) error {
    // Compile pattern
    if rule.Pattern != "" && rule.Pattern != "*" {
        _, err := NewWildcardMatcher(rule.Pattern)
        if err != nil {
            return fmt.Errorf("invalid pattern: %w", err)
        }
    }

    // Set metadata
    rule.CreatedAt = time.Now()
    if rule.Mode == AllowSession {
        rule.SessionID = &e.sessionID
    } else if rule.Mode == AllowWorkspace {
        rule.Workspace = &e.workspace
    }

    // Store in database
    if err := e.store.CreatePermissionRule(ctx, &rule); err !=
        nil {

```

```

        return fmt.Errorf("storing rule: %w", err)
    }

    // Reload rules
    return e.reloadRules(ctx)
}

func (e *DefaultPermissionEngine) reloadRules(ctx
    context.Context) error {
    rules, err := e.store.ListPermissionRules(ctx, RuleFilter{
        SessionID: &e.sessionID,
        Workspace: &e.workspace,
        ActiveOnly: true,
    })
    if err != nil {
        return err
    }

    e.rules = rules
    return nil
}

// RevokePermission, ListRules, UpdateRule, CleanupExpired
// implementations follow...
// ExportRules and ImportRules for YAML serialization

```

Step 4: Tool integration

```

// internal/tool/executor.go
func (e *ToolExecutor) Execute(ctx context.Context, req
    ToolRequest) (*ToolResult, error) {
    // Build permission request
    permReq := permissions.PermissionRequest{
        SessionID: e.sessionID,
        Workspace: e.workspace,
        Category: req.Category,
        ToolName: req.Name,
        Arguments: req.Arguments,
        Description: req.Description,
        Timestamp: time.Now(),
    }

    result, err := e.permissionEngine.CheckPermission(ctx,
        permReq)
    if err != nil {
        return nil, fmt.Errorf("permission check: %w", err)
    }

    if !result.Allowed {

```

```

    if e.ui != nil {
        // Show interactive prompt
        userChoice, err := e.ui.PromptPermission(ctx,
            permReq, result)
        if err != nil {
            return nil, fmt.Errorf("permission prompt: %w",
                err)
        }

        if userChoice.Granted {
            // Create rule based on user choice
            rule := permissions.PermissionRule{
                Category: permReq.Category,
                ToolName:   permReq.ToolName,
                Mode:       userChoice.Mode,
            }
            if err :=
                e.permissionEngine.GrantPermission(ctx, rule); err !=
                nil {
                return nil, err
            }
        } else {
            return &ToolResult{Error: "Permission denied by
                user"}, nil
        }
    } else {
        return &ToolResult{Error:
            "Permission required but no UI available"}, nil
    }
}

// Execute the tool
return e.doExecute(ctx, req)
}

```

2.6 Testing Approach

```

func TestPermissionEngine_WildcardMatching(t *testing.T) {
    engine := setupTestEngine()

    // Test **/*.go pattern
    matcher, _ := NewWildcardMatcher("**/*.go")
    assert.True(t, matcher.Match("src/main.go"))
    assert.True(t, matcher.Match("a/b/c/deep.go"))
    assert.False(t, matcher.Match("README.md"))

    // Test src/**/*.go
    matcher2, _ := NewWildcardMatcher("src/**/*.go")

```



```

    assert.True(t, matcher2.Match("src/pkg/foo/test_bar.go"))
    assert.False(t, matcher2.Match("other/test_bar.go"))
}

func TestPermissionEngine_CheckPermission(t *testing.T) {
    engine, store := setupTestEngine()

    // Grant workspace permission for file writes
    engine.GrantPermission(ctx, PermissionRule{
        Category: CategoryFileWrite,
        ToolName: "write_file",
        Pattern:   "src/**/*.go",
        Mode:     AllowWorkspace,
    })

    // Should allow write to src/main.go
    result, err := engine.CheckPermission(ctx, PermissionRequest{
        Category: CategoryFileWrite,
        ToolName:  "write_file",
        Workspace: "/project",
        Arguments: map[string]any{"path": "src/main.go"},
    })
    assert.NoError(t, err)
    assert.True(t, result.Allowed)

    // Should deny write to vendor/ (not matching pattern)
    result2, err := engine.CheckPermission(ctx,
        PermissionRequest{
            Category: CategoryFileWrite,
            ToolName:  "write_file",
            Workspace: "/project",
            Arguments: map[string]any{"path": "vendor/lib.go"},
        })
    assert.NoError(t, err)
    assert.False(t, result2.Allowed)
}

func TestPermissionEngine_AllowOnce(t *testing.T) {
    engine, _ := setupTestEngine()

    // Grant allow_once
    engine.GrantPermission(ctx, PermissionRule{
        Category: CategoryShellExec,
        ToolName:  "run_command",
        Mode:     AllowOnce,
    })

    // First use should be allowed

```

```

    result1, _ := engine.CheckPermission(ctx, PermissionRequest{
        Category: CategoryShellExec,
        ToolName: "run_command",
    })
    assert.True(t, result1.Allowed)

    // Second use should be denied (rule consumed)
    result2, _ := engine.CheckPermission(ctx, PermissionRequest{
        Category: CategoryShellExec,
        ToolName: "run_command",
    })
    assert.False(t, result2.Allowed)
}

```

2.7 Edge Cases

Edge Case	Handling
Conflicting rules (allow + deny same tool)	Specificity-based resolution: tool name > category > pattern
Rule for non-existent session	Session-scoped rules auto-expire with session
Path traversal in glob patterns	Normalize paths, reject . . sequences
Shell command injection in rules	Validate command names against whitelist
Permission prompt during batch operation	Queue prompts, fail batch if interactive needed
Rule file corruption on import	Validate schema, skip invalid rules with warnings
Timezone issues with expiry	Store all times in UTC
Concurrent rule modification	Optimistic locking with version field
Rule explosion (too many rules)	LRU cache, pagination for listing
Default mode for new workspaces	Inherit from global config, prompt on first use

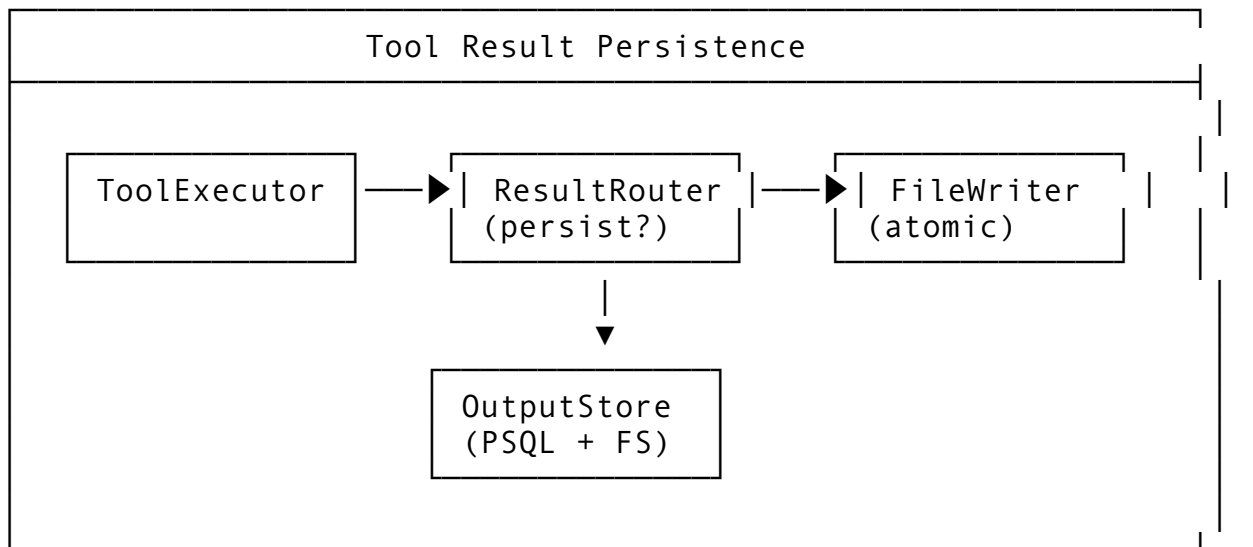
Feature 3: Tool Result Persistence (persistedOutputPath)

3.1 Feature Description

Tool Result Persistence saves the output of tool executions to a specified file path (`persistedOutputPath`), allowing the agent and user to reference previous results without re-executing expensive operations. This is essential for build logs, test outputs, search results, and analysis reports.

Why it matters: Re-running tests, builds, or searches wastes tokens and time. Persisted outputs create an audit trail and enable incremental workflows.

3.2 Architecture Design



3.3 API Design

```
// Package: internal/tool/persist
package persist

import (
    "context"
    "fmt"
    "io"
    "os"
    "path/filepath"
    "time"
)

// PersistConfig configures output persistence
type PersistConfig struct {
    Enabled          bool    `mapstructure:"enabled"`
    DefaultDir       string  `mapstructure:"default_dir" ` //
        e.g., ".helix/outputs"
    MaxFileSize      int64   `mapstructure:"max_file_size" ` //
        Max bytes per output
    MaxTotalSize     int64   `mapstructure:"max_total_size" ` //
        Max total bytes per session
    AutoCleanup      bool    `mapstructure:"auto_cleanup" ` //
        Remove old outputs
    RetentionDays    int     `mapstructure:"retention_days" `
    Compression      bool    `mapstructure:"compression" ` //
        gzip large outputs
    CompressionThreshold int64   `mapstructure:"compression_threshold" `
}
```

```

// ToolOutput represents a persisted tool execution result
type ToolOutput struct {
    ID            string    `json:"id" db:"id"`
    SessionID     string    `json:"session_id" db:"session_id"`
    ToolName      string    `json:"tool_name" db:"tool_name"`
    ToolArgs      map[string]any `json:"tool_args" db:"tool_args"`

    // Output metadata
    OutputPath    string    `json:"output_path" db:"output_path" // Relative to workspace`
    AbsolutePath  string    `json:"absolute_path" db:"absolute_path" // Full filesystem path`
    Size          int64     `json:"size" db:"size"`
    IsCompressed  bool      `json:"is_compressed" db:"is_compressed"`
    Checksum      string    `json:"checksum" db:"checksum" // SHA-256`
    ContentType   string    `json:"content_type" db:"content_type" // e.g., "text/plain", "application/json"`

    // Execution metadata
    StartedAt     time.Time `json:"started_at" db:"started_at"`
    CompletedAt   time.Time `json:"completed_at" db:"completed_at"`
    DurationMs    int64     `json:"duration_ms" db:"duration_ms"`
    ExitCode      int       `json:"exit_code" db:"exit_code"`
    WasTruncated  bool      `json:"was_truncated" db:"was_truncated"`
    TruncatedAt   int64     `json:"truncated_at,omitempty" db:"truncated_at"`

    // Reference management
    ReferenceCount int       `json:"reference_count" db:"reference_count"`
    LastAccessed   *time.Time `json:"last_accessed,omitempty" db:"last_accessed"`

    CreatedAt     time.Time `json:"created_at" db:"created_at"`
}

// PersistedOutputManager handles saving and retrieving tool outputs
type PersistedOutputManager interface {
    // SaveOutput persists a tool execution result

```

```

SaveOutput(ctx context.Context, req SaveRequest)
    (*ToolOutput, error)

// GetOutput retrieves a persisted output
GetOutput(ctx context.Context, outputID string)
    (*ToolOutput, io.ReadCloser, error)

// GetOutputByTool finds the most recent output for a
    tool+args combination
GetOutputByTool(ctx context.Context, sessionID, toolName
    string, args map[string]any) (*ToolOutput, error)

// ListOutputs returns outputs for a session
ListOutputs(ctx context.Context, sessionID string, filter
    OutputFilter) ([]ToolOutput, error)

// DeleteOutput removes a persisted output
DeleteOutput(ctx context.Context, outputID string) error

// CleanupOldOutputs removes outputs older than retention
    policy
CleanupOldOutputs(ctx context.Context) error

// GetStorageStats returns storage usage statistics
GetStorageStats(ctx context.Context, sessionID string)
    (*StorageStats, error)
}

// SaveRequest parameters for persisting output
type SaveRequest struct {
    SessionID      string          `json:"session_id"`
    ToolName       string          `json:"tool_name"`
    ToolArgs       map[string]any `json:"tool_args"`
    OutputPath     string          `json:"output_path,omitempty"` // User-specified or auto-
                                generated
    Content        []byte          `json:"content"`
    ContentType    string          `json:"content_type,omitempty"`
    ExitCode       int             `json:"exit_code"`
    StartedAt      time.Time       `json:"started_at"`
    CompletedAt    time.Time       `json:"completed_at"`
    ForcePath      bool            `json:"force_path"` // Use
                                exact path vs auto-generate
}

// OutputFilter filters output listings
type OutputFilter struct {

```

```

ToolName      *string    `json:"tool_name,omitempty"`
Since         *time.Time `json:"since,omitempty"`
Before        *time.Time `json:"before,omitempty"`
MinSize       *int64     `json:"min_size,omitempty"`
MaxSize       *int64     `json:"max_size,omitempty"`
ContentType   *string    `json:"content_type,omitempty"`
}

// StorageStats reports storage usage
type StorageStats struct {
    TotalOutputs  int64 `json:"total_outputs"`
    TotalSize     int64 `json:"total_size_bytes"`
    CompressedSize int64 `json:"compressed_size_bytes"`
    OldestOutput  *time.Time `json:"oldest_output,omitempty"`
    NewestOutput  *time.Time `json:"newest_output,omitempty"`
}

```

3.4 Integration Points

Package	Modification	Description
internal/tool/persist/	NEW PACKAGE	Output persistence manager
internal/tool/executor.go	Post-execution hook	Auto-save after tool execution
internal/db/migrations/	Add tool_outputs table	Metadata persistence
internal/fs/	Add atomic write helper	Safe file writing with temp+rename
cmd/helix/main.go	Add --output-dir flag	Configure output directory
internal/config/	Add persist config	Viper integration

3.5 Implementation Steps

Step 1: Atomic file writer

```

// internal/fs/atomic_writer.go
package fs

import (
    "crypto/sha256"
    "fmt"
    "io"
    "os"
    "path/filepath"
)

```

```

// WriteAtomic writes data to a file atomically using temp+rename
func WriteAtomic(path string, data []byte) (checksum string,
    size int64, err error) {
    dir := filepath.Dir(path)
    if err := os.MkdirAll(dir, 0755); err != nil {
        return "", 0, fmt.Errorf("creating directory: %w", err)
    }

    // Create temp file in same directory (for atomic rename)
    tmpFile, err := os.CreateTemp(dir, ".tmp-write-*")
    if err != nil {
        return "", 0, fmt.Errorf("creating temp file: %w", err)
    }
    tmpPath := tmpFile.Name()

    // Ensure cleanup on error
    defer func() {
        if err != nil {
            os.Remove(tmpPath)
        }
    }()

    // Write data and compute checksum
    h := sha256.New()
    w := io.MultiWriter(tmpFile, h)

    n, err := w.Write(data)
    if err != nil {
        return "", 0, fmt.Errorf("writing data: %w", err)
    }

    if err := tmpFile.Close(); err != nil {
        return "", 0, fmt.Errorf("closing temp file: %w", err)
    }

    // Atomic rename
    if err := os.Rename(tmpPath, path); err != nil {
        return "", 0, fmt.Errorf("renaming file: %w", err)
    }

    checksum = fmt.Sprintf("sha256:%x", h.Sum(nil))
    return checksum, int64(n), nil
}

```

Step 2: PersistedOutputManager implementation

```

// internal/tool/persist/manager.go
package persist

```

```

import (
    "bytes"
    "compress/gzip"
    "context"
    "crypto/sha256"
    "encoding/json"
    "fmt"
    "io"
    "os"
    "path/filepath"
    "strings"
    "time"

    "dev.helix.code/internal/db"
    "dev.helix.code/internal/fs"
    "dev.helix.code/pkg/logger"
)

type DefaultPersistedOutputManager struct {
    config *PersistConfig
    store  db.Store
    log    logger.Logger
}

func NewPersistedOutputManager(config *PersistConfig, store
    db.Store, log logger.Logger)
    (*DefaultPersistedOutputManager, error) {
    // Ensure default directory exists
    if config.Enabled && config.DefaultDir != "" {
        if err := os.MkdirAll(config.DefaultDir, 0755); err !=
            nil {
            return nil, fmt.Errorf("creating output directory:
                %w", err)
        }
    }

    return &DefaultPersistedOutputManager{
        config: config,
        store:  store,
        log:    log,
    }, nil
}

func (m *DefaultPersistedOutputManager) SaveOutput(
    ctx context.Context,
    req SaveRequest,
) (*ToolOutput, error) {

```



```

if !m.config.Enabled {
    return nil, fmt.Errorf("output persistence is disabled")
}

// Determine output path
outputPath := req.OutputPath
if outputPath == "" || !req.ForcePath {
    outputPath = m.generateOutputPath(req)
}

absPath := m.resolvePath(outputPath)

// Check size limits
contentSize := int64(len(req.Content))
if m.config.MaxFileSize > 0 && contentSize >
    m.config.MaxFileSize {
    // Truncate with notice
    req.Content = m.truncateContent(req.Content,
        m.config.MaxFileSize)
    contentSize = int64(len(req.Content))
}

// Check session total size
stats, _ := m.GetStorageStats(ctx, req.SessionID)
if m.config.MaxTotalSize > 0 && stats != nil &&
    stats.TotalSize+contentSize > m.config.MaxTotalSize {
    return nil, fmt.Errorf("session storage limit exceeded:
        %d bytes", m.config.MaxTotalSize)
}

// Compress if large
var isCompressed bool
var finalContent []byte = req.Content
if m.config.Compression && contentSize >
    m.config.CompressionThreshold {
    compressed, err := m.compress(req.Content)
    if err == nil && int64(len(compressed)) < contentSize {
        finalContent = compressed
        isCompressed = true
    }
}

// Write atomically
checksum, size, err := fs.WriteAtomic(absPath, finalContent)
if err != nil {
    return nil, fmt.Errorf("writing output file: %w", err)
}

```

```

// Determine content type
contentType := req.ContentType
if contentType == "" {
    contentType = m.detectContentType(req.Content)
}

// Create record
output := &ToolOutput{
    ID:                generateID(),
    SessionID:         req.SessionID,
    ToolName:          req.ToolName,
    ToolArgs:          req.ToolArgs,
    OutputPath:        outputPath,
    AbsolutePath:      absPath,
    Size:              contentType, // Original size, not
                        compressed
    IsCompressed:       isCompressed,
    Checksum:           checksum,
    ContentType:        contentType,
    StartedAt:          req.StartedAt,
    CompletedAt:        req.CompletedAt,
    DurationMs:         req.CompletedAt.Sub(req.StartedAt).Milliseconds(),
    ExitCode:           req.ExitCode,
    WasTruncated:       len(req.Content) < len(req.Content), //
                        Actually compare
    CreatedAt:          time.Now(),
}

// Store in database
if err := m.store.CreateToolOutput(ctx, output); err != nil {
    // Attempt cleanup
    os.Remove(absPath)
    return nil, fmt.Errorf("storing output metadata: %w",
        err)
}

m.log.Info("output persisted",
    "output_id", output.ID,
    "path", outputPath,
    "size", size,
    "compressed", isCompressed,
)

return output, nil
}

```

```

func (m *DefaultPersistedOutputManager) generateOutputPath(req
    SaveRequest) string {
    // Generate path based on tool and timestamp
    // e.g., .helix/outputs/2024-01-15/shell_exec_14-30-22.log
    now := time.Now()
    dateDir := now.Format("2006-01-02")

    // Sanitize tool name
    safeToolName := strings.ReplaceAll(req.ToolName, "/", "_")

    // Generate filename
    ext := m.guessExtension(req.ContentType)
    filename := fmt.Sprintf("%s_%s%s", safeToolName,
        now.Format("15-04-05"), ext)

    // Include hash of args for uniqueness
    argsHash := m.hashArgs(req.ToolArgs)
    if argsHash != "" {
        filename = fmt.Sprintf("%s_%s%s", safeToolName,
            argsHash[:8], ext)
    }

    return filepath.Join(m.config.DefaultDir, dateDir, filename)
}

func (m *DefaultPersistedOutputManager) resolvePath(relPath
    string) string {
    if filepath.IsAbs(relPath) {
        return relPath
    }
    // Resolve relative to workspace
    return filepath.Join(m.config.DefaultDir, relPath)
}

func (m *DefaultPersistedOutputManager) compress(data []byte)
    ([]byte, error) {
    var buf bytes.Buffer
    gz := gzip.NewWriter(&buf)
    if _, err := gz.Write(data); err != nil {
        return nil, err
    }
    if err := gz.Close(); err != nil {
        return nil, err
    }
    return buf.Bytes(), nil
}

```

```

func (m *DefaultPersistedOutputManager) decompress(data []byte)
    ([]byte, error) {
    r, err := gzip.NewReader(bytes.NewReader(data))
    if err != nil {
        return nil, err
    }
    defer r.Close()
    return io.ReadAll(r)
}

func (m *DefaultPersistedOutputManager) hashArgs(args
    map[string]any) string {
    if len(args) == 0 {
        return ""
    }
    data, _ := json.Marshal(args)
    h := sha256.Sum256(data)
    return fmt.Sprintf("%x", h[:8])
}

func (m *DefaultPersistedOutputManager) detectContentType(data
    []byte) string {
    // Simple detection
    if len(data) > 0 && data[0] == '{' || data[0] == '[' {
        return "application/json"
    }
    if bytes.Contains(data, []byte("<?xml")) {
        return "application/xml"
    }
    // Check for binary content
    if bytes.Contains(data, []byte{0}) {
        return "application/octet-stream"
    }
    return "text/plain"
}

func (m *DefaultPersistedOutputManager)
    guessExtension(contentType string) string {
    switch contentType {
    case "application/json": return ".json"
    case "application/xml": return ".xml"
    case "text/html": return ".html"
    case "application/octet-stream": return ".bin"
    default: return ".txt"
    }
}

```

```

func (m *DefaultPersistedOutputManager) truncateContent(content
    [][]byte, maxSize int64) [][]byte {
    if int64(len(content)) <= maxSize {
        return content
    }

    // Truncate and add notice
    notice := [][]byte("\n\n[OUTPUT TRUNCATED: exceeded max size]
        \n")
    cutPoint := int(maxSize) - len(notice) - 100
    if cutPoint < 100 {
        cutPoint = int(maxSize) - len(notice)
    }

    result := make([][]byte, 0, maxSize)
    result = append(result, content[:cutPoint]...)
    result = append(result, notice...)
    return result
}

// GetOutput, GetOutputByTool, ListOutputs, DeleteOutput,
// CleanupOldOutputs, GetStorageStats
// implementations follow similar patterns...

func generateID() string {
    // Use UUID or similar
    return fmt.Sprintf("out_%d", time.Now().UnixNano())
}

```

3.6 Testing Approach

```

func TestPersistedOutputManager_SaveOutput(t *testing.T) {
    manager, store := setupTestManager()

    content := [][]byte("Test output from command execution\nLine
        2\nLine 3")
    req := SaveRequest{
        SessionID: "session-1",
        ToolName: "shell_exec",
        ToolArgs: map[string]any{"command": "ls -la"},
        OutputPath: "test_output.txt",
        Content: content,
        ContentType: "text/plain",
        ExitCode: 0,
        StartedAt: time.Now().Add(-time.Second),
        CompletedAt: time.Now(),
        ForcePath: true,
    }
}

```

```

output, err := manager.SaveOutput(ctx, req)
assert.NoError(t, err)
assert.NotEmpty(t, output.ID)
assert.Equal(t, int64(len(content)), output.Size)
assert.True(t, strings.HasPrefix(output.Checksum, "sha256:"))

// Verify file exists
_, err = os.Stat(output.AbsolutePath)
assert.NoError(t, err)

// Verify database record
stored, err := store.GetToolOutput(ctx, output.ID)
assert.NoError(t, err)
assert.Equal(t, output.ToolName, stored.ToolName)
}

func TestPersistedOutputManager_Compression(t *testing.T) {
manager, _ := setupTestManager()
manager.config.Compression = true
manager.config.CompressionThreshold = 100

// Large content should be compressed
largeContent := bytes.Repeat([]byte("Large content block "),
    1000)
req := SaveRequest{
    SessionID: "session-1",
    ToolName:  "large_output",
    Content:   largeContent,
}

output, err := manager.SaveOutput(ctx, req)
assert.NoError(t, err)
assert.True(t, output.IsCompressed)
}

func TestPersistedOutputManager_SizeLimit(t *testing.T) {
manager, _ := setupTestManager()
manager.config.MaxFileSize = 50

content := bytes.Repeat([]byte("x"), 100)
req := SaveRequest{
    SessionID: "session-1",
    ToolName:  "test",
    Content:   content,
}

output, err := manager.SaveOutput(ctx, req)

```

```

    assert.NoError(t, err)
    assert.True(t, output.WasTruncated)
    assert.LessOrEqual(t, output.Size,
        manager.config.MaxFileSize)
}

```

3.7 Edge Cases

Edge Case	Handling
Disk full during write	Pre-check available space, fallback to memory-only storage
Concurrent writes to same path	Atomic write with temp+rename prevents corruption
Path traversal attack	Sanitize and validate all paths, reject . .
Very large outputs (>1GB)	Stream instead of buffering, chunked storage
Binary output content	Store as-is, detect content type, skip compression if already compressed
Network filesystem latency	Use local temp directory for atomic write, then move
Database write succeeds but file fails	Two-phase commit or cleanup on failure
Session storage exceeded	LRU eviction or reject new saves with clear error
Output path already exists	Append timestamp suffix or prompt for overwrite
Unicode filenames	Normalize with NFKC, reject non-portable characters

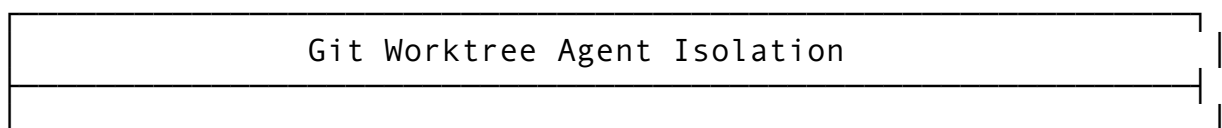
Feature 4: Git Worktree Agent Isolation

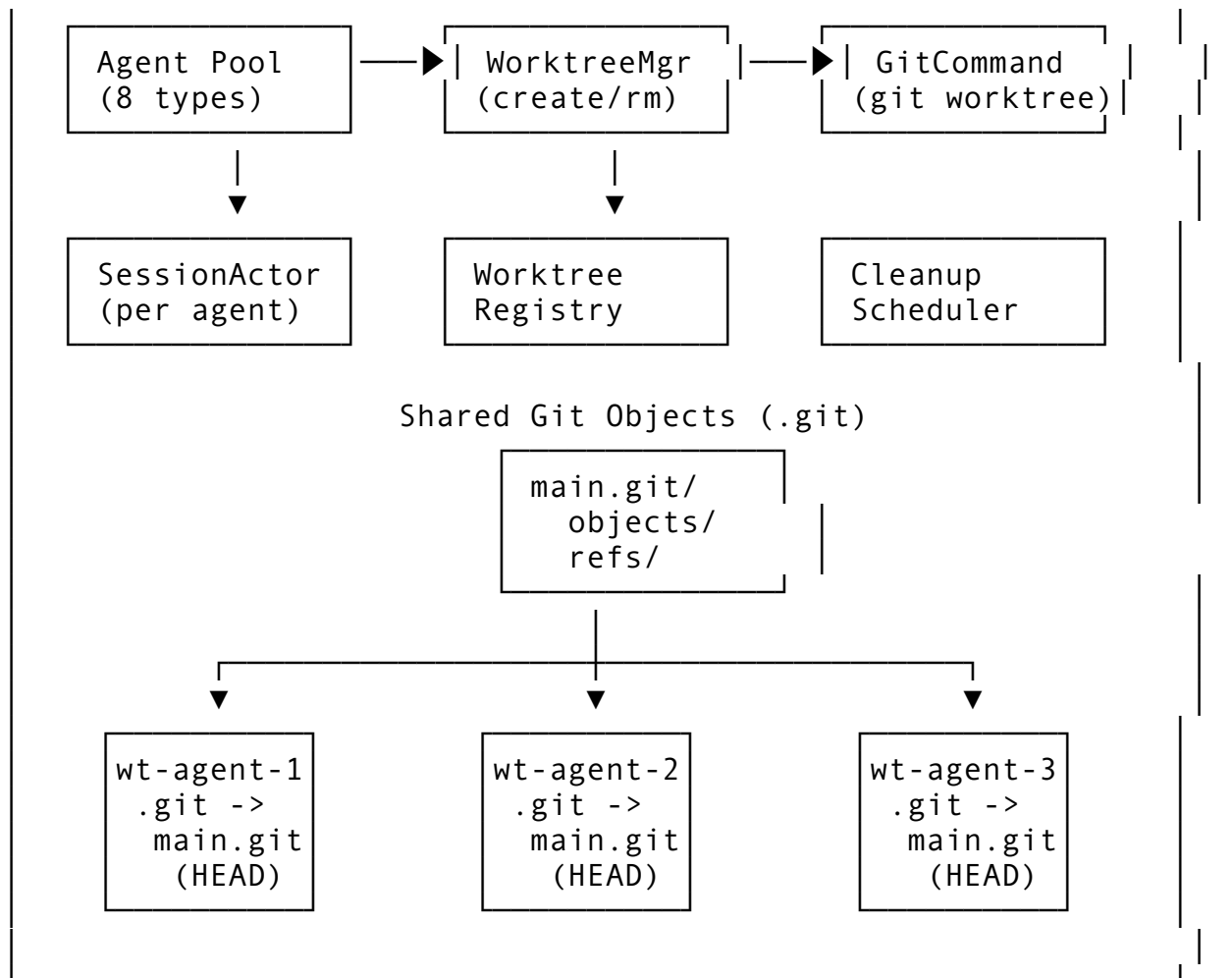
4.1 Feature Description

Git Worktree Agent Isolation creates independent working trees for each agent session, allowing multiple agents to work on the same repository simultaneously without interfering with each other. Each worktree has its own checkout but shares the same underlying Git object database.

Why it matters: Multiple agents working on the same codebase need isolation to prevent conflicts. Worktrees are lightweight (no duplicate object storage) and provide proper Git context for each agent.

4.2 Architecture Design





4.3 API Design

```
// Package: internal/git/worktree
package worktree

import (
    "context"
    "fmt"
    "os"
    "path/filepath"
    "strings"
    "time"
)

// WorktreeConfig configures worktree management
type WorktreeConfig struct {
    Enabled      bool          `mapstructure:"enabled"`
    BaseDir      string        `mapstructure:"base_dir" // Where to create
                        worktrees`
}
```



```

NamingPattern      string
    `mapstructure:"naming_pattern"` // e.g., "helix-wt-
    {agent_id}-{timestamp}"
DefaultBranch      string
    `mapstructure:"default_branch"` // Branch to base
    worktrees on
AutoCleanup        bool
    `mapstructure:"auto_cleanup"`   // Remove on session
    end
MaxWorktrees       int
    `mapstructure:"max_worktrees"`   // Limit per repo
LockTimeout        time.Duration
    `mapstructure:"lock_timeout"`    // How long to hold
    worktree
StashOnCleanup     bool
    `mapstructure:"stash_on_cleanup"` // Stash changes before
    removing
}

// WorktreeInfo tracks an active worktree
type WorktreeInfo struct {
    ID          string `json:"id" db:"id"`
    SessionID   string `json:"session_id" db:"session_id"`
    AgentID     string `json:"agent_id" db:"agent_id"`
    AgentType   string `json:"agent_type" db:"agent_type"`

    // Git metadata
    RepoPath    string `json:"repo_path" db:"repo_path"` // Path to main repo
    WorktreePath string `json:"worktree_path" db:"worktree_path"`
    Branch      string `json:"branch" db:"branch"`
    CommitHash  string `json:"commit_hash" db:"commit_hash"`
    IsBare      bool   `json:"is_bare" db:"is_bare"`

    // Lifecycle
    CreatedAt    time.Time `json:"created_at" db:"created_at"`
    LastAccessed time.Time `json:"last_accessed" db:"last_accessed"`
    ExpiresAt    *time.Time `json:"expires_at,omitempty" db:"expires_at"`
    IsActive     bool       `json:"is_active" db:"is_active"`

    // State tracking
    HasChanges    bool `json:"has_changes" db:"has_changes"`
    StashedRef    *string `json:"stashed_ref,omitempty" db:"stashed_ref"`
}

```

```

// WorktreeManager manages Git worktrees for agent isolation
type WorktreeManager interface {
    // CreateWorktree sets up a new worktree for an agent session
    CreateWorktree(ctx context.Context, req
        CreateWorktreeRequest) (*WorktreeInfo, error)

    // GetWorktree retrieves worktree info by ID
    GetWorktree(ctx context.Context, worktreeID string)
        (*WorktreeInfo, error)

    // GetOrCreateWorktree gets existing or creates new for
    session
    GetOrCreateWorktree(ctx context.Context, req
        CreateWorktreeRequest) (*WorktreeInfo, error)

    // ListWorktrees returns all worktrees for a repository
    ListWorktrees(ctx context.Context, repoPath string)
        ([]WorktreeInfo, error)

    // RemoveWorktree deletes a worktree and cleans up
    RemoveWorktree(ctx context.Context, worktreeID string, force
        bool) error

    // StashWorktree saves current changes and cleans
    StashWorktree(ctx context.Context, worktreeID string,
        message string) error

    // SyncWorktree pulls latest changes from main
    SyncWorktree(ctx context.Context, worktreeID string) error

    // GetWorktreeStatus returns git status of worktree
    GetWorktreeStatus(ctx context.Context, worktreeID string)
        (*WorktreeStatus, error)

    // CleanupExpired removes old/inactive worktrees
    CleanupExpired(ctx context.Context) error

    // LockWorktree prevents cleanup
    LockWorktree(ctx context.Context, worktreeID
        string, duration time.Duration) error

    // UnlockWorktree allows cleanup
    UnlockWorktree(ctx context.Context, worktreeID string) error
}

// CreateWorktreeRequest parameters for worktree creation
type CreateWorktreeRequest struct {

```

```

    SessionID    string `json:"session_id"`
    AgentID      string `json:"agent_id"`
    AgentType    string `json:"agent_type"`
    RepoPath     string `json:"repo_path"`
    BaseBranch   string `json:"base_branch,omitempty"` // Branch
    to create from
    NewBranch    string `json:"new_branch,omitempty"` // Create
    and checkout this branch
    TrackRemote  bool   `json:"track_remote"`           // Set up
    tracking branch
}

// WorktreeStatus reports git status
type WorktreeStatus struct {
    Branch          string          `json:"branch"`
    AheadBehind     [2]int         `json:"ahead_behind"` //
    [ahead, behind]
    ChangedFiles    []FileChange    `json:"changed_files"`
    UntrackedFiles  []string        `json:"untracked_files"`
    IsClean         bool            `json:"is_clean"`
    LastCommitMsg   string          `json:"last_commit_msg"`
    LastCommitHash  string          `json:"last_commit_hash"`
}

// FileChange represents a changed file
type FileChange struct {
    Path    string `json:"path"`
    Status  string `json:"status"` // M, A, D, R, C, U
    Staged  bool   `json:"staged"`
    OldPath string `json:"old_path,omitempty"` // For renames
}

```

4.4 Integration Points

Package	Modification	Description
internal/git/worktree/	NEW PACKAGE	Worktree manager
internal/actor/agent.go	Worktree assignment	Each agent gets its own worktree
internal/actor/session.go	Session lifecycle	Create on start, cleanup on end
internal/db/migrations/	Add worktrees table	Track active worktrees
cmd/helix/main.go	Add --worktree flag	Enable/disable worktree isolation

Package	Modification	Description
internal/ config/	Add worktree config	Viper integration

4.5 Implementation Steps

Step 1: Worktree creation

```
// internal/git/worktree/manager.go
package worktree

import (
    "context"
    "fmt"
    "os"
    "os/exec"
    "path/filepath"
    "strings"
    "time"

    "dev.helix.code/internal/db"
    "dev.helix.code/pkg/logger"
)

type GitWorktreeManager struct {
    config *WorktreeConfig
    store  db.Store
    log    logger.Logger
}

func NewGitWorktreeManager(config *WorktreeConfig, store
    db.Store, log logger.Logger) *GitWorktreeManager {
    return &GitWorktreeManager{
        config: config,
        store:  store,
        log:    log,
    }
}

func (m *GitWorktreeManager) CreateWorktree(
    ctx context.Context,
    req CreateWorktreeRequest,
) (*WorktreeInfo, error) {
    if !m.config.Enabled {
        // Return repo path directly if worktrees disabled
        return &WorktreeInfo{
            RepoPath:      req.RepoPath,
            WorktreePath:  req.RepoPath,
        }
    }
}
```

```

        Branch:          m.config.DefaultBranch,
    }, nil
}

// Validate repo
gitDir := filepath.Join(req.RepoPath, ".git")
if _, err := os.Stat(gitDir); err != nil {
    return nil, fmt.Errorf("not a git repository: %w", err)
}

// Check max worktrees
existing, err := m.ListWorktrees(ctx, req.RepoPath)
if err != nil {
    return nil, err
}
if len(existing) >= m.config.MaxWorktrees {
    return nil, fmt.Errorf("max worktrees (%d) reached for repo", m.config.MaxWorktrees)
}

// Generate worktree path
worktreeID := fmt.Sprintf("wt-%s-%s-%d", req.AgentType,
    req.AgentID[:8], time.Now().Unix())
if m.config.BaseDir != "" {
    req.RepoPath = m.config.BaseDir
}
worktreePath := filepath.Join(req.RepoPath, ".helix",
    "worktrees", worktreeID)

// Ensure directory exists
if err := os.MkdirAll(filepath.Dir(worktreePath), 0755);
    err != nil {
    return nil, fmt.Errorf("creating worktree directory: %w", err)
}

// Determine branch setup
baseBranch := req.BaseBranch
if baseBranch == "" {
    baseBranch = m.config.DefaultBranch
}

// Build git worktree add command
var cmd *exec.Cmd
if req.NewBranch != "" {
    // Create new branch and worktree
    cmd = exec.CommandContext(ctx, "git", "worktree", "add",
        "-b", req.NewBranch, worktreePath, baseBranch)
}

```

```

} else {
    // Create worktree from existing branch
    cmd = exec.CommandContext(ctx, "git", "worktree", "add",
        worktreePath, baseBranch)
}
cmd.Dir = req.RepoPath

output, err := cmd.CombinedOutput()
if err != nil {
    return nil, fmt.Errorf("git worktree add failed:
        %w\nOutput: %s", err, string(output))
}

// Get current commit hash
commitHash, err := m.getCommitHash(ctx, worktreePath)
if err != nil {
    return nil, fmt.Errorf("getting commit hash: %w", err)
}

// Get actual branch name
branch, err := m.getBranchName(ctx, worktreePath)
if err != nil {
    return nil, fmt.Errorf("getting branch name: %w", err)
}

// Create info record
info := &WorktreeInfo{
    ID:          worktreeID,
    SessionID:   req.SessionID,
    AgentID:     req.AgentID,
    AgentType:   req.AgentType,
    RepoPath:    req.RepoPath,
    WorktreePath: worktreePath,
    Branch:      branch,
    CommitHash:  commitHash,
    CreatedAt:   time.Now(),
    LastAccessed: time.Now(),
    IsActive:    true,
}

// Store in database
if err := m.store.CreateWorktree(ctx, info); err != nil {
    // Attempt cleanup
    m.runGit(ctx, req.RepoPath, "worktree", "remove", "--
        force", worktreePath)
    return nil, fmt.Errorf("storing worktree info: %w", err)
}

```

```

m.log.Info("worktree created",
    "id", worktreeID,
    "path", worktreePath,
    "branch", branch,
    "agent", req.AgentType,
)

return info, nil
}

func (m *GitWorktreeManager) RemoveWorktree(ctx context.Context,
    worktreeID string, force bool) error {
    info, err := m.GetWorktree(ctx, worktreeID)
    if err != nil {
        return err
    }

    // Check for uncommitted changes
    if !force {
        status, err := m.GetWorktreeStatus(ctx, worktreeID)
        if err != nil {
            return err
        }

        if !status.IsClean {
            if m.config.StashOnCleanup {
                stashMsg := fmt.Sprintf("helix-auto-stash:
worktree %s", worktreeID)
                if err := m.StashWorktree(ctx, worktreeID,
stashMsg); err != nil {
                    return fmt.Errorf("stashing changes: %w",
err)
                }
            } else {
                return fmt.Errorf("worktree has uncommitted
changes, use force=true or stash first")
            }
        }
    }

    // Remove worktree
    args := []string{"worktree", "remove"}
    if force {
        args = append(args, "--force")
    }
    args = append(args, info.WorktreePath)

    output, err := m.runGit(ctx, info.RepoPath, args...)

```

```

if err != nil {
    // If worktree already removed from git but dir exists,
    // clean up manually
    if strings.Contains(string(output), "is not a working
    tree") {
        os.RemoveAll(info.WorktreePath)
    } else {
        return fmt.Errorf("removing worktree: %w\nOutput:
        %s", err, string(output))
    }
}

// Clean up empty parent directories
parent := filepath.Dir(info.WorktreePath)
if filepath.Base(parent) == "worktrees" {
    // Don't remove the worktrees directory itself
}

// Update database
if err := m.store.DeactivateWorktree(ctx, worktreeID); err !=
nil {
    m.log.Warn("failed to deactivate worktree in db", "id",
    worktreeID, "error", err)
}

m.log.Info("worktree removed", "id", worktreeID)
return nil
}

func (m *GitWorktreeManager) SyncWorktree(ctx context.Context,
worktreeID string) error {
    info, err := m.GetWorktree(ctx, worktreeID)
    if err != nil {
        return err
    }

    // Stash any local changes temporarily
    hadChanges := false
    status, _ := m.GetWorktreeStatus(ctx, worktreeID)
    if status != nil && !status.IsClean {
        hadChanges = true
        if _, err := m.runGit(ctx, info.WorktreePath, "stash",
        "push", "-m", "helix-sync-stash"); err != nil {
            return fmt.Errorf("stashing changes: %w", err)
        }
    }

    // Fetch and pull from tracking branch

```



```

    if _, err := m.runGit(ctx, info.WorktreePath, "fetch",
        "origin", info.Branch); err != nil {
        return fmt.Errorf("fetching: %w", err)
    }

    if _, err := m.runGit(ctx, info.WorktreePath, "pull",
        "origin", info.Branch); err != nil {
        return fmt.Errorf("pulling: %w", err)
    }

    // Restore stashed changes
    if hadChanges {
        if _, err := m.runGit(ctx, info.WorktreePath, "stash",
            "pop"); err != nil {
            return fmt.Errorf("restoring stashed changes: %w",
                err)
        }
    }

    // Update last accessed
    m.store.UpdateWorktreeAccess(ctx, worktreeID, time.Now())

    return nil
}

func (m *GitWorktreeManager) getCommitHash(ctx context.Context,
    worktreePath string) (string, error) {
    output, err := m.runGit(ctx, worktreePath, "rev-parse",
        "HEAD")
    if err != nil {
        return "", err
    }
    return strings.TrimSpace(string(output)), nil
}

func (m *GitWorktreeManager) getBranchName(ctx context.Context,
    worktreePath string) (string, error) {
    output, err := m.runGit(ctx, worktreePath, "rev-parse", "--
        abbrev-ref", "HEAD")
    if err != nil {
        return "", err
    }
    return strings.TrimSpace(string(output)), nil
}

func (m *GitWorktreeManager) runGit(ctx context.Context, dir
    string, args ...string) ([]byte, error) {
    cmd := exec.CommandContext(ctx, "git", args...)

```

```

    cmd.Dir = dir
    return cmd.CombinedOutput()
}

// Additional methods: GetWorktree, GetOrCreateWorktree,
// ListWorktrees, StashWorktree,
// GetWorktreeStatus, CleanupExpired, LockWorktree,
// UnlockWorktree

```

Step 2: Agent integration

```

// internal/actor/agent.go
func (a *AgentActor) Initialize(ctx context.Context) error {
    // Create worktree for this agent
    if a.config.WorktreeEnabled {
        wt, err := a.worktreeManager.GetOrCreateWorktree(ctx,
            worktree.CreateWorktreeRequest{
                SessionID: a.sessionID,
                AgentID:     a.agentID,
                AgentType:   string(a.agentType),
                RepoPath:   a.workspacePath,
                NewBranch:   fmt.Sprintf("helix/%s/%s", a.agentType,
                    a.agentID[:8]),
            })
        if err != nil {
            return fmt.Errorf("creating worktree: %w", err)
        }
        a.worktree = wt
        a.effectiveWorkspace = wt.WorktreePath
    }

    return nil
}

func (a *AgentActor) Cleanup(ctx context.Context) error {
    if a.worktree != nil && a.config.AutoCleanup {
        if err := a.worktreeManager.RemoveWorktree(ctx,
            a.worktree.ID, false); err != nil {
            a.log.Warn("worktree cleanup failed", "error", err)
        }
    }
    return nil
}

```

4.6 Testing Approach

```

func TestGitWorktreeManager_CreateWorktree(t *testing.T) {
    // Setup temp git repo
    repoDir := t.TempDir()

```

```

runGit(t, repoDir, "init")
runGit(t, repoDir, "commit", "--allow-empty", "-m",
    "initial")

manager := setupTestManager(repoDir)

wt, err := manager.CreateWorktree(ctx,
    worktree.CreateWorktreeRequest{
        SessionID: "session-1",
        AgentID:    "agent-123",
        AgentType:  "coder",
        RepoPath:   repoDir,
        NewBranch:  "feature/test",
    })

assert.NoError(t, err)
assert.NotEmpty(t, wt.ID)
assert.Equal(t, "feature/test", wt.Branch)
assert.NotEmpty(t, wt.CommitHash)

// Verify worktree directory exists
_, err = os.Stat(wt.WorktreePath)
assert.NoError(t, err)

// Verify it's a real worktree (has .git file pointing to
// main repo)
gitLink, err := os.ReadFile(filepath.Join(wt.WorktreePath,
    ".git"))
assert.NoError(t, err)
assert.Contains(t, string(gitLink), "gitdir:")
}

func TestGitWorktreeManager_CleanupExpired(t *testing.T) {
    manager, _ := setupTestManager(repoDir)

    // Create expired worktree
    wt, _ := manager.CreateWorktree(ctx, req)
    // Simulate expiration by updating database directly

    err := manager.CleanupExpired(ctx)
    assert.NoError(t, err)

    // Verify removed
    _, err = manager.GetWorktree(ctx, wt.ID)
    assert.Error(t, err) // Should not exist
}

```

4.7 Edge Cases

Edge Case	Handling
Repo not initialized	Error out with clear message, offer to init
Main repo has uncommitted changes	Block worktree creation until clean
Worktree path already exists	Append suffix or use unique ID
Branch already exists	Reuse branch or error with suggestion
Git worktree command unavailable	Error, require git >= 2.5
Disk space low during create	Pre-check, error before git command
Submodule in worktree	Run <code>git submodule update --init</code> after creation
Large repo (GB+)	Warn about shared object store, no duplication
Worktree locked by another process	Wait with timeout, then force
Network filesystem	Use local temp for git operations, warn about performance
Windows path length limits	Use shorter IDs, configurable base path

Feature 5: Hook-Based Extensibility (9+ Event Types)

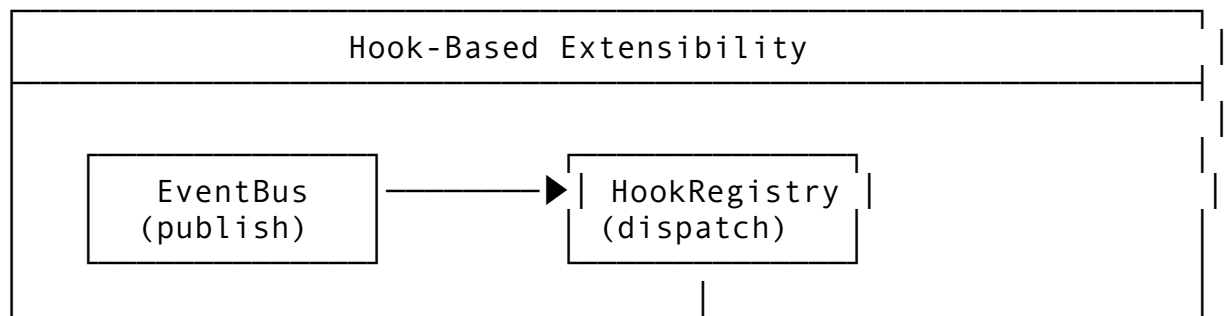
5.1 Feature Description

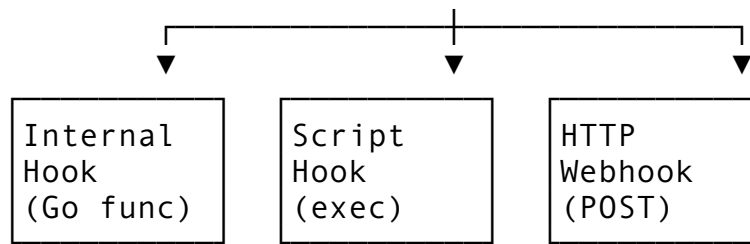
The Hook-Based Extensibility system allows external scripts and internal modules to register callbacks for lifecycle events. This enables custom logging, notifications, pre/post-processing, and integration with external systems.

Why it matters: Users need to integrate agent behavior with CI/CD, Slack, custom analytics, and team workflows without modifying core code.

The 9+ Event Types: 1. `session_start / session_end` 2. `message_send / message_receive` 3. `tool_call / tool_result` 4. `file_change / git_operation` 5. `permission_request / permission_grant` 6. `compaction / summary_generated` 7. `error / warning` 8. `plan_created / task_completed` 9. `subagent_spawned / subagent_completed`

5.2 Architecture Design





Hook Types:

- before: Can veto (return error to cancel)
- after: Observation only
- around: Wraps execution (before + after)

5.3 API Design

```
// Package: internal/hooks
package hooks

import (
    "context"
    "fmt"
    "os/exec"
    "time"
)

// EventType defines the available hook events
type EventType string

const (
    EventSessionStart      EventType = "session_start"
    EventSessionEnd        EventType = "session_end"
    EventMessageSend       EventType = "message_send"
    EventMessageReceive    EventType = "message_receive"
    EventToolCall          EventType = "tool_call"
    EventToolResult        EventType = "tool_result"
    EventFileChange        EventType = "file_change"
    EventGitOperation      EventType = "git_operation"
    EventPermissionRequest EventType = "permission_request"
    EventPermissionGrant   EventType = "permission_grant"
    EventCompaction        EventType = "compaction"
    EventSummaryGenerated  EventType = "summary_generated"
    EventError             EventType = "error"
    EventWarning           EventType = "warning"
    EventPlanCreated       EventType = "plan_created"
    EventTaskCompleted     EventType = "task_completed"
    EventSubagentSpawned   EventType = "subagent_spawned"
    EventSubagentCompleted EventType = "subagent_completed"
    EventLlmCall           EventType = "llm_call"
```

```

    EventLlmResponse      EventType = "llm_response"
)

// HookPhase determines when hook runs relative to event
type HookPhase string

const (
    PhaseBefore HookPhase = "before" // Can cancel event
    PhaseAfter  HookPhase = "after"  // Observation only
    PhaseAround HookPhase = "around" // Both before and after
)

// HookConfig configures a hook registration
type HookConfig struct {
    Name          string          `json:"name" db:"name"`
    EventType     EventType          `json:"event_type" db:"event_type"`
    Phase         HookPhase          `json:"phase" db:"phase"`
    HookType      HookImplementation `json:"hook_type" db:"hook_type"`

    // For script hooks
    ScriptPath  string          `json:"script_path,omitempty" db:"script_path"`
    ScriptArgs  []string        `json:"script_args,omitempty" db:"script_args"`
    Timeout     time.Duration   `json:"timeout,omitempty" db:"timeout"`

    // For webhook hooks
    WebhookURL  string          `json:"webhook_url,omitempty" db:"webhook_url"`
    WebhookHeaders map[string]string `json:"webhook_headers,omitempty" db:"webhook_headers"`

    // For internal hooks
    Handler      InternalHandler `json:"- " db:"- "`

    // Conditions
    Filter      HookFilter      `json:"filter,omitempty" db:"filter"`
    Priority     int             `json:"priority" db:"priority" // Execution order`
    Enabled     bool           `json:"enabled" db:"enabled"`

    // Metadata
    CreatedAt   time.Time       `json:"created_at" db:"created_at"`

```

```

}

// HookImplementation defines how the hook is implemented
type HookImplementation string

const (
    HookInternal HookImplementation = "internal"
    HookScript   HookImplementation = "script"
    HookWebhook  HookImplementation = "webhook"
    HookPlugin   HookImplementation = "plugin" // Shared
        library / Go plugin
)

// HookFilter conditions for selective triggering
type HookFilter struct {
    AgentTypes []string `json:"agent_types,omitempty"` //
        Only for these agent types
    ToolNames  []string `json:"tool_names,omitempty"` //
        Only for these tools
    Categories []string `json:"categories,omitempty"` //
        Only for these categories
    MinRiskLevel string `json:"min_risk_level,omitempty"` //
        Only above this risk
    Workspaces  []string `json:"workspaces,omitempty"` //
        Only for these workspaces
}

// EventPayload is the data passed to hooks
type EventPayload struct {
    EventType EventType `json:"event_type"`
    Timestamp  time.Time  `json:"timestamp"`
    SessionID  string     `json:"session_id,omitempty"`
    AgentID    string     `json:"agent_id,omitempty"`
    AgentType  string     `json:"agent_type,omitempty"`
    Workspace  string     `json:"workspace,omitempty"`
    Data       map[string]any `json:"data"` //
        Event-specific data
    Context    map[string]string `json:"context"` //
        Request context
}

// HookResult is returned by hook execution
type HookResult struct {
    HookName string `json:"hook_name"`
    Success  bool  `json:"success"`
    Error    string `json:"error,omitempty"`
    Veto     bool  `json:"veto,omitempty"` //
        Before hook cancelled event

```

```

    VetoReason    string           `json:"veto_reason,omitempty"`
    DurationMs    int64            `json:"duration_ms"`
    Output        []byte           `json:"output,omitempty"` // For
        script/webhook hooks
}

// InternalHandler is a Go function hook
type InternalHandler func(ctx context.Context, payload
    EventPayload) (*HookResult, error)

// HookManager manages hook registration and execution
type HookManager interface {
    // RegisterHook adds a hook for an event
    RegisterHook(ctx context.Context, config HookConfig) error

    // UnregisterHook removes a hook
    UnregisterHook(ctx context.Context, hookName string) error

    // ExecuteHooks runs all hooks for an event
    ExecuteHooks(ctx context.Context, eventType EventType, phase
        HookPhase, payload EventPayload) ([]HookResult, error)

    // ExecuteBeforeHooks runs before hooks, can cancel event
    ExecuteBeforeHooks(ctx context.Context, eventType EventType,
        payload EventPayload) (*BeforeHookResult, error)

    // ListHooks returns registered hooks
    ListHooks(ctx context.Context, filter HookListFilter)
        ([]HookConfig, error)

    // ReloadHooks reloads from configuration
    ReloadHooks(ctx context.Context) error

    // EnableHook / DisableHook toggle hooks
    EnableHook(ctx context.Context, hookName string) error
    DisableHook(ctx context.Context, hookName string) error
}

// BeforeHookResult indicates if event should proceed
type BeforeHookResult struct {
    Proceed        bool           `json:"proceed"`
    VetoReason     string        `json:"veto_reason,omitempty"`
    Results        []HookResult  `json:"results"`
}

```


5.4 Integration Points

Package	Modification	Description
internal/hooks/	NEW PACKAGE	Hook manager
internal/actor/	Add BeforeEvent/ AfterEvent calls	Instrument all actor lifecycle
internal/tool/	Pre/post execution hooks	Around tool execution
internal/llm/	Before/after LLM calls	Logging, rate limiting
internal/config/	Add hooks : config section	YAML hook definitions
cmd/helix/main.go	Add --hook-dir flag	Auto-load hooks from directory

5.5 Implementation Steps

Step 1: Hook manager core

```
// internal/hooks/manager.go
package hooks

import (
    "bytes"
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "os"
    "os/exec"
    "sort"
    "time"

    "dev.helix.code/pkg/logger"
)

type DefaultHookManager struct {
    hooks    map[EventType][]HookConfig
    log      logger.Logger
    httpClient *http.Client
}

func NewDefaultHookManager(log logger.Logger)
    *DefaultHookManager {
    return &DefaultHookManager{
        hooks:    make(map[EventType][]HookConfig),
        log:      log,
        httpClient: &http.Client{Timeout: 30 * time.Second},
    }
}
```

```

    }
}

func (m *DefaultHookManager) RegisterHook(ctx context.Context,
    config HookConfig) error {
    if config.EventType == "" {
        return fmt.Errorf("event type is required")
    }
    if config.HookType == HookInternal && config.Handler == nil {
        return fmt.Errorf("internal hook requires handler")
    }
    if config.HookType == HookScript && config.ScriptPath == "" {
        return fmt.Errorf("script hook requires script path")
    }
    if config.HookType == HookWebhook && config.WebhookURL == ""
    {
        return fmt.Errorf("webhook hook requires URL")
    }

    config.CreatedAt = time.Now()
    m.hooks[config.EventType] =
        append(m.hooks[config.EventType], config)

    // Sort by priority
    m.sortHooksByPriority(config.EventType)

    m.log.Info("hook registered",
        "name", config.Name,
        "event", config.EventType,
        "type", config.HookType,
        "phase", config.Phase,
    )

    return nil
}

func (m *DefaultHookManager) ExecuteHooks(
    ctx context.Context,
    eventType EventType,
    phase HookPhase,
    payload EventPayload,
) ([]HookResult, error) {
    hooks := m.getHooksForEvent(eventType, phase)
    if len(hooks) == 0 {
        return nil, nil
    }

    var results []HookResult

```

```

for _, hook := range hooks {
    // Check filter
    if !m.matchesFilter(hook.Filter, payload) {
        continue
    }

    result, err := m.executeHook(ctx, hook, payload)
    if err != nil {
        result = &HookResult{
            HookName: hook.Name,
            Success:   false,
            Error:     err.Error(),
        }
    }

    results = append(results, *result)

    m.log.Debug("hook executed",
        "name", hook.Name,
        "success", result.Success,
        "duration_ms", result.DurationMs,
    )
}

return results, nil
}

func (m *Default
HookManager) executeHook(ctx context.Context, hook HookConfig,
    payload EventPayload) (*HookResult, error) {
    start := time.Now()

    switch hook.HookType {
    case HookInternal:
        return hook.Handler(ctx, payload)

    case HookScript:
        return m.executeScriptHook(ctx, hook, payload)

    case HookWebhook:
        return m.executeWebhookHook(ctx, hook, payload)

    case HookPlugin:
        return m.executePluginHook(ctx, hook, payload)

    default:

```

```

        return nil, fmt.Errorf("unknown hook type: %s",
            hook.HookType)
    }
}

func (m *DefaultHookManager) executeScriptHook(ctx
    context.Context, hook HookConfig, payload EventPayload)
    (*HookResult, error) {
    start := time.Now()

    // Serialize payload to JSON
    data, err := json.Marshal(payload)
    if err != nil {
        return nil, err
    }

    // Set up command
    args := append([]string{string(hook.EventType)},
        hook.ScriptArgs...)
    cmd := exec.CommandContext(ctx, hook.ScriptPath, args...)
    cmd.Stdin = bytes.NewReader(data)

    // Set timeout
    if hook.Timeout > 0 {
        var cancel context.CancelFunc
        ctx, cancel = context.WithTimeout(ctx, hook.Timeout)
        defer cancel()
        cmd = exec.CommandContext(ctx, hook.ScriptPath, args...)
        cmd.Stdin = bytes.NewReader(data)
    }

    output, err := cmd.CombinedOutput()

    result := &HookResult{
        HookName:    hook.Name,
        DurationMs:   time.Since(start).Milliseconds(),
        Output:       output,
    }

    if err != nil {
        result.Success = false
        result.Error = err.Error()

        // Check for veto signal (exit code 42)
        if exitErr, ok := err.(*exec.ExitError); ok &&
            exitErr.ExitCode() == 42 {
            result.Veto = true
            result.VetoReason = string(output)
        }
    }
}

```

```

    }
} else {
    result.Success = true
}

return result, nil
}

func (m *DefaultHookManager) executeWebhookHook(ctx
    context.Context, hook HookConfig, payload EventPayload)
    (*HookResult, error) {
    start := time.Now()

    data, err := json.Marshal(payload)
    if err != nil {
        return nil, err
    }

    req, err := http.NewRequestWithContext(ctx, "POST",
        hook.WebhookURL, bytes.NewReader(data))
    if err != nil {
        return nil, err
    }

    req.Header.Set("Content-Type", "application/json")
    req.Header.Set("X-Helix-Event", string(hook.EventType))
    req.Header.Set("X-Helix-Hook", hook.Name)

    for k, v := range hook.WebhookHeaders {
        req.Header.Set(k, v)
    }

    resp, err := m.httpClient.Do(req)
    if err != nil {
        return &HookResult{
            HookName:    hook.Name,
            Success:      false,
            Error:        err.Error(),
            DurationMs:   time.Since(start).Milliseconds(),
        }, nil
    }
    defer resp.Body.Close()

    output, _ := io.ReadAll(resp.Body)

    result := &HookResult{
        HookName:    hook.Name,

```

```

        Success:    resp.StatusCode >= 200 && resp.StatusCode <
                    300,
        DurationMs: time.Since(start).Milliseconds(),
        Output:     output,
    }

    if !result.Success {
        result.Error = fmt.Sprintf("HTTP %d: %s",
            resp.StatusCode, string(output))
    }

    // Check for veto (custom header)
    if resp.Header.Get("X-Helix-Veto") == "true" {
        result.Veto = true
        result.VetoReason = resp.Header.Get("X-Helix-Veto-Reason")
    }

    return result, nil
}

func (m *DefaultHookManager) matchesFilter(filter HookFilter,
    payload EventPayload) bool {
    if len(filter.AgentTypes) > 0 {
        found := false
        for _, at := range filter.AgentTypes {
            if at == payload.AgentType {
                found = true
                break
            }
        }
        if !found {
            return false
        }
    }

    if len(filter.Workspaces) > 0 {
        found := false
        for _, ws := range filter.Workspaces {
            if ws == payload.Workspace {
                found = true
                break
            }
        }
        if !found {
            return false
        }
    }
}

```

```

        return true
    }

    func (m *DefaultHookManager) getHooksForEvent(eventType
        EventType, phase HookPhase) []HookConfig {
        var result []HookConfig
        for _, hook := range m.hooks[eventType] {
            if hook.Enabled && (hook.Phase == phase || hook.Phase ==
                PhaseAround) {
                result = append(result, hook)
            }
        }
        return result
    }

    func (m *DefaultHookManager) sortHooksByPriority(eventType
        EventType) {
        sort.Slice(m.hooks[eventType], func(i, j int) bool {
            return m.hooks[eventType][i].Priority <
                m.hooks[eventType][j].Priority
        })
    }

    // Other interface methods follow...

```

Step 2: Event integration points

```

// internal/actor/session.go
func (a *SessionActor) sendMessage(ctx context.Context, msg
    Message) error {
    // Execute before hooks
    payload := hooks.EventPayload{
        EventType: hooks.EventMessageSend,
        SessionID: a.sessionID,
        AgentID:   a.agentID,
        Data: map[string]any{
            "message_role": msg.Role,
            "message_length": len(msg.Content),
        },
    }

    beforeResult, err := a.hookManager.ExecuteBeforeHooks(ctx,
        hooks.EventMessageSend, payload)
    if err != nil {
        a.log.Warn("before hook error", "error", err)
    }

    if beforeResult != nil && !beforeResult.Proceed {

```

```

        return fmt.Errorf("message blocked by hook: %s",
            beforeResult.VetoReason)
    }

    // Actually send
    err = a.doSend(ctx, msg)

    // Execute after hooks
    payload.Data["error"] = err
    a.hookManager.ExecuteHooks(ctx, hooks.EventMessageSend,
        hooks.PhaseAfter, payload)

    return err
}

```

5.6 Testing Approach

```

func TestHookManager_BeforeHookVeto(t *testing.T) {
    manager := NewDefaultHookManager(logger.NewNop())

    // Register a before hook that vetoes
    manager.RegisterHook(ctx, HookConfig{
        Name:      "veto-hook",
        EventType: EventToolCall,
        Phase:     PhaseBefore,
        HookType:  HookInternal,
        Handler: func(ctx context.Context, p EventPayload)
            (*HookResult, error) {
                return &HookResult{
                    Success: true,
                    Veto:    true,
                    VetoReason: "testing veto",
                }, nil
            }, nil
    })

    result, err := manager.ExecuteBeforeHooks(ctx,
        EventToolCall, EventPayload{
            EventType: EventToolCall,
            Data: map[string]any{"tool": "test"},
        })

    assert.NoError(t, err)
    assert.False(t, result.Proceed)
    assert.Equal(t, "testing veto", result.VetoReason)
}

func TestHookManager_ScriptHook(t *testing.T) {

```



```

// Create test script
scriptPath := filepath.Join(t.TempDir(), "hook.sh")
script := `#!/bin/bash
read input
echo "Received: $input"
exit 0
`

os.WriteFile(scriptPath, []byte(script), 0755)

manager := NewDefaultHookManager(logger.NewNop())
manager.RegisterHook(ctx, HookConfig{
    Name:      "test-script",
    EventType: EventSessionStart,
    Phase:     PhaseAfter,
    HookType:  HookScript,
    ScriptPath: scriptPath,
    Timeout:   5 * time.Second,
})

results, err := manager.ExecuteHooks(ctx, EventSessionStart,
    hooks.PhaseAfter, EventPayload{
        SessionID: "test-session",
        Data:      map[string]any{"workspace": "/test"},
    })

assert.NoError(t, err)
assert.Len(t, results, 1)
assert.True(t, results[0].Success)
assert.Contains(t, string(results[0].Output), "Received:")
}

```

5.7 Edge Cases

Edge Case	Handling
Hook times out	Kill process, mark as failed, don't block main flow (after hooks)
Hook panics	Recover, log, continue with other hooks
Infinite loop in hooks	Max hook chain depth (10), circular reference detection
Script not executable	Log error, skip hook
Webhook endpoint down	Retry with backoff (3 attempts), then skip
Payload too large for script	Stream to temp file, pass path as arg
Hook modifies working directory	Save/restore cwd in script wrapper
Before hook crashes after partial state change	Document: hooks should be idempotent
Hook returns non-UTF8	

Edge Case	Handling
	Handle binary output, base64 encode for JSON
Concurrent hook registration	Mutex on hooks map

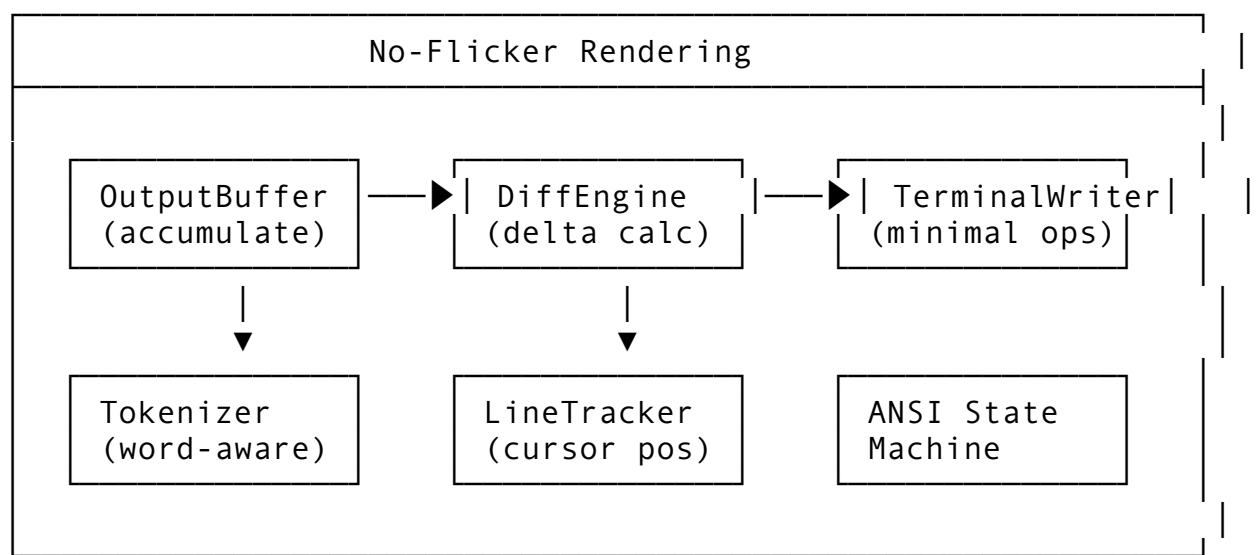
Feature 6: No-Flicker Rendering Mode

6.1 Feature Description

The No-Flicker Rendering Mode provides smooth, incremental terminal updates without screen flashing or cursor jumping. It uses delta-based rendering with intelligent frame buffering to minimize visual noise during streaming LLM outputs.

Why it matters: Frequent screen clear/redraw operations cause eye strain and make it hard to follow streaming output. Smooth rendering is essential for professional developer experience.

6.2 Architecture Design



6.3 API Design

```
// Package: internal/ui/render
package render

import (
    "bytes"
    "fmt"
    "io"
    "strings"
    "sync"
    "time"
)
```

```

// RenderMode defines rendering strategy
type RenderMode int

const (
    RenderModePlain RenderMode = iota
    RenderModeFancy    // Default HelixCode mode
    RenderModeNoFlicker // Delta-based
    RenderModeRaw      // Pass through
)

// TerminalCapabilities describes terminal features
type TerminalCapabilities struct {
    SupportsANSI          bool
    Supports256Color      bool
    SupportsTrueColor     bool
    SupportsAlternateBuffer bool
    SupportsMouse         bool
    Width                 int
    Height                int
    IsTerminal            bool
    IsWindows             bool
}

// RenderEngine is the main rendering interface
type RenderEngine interface {
    // Write appends content to the buffer
    Write(content string) error

    // Flush renders accumulated changes to terminal
    Flush() error

    // Clear clears the current output area
    Clear() error

    // SetMode changes rendering mode
    SetMode(mode RenderMode) error

    // GetRendered returns the currently rendered text
    GetRendered() string

    // SetDimensions updates terminal size
    SetDimensions(width, height int) error

    // Pause pauses rendering (for critical sections)
    Pause()

    // Resume resumes rendering
    Resume()
}

```

```

    // Close cleans up terminal state
    Close() error
}

// DeltaCalculator computes minimal terminal operations
type DeltaCalculator struct {
    mu          sync.Mutex
    previous    []string    // Previous frame lines
    current     []string    // Current frame lines
    cursorLine  int
    cursorCol   int
    width       int
    ansiState   *ANSIState
}

// ANSIState tracks ANSI escape sequence state
type ANSIState struct {
    ForegroundColor string
    BackgroundColor string
    Bold             bool
    Italic           bool
    Underline        bool
    Strikethrough    bool
    InLink           bool
}

// RenderConfig configures rendering
type RenderConfig struct {
    Mode                RenderMode        `mapstructure:"mode"`
    MinFlushInterval    time.Duration     `mapstructure:"min_flush_interval" // Debounce`
    FrameBufferSize     int               `mapstructure:"frame_buffer_size"`
    EnableLineNumbers   bool              `mapstructure:"enable_line_numbers"`
    Theme               string            `mapstructure:"theme"`
    TabWidth            int               `mapstructure:"tab_width"`
    WordWrap            bool              `mapstructure:"word_wrap"`
    ShowSpinner         bool              `mapstructure:"show_spinner"`
}

// MinimalRenderEngine implements no-flicker rendering
type MinimalRenderEngine struct {
    config *RenderConfig

```

```

    cap      *TerminalCapabilities
    output   io.Writer
    buffer   *bytes.Buffer
    rendered string
    mu       sync.Mutex
    paused   bool
    dirty    bool
    flushTimer *time.Timer
}

```

6.4 Integration Points

Package	Modification	Description
internal/ui/render/	NEW PACKAGE	Rendering engine
internal/ui/terminal.go	Replace direct writes	Route through render engine
cmd/helix/main.go	Add --render-mode flag	Select rendering mode
internal/config/	Add render config	Viper integration

6.5 Implementation Steps

```

// internal/ui/render/minimal.go
package render

import (
    "bytes"
    "fmt"
    "io"
    "strings"
    "sync"
    "time"
)

func NewMinimalRenderEngine(config *RenderConfig, cap
    *TerminalCapabilities, output io.Writer)
    *MinimalRenderEngine {
    return &MinimalRenderEngine{
        config:    config,
        cap:       cap,
        output:    output,
        buffer:    &bytes.Buffer{},
        tabWidth:  config.TabWidth,
    }
}

```

```

func (e *MinimalRenderEngine) Write(content string) error {
    e.mu.Lock()
    defer e.mu.Unlock()

    e.buffer.WriteString(content)
    e.dirty = true

    // Debounce: schedule flush
    if e.flushTimer != nil {
        e.flushTimer.Stop()
    }
    e.flushTimer = time.AfterFunc(e.config.MinFlushInterval,
        func() {
            e.Flush()
        })

    return nil
}

func (e *MinimalRenderEngine) Flush() error {
    e.mu.Lock()
    defer e.mu.Unlock()

    if !e.dirty || e.paused {
        return nil
    }

    newContent := e.buffer.String()
    e.buffer.Reset()
    e.dirty = false

    if !e.cap.IsTerminal || e.cap.SupportsANSI {
        // Full redraw (fallback)
        _, err := e.output.Write([]byte(newContent))
        e.rendered = newContent
        return err
    }

    // Delta-based rendering
    delta := e.computeDelta(e.rendered, newContent)
    _, err := e.output.Write([]byte(delta))
    e.rendered = newContent

    return err
}

```

```

func (e *MinimalRenderEngine) computeDelta(old, new string)
    string {
    if old == "" {
        return new
    }

    // Find common prefix
    minLen := len(old)
    if len(new) < minLen {
        minLen = len(new)
    }

    prefixLen := 0
    for prefixLen < minLen && old[prefixLen] == new[prefixLen] {
        prefixLen++
    }

    // If content only appended, just write suffix
    if prefixLen == len(old) {
        return new[prefixLen:]
    }

    // Find common suffix
    oldSuffixStart := len(old)
    newSuffixStart := len(new)
    for oldSuffixStart > prefixLen && newSuffixStart > prefixLen
        &&
        old[oldSuffixStart-1] == new[newSuffixStart-1] {
        oldSuffixStart--
        newSuffixStart--
    }

    // Build delta operations
    var ops []string

    // Position cursor at start of changed region
    prefixLines := strings.Count(old[:prefixLen], "\n")
    prefixLastNL := strings.LastIndex(old[:prefixLen], "\n")
    prefixCol := prefixLen - prefixLastNL - 1
    if prefixLastNL == -1 {
        prefixCol = prefixLen
    }

    ops = append(ops, fmt.Sprintf("\033[%d;%dH", prefixLines+1,
        prefixCol+1))

    // Clear from cursor to end of changed region
    if oldSuffixStart < len(old) {

```

```

        // Clear to end of line and down
        ops = append(ops, "\033[J")
    }

    // Write new content for changed region
    ops = append(ops, new[prefixLen:newSuffixStart])

    // If suffix changed, write remainder
    if newSuffixStart < len(new) {
        ops = append(ops, new[newSuffixStart:])
    }

    return strings.Join(ops, "")
}

// Alternative: Line-based diff for multi-line content
func (e *MinimalRenderEngine) computeLineDelta(oldLines,
    newLines []string) string {
    var ops []string

    // Simple approach: compare line by line
    maxLines := len(oldLines)
    if len(newLines) > maxLines {
        maxLines = len(newLines)
    }

    for i := 0; i < maxLines; i++ {
        if i >= len(oldLines) {
            // New line
            ops = append(ops, fmt.Sprintf("\033[%d;1H%s", i+1,
                newLines[i]))
        } else if i >= len(newLines) {
            // Removed line
            ops = append(ops, fmt.Sprintf("\033[%d;1H\033[K",
                i+1))
        } else if oldLines[i] != newLines[i] {
            // Changed line
            ops = append(ops, fmt.Sprintf("\033[%d;1H\033[K%s",
                i+1, newLines[i]))
        }
    }

    return strings.Join(ops, "")
}

func (e *MinimalRenderEngine) Clear() error {
    e.mu.Lock()
    defer e.mu.Unlock()

```



```

    if e.cap.SupportsANSI {
        _, err := e.output.Write([]byte("\033[2J\033[H"))
        e.rendered = ""
        e.buffer.Reset()
        return err
    }

    // Fallback: lots of newlines
    _, err := e.output.Write(bytes.Repeat([]byte("\n"),
        e.cap.Height))
    e.rendered = ""
    e.buffer.Reset()
    return err
}

func (e *MinimalRenderEngine) Pause() {
    e.mu.Lock()
    e.paused = true
    e.mu.Unlock()
}

func (e *MinimalRenderEngine) Resume() {
    e.mu.Lock()
    e.paused = false
    e.mu.Unlock()
    e.Flush()
}

```

6.6 Testing Approach

```

func TestMinimalRenderEngine_Delta(t *testing.T) {
    var buf bytes.Buffer
    cap := &TerminalCapabilities{SupportsANSI: true, IsTerminal:
        true, Width: 80, Height: 24}
    engine :=
        NewMinimalRenderEngine(&RenderConfig{MinFlushInterval: 1
            * time.Millisecond}, cap, &buf)

    // First write
    engine.Write("Hello world")
    engine.Flush()
    assert.Equal(t, "Hello world", buf.String())

    // Append
    buf.Reset()
    engine.Write("!")
    engine.Flush()
}

```

```

    // Should only write "!"
    assert.Equal(t, "!", buf.String())

    // Middle edit
    buf.Reset()
    engine.Write("Hello beautiful world!")
    engine.Flush()
    // Should compute minimal operations
    assert.Contains(t, buf.String(), "beautiful")
}

func TestMinimalRenderEngine_PauseResume(t *testing.T) {
    var buf bytes.Buffer
    engine := NewMinimalRenderEngine(defaultConfig, defaultCap,
        &buf)

    engine.Pause()
    engine.Write("test")
    engine.Flush()
    assert.Equal(t, "", buf.String()) // Should not render while
        paused

    engine.Resume()
    assert.Contains(t, buf.String(), "test")
}

```

6.7 Edge Cases

Edge Case	Handling
ANSI codes in content	Parse and preserve, track state across deltas
Multi-byte UTF-8	Count runes, not bytes, for cursor positioning
Terminal resize	Redraw from scratch, recompute line wrapping
Very fast updates	Debounce with max frequency (60fps cap)
Cursor in middle of content	Track absolute position, not relative
Terminal doesn't support ANSI	Fall back to simple append-only
Content shorter than previous	Clear trailing characters
Wide characters (CJK)	Account for double-width in column calculation
Tab characters	Expand to spaces based on tab width setting
Nested color codes	Stack-based ANSI state machine

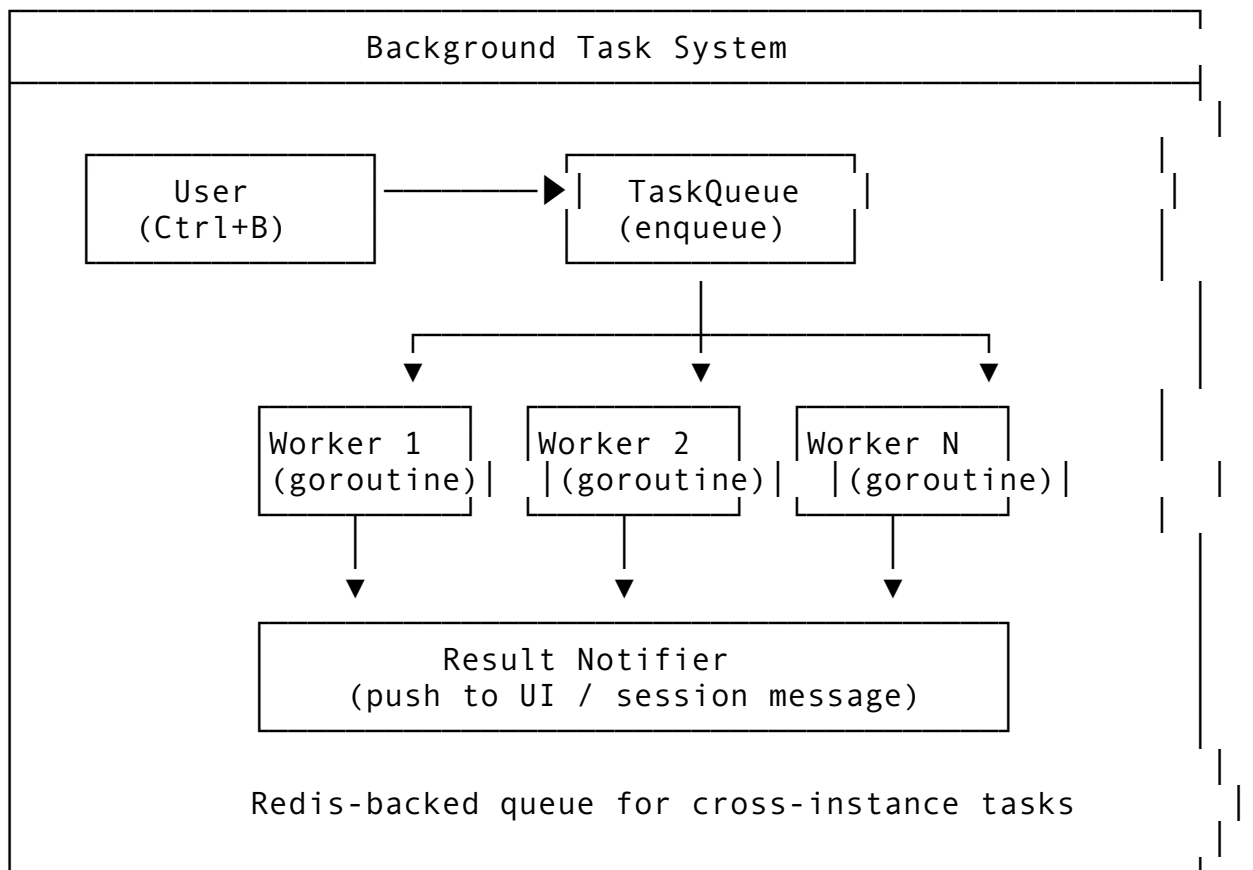
Feature 7: Background Task System (Ctrl+B)

7.1 Feature Description

The Background Task System allows users to dispatch long-running operations (builds, tests, searches, analysis) to run asynchronously while continuing their interactive session. Tasks report progress and results through a job queue.

Why it matters: Long-running operations block the UI. Background tasks enable productive multitasking - start a build and continue coding while it runs.

7.2 Architecture Design



7.3 API Design

```
// Package: internal/task
package task

import (
    "context"
    "fmt"
    "time"
)

// TaskPriority determines execution order
```

```

type TaskPriority int

const (
    PriorityLow TaskPriority = iota
    PriorityNormal
    PriorityHigh
    PriorityCritical
)

// TaskStatus tracks task lifecycle
type TaskStatus string

const (
    StatusPending    TaskStatus = "pending"
    StatusQueued      TaskStatus = "queued"
    StatusRunning     TaskStatus = "running"
    StatusPaused      TaskStatus = "paused"
    StatusCompleted   TaskStatus = "completed"
    StatusFailed       TaskStatus = "failed"
    StatusCancelled   TaskStatus = "cancelled"
    StatusTimedOut    TaskStatus = "timed_out"
)

// TaskType categorizes tasks
type TaskType string

const (
    TaskTypeShell      TaskType = "shell"
    TaskTypeBuild       TaskType = "build"
    TaskTypeTest        TaskType = "test"
    TaskTypeSearch      TaskType = "search"
    TaskTypeAnalysis    TaskType = "analysis"
    TaskTypeLSP         TaskType = "lsp"
    TaskTypeGit         TaskType = "git"
    TaskTypeCustom      TaskType = "custom"
)

// BackgroundTask represents an async task
type BackgroundTask struct {
    ID          string    `json:"id" db:"id"`
    SessionID   string    `json:"session_id" db:"session_id"`
    AgentID     string    `json:"agent_id" db:"agent_id"`

    TaskType    TaskType    `json:"task_type" db:"task_type"`
    Name       string    `json:"name" db:"name"`

```

```

Description    string           `json:"description"
               db:"description"`

// Execution
Command        string           `json:"command,omitempty"
               db:"command"`
Arguments      []string         `json:"arguments,omitempty"
               db:"arguments"`
Environment     map[string]string
               `json:"environment,omitempty" db:"environment"`
WorkingDir     string           `json:"working_dir"
               db:"working_dir"`

// State
Status         TaskStatus       `json:"status" db:"status"`
Priority        TaskPriority     `json:"priority" db:"priority"`
Progress        float64         `json:"progress"
               db:"progress" // 0.0 - 1.0

// Timing
CreatedAt       time.Time       `json:"created_at"
               db:"created_at"`
StartedAt       *time.Time     `json:"started_at,omitempty"
               db:"started_at"`
CompletedAt     *time.Time     `json:"completed_at,omitempty"
               db:"completed_at"`
Timeout         time.Duration   `json:"timeout" db:"timeout"`

// Results
ExitCode        *int           `json:"exit_code,omitempty"
               db:"exit_code"`
Output          *string        `json:"output,omitempty"
               db:"output"`
OutputPath      *string        `json:"output_path,omitempty"
               db:"output_path"`
Error           *string        `json:"error,omitempty"
               db:"error"`

// Metadata
Tags            []string         `json:"tags,omitempty"
               db:"tags"`
Annotations     map[string]any `json:"annotations,omitempty"
               db:"annotations"`
ParentTaskID    *string        `json:"parent_task_id,omitempty" db:"parent_task_id"`

// Cancellation
CancelFunc      context.CancelFunc `json:"-" db:"-"`

```

```

}

// TaskQueue manages background task execution
type TaskQueue interface {
    // Submit adds a task to the queue
    Submit(ctx context.Context, task *BackgroundTask) (string,
        error)

    // Cancel stops a running or pending task
    Cancel(ctx context.Context, taskID string) error

    // GetTask retrieves task status
    GetTask(ctx context.Context, taskID string)
        (*BackgroundTask, error)

    // ListTasks returns tasks, optionally filtered
    ListTasks(ctx context.Context, filter TaskFilter)
        ([]BackgroundTask, error)

    // WaitForTask blocks until task completes
    WaitForTask(ctx context.Context, taskID string)
        (*BackgroundTask, error)

    // Pause pauses the queue
    Pause(ctx context.Context) error

    // Resume resumes the queue
    Resume(ctx context.Context) error

    // GetQueueStats returns queue statistics
    GetQueueStats(ctx context.Context) (*QueueStats, error)

    // RegisterHandler registers a custom task executor
    RegisterHandler(taskType TaskType, handler TaskHandler) error

    // SubscribeToUpdates subscribes to task status changes
    SubscribeToUpdates(ctx context.Context, taskID string)
        (<-chan TaskUpdate, error)
}

// TaskHandler executes a specific task type
type TaskHandler interface {
    Execute(ctx context.Context, task *BackgroundTask, progress
        ProgressReporter) (*TaskResult, error)
    Supports(taskType TaskType) bool
}

// ProgressReporter reports task progress

```

```

type ProgressReporter interface {
    ReportProgress(progress float64, message string)
    ReportOutput(chunk string)
}

// TaskResult is the outcome of task execution
type TaskResult struct {
    ExitCode    int           `json:"exit_code"`
    Output      string        `json:"output"`
    OutputPath  string        `json:"output_path,omitempty"`
    Error       string        `json:"error,omitempty"`
    Artifacts   []TaskArtifact `json:"artifacts,omitempty"`
}

// TaskArtifact is a file produced by a task
type TaskArtifact struct {
    Path          string `json:"path"`
    ContentType   string `json:"content_type"`
    Size          int64  `json:"size"`
}

// TaskUpdate is a status change notification
type TaskUpdate struct {
    TaskID      string    `json:"task_id"`
    Status      TaskStatus `json:"status"`
    Progress    float64   `json:"progress"`
    Message     string    `json:"message"`
    Timestamp   time.Time `json:"timestamp"`
}

// QueueStats reports queue health
type QueueStats struct {
    PendingCount  int `json:"pending_count"`
    RunningCount  int `json:"running_count"`
    CompletedCount int `json:"completed_count"`
    FailedCount   int `json:"failed_count"`
    WorkersTotal  int `json:"workers_total"`
    WorkersBusy   int `json:"workers_busy"`
    AvgWaitTime   time.Duration `json:"avg_wait_time"`
    AvgExecTime   time.Duration `json:"avg_exec_time"`
}

// TaskFilter filters task queries
type TaskFilter struct {
    SessionID *string `json:"session_id,omitempty"`
    AgentID   *string `json:"agent_id,omitempty"`
    Status    *TaskStatus `json:"status,omitempty"`
    TaskType  *TaskType  `json:"task_type,omitempty"`
}

```

```

    Since      *time.Time `json:"since,omitempty"`
    Tags        []string   `json:"tags,omitempty"`
}

```

7.4 Integration Points

Package	Modification	Description
internal/task/	NEW PACKAGE	Task queue and workers
internal/ actor/ session.go	Background task dispatch	Ctrl+B handler
internal/ui/	Add task status UI	Show running tasks
internal/ redis/	Redis-backed queue	Cross-instance task distribution
internal/db/ migrations/	Add background_tasks table	Persist task metadata
cmd/helix/ main.go	Add --max-workers flag	Configure concurrency

7.5 Implementation Steps

```

// internal/task/queue.go
package task

import (
    "context"
    "fmt"
    "os/exec"
    "sync"
    "time"

    "dev.helix.code/internal/db"
    "dev.helix.code/pkg/logger"
)

type DefaultTaskQueue struct {
    store      db.Store
    log        logger.Logger
    workers    int
    handlers   map[TaskType]TaskHandler
    tasks      map[string]*BackgroundTask
    queue      chan *BackgroundTask
    mu         sync.RWMutex
    wg         sync.WaitGroup
    ctx        context.Context
    cancel     context.CancelFunc
}

```



```

    updates    map[string][]chan TaskUpdate
    updatesMu  sync.RWMutex
}

func NewDefaultTaskQueue(store db.Store, log logger.Logger,
    workers int) *DefaultTaskQueue {
    ctx, cancel := context.WithCancel(context.Background())
    q := &DefaultTaskQueue{
        store:    store,
        log:      log,
        workers:  workers,
        handlers: make(map[TaskType]TaskHandler),
        tasks:    make(map[string]*BackgroundTask),
        queue:    make(chan *BackgroundTask, workers*2),
        ctx:      ctx,
        cancel:   cancel,
        updates:  make(map[string][]chan TaskUpdate),
    }

    // Register default handlers
    q.RegisterHandler(TaskTypeShell, &ShellTaskHandler{})

    // Start workers
    for i := 0; i < workers; i++ {
        q.wg.Add(1)
        go q.worker(i)
    }

    return q
}

func (q *DefaultTaskQueue) worker(id int) {
    defer q.wg.Done()

    q.log.Info("task worker started", "worker_id", id)

    for {
        select {
        case <-q.ctx.Done():
            return
        case task := <-q.queue:
            q.executeTask(task)
        }
    }
}

func (q *DefaultTaskQueue) executeTask(task *BackgroundTask) {
    now := time.Now()

```

```

task.StartedAt = &now
task.Status = StatusRunning
q.store.UpdateTask(q.ctx, task)
q.notifyUpdate(task, "Task started")

// Create timeout context
taskCtx, cancel := context.WithTimeout(q.ctx, task.Timeout)
task.CancelFunc = cancel
defer cancel()

// Find handler
handler, ok := q.handlers[task.TaskType]
if !ok {
    task.Status = StatusFailed
    errMsg := fmt.Sprintf("no handler for task type: %s",
        task.TaskType)
    task.Error = &errMsg
    q.finalizeTask(task)
    return
}

// Execute
progress := &defaultProgressReporter{queue: q, task: task}
result, err := handler.Execute(taskCtx, task, progress)

// Handle result
if err != nil {
    if taskCtx.Err() == context.DeadlineExceeded {
        task.Status = StatusTimedOut
    } else {
        task.Status = StatusFailed
    }
    errStr := err.Error()
    task.Error = &errStr
} else {
    task.Status = StatusCompleted
    task.ExitCode = &result.ExitCode
    if result.Output != "" {
        task.Output = &result.Output
    }
    if result.OutputPath != "" {
        task.OutputPath = &result.OutputPath
    }
}

completedAt := time.Now()
task.CompletedAt = &completedAt

```

```

    q.finalizeTask(task)
}

func (q *DefaultTaskQueue) finalizeTask(task *BackgroundTask) {
    q.store.UpdateTask(q.ctx, task)

    msg := fmt.Sprintf("Task %s completed with status %s",
        task.Name, task.Status)
    if task.Error != nil {
        msg = fmt.Sprintf("Task %s failed: %s", task.Name,
            *task.Error)
    }
    q.notifyUpdate(task, msg)

    q.log.Info("task completed",
        "task_id", task.ID,
        "status", task.Status,
        "duration", task.CompletedAt.Sub(*task.StartedAt),
    )
}

func (q *DefaultTaskQueue) Submit(ctx context.Context, task
    *BackgroundTask) (string, error) {
    task.ID = generateTaskID()
    task.Status = StatusQueued
    task.CreatedAt = time.Now()
    if task.Timeout == 0 {
        task.Timeout = 5 * time.Minute
    }

    // Store in database
    if err := q.store.CreateTask(ctx, task); err != nil {
        return "", fmt.Errorf("storing task: %w", err)
    }

    // Add to in-memory tracking
    q.mu.Lock()
    q.tasks[task.ID] = task
    q.mu.Unlock()

    // Queue for execution
    select {
    case q.queue <- task:
        q.log.Info("task queued", "task_id", task.ID, "type",
            task.TaskType)
    case <- ctx.Done():
        return "", ctx.Err()
    }
}

```

```

    return task.ID, nil
}

func (q *DefaultTaskQueue) Cancel(ctx context.Context, taskID
    string) error {
    q.mu.Lock()
    task, ok := q.tasks[taskID]
    q.mu.Unlock()

    if !ok {
        return fmt.Errorf("task not found: %s", taskID)
    }

    if task.Status != StatusRunning && task.Status !=
        StatusQueued && task.Status != StatusPending {
        return fmt.Errorf("task cannot be cancelled in status:
            %s", task.Status)
    }

    if task.CancelFunc != nil {
        task.CancelFunc()
    }

    task.Status = StatusCancelled
    q.store.UpdateTask(ctx, task)
    q.notifyUpdate(task, "Task cancelled")

    return nil
}

func (q *DefaultTaskQueue) notifyUpdate(task *BackgroundTask,
    message string) {
    q.updatesMu.RLock()
    chans := q.updates[task.ID]
    q.updatesMu.RUnlock()

    update := TaskUpdate{
        TaskID:    task.ID,
        Status:    task.Status,
        Progress:  task.Progress,
        Message:   message,
        Timestamp: time.Now(),
    }

    for _, ch := range chans {
        select {
        case ch <- update:

```

```

        default:
            // Channel full, skip
        }
    }
}

func (q *DefaultTaskQueue) RegisterHandler(taskType TaskType,
    handler TaskHandler) error {
    q.handlers[taskType] = handler
    return nil
}

func (q *DefaultTaskQueue) SubscribeToUpdates(ctx
    context.Context, taskID string) (<-chan TaskUpdate,
    error) {
    ch := make(chan TaskUpdate, 10)

    q.updatesMu.Lock()
    q.updates[taskID] = append(q.updates[taskID], ch)
    q.updatesMu.Unlock()

    // Cleanup on context done
    go func() {
        <-ctx.Done()
        q.updatesMu.Lock()
        taskChans := q.updates[taskID]
        for i, c := range taskChans {
            if c == ch {
                q.updates[taskID] = append(taskChans[:i],
                    taskChans[i+1:]...)
                break
            }
        }
        q.updatesMu.Unlock()
        close(ch)
    }()

    return ch, nil
}

// ShellTaskHandler default implementation

type ShellTaskHandler struct{}

func (h *ShellTaskHandler) Supports(taskType TaskType) bool {
    return taskType == TaskTypeShell || taskType ==
        TaskTypeBuild || taskType == TaskTypeTest
}

```

```

func (h *ShellTaskHandler) Execute(ctx context.Context, task
    *BackgroundTask, progress ProgressReporter)
    (*TaskResult, error) {
    cmd := exec.CommandContext(ctx, task.Command,
        task.Arguments...)
    cmd.Dir = task.WorkingDir
    cmd.Env = h.buildEnv(task.Environment)

    // Capture output
    output, err := cmd.CombinedOutput()

    result := &TaskResult{
        ExitCode: cmd.ProcessState.ExitCode(),
        Output:    string(output),
    }

    if err != nil {
        result.Error = err.Error()
    }

    return result, err
}

func (h *ShellTaskHandler) buildEnv(env map[string]string)
    []string {
    base := os.Environ()
    for k, v := range env {
        base = append(base, fmt.Sprintf("%s=%s", k, v))
    }
    return base
}

type defaultProgressReporter struct {
    queue *DefaultTaskQueue
    task  *BackgroundTask
}

func (r *defaultProgressReporter) ReportProgress(progress
    float64, message string) {
    r.task.Progress = progress
    r.queue.store.UpdateTask(r.queue.ctx, r.task)
    r.queue.notifyUpdate(r.task, message)
}

func (r *defaultProgressReporter) ReportOutput(chunk string) {
    if r.task.Output == nil {
        s := ""

```

```

        r.task.Output = &s
    }
    combined := *r.task.Output + chunk
    r.task.Output = &combined
}

func generateTaskID() string {
    return fmt.Sprintf("task_%d", time.Now().UnixNano())
}

```

7.6 Testing Approach

```

func TestDefaultTaskQueue_SubmitAndComplete(t *testing.T) {
    queue := NewDefaultTaskQueue(mockStore, logger.NewNop(), 2)
    defer queue.cancel()

    task := &BackgroundTask{
        TaskType:    TaskTypeShell,
        Name:         "test-echo",
        Command:      "echo",
        Arguments:    []string{"hello world"},
        WorkingDir:   t.TempDir(),
        Timeout:      10 * time.Second,
    }

    taskID, err := queue.Submit(ctx, task)
    assert.NoError(t, err)
    assert.NotEmpty(t, taskID)

    // Wait for completion
    completed, err := queue.WaitForTask(ctx, taskID)
    assert.NoError(t, err)
    assert.Equal(t, StatusCompleted, completed.Status)
    assert.Equal(t, 0, *completed.ExitCode)
    assert.Contains(t, *completed.Output, "hello world")
}

func TestDefaultTaskQueue_Cancel(t *testing.T) {
    queue := NewDefaultTaskQueue(mockStore, logger.NewNop(), 1)
    defer queue.cancel()

    // Long-running task
    task := &BackgroundTask{
        TaskType:    TaskTypeShell,
        Name:         "sleep",
        Command:      "sleep",
        Arguments:    []string{"10"},
        Timeout:      30 * time.Second,
    }
}

```

```

}

taskID, _ := queue.Submit(ctx, task)

// Small delay to ensure task starts
time.Sleep(100 * time.Millisecond)

err := queue.Cancel(ctx, taskID)
assert.NoError(t, err)

// Verify cancelled
updated, _ := queue.GetTask(ctx, taskID)
assert.Equal(t, StatusCancelled, updated.Status)
}

```

7.7 Edge Cases

Edge Case	Handling
Task exceeds timeout	Context cancellation kills process tree
Worker panic	Recover, mark task failed, restart worker goroutine
Queue full	Block submit with ctx timeout, or error immediately
Task dependency cycle	Detect during submit, error with cycle path
Zombie child processes	Use process group, kill -pgid on cancel
Output buffer overflow	Stream to file after threshold
Redis connection lost	Fall back to in-memory queue, reconnect with backoff
Task produces huge output	Streaming, file-based storage
Concurrent cancel and complete	Atomic status update, first writer wins
Task handler deadlock	Timeout in wrapper, force kill after 2x timeout

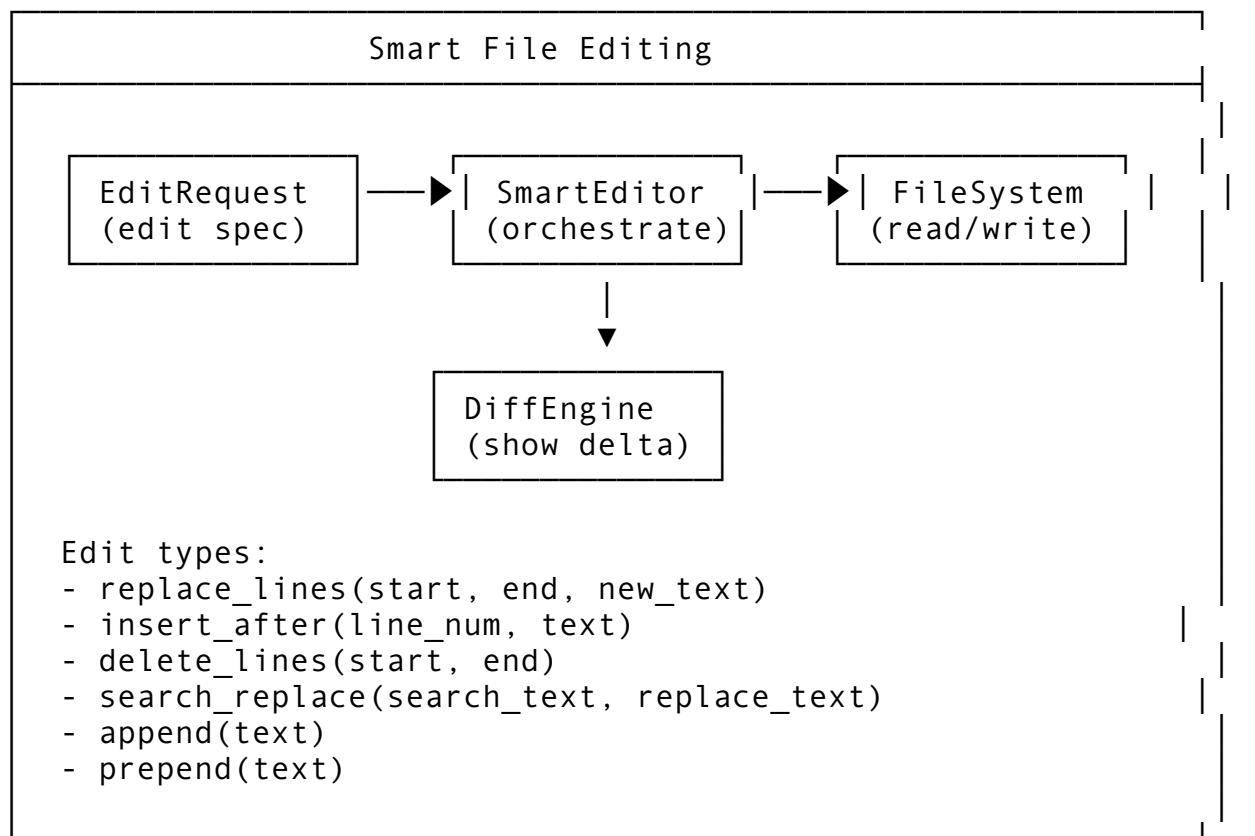
Feature 8: Smart File Editing (Edit Without Separate Read)

8.1 Feature Description

Smart File Editing allows the agent to modify files directly without first reading them into the conversation context. The system automatically reads the file, applies the edit, and presents a summary of changes, reducing token usage and improving efficiency.

Why it matters: Reading a file just to edit it wastes context window tokens. Smart editing reads the file internally, applies changes, and only reports what changed.

8.2 Architecture Design



8.3 API Design

```
// Package: internal/edit
package edit

import (
    "context"
    "fmt"
    "strings"
)

// EditType defines the kind of edit operation
type EditType string

const (
    EditReplaceLines EditType = "replace_lines"
    EditInsertAfter  EditType = "insert_after"
    EditInsertBefore EditType = "insert_before"
    EditDeleteLines  EditType = "delete_lines"
    EditSearchReplace EditType = "search_replace"
    EditAppend       EditType = "append"
    EditPrepend      EditType = "prepend"
    EditCreateFile   EditType = "create_file"
    EditRegexReplace EditType = "regex_replace"
```

)

// EditRequest specifies a file edit

```
type EditRequest struct {
    FilePath      string    `json:"file_path"`
    EditType      EditType  `json:"edit_type"`

    // For line-based edits
    StartLine     *int     `json:"start_line,omitempty"`
    EndLine       *int     `json:"end_line,omitempty"`

    // For search/replace
    SearchText     string    `json:"search_text,omitempty"`
    ReplaceText    string    `json:"replace_text,omitempty"`

    // For insert/append/prepend
    NewText        string    `json:"new_text,omitempty"`

    // For regex
    RegexPattern   string    `json:"regex_pattern,omitempty"`

    // Options
    DryRun         bool      `json:"dry_run"`
    CreateIfMissing bool      `json:"create_if_missing"`
    Backup         bool      `json:"backup"` // Create .bak file
}
```

// EditResult reports what changed

```
type EditResult struct {
    FilePath      string    `json:"file_path"`
    Success       bool      `json:"success"`
    EditType      EditType  `json:"edit_type"`

    // Change details
    OriginalLines []string  `json:"original_lines,omitempty"`
    NewLines      []string  `json:"new_lines,omitempty"`
    LinesChanged  int       `json:"lines_changed"`
    LinesAdded    int       `json:"lines_added"`
    LinesRemoved  int       `json:"lines_removed"`

    // File state
    OriginalHash  string    `json:"original_hash"`
    NewHash       string    `json:"new_hash"`
    OriginalSize  int64     `json:"original_size"`
    NewSize       int64     `json:"new_size"`

    // Error
```

```

    Error            string            `json:"error,omitempty"`
    Suggestion       string
    `json:"suggestion,omitempty"` // AI suggestion if edit
    failed

    // Diff
    UnifiedDiff      string            `json:"unified_diff,omitempty"`
}

// SmartEditor provides intelligent file editing
type SmartEditor interface {
    // EditFile applies an edit to a file
    EditFile(ctx context.Context, req EditRequest) (*EditResult,
        error)

    // BatchEdit applies multiple edits atomically
    BatchEdit(ctx context.Context, reqs []EditRequest)
        (*BatchEditResult, error)

    // PreviewEdit shows what would change without applying
    PreviewEdit(ctx context.Context, req EditRequest)
        (*EditResult, error)

    // ReadFile reads file for LLM context (with optional line
    range)
    ReadFile(ctx context.Context, filePath string, startLine,
        endLine *int) (*FileContent, error)

    // GetFileSummary returns a concise file overview
    GetFileSummary(ctx context.Context, filePath string)
        (*FileSummary, error)
}

// BatchEditResult reports batch operation outcome
type BatchEditResult struct {
    Results          []EditResult `json:"results"`
    Success          bool         `json:"success"`
    RolledBack       bool         `json:"rolled_back"`
    TotalChanges     int          `json:"total_changes"`
    FilesAffected    int          `json:"files_affected"`
}

// FileContent is the result of reading a file
type FileContent struct {
    FilePath         string       `json:"file_path"`
    Content          string       `json:"content"`
    Lines            []string    `json:"lines"`
    LineCount        int          `json:"line_count"`
}

```

```

        ByteSize      int64      `json:"byte_size"`
        Encoding      string     `json:"encoding"`
        Hash           string     `json:"hash"`
    }

    // FileSummary provides a concise overview
    type FileSummary struct {
        FilePath      string     `json:"file_path"`
        LineCount      int        `json:"line_count"`
        Language       string     `json:"language"`
        Imports        []string   `json:"imports,omitempty"`
        TopLevelDefs   []string   `json:"top_level_defs,omitempty"`
        KeyFunctions   []string   `json:"key_functions,omitempty"`
        Summary        string     `json:"summary"` // Tree-sitter
                    generated
    }

```

8.4 Integration Points

Package	Modification	Description
internal/edit/	NEW PACKAGE	Smart editing engine
internal/tool/ file.go	Replace with SmartEditor	Use smart editing instead of separate read+write
internal/llm/ prompts.go	Add edit tool descriptions	Teach LLM about smart edit format
internal/fs/	Add atomic read-write	Safe file operations
internal/git/	Auto-stage on edit	Optional git integration

8.5 Implementation Steps

```

// internal/edit/editor.go
package edit

import (
    "context"
    "crypto/sha256"
    "fmt"
    "io"
    "os"
    "path/filepath"
    "regexp"
    "strings"
)

type DefaultSmartEditor struct {
    fs FSProvider
}

```

```

func NewDefaultSmartEditor(fs FSProvider) *DefaultSmartEditor {
    return &DefaultSmartEditor{fs: fs}
}

func (e *DefaultSmartEditor) EditFile(ctx context.Context, req
    EditRequest) (*EditResult, error) {
    // Read existing file
    content, err := e.fs.ReadFile(req.FilePath)
    if err != nil {
        if req.CreateIfMissing && os.IsNotExist(err) {
            return e.createFile(ctx, req)
        }
        return &EditResult{FilePath: req.FilePath, Success:
            false, Error: err.Error()}, err
    }

    originalHash := e.hash(content)
    lines := strings.Split(string(content), "\n")

    if req.Backup {
        if err := e.fs.WriteFile(req.FilePath+".bak", content,
            0644); err != nil {
            return nil, fmt.Errorf("creating backup: %w", err)
        }
    }

    var newLines []string
    var result *EditResult

    switch req.EditType {
    case EditReplaceLines:
        result = e.replaceLines(lines, req)
    case EditInsertAfter:
        result = e.insertAfter(lines, req)
    case EditInsertBefore:
        result = e.insertBefore(lines, req)
    case EditDeleteLines:
        result = e.deleteLines(lines, req)
    case EditSearchReplace:
        result = e.searchReplace(lines, req)
    case EditAppend:
        result = e.appendLines(lines, req)
    case EditPrepend:
        result = e.prependLines(lines, req)
    case EditRegexReplace:
        result = e.regexReplace(lines, req)
    default:

```

```

        return &EditResult{FilePath: req.FilePath, Success:
            false, Error: fmt.Sprintf("unknown edit type: %s",
                req.EditType)}, nil
    }

    if !result.Success {
        return result, nil
    }

    newLines = result.NewLines
    newContent := strings.Join(newLines, "\n")

    if !req.DryRun {
        if err := e.fs.WriteFile(req.FilePath,
            []byte(newContent), 0644); err != nil {
            result.Success = false
            result.Error = err.Error()
            return result, err
        }
    }

    // Calculate diff
    result.UnifiedDiff = generateUnifiedDiff(req.FilePath,
        lines, newLines)
    result.OriginalHash = originalHash
    result.NewHash = e.hash([]byte(newContent))
    result.OriginalSize = int64(len(content))
    result.NewSize = int64(len(newContent))
    result.FilePath = req.FilePath
    result.EditType = req.EditType

    return result, nil
}

func (e *DefaultSmartEditor) replaceLines(lines []string, req
    EditRequest) *EditResult {
    if req.StartLine == nil || req.EndLine == nil {
        return &EditResult{Success: false, Error:
            "start_line and end_line required for replace_lines"}
    }

    start := *req.StartLine - 1 // Convert to 0-indexed
    end := *req.EndLine

    if start < 0 || end > len(lines) || start >= end {
        return &EditResult{Success: false, Error:
            fmt.Sprintf("invalid line range: %d-%d", *req.StartLine,
                *req.EndLine)}
    }

```

```

    }

    newLines := append([]string{}, lines[:start]...)
    newTextLines := strings.Split(req.NewText, "\n")
    newLines = append(newLines, newTextLines...)
    newLines = append(newLines, lines[end:]...)

    return &EditResult{
        Success:      true,
        OriginalLines: lines[start:end],
        NewLines:      newLines,
        LinesChanged:  end - start + len(newTextLines),
        LinesAdded:    len(newTextLines),
        LinesRemoved:  end - start,
    }
}

func (e *DefaultSmartEditor) searchReplace(lines []string, req
    EditRequest) *EditResult {
    content := strings.Join(lines, "\n")

    if !strings.Contains(content, req.SearchText) {
        return &EditResult{
            Success:      false,
            Error:        "search text not found",
            Suggestion:   "The text to search for was not found in
the file. Check for exact matching including
whitespace.",
        }
    }

    // Count occurrences
    count := strings.Count(content, req.SearchText)
    if count > 1 {
        return &EditResult{
            Success:      false,
            Error:        fmt.Sprintf("search text found %d times,
be more specific", count),
            Suggestion:   "The search text appears multiple times.
Include more surrounding context for a unique match.",
        }
    }

    newContent := strings.Replace(content, req.SearchText,
        req.ReplaceText, 1)
    newLines := strings.Split(newContent, "\n")

    return &EditResult{

```

```

        Success:    true,
        NewLines:   newLines,
        LinesAdded: strings.Count(req.ReplaceText, "\n") -
            strings.Count(req.SearchText, "\n") + 1,
    }
}

func (e *DefaultSmartEditor) insertAfter(lines []string, req
    EditRequest) *EditResult {
    if req.StartLine == nil {
        return &EditResult{Success: false, Error: "start_line
            required for insert_after"}
    }

    line := *req.StartLine - 1
    if line < 0 || line >= len(lines) {
        return &EditResult{Success: false, Error:
            fmt.Sprintf("line %d out of range", *req.StartLine)}
    }

    newLines := append([]string{}, lines[:line+1]...)
    newTextLines := strings.Split(req.NewText, "\n")
    newLines = append(newLines, newTextLines...)
    newLines = append(newLines, lines[line+1:]...)

    return &EditResult{
        Success:    true,
        NewLines:   newLines,
        LinesAdded: len(newTextLines),
    }
}

func (e *DefaultSmartEditor) regexReplace(lines []string, req
    EditRequest) *EditResult {
    re, err := regexp.Compile(req.RegexPattern)
    if err != nil {
        return &EditResult{Success: false, Error:
            fmt.Sprintf("invalid regex: %s", err)}
    }

    content := strings.Join(lines, "\n")
    if !re.MatchString(content) {
        return &EditResult{Success: false, Error: "regex pattern
            not found"}
    }

    newContent := re.ReplaceAllString(content, req.ReplaceText)
    newLines := strings.Split(newContent, "\n")

```



```

    return &EditResult{
        Success: true,
        NewLines: newLines,
    }
}

func (e *DefaultSmartEditor) createFile(ctx context.Context, req
    EditRequest) (*EditResult, error) {
    content := []byte(req.NewText)
    if !req.DryRun {
        if err := os.MkdirAll(filepath.Dir(req.FilePath), 0755);
            err != nil {
                return nil, err
            }
        if err := e.fs.WriteFile(req.FilePath, content, 0644);
            err != nil {
                return nil, err
            }
    }

    newLines := strings.Split(req.NewText, "\n")
    return &EditResult{
        Success: true,
        FilePath: req.FilePath,
        EditType: EditCreateFile,
        NewLines: newLines,
        LinesAdded: len(newLines),
        OriginalHash: "",
        NewHash: e.hash(content),
    }, nil
}

func (e *DefaultSmartEditor) hash(data []byte) string {
    h := sha256.Sum256(data)
    return fmt.Sprintf("%x", h[:8])
}

func generateUnifiedDiff(filename string, oldLines, newLines
    []string) string {
    // Simple unified diff generation
    var b strings.Builder
    b.WriteString(fmt.Sprintf("--- %s\n", filename))
    b.WriteString(fmt.Sprintf("+++ %s\n", filename))

    // This is a simplified diff - in production use a proper
    diff library

```

```

b.WriteString("@@ -1," + fmt.Sprintf("%d", len(oldLines)) +
    " +1," + fmt.Sprintf("%d", len(newLines)) + " @@\\n")

// A real implementation would use Myers diff algorithm
// For brevity, showing a simple marker approach
for _, line := range oldLines {
    b.WriteString("-" + line + "\\n")
}
for _, line := range newLines {
    b.WriteString"+" + line + "\\n")
}

return b.String()
}

func (e *DefaultSmartEditor) BatchEdit(ctx context.Context, reqs
    []EditRequest) (*BatchEditResult, error) {
    results := make([]EditResult, 0, len(reqs))
    allSuccess := true

    for _, req := range reqs {
        result, err := e.EditFile(ctx, req)
        if err != nil {
            allSuccess = false
            // Rollback previously applied edits
            // ... rollback logic
        }
        results = append(results, *result)
        if !result.Success {
            allSuccess = false
        }
    }

    return &BatchEditResult{
        Results:      results,
        Success:      allSuccess,
        TotalChanges: len(results),
        FilesAffected: len(uniqueFiles(reqs)),
    }, nil
}

// FSProvider interface for file operations
type FSProvider interface {
    ReadFile(path string) ([]byte, error)
    WriteFile(path string, data []byte, perm os.FileMode) error
}

```

8.6 Testing Approach

```
func TestSmartEditor_ReplaceLines(t *testing.T) {
    fs := newMockFS(map[string]string{
        "test.go": "line1\nline2\nline3\nline4\nline5",
    })
    editor := NewDefaultSmartEditor(fs)

    startLine := 2
    endLine := 4
    result, err := editor.EditFile(ctx, EditRequest{
        FilePath: "test.go",
        EditType: EditReplaceLines,
        StartLine: &startLine,
        EndLine:   &endLine,
        NewText:   "new_line2\nnew_line3",
    })

    assert.NoError(t, err)
    assert.True(t, result.Success)
    assert.Equal(t, 2, result.LinesAdded)
    assert.Equal(t, 2, result.LinesRemoved)
}

func TestSmartEditor_SearchReplace_MultipleMatches(t *testing.T)
{
    fs := newMockFS(map[string]string{
        "test.go": "func foo() {} \nfunc bar() {} \nfunc baz() {}",
    })
    editor := NewDefaultSmartEditor(fs)

    result, err := editor.EditFile(ctx, EditRequest{
        FilePath:   "test.go",
        EditType:    EditSearchReplace,
        SearchText:  "func",
        ReplaceText: "function",
    })

    assert.NoError(t, err)
    assert.False(t, result.Success)
    assert.Contains(t, result.Error, "3 times")
}
```

8.7 Edge Cases

Edge Case	Handling
File doesn't exist and !create_if_missing	Error with file list suggestion

Edge Case	Handling
Search text spans multiple lines	Support multi-line search with \n
Binary file edit	Error, refuse to edit binary
File modified after read (race)	Hash check before write, retry with fresh read
Edit exceeds file bounds	Clamp to valid range, report adjustment
Empty file	Handle gracefully, all operations valid
Very large file (100K+ lines)	Read partially, report line range
No-op edit (search=text)	Report no changes needed
Permission denied	Error with sudo suggestion if applicable
Edit creates invalid syntax	Let LLM know, suggest fix

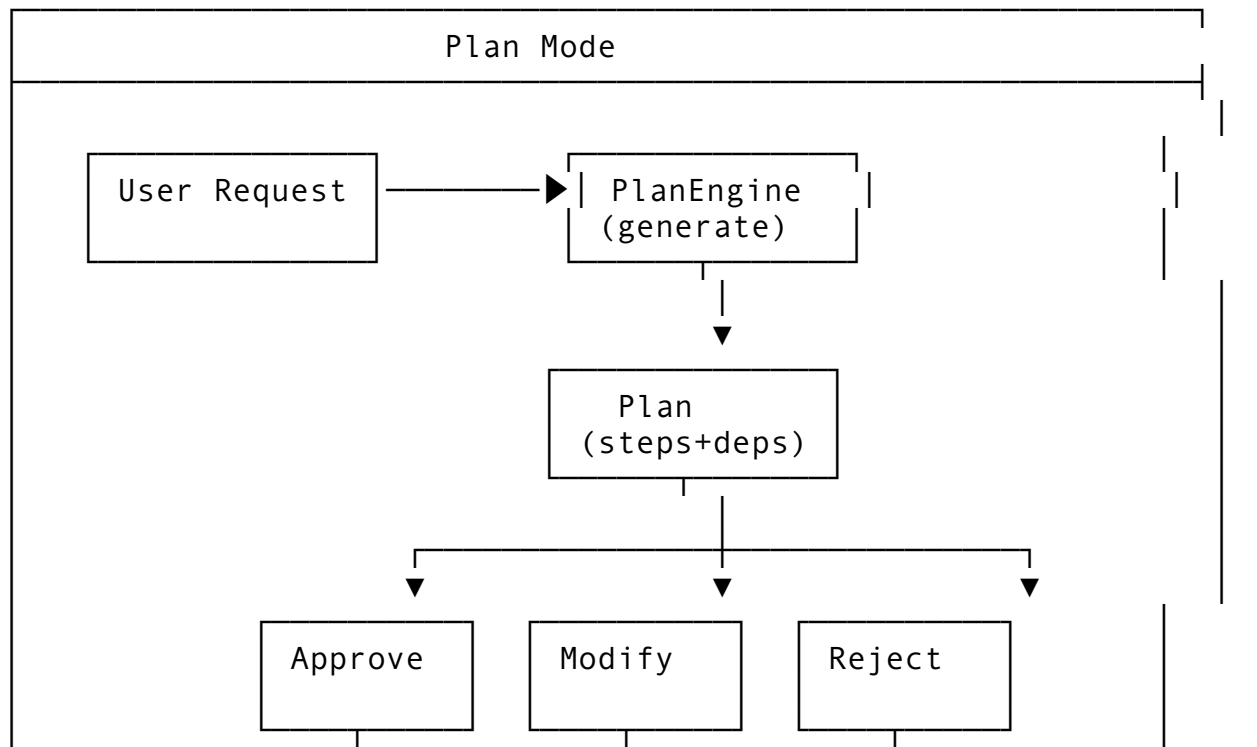
Feature 9: Plan Mode

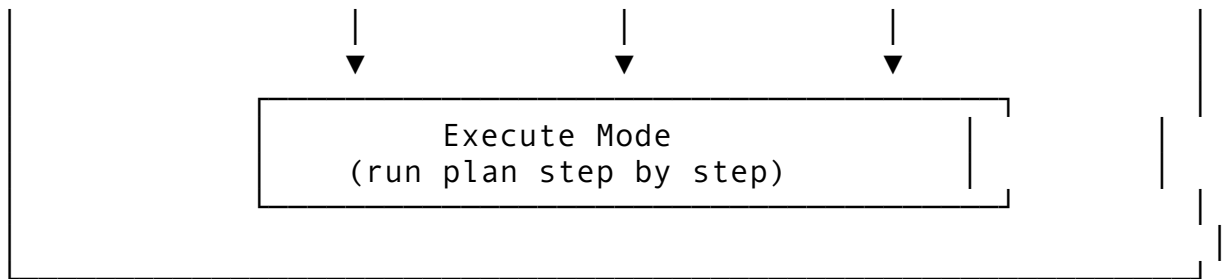
9.1 Feature Description

Plan Mode switches the agent from execution mode to planning mode. In this mode, the agent generates a structured plan of actions (with reasoning) before executing any tools. The user reviews and approves the plan before execution proceeds.

Why it matters: Complex multi-step tasks benefit from upfront planning. Users can catch misunderstandings early, and the agent can reason about dependencies before taking irreversible actions.

9.2 Architecture Design





9.3 API Design

```
// Package: internal/plan
package plan

import (
    "context"
    "fmt"
    "time"
)

// PlanStatus tracks plan lifecycle
type PlanStatus string

const (
    PlanStatusDrafting    PlanStatus = "drafting"
    PlanStatusPending     PlanStatus = "pending"      // Awaiting
    PlanStatusApproval    PlanStatus = "approval"
    PlanStatusApproved    PlanStatus = "approved"
    PlanStatusExecuting    PlanStatus = "executing"
    PlanStatusPaused      PlanStatus = "paused"
    PlanStatusCompleted   PlanStatus = "completed"
    PlanStatusPartial     PlanStatus = "partial"      // Some
    PlanStatusStepsFailed PlanStatus = "steps failed"
    PlanStatusFailed      PlanStatus = "failed"
    PlanStatusRejected     PlanStatus = "rejected"
    PlanStatusModified     PlanStatus = "modified"    // User
    PlanStatusEdited      PlanStatus = "edited"
)

// StepStatus tracks individual step state
type StepStatus string

const (
    StepStatusPending    StepStatus = "pending"
    StepStatusReady      StepStatus = "ready"      //
    StepStatusDependenciesMet StepStatus = "dependencies met"
    StepStatusRunning    StepStatus = "running"
    StepStatusCompleted  StepStatus = "completed"
    StepStatusFailed     StepStatus = "failed"
)
```

```

        StepStatusSkipped    StepStatus = "skipped"
        StepStatusBlocked    StepStatus = "blocked"           // Dependency
            failed
    )

    // PlanStep represents a single planned action
    type PlanStep struct {
        ID            string            `json:"id" db:"id"`
        PlanID        string            `json:"plan_id" db:"plan_id"`
        Order         int               `json:"order" db:"order"`

        Description    string            `json:"description" db:"description"`
        Reasoning      string            `json:"reasoning" db:"reasoning" // Why this step

        // Action specification
        ActionType     string            `json:"action_type" db:"action_type" // "tool_call", "file_edit", "message"
        ActionConfig   map[string]any `json:"action_config" db:"action_config"`

        // Dependencies
        DependsOn      []string          `json:"depends_on,omitempty" db:"depends_on" // Step IDs

        // State
        Status         StepStatus        `json:"status" db:"status"`
        Result         *StepResult       `json:"result,omitempty" db:"result"`

        // Risk
        RiskLevel      string            `json:"risk_level" db:"risk_level" // "low", "medium", "high"
        IsReversible   bool              `json:"is_reversible" db:"is_reversible"`

        // Timing
        StartedAt      *time.Time        `json:"started_at,omitempty" db:"started_at"`
        CompletedAt    *time.Time        `json:"completed_at,omitempty" db:"completed_at"`

        // Metadata
        AgentID        string            `json:"agent_id" db:"agent_id" // Which agent executes
        Tags           []string          `json:"tags,omitempty" db:"tags"`
    }

```

```

// StepResult is the outcome of executing a step
type StepResult struct {
    Success      bool           `json:"success"`
    Output        string         `json:"output,omitempty"`
    Error         string         `json:"error,omitempty"`
    Artifacts     []string       `json:"artifacts,omitempty" //
        File paths created
}

// Plan represents a complete execution plan
type Plan struct {
    ID            string         `json:"id" db:"id"`
    SessionID     string         `json:"session_id"
        db:"session_id"`
    AgentID       string         `json:"agent_id" db:"agent_id"`

    // Content
    Title         string         `json:"title" db:"title"`
    Description    string         `json:"description"
        db:"description"`
    Goal          string         `json:"goal" db:"goal" //
        Original user request

    Steps         []PlanStep     `json:"steps" db:"- "`

    // State
    Status        PlanStatus     `json:"status" db:"status"`
    CurrentStep   *string        `json:"current_step,omitempty"
        db:"current_step"`

    // User interaction
    UserNotes     string         `json:"user_notes,omitempty"
        db:"user_notes"`
    ApprovedBy    *string        `json:"approved_by,omitempty"
        db:"approved_by"`
    ApprovedAt    *time.Time     `json:"approved_at,omitempty"
        db:"approved_at"`

    // Metrics
    CreatedAt     time.Time      `json:"created_at"
        db:"created_at"`
    StartedAt     *time.Time     `json:"started_at,omitempty"
        db:"started_at"`
    CompletedAt   *time.Time     `json:"completed_at,omitempty"
        db:"completed_at"`

    // Configuration

```

```

    AutoApprove bool           `json:"auto_approve"
        db:"auto_approve" ` // Skip approval for low-risk
    MaxRetries int             `json:"max_retries"
        db:"max_retries" `
}

// PlanEngine generates and executes plans
type PlanEngine interface {
    // GeneratePlan creates a plan from a user request
    GeneratePlan(ctx context.Context, sessionID, userRequest
        string) (*Plan, error)

    // ApprovePlan marks plan as approved and begins execution
    ApprovePlan(ctx context.Context, planID string, userNotes
        string) error

    // RejectPlan cancels the plan
    RejectPlan(ctx context.Context, planID string, reason
        string) error

    // ModifyPlan allows user to edit steps before approval
    ModifyPlan(ctx context.Context, planID string, modifications
        []StepModification) (*Plan, error)

    // ExecutePlan runs an approved plan
    ExecutePlan(ctx context.Context, planID string) error

    // ExecuteStep runs a single step
    ExecuteStep(ctx context.Context, planID, stepID string)
        (*StepResult, error)

    // PausePlan pauses execution
    PausePlan(ctx context.Context, planID string) error

    // ResumePlan continues execution
    ResumePlan(ctx context.Context, planID string) error

    // GetPlan retrieves plan status
    GetPlan(ctx context.Context, planID string) (*Plan, error)

    // ListPlans returns plans for a session
    ListPlans(ctx context.Context, sessionID string) ([]Plan,
        error)

    // GetNextReadyStep returns steps that can execute
    // (dependencies met)
    GetNextReadySteps(ctx context.Context, planID string)
        ([]PlanStep, error)

```



```

}

// StepModification describes a user edit to a plan step
type StepModification struct {
    StepID      string    `json:"step_id"`
    Operation    string    `json:"operation" // "update",
                  "delete", "insert_after", "reorder"
    Updates      *PlanStep `json:"updates,omitempty"`
    NewOrder     *int      `json:"new_order,omitempty"`
}

```

9.4 Integration Points

Package	Modification	Description
internal/plan/	NEW PACKAGE	Plan engine
internal/ actor/ session.go	Plan mode toggle	Switch between plan/act modes
internal/llm/ prompts.go	Add planning prompt	Teach LLM to generate plans
internal/ui/	Plan display UI	Show plan steps with checkboxes
internal/db/ migrations/	Add plans, plan_steps tables	Persist plans
internal/ permissions/	Plan-level permissions	Approve all steps at once

9.5 Implementation Steps

```

// internal/plan/engine.go
package plan

import (
    "context"
    "fmt"
    "strings"
    "time"

    "dev.helix.code/internal/db"
    "dev.helix.code/internal/llm"
    "dev.helix.code/pkg/logger"
)

type DefaultPlanEngine struct {
    store      db.Store
    llmClient  llm.Client
    log        logger.Logger
    executors  map[string]StepExecutor
}

```

```

}

func NewDefaultPlanEngine(store db.Store, llmClient llm.Client,
    log logger.Logger) *DefaultPlanEngine {
    return &DefaultPlanEngine{
        store:      store,
        llmClient:  llmClient,
        log:        log,
        executors:  make(map[string]StepExecutor),
    }
}

func (e *DefaultPlanEngine) GeneratePlan(ctx context.Context,
    sessionID, userRequest string) (*Plan, error) {
    // Build planning prompt
    prompt := fmt.Sprintf(`You are a planning assistant. Break
        down the following task into concrete, executable steps.

```

User request: %s

Generate a plan with:

1. A title and brief description
2. Numbered steps, each with:
 - Clear description of what to do
 - Reasoning for why this step is needed
 - Action type (tool_call, file_edit, message)
 - Risk level (low/medium/high)
 - Whether it's reversible
 - Dependencies on previous steps (if any)

Format as JSON:

```

{
    "title": "...",
    "description": "...",
    "steps": [
        {
            "order": 1,
            "description": "...",
            "reasoning": "...",
            "action_type": "tool_call|file_edit|message",
            "action_config": {...},
            "risk_level": "low|medium|high",
            "is_reversible": true|false,
            "depends_on": []
        }
    ]
}, userRequest)

```

```

response, err := e.llmClient.Complete(ctx,
    llm.CompletionRequest{
        Model:      "claude-sonnet-4", // Use capable model
        Prompt:     prompt,
        MaxTokens:  4000,
        Temperature: 0.2,
    })
if err != nil {
    return nil, fmt.Errorf("generating plan: %w", err)
}

// Parse JSON response
plan, err := e.parsePlanJSON(response.Text, sessionID)
if err != nil {
    return nil, fmt.Errorf("parsing plan: %w", err)
}

// Store plan
plan.Status = PlanStatusPending
plan.CreatedAt = time.Now()
if err := e.store.CreatePlan(ctx, plan); err != nil {
    return nil, fmt.Errorf("storing plan: %w", err)
}

// Store steps
for i := range plan.Steps {
    plan.Steps[i].PlanID = plan.ID
    plan.Steps[i].Status = StepStatusPending
    if err := e.store.CreatePlanStep(ctx, &plan.Steps[i]);
    err != nil {
        return nil, fmt.Errorf("storing step: %w", err)
    }
}

return plan, nil
}

func (e *DefaultPlanEngine) ExecutePlan(ctx context.Context,
    planID string) error {
    plan, err := e.GetPlan(ctx, planID)
    if err != nil {
        return err
    }

    if plan.Status != PlanStatusApproved {
        return fmt.Errorf("plan must be approved before
            execution, current status: %s", plan.Status)
    }
}

```

```

plan.Status = PlanStatusExecuting
now := time.Now()
plan.StartedAt = &now
e.store.UpdatePlan(ctx, plan)

// Execute steps respecting dependencies
for {
    readySteps, err := e.GetNextReadySteps(ctx, planID)
    if err != nil {
        return err
    }

    if len(readySteps) == 0 {
        // Check if any steps still pending (blocked) or
        // running
        allDone := e.checkAllDone(plan)
        if allDone {
            break
        }
        // Wait for running steps
        time.Sleep(100 * time.Millisecond)
        continue
    }

    // Execute ready steps (concurrently if independent)
    for _, step := range readySteps {
        go e.executeStepWithTracking(ctx, planID, step.ID)
    }
}

// Finalize
completedAt := time.Now()
plan.CompletedAt = &completedAt

// Determine final status
hasFailures := false
for _, step := range plan.Steps {
    if step.Status == StepStatusFailed {
        hasFailures = true
        break
    }
}

if hasFailures {
    plan.Status = PlanStatusPartial
} else {
    plan.Status = PlanStatusCompleted
}

```

```

    }

    e.store.UpdatePlan(ctx, plan)

    return nil
}

func (e *DefaultPlanEngine) GetNextReadySteps(ctx
    context.Context, planID string) ([]PlanStep, error) {
    allSteps, err := e.store.GetPlanSteps(ctx, planID)
    if err != nil {
        return nil, err
    }

    // Build status map
    statusMap := make(map[string]StepStatus)
    for _, s := range allSteps {
        statusMap[s.ID] = s.Status
    }

    var ready []PlanStep
    for _, step := range allSteps {
        if step.Status != StepStatusPending {
            continue
        }

        // Check all dependencies completed
        depsMet := true
        for _, depID := range step.DependsOn {
            if statusMap[depID] != StepStatusCompleted {
                // If dependency failed, mark this as blocked
                if statusMap[depID] == StepStatusFailed ||
                    statusMap[depID] == StepStatusBlocked {
                    step.Status = StepStatusBlocked
                    e.store.UpdatePlanStep(ctx, &step)
                }
                depsMet = false
                break
            }
        }

        if depsMet {
            step.Status = StepStatusReady
            ready = append(ready, step)
        }
    }

    return ready, nil
}

```

```

}

func (e *DefaultPlanEngine) executeStepWithTracking(ctx
    context.Context, planID, stepID string) {
    step, err := e.store.GetPlanStep(ctx, stepID)
    if err != nil {
        e.log.Error("fetching step", "error", err)
        return
    }

    now := time.Now()
    step.StartedAt = &now
    step.Status = StepStatusRunning
    step.CurrentStep = &stepID
    e.store.UpdatePlanStep(ctx, step)

    // Execute
    result, err := e.ExecuteStep(ctx, planID, stepID)

    completedAt := time.Now()
    step.CompletedAt = &completedAt

    if err != nil || !result.Success {
        step.Status = StepStatusFailed
        step.Result = &StepResult{Success: false, Error:
            err.Error()}
    } else {
        step.Status = StepStatusCompleted
        step.Result = result
    }

    e.store.UpdatePlanStep(ctx, step)
}

func (e *DefaultPlanEngine) ApprovePlan(ctx context.Context,
    planID, userNotes string) error {
    plan, err := e.GetPlan(ctx, planID)
    if err != nil {
        return err
    }

    if plan.Status != PlanStatusPending && plan.Status !=
        PlanStatusModified {
        return fmt.Errorf("plan cannot be approved in status:
            %s", plan.Status)
    }

    plan.Status = PlanStatusApproved

```

```

    plan.UserNotes = userNotes
    now := time.Now()
    plan.ApprovedAt = &now
    // plan.ApprovedBy = &userID // If user auth available

    return e.store.UpdatePlan(ctx, plan)
}

func (e *DefaultPlanEngine) ModifyPlan(ctx context.Context,
    planID string, modifications []StepModification) (*Plan,
    error) {
    plan, err := e.GetPlan(ctx, planID)
    if err != nil {
        return nil, err
    }

    if plan.Status != PlanStatusPending && plan.Status !=
        PlanStatusRejected {
        return nil, fmt.Errorf("plan can only be modified when
            pending or rejected")
    }

    // Apply modifications
    for _, mod := range modifications {
        switch mod.Operation {
        case "update":
            for i := range plan.Steps {
                if plan.Steps[i].ID == mod.StepID {
                    if mod.Updates != nil {
                        plan.Steps[i].Description =
                            mod.Updates.Description
                        plan.Steps[i].Reasoning =
                            mod.Updates.Reasoning
                        plan.Steps[i].ActionConfig =
                            mod.Updates.ActionConfig
                        plan.Steps[i].RiskLevel =
                            mod.Updates.RiskLevel
                    }
                    break
                }
            }
        case "delete":
            // Remove step and update dependencies
            var newSteps []PlanStep
            for _, step := range plan.Steps {
                if step.ID != mod.StepID {
                    // Remove this step from dependency lists
                    var newDeps []string

```

```

        for _, dep := range step.DependsOn {
            if dep != mod.StepID {
                newDeps = append(newDeps, dep)
            }
        }
        step.DependsOn = newDeps
        newSteps = append(newSteps, step)
    }
    plan.Steps = newSteps
case "insert_after":
    // Insert new step after specified step
    // ... implementation
case "reorder":
    // Change step order
    // ... implementation
}
}

plan.Status = PlanStatusModified
e.store.UpdatePlan(ctx, plan)

// Re-store steps
for _, step := range plan.Steps {
    e.store.UpdatePlanStep(ctx, &step)
}

return plan, nil
}

func (e *DefaultPlanEngine) parsePlanJSON(text, sessionID
    string) (*Plan, error) {
    // Extract JSON from response (may have markdown fences)
    jsonStart := strings.Index(text, "{")
    jsonEnd := strings.LastIndex(text, "}")
    if jsonStart == -1 || jsonEnd == -1 {
        return nil, fmt.Errorf("no JSON found in response")
    }
    jsonStr := text[jsonStart : jsonEnd+1]

    // Parse using encoding/json
    // ... standard JSON unmarshaling into Plan structure

    // For brevity, showing pseudo-code:
    var rawPlan struct {
        Title      string `json:"title"`
        Description string `json:"description"`
        Steps      []struct {

```



```

        Order      int           `json:"order"`
        Description string       `json:"description"`
        Reasoning   string       `json:"reasoning"`
        ActionType  string       `json:"action_type"`
        ActionConfig map[string]any `json:"action_config"`
        RiskLevel   string       `json:"risk_level"`
        IsReversible bool        `json:"is_reversible"`
        DependsOn   []string     `json:"depends_on"`
    } `json:"steps"`
}

// Unmarshal and convert to Plan structure
// ...

return &Plan{}, nil
}

func (e *DefaultPlanEngine) checkAllDone(plan *Plan) bool {
    for _, step := range plan.Steps {
        if step.Status == StepStatusPending || step.Status ==
            StepStatusReady ||
            step.Status == StepStatusRunning {
            return false
        }
    }
    return true
}

// StepExecutor interface for different action types
type StepExecutor interface {
    Execute(ctx context.Context, step PlanStep) (*StepResult,
        error)
    Supports(actionType string) bool
}

```

9.6 Testing Approach

```

func TestPlanEngine_GeneratePlan(t *testing.T) {
    engine, mockLLM, store := setupTestEngine()

    mockLLM.On("Complete", mock.Anything,
        mock.Anything).Return(&llm.CompletionResponse{
        Text: `{"title":"Test Plan","description":"A test plan","steps":
        [{"order":1,"description":"Step
        1","reasoning":"Because","action_type":"tool_call","action_config"
        {"tool":"test"},"risk_level":"low","is_reversible":true}]}`,
        }, nil)
}

```

```

plan, err := engine.GeneratePlan(ctx, "session-1", "Create a
    hello world program")

assert.NoError(t, err)
assert.NotNil(t, plan)
assert.Equal(t, "Test Plan", plan.Title)
assert.Equal(t, PlanStatusPending, plan.Status)
assert.Len(t, plan.Steps, 1)
}

func TestPlanEngine_DependencyResolution(t *testing.T) {
    engine, _, _ := setupTestEngine()

    plan := &Plan{
        ID: "plan-1",
        Steps: []PlanStep{
            {ID: "step-1", Order: 1, Status:
                StepStatusCompleted},
            {ID: "step-2", Order: 2, Status: StepStatusPending,
                DependsOn: []string{"step-1"}},
            {ID: "step-3", Order: 3, Status: StepStatusPending,
                DependsOn: []string{"step-2"}},
            {ID: "step-4", Order: 4, Status:
                StepStatusPending}, // No dependencies
        },
    }

    // After step-1 completes, step-2 and step-4 should be ready
    ready, err := engine.GetNextReadySteps(ctx, plan.ID)
    assert.NoError(t, err)
    assert.Len(t, ready, 2) // step-2 and step-4
}

```

9.7 Edge Cases

Edge Case	Handling
Circular dependencies	Detect during generation, error with cycle path
Step fails and blocks dependents	Mark dependents blocked, report to user
LLM generates invalid plan JSON	Retry with structured prompt, fallback to error
Plan too large (100+ steps)	Paginate, suggest splitting into sub-plans
All steps are high-risk	Require explicit per-step approval even with auto_approve
User modifies plan during execution	Lock plan during execution, reject modifications

Edge Case	Handling
Step exceeds timeout	Mark failed, allow retry or skip
Plan partially approved	Allow approving subset of steps
Dependency on external event	Support “wait” action type with polling

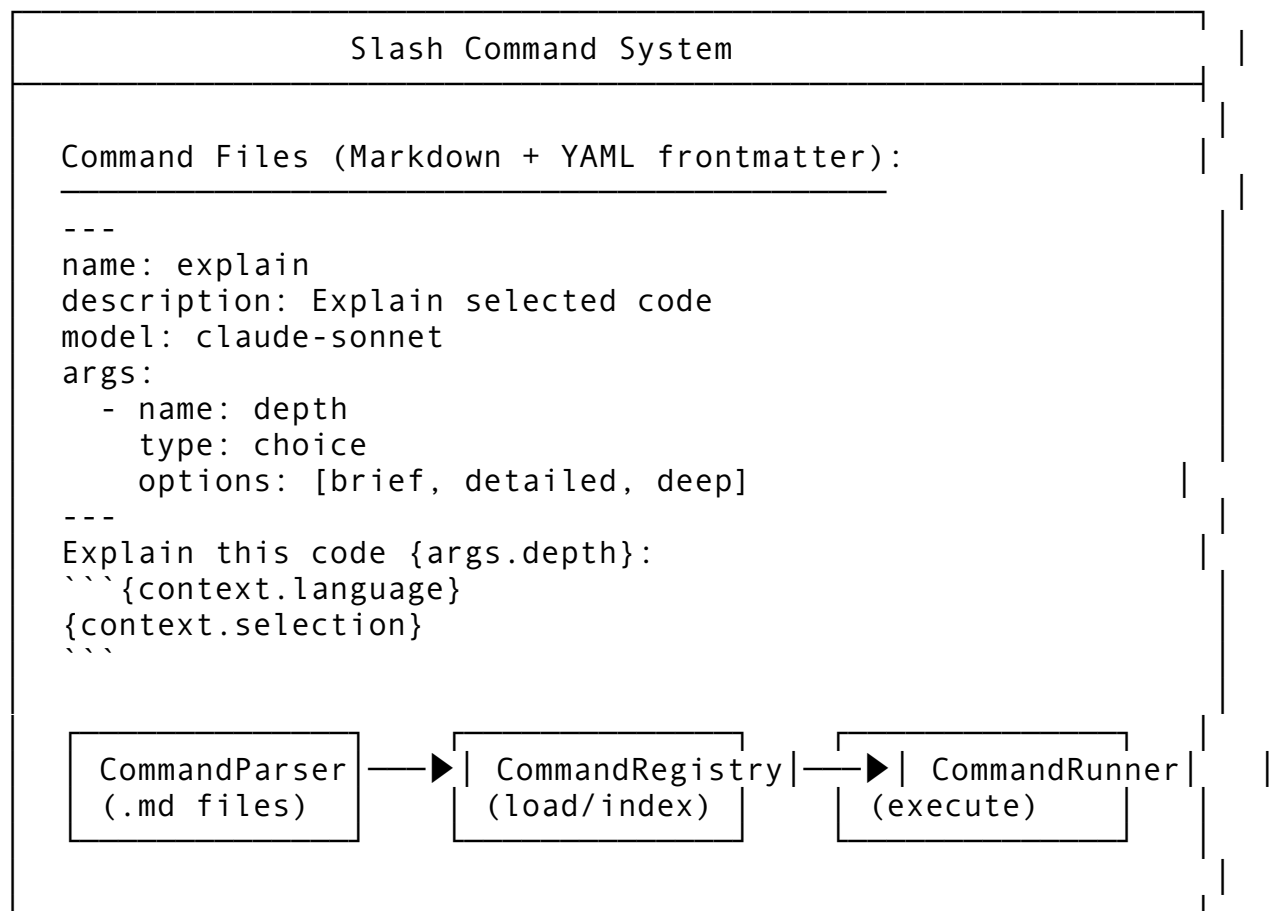
Feature 10: Slash Command System (Markdown + YAML Frontmatter)

10.1 Feature Description

The Slash Command System provides user-triggered commands via `/command` syntax. Commands are defined in Markdown files with YAML frontmatter metadata, making them self-documenting and easy to extend.

Why it matters: Power users need quick access to common operations. A declarative command format allows users and teams to create custom workflows without writing code.

10.2 Architecture Design



10.3 API Design

```
// Package: internal/commands
package commands

import (
    "context"
    "fmt"
    "path/filepath"
    "strings"
    "time"
)

// SlashCommand represents a user-defined command
type SlashCommand struct {
    // YAML Frontmatter
    Name          string          `yaml:"name" json:"name"`
    Description    string          `yaml:"description" json:"description"`
    Category      string          `yaml:"category,omitempty" json:"category,omitempty"`
    Author        string          `yaml:"author,omitempty" json:"author,omitempty"`
    Version       string          `yaml:"version,omitempty" json:"version,omitempty"`

    // Execution
    Model         string          `yaml:"model,omitempty" json:"model,omitempty"` // Preferred LLM model
    Temperature   *float64       `yaml:"temperature,omitempty" json:"temperature,omitempty"`
    MaxTokens     *int           `yaml:"max_tokens,omitempty" json:"max_tokens,omitempty"`

    // Arguments
    Args          []CommandArg   `yaml:"args,omitempty" json:"args,omitempty"`

    // Context
    Context       CommandContext `yaml:"context,omitempty" json:"context,omitempty"`

    // The prompt template (Markdown body after frontmatter)
    Prompt        string          `yaml:"- " json:"- " // Parsed from markdown body`

    // Metadata
    SourcePath    string          `yaml:"- " json:"source_path"`
}
```

```

    IsBuiltIn    bool           `yaml:"- " json:"is_built_in"`
    IsEnabled    bool           `yaml:"- " json:"is_enabled"`
    LastLoaded   time.Time      `yaml:"- " json:"- "`
}

// CommandArg defines an argument the command accepts
type CommandArg struct {
    Name          string    `yaml:"name" json:"name"`
    Description    string    `yaml:"description" json:"description"`
    Type          string    `yaml:"type" json:"type" // "string",
    "number", "boolean", "choice", "file", "directory"
    Required      bool      `yaml:"required,omitempty"
    json:"required,omitempty"`
    Default       any       `yaml:"default,omitempty"
    json:"default,omitempty"`
    Options       []string  `yaml:"options,omitempty"
    json:"options,omitempty" // For choice type
    Validation    string    `yaml:"validation,omitempty"
    json:"validation,omitempty" // Regex pattern
}

// CommandContext defines what context is available to the
// command
type CommandContext struct {
    IncludeSelection bool      `yaml:"include_selection,omitempty"
    json:"include_selection,omitempty"`
    IncludeFile      bool      `yaml:"include_file,omitempty"
    json:"include_file,omitempty"`
    IncludeDirectory bool      `yaml:"include_directory,omitempty"
    json:"include_directory,omitempty"`
    IncludeWorkspace bool      `yaml:"include_workspace,omitempty"
    json:"include_workspace,omitempty"`
    FilePatterns     []string  `yaml:"file_patterns,omitempty"
    json:"file_patterns,omitempty"`
    MaxFiles         int       `yaml:"max_files,omitempty"
    json:"max_files,omitempty"`
}

// CommandInvocation represents a user's command call
type CommandInvocation struct {
    Command    string    `json:"command"`
    Args       map[string]any `json:"args"`
    RawInput   string    `json:"raw_input" // The full /
    command ... text
    Context    InvocationContext `json:"context"`
}

```

```

    SessionID string          `json:"session_id"`
    AgentID   string          `json:"agent_id"`
}

// InvocationContext provides runtime context
type InvocationContext struct {
    Selection    string    `json:"selection,omitempty"`    //
        Selected text
    CurrentFile string    `json:"current_file,omitempty"` //
        Active file
    CurrentDir  string    `json:"current_dir,omitempty"`  //
        Working directory
    Workspace  string    `json:"workspace,omitempty"`    //
        Project root
    Language   string    `json:"language,omitempty"`     //
        Detected language
}

// CommandResult is the outcome of running a command
type CommandResult struct {
    Success    bool    `json:"success"`
    Response   string  `json:"response,omitempty"`
    Error      string  `json:"error,omitempty"`
    Actions    []SuggestedAction `json:"actions,omitempty"`
}

// SuggestedAction is a follow-up the command suggests
type SuggestedAction struct {
    Type      string `json:"type"`    // "run_command",
        "open_file", "edit_file"
    Label     string `json:"label"`
    Payload   map[string]any `json:"payload"`
}

// CommandRegistry manages slash commands
type CommandRegistry interface {
    // LoadCommands discovers and loads command definitions
    LoadCommands(ctx context.Context, dirs []string) error

    // RegisterCommand adds a command programmatically
    RegisterCommand(ctx context.Context, cmd SlashCommand) error

    // UnregisterCommand removes a command
    UnregisterCommand(ctx context.Context, name string) error

    // GetCommand retrieves a command definition
    GetCommand(ctx context.Context, name string) (*SlashCommand,
        error)
}

```

```

// ListCommands returns available commands
ListCommands(ctx context.Context, filter CommandFilter)
    ([]SlashCommand, error)

// Execute runs a command invocation
Execute(ctx context.Context, invocation CommandInvocation)
    (*CommandResult, error)

// Complete suggests command completions
Complete(ctx context.Context, partial string)
    []CommandCompletion

// Reload refreshes command definitions
Reload(ctx context.Context) error
}

// CommandFilter filters command listings
type CommandFilter struct {
    Category    *string `json:"category,omitempty"`
    BuiltIn     *bool   `json:"built_in,omitempty"`
    Enabled     *bool   `json:"enabled,omitempty"`
    Search      *string `json:"search,omitempty"`
}

// CommandCompletion is an autocomplete suggestion
type CommandCompletion struct {
    Name          string `json:"name"`
    Description    string `json:"description"`
    InsertText     string `json:"insert_text"`
}

```

10.4 Integration Points

Package	Modification	Description
internal/ commands/	NEW PACKAGE	Command registry and executor
internal/llm/	Command LLM calls	Route command prompts to LLM
cmd/helix/ main.go	Add --commands-dir flag	Load custom commands
internal/ config/	Add commands config	Enable/disable built-ins
internal/ui/	Command completion UI	/ triggers autocomplete

10.5 Implementation Steps

```
// internal/commands/registry.go
package commands

import (
    "bufio"
    "context"
    "fmt"
    "os"
    "path/filepath"
    "strings"
    "time"

    "github.com/goccy/go-yaml" // or similar YAML library
    "dev.helix.code/internal/llm"
    "dev.helix.code/pkg/logger"
)

type DefaultCommandRegistry struct {
    commands    map[string]*SlashCommand
    llmClient   llm.Client
    log         logger.Logger
    commandDirs []string
}

func NewDefaultCommandRegistry(llmClient llm.Client, log
    logger.Logger) *DefaultCommandRegistry {
    return &DefaultCommandRegistry{
        commands:    make(map[string]*SlashCommand),
        llmClient:   llmClient,
        log:         log,
    }
}

func (r *DefaultCommandRegistry) LoadCommands(ctx
    context.Context, dirs []string) error {
    r.commandDirs = dirs

    for _, dir := range dirs {
        if err := r.loadFromDir(ctx, dir); err != nil {
            r.log.Warn("failed to load commands from dir",
                "dir", dir, "error", err)
        }
    }

    // Load built-in commands
    if err := r.loadBuiltins(ctx); err != nil {
```



```

        return err
    }

    return nil
}

func (r *DefaultCommandRegistry) loadFromDir(ctx
    context.Context, dir string) error {
    entries, err := os.ReadDir(dir)
    if err != nil {
        return err
    }

    for _, entry := range entries {
        if entry.IsDir() || !strings.HasSuffix(entry.Name(),
            ".md") {
            continue
        }

        path := filepath.Join(dir, entry.Name())
        if err := r.loadCommandFile(ctx, path); err != nil {
            r.log.Warn("failed to load command file", "path",
                path, "error", err)
            continue
        }
    }

    return nil
}

func (r *DefaultCommandRegistry) loadCommandFile(ctx
    context.Context, path string) error {
    data, err := os.ReadFile(path)
    if err != nil {
        return err
    }

    content := string(data)

    // Parse frontmatter
    if !strings.HasPrefix(content, "---") {
        return fmt.Errorf("no YAML frontmatter found")
    }

    // Find end of frontmatter
    endIdx := strings.Index(content[3:], "---")
    if endIdx == -1 {
        return fmt.Errorf("unterminated YAML frontmatter")
    }

```

```

}

frontmatter := content[3 : endIdx+3]
prompt := strings.TrimSpace(content[endIdx+6:])

// Parse YAML
var cmd SlashCommand
if err := yaml.Unmarshal([]byte(frontmatter), &cmd); err !=
    nil {
    return fmt.Errorf("parsing frontmatter: %w", err)
}

cmd.Prompt = prompt
cmd.SourcePath = path
cmd.IsBuiltin = false
cmd.IsEnabled = true
cmd.LastLoaded = time.Now()

// Validate
if cmd.Name == "" {
    return fmt.Errorf("command name is required")
}

// Register
r.commands[cmd.Name] = &cmd
r.log.Info("loaded command", "name", cmd.Name, "path", path)

return nil
}

func (r *DefaultCommandRegistry) loadBuiltins(ctx
context.Context) error {
    builtins := []SlashCommand{
        {
            Name: "explain",
            Description: "Explain the selected code or concept",
            Category: "code",
            IsBuiltin: true,
            IsEnabled: true,
            Args: []CommandArg{
                {
                    Name: "depth",
                    Description: "Explanation depth",
                    Type: "choice",
                    Required: false,
                    Default: "detailed",
                    Options: []string{"brief", "detailed",
"deep"},
                },
            },
        },
    }

```

```

    },
  },
  Context: CommandContext{
    IncludeSelection: true,
    IncludeFile: true,
  },
  Prompt: "Explain the following code at {args.depth}
level:\n\n```${context.language}\n{context.selection}
\n```\n\nFocus on:\n1. What this code does\n2. Key
patterns or algorithms used\n3. Potential edge cases or
issues",
},
{
  Name: "refactor",
  Description: "Suggest refactoring for selected code",
  Category: "code",
  IsBuiltIn: true,
  IsEnabled: true,
  Args: []CommandArg{
    {
      Name: "focus",
      Description: "Refactoring focus",
      Type: "choice",
      Required: false,
      Default: "readability",
      Options: []string{"readability",
"performance", "safety", "simplicity"},
    },
  },
},
  Context: CommandContext{
    IncludeSelection: true,
    IncludeFile: true,
  },
  Prompt: "Refactor the following code focusing on
{args.focus}:\n\n```${context.language}
\n{context.selection}\n```\n\nProvide:
\n1. The refactored code\n2. Explanation of changes\n3.
Trade-offs considered",
},
{
  Name: "test",
  Description: "Generate tests for selected code",
  Category: "testing",
  IsBuiltIn: true,
  IsEnabled: true,
  Args: []CommandArg{
    {
      Name: "framework",

```

```

        Description: "Test framework",
        Type:        "choice",
        Required:    false,
        Default:     "auto",
        Options:     []string{"auto", "standard",
"table_driven"},
    },
    },
    Context: CommandContext{
        IncludeSelection: true,
        IncludeFile:      true,
    },
    Prompt: "Generate comprehensive tests for:
\n\n```${context.language}\n{context.selection}
\n```\n\nInclude:\n1. Happy path tests\n2. Edge case
tests\n3. Error condition tests\n4. Table-driven tests
where appropriate",
},
}

for i := range builtins {
    builtins[i].LastLoaded = time.Now()
    r.commands[builtins[i].Name] = &builtins[i]
}

return nil
}

func (r *DefaultCommandRegistry) Execute(ctx context.Context,
    invocation CommandInvocation) (*CommandResult, error) {
    cmd, ok := r.commands[invocation.Command]
    if !ok {
        return &CommandResult{
            Success: false,
            Error:    fmt.Sprintf("unknown command: /%s",
invocation.Command),
        }, nil
    }

    if !cmd.IsEnabled {
        return &CommandResult{
            Success: false,
            Error:    fmt.Sprintf("command /%s is disabled",
invocation.Command),
        }, nil
    }

    // Validate arguments

```

```

validatedArgs, err := r.validateArgs(cmd, invocation.Args)
if err != nil {
    return &CommandResult{
        Success: false,
        Error:    fmt.Sprintf("argument error: %s", err),
    }, nil
}

// Build prompt from template
prompt, err := r.renderPrompt(cmd.Prompt, validatedArgs,
    invocation.Context)
if err != nil {
    return nil, fmt.Errorf("rendering prompt: %w", err)
}

// Determine model
model := cmd.Model
if model == "" {
    model = "claude-sonnet-4" // Default
}

// Call LLM
req := llm.CompletionRequest{
    Model: model,
    Prompt: prompt,
}
if cmd.Temperature != nil {
    req.Temperature = *cmd.Temperature
}
if cmd.MaxTokens != nil {
    req.MaxTokens = *cmd.MaxTokens
}

response, err := r.llmClient.Complete(ctx, req)
if err != nil {
    return nil, fmt.Errorf("LLM call: %w", err)
}

return &CommandResult{
    Success: true,
    Response: response.Text,
}, nil
}

func (r *DefaultCommandRegistry) renderPrompt(template string,
    args map[string]any, ctx InvocationContext) (string,
    error) {
    result := template

```

```

// Replace {args.name}
for k, v := range args {
    placeholder := fmt.Sprintf("{args.%s}", k)
    result = strings.ReplaceAll(result, placeholder,
        fmt.Sprintf("%v", v))
}

// Replace {context.field}
result = strings.ReplaceAll(result, "{context.selection}",
    ctx.Selection)
result = strings.ReplaceAll(result,
    "{context.current_file}", ctx.CurrentFile)
result = strings.ReplaceAll(result, "{context.current_dir}",
    ctx.CurrentDir)
result = strings.ReplaceAll(result, "{context.workspace}",
    ctx.Workspace)
result = strings.ReplaceAll(result, "{context.language}",
    ctx.Language)

// Clean up unreplaced placeholders
// (optional: could error on unreplaced required ones)

return result, nil
}

func (r *DefaultCommandRegistry) validateArgs(cmd *SlashCommand,
    provided map[string]any) (map[string]any, error) {
    result := make(map[string]any)

    for _, arg := range cmd.Args {
        val, provided := provided[arg.Name]

        if !provided {
            if arg.Required {
                return nil, fmt.Errorf("required argument '%s'
not provided", arg.Name)
            }
            if arg.Default != nil {
                val = arg.Default
            } else {
                continue
            }
        }

        // Type validation
        switch arg.Type {
        case "choice":

```

```

        strVal, ok := val.(string)
        if !ok {
            return nil, fmt.Errorf("argument '%s' must be a
string", arg.Name)
        }
        valid := false
        for _, opt := range arg.Options {
            if opt == strVal {
                valid = true
                break
            }
        }
        if !valid {
            return nil,
fmt.Errorf("argument '%s' must be one of: %v", arg.Name,
arg.Options)
        }
    case "number":
        // Validate numeric
    case "boolean":
        // Validate bool
    case "string":
        // Validate string
    }

    // Regex validation
    if arg.Validation != "" {
        strVal := fmt.Sprintf("%v", val)
        matched, err := regexp.MatchString(arg.Validation,
strVal)
        if err != nil {
            return nil, fmt.Errorf("invalid validation
pattern for '%s': %w", arg.Name, err)
        }
        if !matched {
            return nil, fmt.Errorf("argument '%s' does not
match pattern '%s'", arg.Name, arg.Validation)
        }
    }

    result[arg.Name] = val
}

return result, nil
}

func (r *DefaultCommandRegistry) Complete(ctx context.Context,
partial string) []CommandCompletion {

```

```

var completions []CommandCompletion

for name, cmd := range r.commands {
    if !cmd.IsEnabled {
        continue
    }
    if strings.HasPrefix(name, partial) {
        completions = append(completions, CommandCompletion{
            Name:      name,
            Description: cmd.Description,
            InsertText: "/" + name,
        })
    }
}

return completions
}

func (r *DefaultCommandRegistry) RegisterCommand(ctx
context.Context, cmd SlashCommand) error {
    r.commands[cmd.Name] = &cmd
    return nil
}

func (r *DefaultCommandRegistry) UnregisterCommand(ctx
context.Context, name string) error {
    delete(r.commands, name)
    return nil
}

func (r *DefaultCommandRegistry) GetCommand(ctx context.Context,
name string) (*SlashCommand, error) {
    cmd, ok := r.commands[name]
    if !ok {
        return nil, fmt.Errorf("command not found: %s", name)
    }
    return cmd, nil
}

func (r *DefaultCommandRegistry) ListCommands(ctx
context.Context, filter CommandFilter) ([]SlashCommand,
error) {
    var result []SlashCommand

    for _, cmd := range r.commands {
        if filter.Category != nil && cmd.Category !=
        *filter.Category {
            continue
        }
    }
}

```



```

    }
    if filter.BuiltIn != nil && cmd.IsBuiltIn !=
        *filter.BuiltIn {
        continue
    }
    if filter.Enabled != nil && cmd.IsEnabled !=
        *filter.Enabled {
        continue
    }
    if filter.Search != nil && !strings.Contains(cmd.Name,
        *filter.Search) && !strings.Contains(cmd.Description,
        *filter.Search) {
        continue
    }

    result = append(result, *cmd)
}

return result, nil
}

func (r *DefaultCommandRegistry) Reload(ctx context.Context)
    error {
    r.commands = make(map[string]*SlashCommand)
    return r.LoadCommands(ctx, r.commandDirs)
}

```

10.6 Testing Approach

```

func TestCommandRegistry_LoadCommandFile(t *testing.T) {
    registry := NewDefaultCommandRegistry(nil, logger.NewNop())

    // Create temp command file
    cmdFile := filepath.Join(t.TempDir(), "explain.
md")
    content := `---
name: explain
description: Explain selected code
model: claude-sonnet
args:
  - name: depth
    type: choice
    options: [brief, detailed]
---
Explain: {context.selection}
`

    os.WriteFile(cmdFile, []byte(content), 0644)
}

```

```

err := registry.loadCommandFile(ctx, cmdFile)
assert.NoError(t, err)

cmd, err := registry.GetCommand(ctx, "explain")
assert.NoError(t, err)
assert.Equal(t, "Explain selected code", cmd.Description)
assert.Equal(t, "Explain: {context.selection}", cmd.Prompt)
assert.Len(t, cmd.Args, 1)
}

func TestCommandRegistry_Execute(t *testing.T) {
    mockLLM := new(MockLLMClient)
    mockLLM.On("Complete", mock.Anything,
        mock.Anything).Return(&llm.CompletionResponse{
            Text: "This code prints hello world",
        }, nil)

    registry := NewDefaultCommandRegistry(mockLLM,
        logger.NewNop())
    registry.loadBuiltins(ctx)

    result, err := registry.Execute(ctx, CommandInvocation{
        Command: "explain",
        Args:     map[string]any{"depth": "brief"},
        Context: InvocationContext{
            Selection:    `fmt.Println("hello world")`,
            Language:     "go",
            CurrentFile: "main.go",
        },
    },
    })

    assert.NoError(t, err)
    assert.True(t, result.Success)
    assert.Contains(t, result.Response, "hello world")
}

func TestCommandRegistry_Complete(t *testing.T) {
    registry := NewDefaultCommandRegistry(nil, logger.NewNop())
    registry.loadBuiltins(ctx)

    completions := registry.Complete(ctx, "ex")
    assert.Len(t, completions, 1)
    assert.Equal(t, "explain", completions[0].Name)
}

```

10.7 Edge Cases

Edge Case	Handling
Invalid YAML frontmatter	Skip file, log warning, continue loading others
Missing required argument	Return error with usage hint
Circular reference in args	Not applicable for simple args
Command name collision	Built-in wins, log warning for custom overrides
Template has unreplaced placeholders	Leave as-is or warn
Command file deleted while loaded	Handle gracefully on reload
Very large command file (>1MB)	Skip, warn about size
User provides unknown args	Ignore extras, or warn
Model specified but unavailable	Fallback to default model
Prompt injection in args	Sanitize/escape before rendering

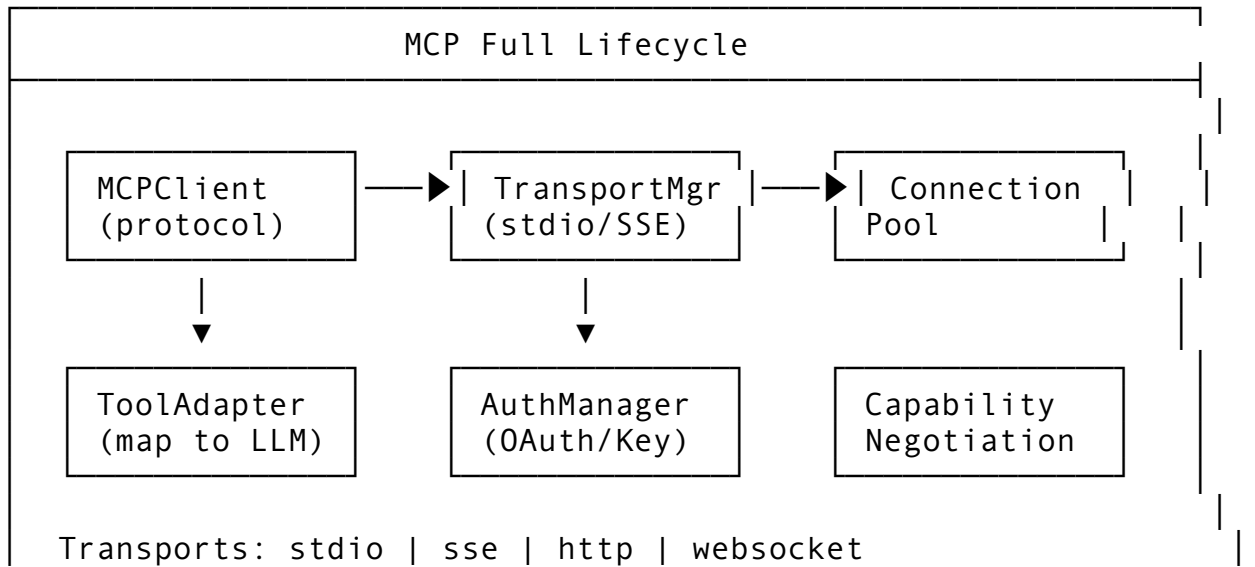
Feature 11: MCP Full Lifecycle (stdio, SSE, HTTP, WebSocket + OAuth)

11.1 Feature Description

The Model Context Protocol (MCP) provides a standardized way for LLMs to interact with external tools and data sources. Full lifecycle support includes stdio, Server-Sent Events (SSE), HTTP, and WebSocket transports with OAuth authentication.

Why it matters: MCP is becoming the standard for LLM tool integration. Supporting all transports ensures compatibility with the ecosystem.

11.2 Architecture Design



Auth: OAuth2 API Key Bearer Basic Capabilities: tools resources prompts sampling

11.3 API Design

```
// Package: internal/mcp
package mcp

import (
    "context"
    "fmt"
    "io"
    "time"
)

// TransportType defines MCP transport protocol
type TransportType string

const (
    TransportStdio      TransportType = "stdio"
    TransportSSE        TransportType = "sse"
    TransportHTTP        TransportType = "http"
    TransportWebSocket TransportType = "websocket"
)

// MCPMessage is the base protocol message
type MCPMessage struct {
    JSONRPC string          `json:"jsonrpc"`
    ID       any                    `json:"id,omitempty"`
    Method   string                `json:"method,omitempty"`
    Params   json.RawMessage       `json:"params,omitempty"`
    Result   json.RawMessage       `json:"result,omitempty"`
    Error    *MCPErrors            `json:"error,omitempty"`
}

type MCPErrors struct {
    Code      int      `json:"code"`
    Message   string   `json:"message"`
    Data      any      `json:"data,omitempty"`
}

// MCPServerConfig defines a server connection
type MCPServerConfig struct {
    Name          string          `json:"name" db:"name"`
    Transport     TransportType   `json:"transport" db:"transport"`
}
```

```

// stdio config
Command      string          `json:"command,omitempty"
           db:"command"`
Args         []string        `json:"args,omitempty"
           db:"args"`
Env          map[string]string `json:"env,omitempty" db:"env"`
WorkingDir   string          `json:"working_dir,omitempty"
           db:"working_dir"`

// HTTP/SSE/WebSocket config
URL          string          `json:"url,omitempty" db:"url"`
Headers      map[string]string `json:"headers,omitempty"
           db:"headers"`

// Auth
AuthType     string          `json:"auth_type,omitempty"
           db:"auth_type" // "oauth", "api_key", "bearer", "none"
AuthConfig   json.RawMessage `json:"auth_config,omitempty"
           db:"auth_config"`

// Connection
Timeout      time.Duration   `json:"timeout" db:"timeout"`
Reconnect    bool           `json:"reconnect"
           db:"reconnect"`
MaxRetries   int            `json:"max_retries"
           db:"max_retries"`

// State
IsConnected  bool           `json:"is_connected"
           db:"is_connected"`
IsEnabled    bool           `json:"is_enabled"
           db:"is_enabled"`

// Capabilities
Tools        []MCPTool      `json:"tools,omitempty" db:"- "`
Resources    []MCPResource  `json:"resources,omitempty"
           db:"- "`
Prompts      []MCPPrompt    `json:"prompts,omitempty"
           db:"- "`
}

// MCPTool is a tool exposed by MCP server
type MCPTool struct {
    Name          string          `json:"name"`
    Description   string          `json:"description"`
    InputSchema   json.RawMessage `json:"inputSchema"`
}

```

```

// MCPResource is a resource exposed by MCP server
type MCPResource struct {
    URI          string `json:"uri"`
    Name         string `json:"name"`
    MIMETYPE     string `json:"mimeType,omitempty"`
    Description   string `json:"description,omitempty"`
}

// MCPPrompt is a prompt template exposed by MCP server
type MCPPrompt struct {
    Name         string          `json:"name"`
    Description   string          `json:"description"`
    Arguments    []MCPPromptArg `json:"arguments,omitempty"`
}

type MCPPromptArg struct {
    Name         string `json:"name"`
    Description   string `json:"description,omitempty"`
    Required     bool   `json:"required,omitempty"`
}

// MCPManager manages MCP server connections
type MCPManager interface {
    // Connect establishes connection to an MCP server
    Connect(ctx context.Context, config MCPServerConfig) error

    // Disconnect closes a connection
    Disconnect(ctx context.Context, serverName string) error

    // CallTool invokes an MCP tool
    CallTool(ctx context.Context, serverName, toolName string,
        args map[string]any) (*ToolCallResult, error)

    // ReadResource reads an MCP resource
    ReadResource(ctx context.Context, serverName, uri string)
        (*ResourceContent, error)

    // GetPrompt retrieves a prompt template
    GetPrompt(ctx context.Context, serverName, promptName
        string, args map[string]any) (*PromptContent, error)

    // ListTools returns available tools from all servers
    ListTools(ctx context.Context) ([]MCPToolInfo, error)

    // ListResources returns available resources
    ListResources(ctx context.Context, serverName string)
        ([]MCPResource, error)
}

```

```

// RefreshCapabilities re-queries server capabilities
RefreshCapabilities(ctx context.Context, serverName string)
    error

// GetConnectionStatus returns connection health
GetConnectionStatus(ctx context.Context, serverName string)
    (*ConnectionStatus, error)

// ListServers returns configured servers
ListServers(ctx context.Context) ([]MCPServerConfig, error)

// RegisterServer adds a server configuration
RegisterServer(ctx context.Context, config MCPServerConfig)
    error

// UnregisterServer removes a server
UnregisterServer(ctx context.Context, serverName string)
    error
}

// ToolCallResult is the outcome of calling an MCP tool
type ToolCallResult struct {
    Content []ToolContent `json:"content"`
    IsError bool           `json:"isError,omitempty"`
}

type ToolContent struct {
    Type string `json:"type"` // "text", "image", "resource"
    Text string `json:"text,omitempty"`
    // Image and Resource fields omitted for brevity
}

// ResourceContent is the content of an MCP resource
type ResourceContent struct {
    URI string `json:"uri"`
    MimeType string `json:"mimeType,omitempty"`
    Text string `json:"text,omitempty"`
    Blob []byte `json:"blob,omitempty"`
}

// PromptContent is the resolved prompt
type PromptContent struct {
    Messages []PromptMessage `json:"messages"`
}

type PromptMessage struct {
    Role string `json:"role"`
    Content PromptMsgContent `json:"content"`
}

```

```

}

type PromptMsgContent struct {
    Type string `json:"type"`
    Text string `json:"text,omitempty"`
}

// ConnectionStatus reports connection health
type ConnectionStatus struct {
    ServerName    string    `json:"server_name"`
    IsConnected   bool      `json:"is_connected"`
    LastPing      time.Time `json:"last_ping"`
    LatencyMs     int64     `json:"latency_ms"`
    ErrorCount    int       `json:"error_count"`
    Capabilities  []string  `json:"capabilities"`
}

// MCPToolInfo combines tool and server info
type MCPToolInfo struct {
    ServerName string `json:"server_name"`
    MCPTool    // embedded
}

// OAuthConfig for OAuth authentication
type OAuthConfig struct {
    ClientID      string `json:"client_id"`
    ClientSecret  string `json:"client_secret,omitempty"`
    AuthURL       string `json:"auth_url"`
    TokenURL      string `json:"token_url"`
    Scopes        []string `json:"scopes,omitempty"`
    RedirectURI   string `json:"redirect_uri"`

    // Token storage
    AccessToken string `json:"access_token,omitempty"`
    RefreshToken string `json:"refresh_token,omitempty"`
    ExpiresAt   time.Time `json:"expires_at,omitempty"`
}

```

11.4 Integration Points

Package	Modification	Description
internal/mcp/	NEW PACKAGE	MCP manager and transports
internal/tool/	MCP tool adapter	Map MCP tools to HelixCode tool interface
internal/auth/	OAuth flow	Implement OAuth2 authorization code flow
	MCP server config	

Package	Modification	Description
internal/ config/		Viper integration for server definitions
cmd/helix/ main.go	--mcp-config flag	Path to MCP server config file
internal/db/ migrations/	Add mcp_servers table	Persist server configs

11.5 Implementation Steps

```
// internal/mcp/manager.go
package mcp

import (
    "bufio"
    "context"
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "os"
    "os/exec"
    "sync"
    "time"

    "dev.helix.code/pkg/logger"
    "golang.org/x/oauth2" // for OAuth
)

type DefaultMCPManager struct {
    servers      map[string]*mcpConnection
    mu           sync.RWMutex
    log          logger.Logger
    httpClient   *http.Client
}

type mcpConnection struct {
    config      MCPServerConfig
    transport   mcpTransport
    tools       []MCPTool
    resources   []MCPResource
    prompts     []MCPPrompt
    mu          sync.RWMutex
}

type mcpTransport interface {
    Send(ctx context.Context, msg MCPMessage) (*MCPMessage,
        error)
```

```

    Close() error
    IsConnected() bool
}

func NewDefaultMCPManager(log logger.Logger) *DefaultMCPManager {
    return &DefaultMCPManager{
        servers:    make(map[string]*mcpConnection),
        log:         log,
        httpClient: &http.Client{Timeout: 30 * time.Second},
    }
}

func (m *DefaultMCPManager) Connect(ctx context.Context, config
    MCPServerConfig) error {
    m.mu.Lock()
    defer m.mu.Unlock()

    if existing, ok := m.servers[config.Name]; ok &&
        existing.transport.IsConnected() {
        return fmt.Errorf("server %s already connected",
            config.Name)
    }

    // Create transport
    transport, err := m.createTransport(ctx, config)
    if err != nil {
        return fmt.Errorf("creating transport: %w", err)
    }

    // Initialize connection
    initResult, err := m.initialize(ctx, transport)
    if err != nil {
        transport.Close()
        return fmt.Errorf("initializing MCP: %w", err)
    }

    conn := &mcpConnection{
        config:    config,
        transport: transport,
    }

    // Parse capabilities
    if initResult != nil {
        // Extract tools, resources, prompts from result
        // ... capability parsing
    }

    m.servers[config.Name] = conn

```

```

        m.log.Info("MCP server connected", "name", config.Name,
            "transport", config.Transport)

        return nil
    }

    func (m *DefaultMCPManager) createTransport(ctx context.Context,
        config MCPServerConfig) (mcpTransport, error) {
        switch config.Transport {
        case TransportStdio:
            return newStdioTransport(ctx, config, m.log)
        case TransportSSE:
            return newSSETransport(ctx, config, m.httpClient, m.log)
        case TransportHTTP:
            return newHTTPTransport(ctx, config, m.httpClient, m.log)
        case TransportWebSocket:
            return newWebSocketTransport(ctx, config, m.log)
        default:
            return nil, fmt.Errorf("unsupported transport: %s",
                config.Transport)
        }
    }

    func (m *DefaultMCPManager) initialize(ctx context.Context,
        transport mcpTransport) (*MCPMessage, error) {
        initMsg := MCPMessage{
            JSONRPC: "2.0",
            ID:       1,
            Method:   "initialize",
            Params: mustJSON(map[string]any{
                "protocolVersion": "2024-11-05",
                "capabilities":      map[string]any{},
                "clientInfo": map[string]any{
                    "name": "helixcode",
                    "version": "1.0.0",
                },
            }),
        }

        result, err := transport.Send(ctx, initMsg)
        if err != nil {
            return nil, err
        }

        // Send initialized notification
        _ = transport.Send(ctx, MCPMessage{
            JSONRPC: "2.0",
            Method:   "notifications/initialized",
        })
    }

```

```

    })

    return result, nil
}

// Stdio transport implementation

type stdioTransport struct {
    cmd      *exec.Cmd
    stdin    io.WriteCloser
    stdout    *bufio.Reader
    stderr    io.ReadCloser
    log       logger.Logger
    mu        sync.Mutex
}

func newStdioTransport(ctx context.Context, config
    MCPServerConfig, log logger.Logger) (*stdioTransport,
    error) {
    cmd := exec.CommandContext(ctx, config.Command,
        config.Args...)
    cmd.Dir = config.WorkingDir
    cmd.Env = os.Environ()
    for k, v := range config.Env {
        cmd.Env = append(cmd.Env, fmt.Sprintf("%s=%s", k, v))
    }

    stdin, err := cmd.StdinPipe()
    if err != nil {
        return nil, err
    }

    stdout, err := cmd.StdoutPipe()
    if err != nil {
        return nil, err
    }

    stderr, err := cmd.StderrPipe()
    if err != nil {
        return nil, err
    }

    if err := cmd.Start(); err != nil {
        return nil, err
    }

    // Log stderr in background
    go func() {

```

```

        scanner := bufio.NewScanner(stderr)
        for scanner.Scan() {
            log.Debug("MCP stderr", "line", scanner.Text())
        }
    }()

    return &stdioTransport{
        cmd:      cmd,
        stdin:    stdin,
        stdout:   bufio.NewReader(stdout),
        stderr:    stderr,
        log:      log,
    }, nil
}

func (t *stdioTransport) Send(ctx context.Context, msg
    MCPMessage) (*MCPMessage, error) {
    t.mu.Lock()
    defer t.mu.Unlock()

    data, err := json.Marshal(msg)
    if err != nil {
        return nil, err
    }

    // Write with newline delimiter
    if _, err := t.stdin.Write(append(data, '\n')); err != nil {
        return nil, err
    }

    // Read response (for requests)
    if msg.ID != nil {
        line, err := t.stdout.ReadString('\n')
        if err != nil {
            return nil, err
        }

        var response MCPMessage
        if err := json.Unmarshal([]byte(line), &response); err !=
            nil {
            return nil, err
        }

        return &response, nil
    }

    return nil, nil // Notifications have no response
}

```

```

func (t *stdioTransport) Close() error {
    if t.stdin != nil {
        t.stdin.Close()
    }
    if t.cmd != nil && t.cmd.Process != nil {
        t.cmd.Process.Kill()
        t.cmd.Wait()
    }
    return nil
}

func (t *stdioTransport) IsConnected() bool {
    return t.cmd != nil && t.cmd.Process != nil &&
        t.cmd.ProcessState == nil
}

// CallTool implementation
func (m *DefaultMCPManager) CallTool(ctx context.Context,
    serverName, toolName string, args map[string]any)
    (*ToolCallResult, error) {
    m.mu.RLock()
    conn, ok := m.servers[serverName]
    m.mu.RUnlock()

    if !ok {
        return nil, fmt.Errorf("server not found: %s",
            serverName)
    }

    msg := MCPMessage{
        JSONRPC: "2.0",
        ID:      generateRequestID(),
        Method:  "tools/call",
        Params: mustJSON(map[string]any{
            "name": toolName,
            "arguments": args,
        })),
    }

    result, err := conn.transport.Send(ctx, msg)
    if err != nil {
        return nil, err
    }

    if result.Error != nil {
        return nil, fmt.Errorf("MCP error %d: %s",
            result.Error.Code, result.Error.Message)
    }
}

```

```

    }

    var toolResult ToolCallResult
    if err := json.Unmarshal(result.Result, &toolResult); err !=
        nil {
        return nil, fmt.Errorf("parsing tool result: %w", err)
    }

    return &toolResult, nil
}

// OAuth implementation
func (m *DefaultMCPManager) setupOAuth(ctx context.Context,
    config *MCPServerConfig) error {
    if config.AuthType != "oauth" {
        return nil
    }

    var oauthConfig OAuthConfig
    if err := json.Unmarshal(config.AuthConfig, &oauthConfig);
        err != nil {
        return err
    }

    // Check if we have a valid token
    if oauthConfig.AccessToken != "" &&
        oauthConfig.ExpiresAt.After(time.Now()) {
        // Token still valid
        config.Headers["Authorization"] = "Bearer " +
            oauthConfig.AccessToken
        return nil
    }

    // Need to refresh or authorize
    if oauthConfig.RefreshToken != "" {
        // Implement refresh token flow
        // ...
    }

    // Start OAuth authorization code flow
    // This requires opening a browser or providing a URL to the
    // user
    // ...

    return fmt.Errorf("OAuth authorization required")
}

func mustJSON(v any) json.RawMessage {

```

```

    data, _ := json.Marshal(v)
    return data
}

func generateRequestID() int {
    // Thread-safe counter or random
    return int(time.Now().UnixNano())
}

// SSE and WebSocket transports follow similar patterns
// HTTP transport uses standard REST calls

```

11.6 Testing Approach

```

func TestMCPManager_StdioTransport(t *testing.T) {
    // Create a mock MCP server as a simple echo script
    scriptPath := filepath.Join(t.TempDir(), "mcp-server")
    script := `#!/bin/bash
while read line; do
    echo '{"jsonrpc":"2.0","id":1,"result":
        {"protocolVersion":"2024-11-05","capabilities":
        {},"serverInfo":{"name":"test","version":"1.0"}}}'
done
`
    os.WriteFile(scriptPath, []byte(script), 0755)

    manager := NewDefaultMCPManager(logger.NewNop())
    err := manager.Connect(ctx, MCPServerConfig{
        Name:      "test-server",
        Transport: TransportStdio,
        Command:   scriptPath,
    })

    assert.NoError(t, err)

    status, err := manager.GetConnectionStatus(ctx, "test-
        server")
    assert.NoError(t, err)
    assert.True(t, status.IsConnected)
}

func TestMCPManager_CallTool(t *testing.T) {
    manager, mockTransport := setupTestManager()

    mockTransport.On("Send", mock.Anything,
        mock.Anything).Return(&MCPMessage{
        Result: mustJSON(ToolCallResult{

```



```

        Content: []ToolContent{{Type: "text", Text:
            "result"}}},
    }},
}, nil)

result, err := manager.CallTool(ctx, "test-server",
    "test_tool", map[string]any{"arg": "value"})

assert.NoError(t, err)
assert.False(t, result.IsError)
assert.Len(t, result.Content, 1)
}

```

11.7 Edge Cases

Edge Case	Handling
Server process crashes	Auto-restart with backoff, notify user
OAuth token expires mid-session	Refresh token, re-request if refresh fails
SSE connection drops	Reconnect with Last-Event-ID header
WebSocket ping timeout	Send ping frames, close on pong timeout
Duplicate request IDs	Use UUIDs, track pending requests
Server returns unexpected format	Graceful error, log raw response
Circular tool dependency between servers	Timeout at 30s, return error
Stdio server outputs non-JSON to stdout	Use JSON-RPC line delimiter, skip non-JSON
HTTP rate limiting	Respect Retry-After header, exponential backoff
Capability negotiation mismatch	Log unsupported features, proceed with subset

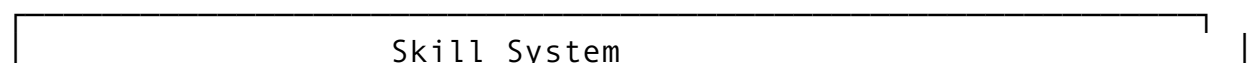
Feature 12: Skill System (Auto-Invocation + Variable Substitution)

12.1 Feature Description

The Skill System packages reusable agent capabilities into declarative units that can auto-invoke based on context detection. Skills support variable substitution for parameterized behavior.

Why it matters: Teams need to encode domain knowledge (e.g., “how we handle security reviews”, “our coding standards”) that the agent can apply automatically.

12.2 Architecture Design



Skill File (.skill.md):

name: security-review

auto_invoke:

patterns: ["auth", "password", "crypto"]

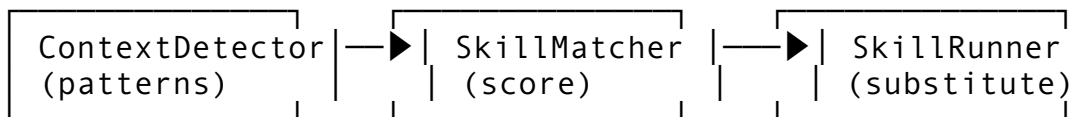
file_types: ["*.go", "*.js"]

variables:

- name: severity_threshold

default: medium

Review for security issues with {severity_threshold}...



12.3 API Design

```
// Package: internal/skills
```

```
package skills
```

```
import (
```

```
    "context"
```

```
    "fmt"
```

```
    "path/filepath"
```

```
    "strings"
```

```
    "time"
```

```
)
```

```
// SkillConfig defines a reusable agent capability
```

```
type SkillConfig struct {
```

```
    // YAML frontmatter
```

```
    Name      string          `yaml:"name" json:"name"`
```

```
    Description string          `yaml:"description"
```

```
        json:"description"`
```

```
    Version   string          `yaml:"version,omitempty"
```

```
        json:"version,omitempty"`
```

```
    Category  string          `yaml:"category,omitempty"
```

```
        json:"category,omitempty"`
```

```
    Tags      []string         `yaml:"tags,omitempty"
```

```
        json:"tags,omitempty"`
```

```
    // Auto-invocation triggers
```

```
    AutoInvoke *AutoInvokeConfig `yaml:"auto_invoke,omitempty"
```

```
        json:"auto_invoke,omitempty"`
```

```

// Variables
Variables    []SkillVariable `yaml:"variables,omitempty"
             json:"variables,omitempty"`

// Execution
Model        string          `yaml:"model,omitempty"
             json:"model,omitempty"`
Temperature  *float64         `yaml:"temperature,omitempty"
             json:"temperature,omitempty"`

// The skill prompt template
Prompt       string          `yaml:"- " json:"- "`

// Metadata
SourcePath   string          `yaml:"- " json:"source_path"`
IsBuiltIn    bool            `yaml:"- " json:"is_built_in"`
IsEnabled    bool            `yaml:"- " json:"is_enabled"`
}

// AutoInvokeConfig defines when a skill triggers automatically
type AutoInvokeConfig struct {
    Patterns    []string `yaml:"patterns,omitempty"
              json:"patterns,omitempty"` // Regex patterns in
              context
    FileTypes    []string `yaml:"file_types,omitempty"
              json:"file_types,omitempty"` // File glob patterns
    Keywords     []string `yaml:"keywords,omitempty"
              json:"keywords,omitempty"` // Literal keywords
    MinConfidence float64 `yaml:"min_confidence,omitempty"
              json:"min_confidence,omitempty"` // 0.0-1.0
    MaxInvocationsPerSession int
              `yaml:"max_invocations_per_session,omitempty"
              json:"max_invocations_per_session,omitempty"`
}

// SkillVariable is a substitutable parameter
type SkillVariable struct {
    Name        string `yaml:"name" json:"name"`
    Description string `yaml:"description,omitempty"
              json:"description,omitempty"`
    Type        string `yaml:"type,omitempty"
              json:"type,omitempty"` // "string", "number", "boolean",
              "choice", "file", "directory"
    Required    bool   `yaml:"required,omitempty"
              json:"required,omitempty"`
    Default     any    `yaml:"default,omitempty"
              json:"default,omitempty"`
}

```

```

Options      []string `yaml:"options,omitempty"
             json:"options,omitempty"` // For choice type
Validation   string   `yaml:"validation,omitempty"
             json:"validation,omitempty"` // Regex
}

// SkillInvocation represents an activated skill
type SkillInvocation struct {
    SkillName    string           `json:"skill_name"`
    Variables    map[string]any   `json:"variables"`
    Context      InvocationContext `json:"context"`
    TriggerType  string           `json:"trigger_type"` //
    "manual", "auto"
    Confidence   float64          `json:"confidence,omitempty"`
}

// SkillResult is the outcome of skill execution
type SkillResult struct {
    Success      bool           `json:"success"`
    Response     string         `json:"response,omitempty"`
    Error        string         `json:"error,omitempty"`
    Actions      []SuggestedAction `json:"actions,omitempty"`
}

// SkillRegistry manages skills
type SkillRegistry interface {
    // LoadSkills discovers skill definitions
    LoadSkills(ctx context.Context, dirs []string) error

    // RegisterSkill adds a skill
    RegisterSkill(ctx context.Context, skill SkillConfig) error

    // GetSkill retrieves a skill
    GetSkill(ctx context.Context, name string) (*SkillConfig,
        error)

    // ListSkills returns available skills
    ListSkills(ctx context.Context, filter SkillFilter)
        ([]SkillConfig, error)

    // Execute runs a skill
    Execute(ctx context.Context, invocation SkillInvocation)
        (*SkillResult, error)

    // DetectSkills finds applicable skills for context
    DetectSkills(ctx context.Context, context string, filePath
        string) ([]SkillMatch, error)

```

```

    // EnableSkill / DisableSkill toggle
    EnableSkill(ctx context.Context, name string) error
    DisableSkill(ctx context.Context, name string) error
}

// SkillMatch is a detected applicable skill
type SkillMatch struct {
    Skill      SkillConfig `json:"skill"`
    Confidence float64      `json:"confidence"`
    Trigger    string      `json:"trigger"` // Which pattern
                triggered
}

// SkillFilter filters skill listings
type SkillFilter struct {
    Category *string `json:"category,omitempty"`
    Tags     []string `json:"tags,omitempty"`
    BuiltIn  *bool   `json:"built_in,omitempty"`
    Enabled  *bool   `json:"enabled,omitempty"`
    Search   *string `json:"search,omitempty"`
}

```

12.4 Integration Points

Package	Modification	Description
internal/ skills/	NEW PACKAGE	Skill registry and executor
internal/llm/ prompts.go	Skill prompt integration	Inject detected skills into system prompt
internal/ actor/ session.go	Auto-invocation	Detect and invoke skills automatically
cmd/helix/ main.go	--skills-dir flag	Load custom skills

12.5 Implementation Steps

```

// internal/skills/registry.go
package skills

import (
    "context"
    "fmt"
    "os"
    "path/filepath"
    "regexp"
    "strings"

```

```

    "time"

    "dev.helix.code/internal/llm"
    "dev.helix.code/pkg/logger"
    "github.com/goccy/go-yaml"
)

type DefaultSkillRegistry struct {
    skills    map[string]*SkillConfig
    llmClient llm.Client
    log       logger.Logger
}

func NewDefaultSkillRegistry(llmClient llm.Client, log
    logger.Logger) *DefaultSkillRegistry {
    return &DefaultSkillRegistry{
        skills:    make(map[string]*SkillConfig),
        llmClient: llmClient,
        log:       log,
    }
}

func (r *DefaultSkillRegistry) DetectSkills(ctx context.Context,
    context string, filePath string) ([]SkillMatch, error) {
    var matches []SkillMatch

    for _, skill := range r.skills {
        if !skill.IsEnabled || skill.AutoInvoke == nil {
            continue
        }

        confidence, trigger :=
            r.calculateConfidence(skill.AutoInvoke, context,
                filePath)
        if confidence >= skill.AutoInvoke.MinConfidence {
            matches = append(matches, SkillMatch{
                Skill:      *skill,
                Confidence: confidence,
                Trigger:     trigger,
            })
        }
    }

    // Sort by confidence descending
    sort.Slice(matches, func(i, j int) bool {
        return matches[i].Confidence > matches[j].Confidence
    })
}

```

```

    return matches, nil
}

func (r *DefaultSkillRegistry) calculateConfidence(config
    *AutoInvokeConfig, context string, filePath string)
    (float64, string) {
var scores []float64
var triggers []string

// File type matching
if len(config.FileTypes) > 0 && filePath != "" {
    matched := false
    for _, pattern := range config.FileTypes {
        if matched, _ := filepath.Match(pattern,
            filepath.Base(filePath)); matched {
            scores = append(scores, 0.3)
            triggers = append(triggers, "file_type")
            break
        }
    }
}

// Pattern matching (regex)
for _, pattern := range config.Patterns {
    re, err := regexp.Compile(pattern)
    if err != nil {
        continue
    }
    if re.MatchString(context) {
        scores = append(scores, 0.5)
        triggers = append(triggers, fmt.Sprintf("pattern:
%s", pattern))
    }
}

// Keyword matching
contextLower := strings.ToLower(context)
for _, keyword := range config.Keywords {
    if strings.Contains(contextLower,
        strings.ToLower(keyword)) {
        scores = append(scores, 0.2)
        triggers = append(triggers, fmt.Sprintf("keyword:
%s", keyword))
    }
}

if len(scores) == 0 {
    return 0, ""
}

```

```

    }

    // Combine scores (sum with diminishing returns)
    total := 0.0
    for _, s := range scores {
        total += s * (1 - total*0.5)
    }

    return min(total, 1.0), strings.Join(triggers, ", ")
}

func (r *DefaultSkillRegistry) Execute(ctx context.Context,
    invocation SkillInvocation) (*SkillResult, error) {
    skill, ok := r.skills[invocation.SkillName]
    if !ok {
        return &SkillResult{Success: false, Error:
            fmt.Sprintf("skill not found: %s",
                invocation.SkillName)}, nil
    }

    // Merge variables with defaults
    variables := make(map[string]any)
    for _, v := range skill.Variables {
        if val, ok := invocation.Variables[v.Name]; ok {
            variables[v.Name] = val
        } else if v.Default != nil {
            variables[v.Name] = v.Default
        } else if v.Required {
            return &SkillResult{Success: false, Error:
                fmt.Sprintf("required variable '%s' not provided",
                    v.Name)}, nil
        }
    }

    // Render prompt
    prompt := skill.Prompt
    for k, v := range variables {
        placeholder := fmt.Sprintf("{%s}", k)
        prompt = strings.ReplaceAll(prompt, placeholder,
            fmt.Sprintf("%v", v))
    }

    // Add context
    prompt = strings.ReplaceAll(prompt, "{context.selection}",
        invocation.Context.Selection)
    prompt = strings.ReplaceAll(prompt, "{context.file}",
        invocation.Context.CurrentFile)

```



```

prompt = strings.ReplaceAll(prompt, "{context.language}",
    invocation.Context.Language)

// Call LLM
req := llm.CompletionRequest{
    Model: skill.Model,
    Prompt: prompt,
}
if skill.Temperature != nil {
    req.Temperature = *skill.Temperature
}
if skill.Model == "" {
    req.Model = "claude-sonnet-4"
}

response, err := r.llmClient.Complete(ctx, req)
if err != nil {
    return nil, err
}

return &SkillResult{
    Success: true,
    Response: response.Text,
}, nil
}

func (r *DefaultSkillRegistry) LoadSkills(ctx context.Context,
    dirs []string) error {
    for _, dir := range dirs {
        entries, err := os.ReadDir(dir)
        if err != nil {
            continue
        }

        for _, entry := range entries {
            if entry.IsDir() || !strings.HasSuffix(entry.Name(),
                ".skill.md") {
                continue
            }

            path := filepath.Join(dir, entry.Name())
            data, err := os.ReadFile(path)
            if err != nil {
                continue
            }

            skill, err := r.parseSkillFile(string(data), path)
            if err != nil {

```

```

        r.log.Warn("failed to parse skill", "path",
path, "error", err)
        continue
    }

    r.skills[skill.Name] = skill
    r.log.Info("loaded skill", "name", skill.Name,
"path", path)
}
}

return nil
}

func (r *DefaultSkillRegistry) parseSkillFile(content, path
string) (*SkillConfig, error) {
    if !strings.HasPrefix(content, "---") {
        return nil, fmt.Errorf("no YAML frontmatter")
    }

    endIdx := strings.Index(content[3:], "---")
    if endIdx == -1 {
        return nil, fmt.Errorf("unterminated frontmatter")
    }

    frontmatter := content[3 : endIdx+3]
    prompt := strings.TrimSpace(content[endIdx+6:])

    var skill SkillConfig
    if err := yaml.Unmarshal([]byte(frontmatter), &skill); err != nil {
        return nil, err
    }

    skill.Prompt = prompt
    skill.SourcePath = path
    skill.IsBuiltIn = false
    skill.IsEnabled = true

    return &skill, nil
}

func (r *DefaultSkillRegistry) RegisterSkill(ctx
context.Context, skill SkillConfig) error {
    r.skills[skill.Name] = &skill
    return nil
}

```

```

func (r *DefaultSkillRegistry) GetSkill(ctx context.Context,
    name string) (*SkillConfig, error) {
    skill, ok := r.skills[name]
    if !ok {
        return nil, fmt.Errorf("skill not found: %s", name)
    }
    return skill, nil
}

func (r *DefaultSkillRegistry) ListSkills(ctx context.Context,
    filter SkillFilter) ([]SkillConfig, error) {
    var result []SkillConfig
    for _, skill := range r.skills {
        if filter.Category != nil && skill.Category !=
            *filter.Category {
            continue
        }
        if filter.BuiltIn != nil && skill.IsBuiltIn !=
            *filter.BuiltIn {
            continue
        }
        if filter.Enabled != nil && skill.IsEnabled !=
            *filter.Enabled {
            continue
        }
        if filter.Search != nil && !strings.Contains(skill.Name,
            *filter.Search) {
            continue
        }
        result = append(result, *skill)
    }
    return result, nil
}

func (r *DefaultSkillRegistry) EnableSkill(ctx context.Context,
    name string) error {
    skill, ok := r.skills[name]
    if !ok {
        return fmt.Errorf("skill not found: %s", name)
    }
    skill.IsEnabled = true
    return nil
}

func (r *DefaultSkillRegistry) DisableSkill(ctx context.Context,
    name string) error {
    skill, ok := r.skills[name]
    if !ok {

```

```

        return fmt.Errorf("skill not found: %s", name)
    }
    skill.IsEnabled = false
    return nil
}

```

12.6 Testing Approach

```

func TestSkillRegistry_DetectSkills(t *testing.T) {
    registry := NewDefaultSkillRegistry(nil, logger.NewNop())

    registry.RegisterSkill(ctx, SkillConfig{
        Name:          "security-review",
        Description:    "Security code review",
        AutoInvoke:    &AutoInvokeConfig{
            Patterns:      []string{`(?:i)password|auth|token`},
            FileTypes:    []string{ "*.go" },
            Keywords:      []string{ "security" },
            MinConfidence: 0.3,
        },
        IsEnabled: true,
    })

    matches, err := registry.DetectSkills(ctx, "Implement JWT
        authentication with password hashing", "auth.go")

    assert.NoError(t, err)
    assert.Len(t, matches, 1)
    assert.Greater(t, matches[0].Confidence, 0.3)
    assert.Contains(t, matches[0].Trigger, "pattern")
}

func TestSkillRegistry_VariableSubstitution(t *testing.T) {
    mockLLM := new(MockLLMClient)
    mockLLM.On("Complete", mock.Anything,
        mock.Anything).Return(&llm.CompletionResponse{
        Text: "Reviewed with medium threshold",
    }, nil)

    registry := NewDefaultSkillRegistry(mockLLM, logger.NewNop())

    registry.RegisterSkill(ctx, SkillConfig{
        Name:          "review",
        Prompt:        "Review with {severity} threshold for
            {context.file}",
        Variables: []SkillVariable{
            {Name: "severity", Type: "string", Default:
                "medium"},
        },
    })
}

```

```

    },
    IsEnabled: true,
  })

  result, err := registry.Execute(ctx, SkillInvocation{
    SkillName: "review",
    Variables: map[string]any{"severity": "high"},
    Context:   InvocationContext{CurrentFile: "main.go"},
  })

  assert.NoError(t, err)
  assert.True(t, result.Success)

  // Verify LLM was called with substituted prompt
  call := mockLLM.Calls[0]
  assert.Contains(t, call.Arguments.Get(1).
    (llm.CompletionRequest).Prompt, "high threshold")
  assert.Contains(t, call.Arguments.Get(1).
    (llm.CompletionRequest).Prompt, "main.go")
}

```

12.7 Edge Cases

Edge Case	Handling
Multiple skills match with high confidence	Show picker, allow multi-select
Circular variable references	Error during validation, reject skill
Variable type mismatch	Coerce or error with expected type
Skill prompts cause context overflow	Summarize context, prioritize skill-relevant parts
Auto-invocation loop	Track invocation count per session, max 3 per skill
Skill file syntax error	Skip, log, load remaining skills
Missing default for optional variable	Leave placeholder as-is or empty string
Skill conflicts with slash command	Prefix skill with ! vs command with /
Very long skill prompt	Chunk, add summary header
Skill requires unavailable model	Fallback to default, log warning

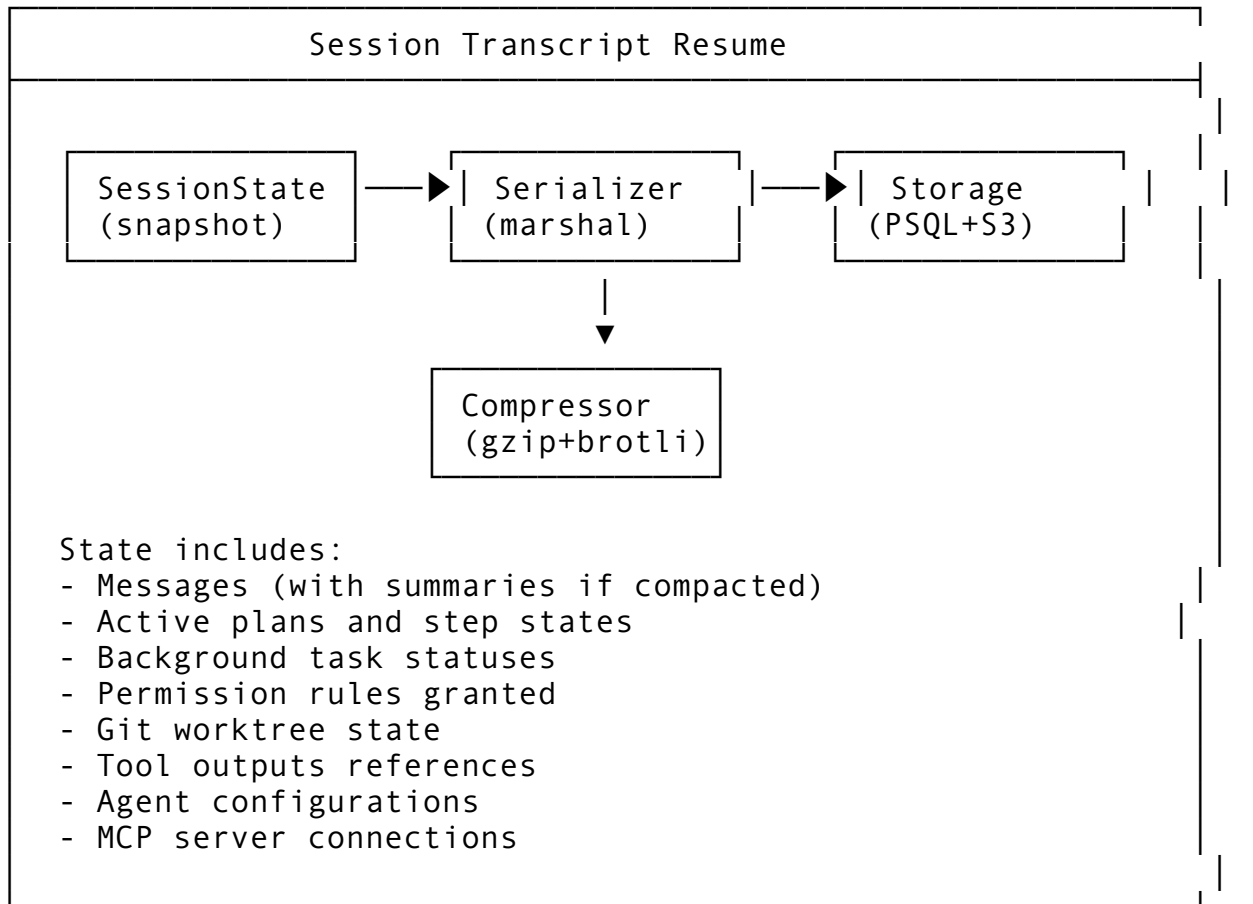
Feature 13: Session Transcript Resume

13.1 Feature Description

Session Transcript Resume allows sessions to be saved, exported, and later resumed from their exact state. This includes conversation history, active plans, file modifications, tool outputs, and agent context.

Why it matters: Work sessions shouldn't be lost on disconnect. Resumable sessions enable async workflows and multi-device continuity.

13.2 Architecture Design



13.3 API Design

```
// Package: internal/session
package session

import (
    "context"
    "time"
)

// SessionSnapshot is a serializable session state
type SessionSnapshot struct {
    // Identity
    ID          string    `json:"id"`
    Name        string    `json:"name"`
    CreatedAt   time.Time `json:"created_at"`
    ResumedFrom *string   `json:"resumed_from,omitempty"`

    // Conversation
```

```

Messages      []MessageSnapshot `json:"messages"`
MessageCount  int                `json:"message_count"`

// Context
Workspace     string            `json:"workspace"`
Branch        string            `json:"branch,omitempty"`
GitCommit     string            `json:"git_commit,omitempty"`

// Active plans
Plans         []PlanSnapshot   `json:"plans,omitempty"`

// Background tasks
Tasks         []TaskSnapshot    `json:"tasks,omitempty"`

// State
Permissions   []PermissionSnapshot `json:"permissions,omitempty"`
Compaction    []CompactionSnapshot `json:"compaction,omitempty"`

// Configuration
Config        SessionConfig    `json:"config"`

// Metadata
ExportedAt    time.Time                `json:"exported_at"`
Version       string                `json:"version"` // Snapshot
              format version
Checksum      string                `json:"checksum"`
}

// MessageSnapshot is a stored message
type MessageSnapshot struct {
    ID          string    `json:"id"`
    Role        string    `json:"role"`
    Content     string    `json:"content"`
    Timestamp   time.Time `json:"timestamp"`
    IsSummary   bool      `json:"is_summary,omitempty"` // If from
                          compaction
    Metadata    map[string]any `json:"metadata,omitempty"`
}

// PlanSnapshot captures plan state
type PlanSnapshot struct {
    ID          string    `json:"id"`
    Title       string    `json:"title"`
    Status      string    `json:"status"`
    Steps       []StepSnapshot `json:"steps"`
    UserNotes   string    `json:"user_notes,omitempty"`
}

```

```

        Approved    bool           `json:"approved"`
    }

    // StepSnapshot captures step state
    type StepSnapshot struct {
        ID            string `json:"id"`
        Order         int    `json:"order"`
        Status        string `json:"status"`
        Result        *StepResultSnapshot `json:"result,omitempty"`
    }

    // StepResultSnapshot is serializable step result
    type StepResultSnapshot struct {
        Success bool    `json:"success"`
        Output  string `json:"output,omitempty"`
        Error   string `json:"error,omitempty"`
    }

    // TaskSnapshot captures background task state
    type TaskSnapshot struct {
        ID            string `json:"id"`
        Name          string `json:"name"`
        Status        string `json:"status"`
        Output        string `json:"output,omitempty"`
    }

    // PermissionSnapshot captures granted permissions
    type PermissionSnapshot struct {
        RuleID        string `json:"rule_id"`
        Category      string `json:"category"`
        ToolName      string `json:"tool_name"`
        Mode          string `json:"mode"`
        Scope         string `json:"scope,omitempty"`
    }

    // CompactionSnapshot captures compaction history
    type CompactionSnapshot struct {
        SummaryID      string `json:"summary_id"`
        OriginalRange  [2]int `json:"original_range"`
        Summary        string `json:"summary"`
        CompactionLevel int    `json:"compaction_level"`
    }

    // SessionConfig is session-specific configuration
    type SessionConfig struct {
        Model          string `json:"model,omitempty"`
        Temperature    float64 `json:"temperature,omitempty"`
        RenderMode     string `json:"render_mode,omitempty"`
    }

```



```

WorktreeEnabled    bool        `json:"worktree_enabled,omitempty"`
PlanMode           bool        `json:"plan_mode,omitempty"`
AutoCompact        bool        `json:"auto_compact,omitempty"`
CustomSettings     map[string]any
                    `json:"custom_settings,omitempty"`
}

// SessionManager handles session lifecycle
type SessionManager interface {
    // CreateSession initializes a new session
    CreateSession(ctx context.Context, workspace string, config
        SessionConfig) (*Session, error)

    // ExportSession serializes session to snapshot
    ExportSession(ctx context.Context, sessionID string)
        (*SessionSnapshot, error)

    // ImportSession restores session from snapshot
    ImportSession(ctx context.Context, snapshot
        *SessionSnapshot) (*Session, error)

    // ResumeSession resumes a previously exported session
    ResumeSession(ctx context.Context, sessionID string)
        (*Session, error)

    // SaveCheckpoint creates an intermediate save point
    SaveCheckpoint(ctx context.Context, sessionID, name string)
        error

    // ListCheckpoints returns available checkpoints
    ListCheckpoints(ctx context.Context, sessionID string)
        ([]CheckpointInfo, error)

    // RestoreCheckpoint restores session to checkpoint
    RestoreCheckpoint(ctx context.Context, sessionID,
        checkpointID string) error

    // DeleteSession removes session and all data
    DeleteSession(ctx context.Context, sessionID string) error

    // ListSessions returns sessions
    ListSessions(ctx context.Context, filter SessionFilter)
        ([]SessionInfo, error)

    // ExportToFile writes snapshot to file
    ExportToFile(ctx context.Context, sessionID, path string)
        error

```

```

    // ImportFromFile reads snapshot from file
    ImportFromFile(ctx context.Context, path string)
        (*SessionSnapshot, error)
}

// Session represents an active session
type Session struct {
    ID          string    `json:"id"`
    Name        string    `json:"name"`
    Workspace   string    `json:"workspace"`
    Status       string    `json:"status" // "active", "paused",
                "resumed", "closed"
    CreatedAt   time.Time `json:"created_at"`
    UpdatedAt   time.Time `json:"updated_at"`
    Config       SessionConfig `json:"config"`
}

// SessionInfo is metadata about a session
type SessionInfo struct {
    ID          string    `json:"id"`
    Name        string    `json:"name"`
    Workspace   string    `json:"workspace"`
    Status       string    `json:"status"`
    CreatedAt    time.Time `json:"created_at"`
    LastActivity time.Time `json:"last_activity"`
    MessageCount int        `json:"message_count"`
    CanResume    bool      `json:"can_resume"`
}

// CheckpointInfo is metadata about a checkpoint
type CheckpointInfo struct {
    ID          string    `json:"id"`
    Name        string    `json:"name"`
    CreatedAt   time.Time `json:"created_at"`
    MessageCount int        `json:"message_count"`
}

// SessionFilter filters session listings
type SessionFilter struct {
    Workspace *string `json:"workspace,omitempty"`
    Status    *string `json:"status,omitempty"`
    Since     *time.Time `json:"since,omitempty"`
    Search    *string `json:"search,omitempty"`
}

```

13.4 Integration Points

Package	Modification	Description
internal/session/	NEW PACKAGE	Session manager
internal/db/migrations/	Add sessions, messages, checkpoints tables	Persistent storage
internal/memory/	Session memory	Store and retrieve session state
internal/actor/	Session lifecycle	Create/resume/cleanup
cmd/helix/main.go	--resume flag	Resume previous session
internal/ui/	Session browser UI	List and select sessions

13.5 Implementation Steps

```
// internal/session/manager.go
package session

import (
    "compress/gzip"
    "context"
    "crypto/sha256"
    "encoding/json"
    "fmt"
    "io"
    "os"
    "time"

    "dev.helix.code/internal/db"
    "dev.helix.code/pkg/logger"
)

type DefaultSessionManager struct {
    store db.Store
    log    logger.Logger
}

func NewDefaultSessionManager(store db.Store, log logger.Logger)
    *DefaultSessionManager {
    return &DefaultSessionManager{store: store, log: log}
}

func (m *DefaultSessionManager) CreateSession(ctx
    context.Context, workspace string, config SessionConfig)
    (*Session, error) {
```

```

session := &Session{
    ID:          generateSessionID(),
    Name:        fmt.Sprintf("Session %s",
        time.Now().Format("2006-01-02 15:04")),
    Workspace:   workspace,
    Status:      "active",
    CreatedAt:   time.Now(),
    UpdatedAt:   time.Now(),
    Config:      config,
}

if err := m.store.CreateSession(ctx, session); err != nil {
    return nil, fmt.Errorf("creating session: %w", err)
}

return session, nil
}

func (m *DefaultSessionManager) ExportSession(ctx
    context.Context, sessionID string) (*SessionSnapshot,
    error) {
    // Gather all session data
    session, err := m.store.GetSession(ctx, sessionID)
    if err != nil {
        return nil, err
    }

    messages, err := m.store.GetSessionMessages(ctx, sessionID)
    if err != nil {
        return nil, err
    }

    plans, err := m.store.GetSessionPlans(ctx, sessionID)
    if err != nil {
        return nil, err
    }

    tasks, err := m.store.GetSessionTasks(ctx, sessionID)
    if err != nil {
        return nil, err
    }

    permissions, err := m.store.GetSessionPermissions(ctx,
        sessionID)
    if err != nil {
        return nil, err
    }
}

```

```

// Build snapshot
snapshot := &SessionSnapshot{
    ID:          sessionID,
    Name:        session.Name,
    CreatedAt:   session.CreatedAt,
    Workspace:   session.Workspace,
    MessageCount: len(messages),
    ExportedAt:  time.Now(),
    Version:     "1.0",
}

// Convert messages
for _, msg := range messages {
    snapshot.Messages = append(snapshot.Messages,
        MessageSnapshot{
            ID:          msg.ID,
            Role:        msg.Role,
            Content:     msg.Content,
            Timestamp:   msg.CreatedAt,
        })
}

// Convert plans
for _, plan := range plans {
    ps := PlanSnapshot{
        ID:          plan.ID,
        Title:       plan.Title,
        Status:      string(plan.Status),
        UserNotes:   plan.UserNotes,
        Approved:   plan.Status == "approved",
    }

    steps, _ := m.store.GetPlanSteps(ctx, plan.ID)
    for _, step := range steps {
        ss := StepSnapshot{
            ID:          step.ID,
            Order:       step.Order,
            Status:      string(step.Status),
        }
        if step.Result != nil {
            ss.Result = &StepResultSnapshot{
                Success: step.Result.Success,
                Output:  step.Result.Output,
                Error:   step.Result.Error,
            }
        }
        ps.Steps = append(ps.Steps, ss)
    }
}

```

```

        snapshot.Plans = append(snapshot.Plans, ps)
    }

    // Convert tasks
    for _, task := range tasks {
        snapshot.Tasks = append(snapshot.Tasks, TaskSnapshot{
            ID:      task.ID,
            Name:    task.Name,
            Status:  string(task.Status),
            Output:  derefString(task.Output),
        })
    }

    // Convert permissions
    for _, perm := range permissions {
        snapshot.Permissions = append(snapshot.Permissions,
            PermissionSnapshot{
                RuleID:    perm.ID,
                Category:  string(perm.Category),
                ToolName:  perm.ToolName,
                Mode:      perm.Mode.String(),
            })
    }

    // Compute checksum
    data, _ := json.Marshal(snapshot)
    h := sha256.Sum256(data)
    snapshot.Checksum = fmt.Sprintf("%x", h)

    return snapshot, nil
}

func (m *DefaultSessionManager) ImportSession(ctx
    context.Context, snapshot *SessionSnapshot) (*Session,
    error) {
    // Verify checksum
    originalChecksum := snapshot.Checksum
    snapshot.Checksum = ""
    data, _ := json.Marshal(snapshot)
    h := sha256.Sum256(data)
    computedChecksum := fmt.Sprintf("%x", h)
    if computedChecksum != originalChecksum {
        return nil, fmt.Errorf("checksum mismatch: snapshot may
            be corrupted")
    }

    // Create new session

```

```

session := &Session{
    ID:          generateSessionID(),
    Name:        snapshot.Name,
    Workspace:   snapshot.Workspace,
    Status:      "resumed",
    CreatedAt:   snapshot.CreatedAt,
    UpdatedAt:   time.Now(),
    Config:      snapshot.Config,
}

if err := m.store.CreateSession(ctx, session); err != nil {
    return nil, fmt.Errorf("creating resumed session: %w",
        err)
}

// Restore messages
for i, msg := range snapshot.Messages {
    m.store.CreateMessage(ctx, session.ID, &Message{
        ID:          fmt.Sprintf("%s-msg-%d", session.ID, i),
        Role:        msg.Role,
        Content:     msg.Content,
        CreatedAt:   msg.Timestamp,
    })
}

// Restore plans
for _, planSnap := range snapshot.Plans {
    // ... restore plan
}

// Restore permissions
for _, permSnap := range snapshot.Permissions {
    // ... restore permission rules
}

m.log.Info("session imported", "from", snapshot.ID, "to",
    session.ID, "messages", len(snapshot.Messages))

return session, nil
}

func (m *DefaultSessionManager) ResumeSession(ctx
    context.Context, sessionID string) (*Session, error) {
    session, err := m.store.GetSession(ctx, sessionID)
    if err != nil {
        return nil, err
    }
}

```

```

    if session.Status == "closed" {
        return nil, fmt.Errorf("session %s is closed and cannot
            be resumed", sessionID)
    }

    session.Status = "resumed"
    session.UpdatedAt = time.Now()
    m.store.UpdateSession(ctx, session)

    m.log.Info("session resumed", "id", sessionID)

    return session, nil
}

func (m *DefaultSessionManager) SaveCheckpoint(ctx
    context.Context, sessionID, name string) error {
    // Export and store as checkpoint
    snapshot, err := m.ExportSession(ctx, sessionID)
    if err != nil {
        return err
    }

    checkpoint := &CheckpointInfo{
        ID:          fmt.Sprintf("chk-%s-%d", sessionID,
            time.Now().Unix()),
        Name:         name,
        CreatedAt:    time.Now(),
        MessageCount: snapshot.MessageCount,
    }

    data, _ := json.Marshal(snapshot)

    if err := m.store.StoreCheckpoint(ctx, sessionID,
        checkpoint, data); err != nil {
        return fmt.Errorf("storing checkpoint: %w", err)
    }

    return nil
}

func (m *DefaultSessionManager) ExportToFile(ctx
    context.Context, sessionID, path string) error {
    snapshot, err := m.ExportSession(ctx, sessionID)
    if err != nil {
        return err
    }

    data, err := json.MarshalIndent(snapshot, "", " ")

```



```

    if err != nil {
        return err
    }

    // Compress
    var buf bytes.Buffer
    gz := gzip.NewWriter(&buf)
    gz.Write(data)
    gz.Close()

    if err := os.WriteFile(path, buf.Bytes(), 0644); err != nil {
        return err
    }

    m.log.Info("session exported", "id", sessionID, "path",
        path, "size", len(data))

    return nil
}

func (m *DefaultSessionManager) ImportFromFile(ctx
    context.Context, path string) (*SessionSnapshot, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, err
    }

    // Try to decompress
    r, err := gzip.NewReader(bytes.NewReader(data))
    var jsonData []byte
    if err != nil {
        // Not gzipped, use raw
        jsonData = data
    } else {
        jsonData, _ = io.ReadAll(r)
        r.Close()
    }

    var snapshot SessionSnapshot
    if err := json.Unmarshal(jsonData, &snapshot); err != nil {
        return nil, fmt.Errorf("parsing snapshot: %w", err)
    }

    return &snapshot, nil
}

func generateSessionID() string {
    return fmt.Sprintf("sess_%d", time.Now().UnixNano())
}

```

```

}

func derefString(s *string) string {
    if s == nil {
        return ""
    }
    return *s
}

```

13.6 Testing Approach

```

func TestSessionManager_ExportImport(t *testing.T) {
    manager, store := setupTestManager()

    // Create session with messages
    session, _ := manager.CreateSession(ctx, "/workspace",
        SessionConfig{Model: "claude"})
    store.CreateMessage(ctx, session.ID, &Message{Role: "user",
        Content: "Hello"})
    store.CreateMessage(ctx, session.ID, &Message{Role:
        "assistant", Content: "Hi there"})

    // Export
    snapshot, err := manager.ExportSession(ctx, session.ID)
    assert.NoError(t, err)
    assert.Equal(t, 2, snapshot.MessageCount)
    assert.NotEmpty(t, snapshot.Checksum)

    // Import
    imported, err := manager.ImportSession(ctx, snapshot)
    assert.NoError(t, err)
    assert.NotEqual(t, session.ID, imported.ID) // New ID
    assert.Equal(t, "resumed", imported.Status)
}

func TestSessionManager_Checkpoint(t *testing.T) {
    manager, _ := setupTestManager()
    session, _ := manager.CreateSession(ctx, "/workspace",
        SessionConfig{})

    err := manager.SaveCheckpoint(ctx, session.ID, "before-
        refactor")
    assert.NoError(t, err)

    checkpoints, err := manager.ListCheckpoints(ctx, session.ID)
    assert.NoError(t, err)
    assert.Len(t, checkpoints, 1)
    assert.Equal(t, "before-refactor", checkpoints[0].Name)
}

```

```

}

func TestSessionManager_ChecksumValidation(t *testing.T) {
    manager, _ := setupTestManager()

    snapshot := &SessionSnapshot{
        ID: "test", Name: "test", Checksum: "invalid",
    }

    _, err := manager.ImportSession(ctx, snapshot)
    assert.Error(t, err)
    assert.Contains(t, err.Error(), "checksum mismatch")
}

```

13.7 Edge Cases

Edge Case	Handling
Snapshot version mismatch	Migration path, or error with upgrade instructions
Workspace no longer exists	Prompt user for new workspace, or create temp
Git commit no longer exists	Use HEAD, warn about commit mismatch
Very large session (10K+ messages)	Compress, paginate, warn about size
Import during active session	Close current, prompt before replacing
Concurrent export and modification	Snapshot at a point-in-time, use DB transaction
Checkpoint explosion	Auto-delete old checkpoints (retain 10)
Cross-platform path issues	Store relative paths, resolve on import
Binary content in messages	Base64 encode, mark content type
Network storage unavailable	Fallback to local, queue for sync

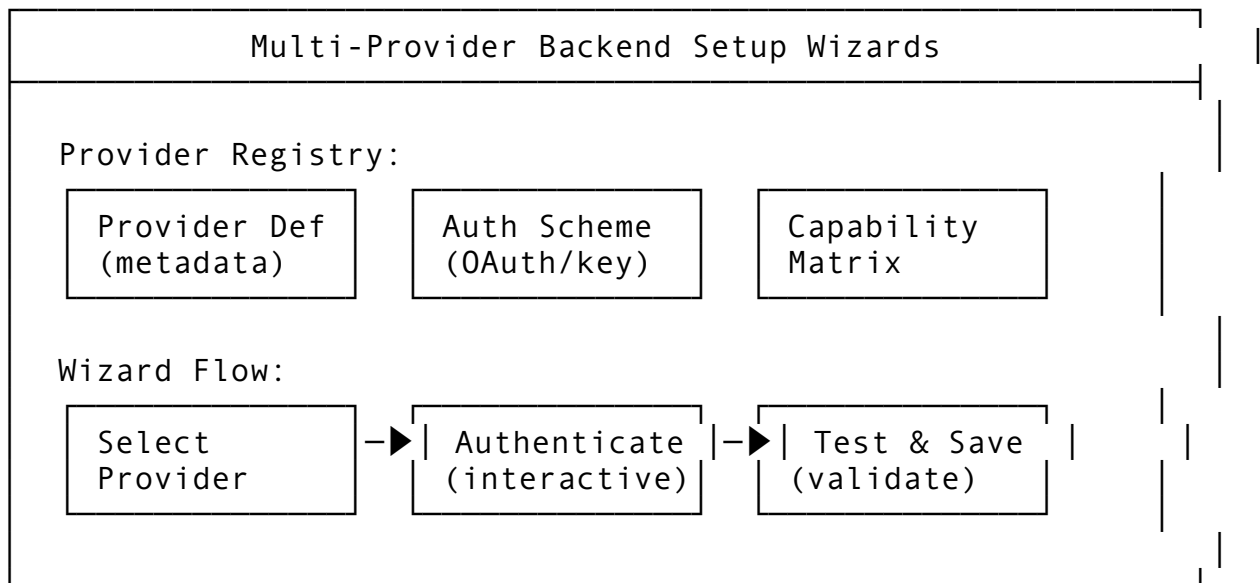
Feature 14: Multi-Provider Backend Setup Wizards

14.1 Feature Description

Multi-Provider Backend Setup Wizards provide interactive, guided configuration for connecting to any of the 29+ LLM providers. Each provider has its own authentication method, endpoint structure, and capability set.

Why it matters: Users shouldn't need to read API docs to connect their first provider. Wizards reduce onboarding friction and misconfiguration.

14.2 Architecture Design



14.3 API Design

```
// Package: internal/providers
package providers

import (
    "context"
    "fmt"
    "time"
)

// ProviderDefinition defines an LLM provider
type ProviderDefinition struct {
    Name           string          `json:"name" db:"name"`
    DisplayName    string          `json:"display_name" db:"display_name"`
    Description    string          `json:"description" db:"description"`
    Website        string          `json:"website,omitempty" db:"website"`

    // Connection
    BaseURL        string          `json:"base_url" db:"base_url"`
    APIVersion     string          `json:"api_version,omitempty" db:"api_version"`

    // Authentication
    AuthType       AuthType        `json:"auth_type" db:"auth_type"}`
```

```

AuthFields    []AuthField    `json:"auth_fields"
              db:"auth_fields"`

// Capabilities
Models        []ProviderModel `json:"models" db:"models"`
Features      ProviderFeatures `json:"features"
              db:"features"`

// Setup
SetupSteps    []SetupStep    `json:"setup_steps"
              db:"setup_steps"`
TestEndpoint  string         `json:"test_endpoint"
              db:"test_endpoint"`

// Status
IsBuiltIn     bool           `json:"is_built_in"
              db:"is_built_in"`
IsEnabled     bool           `json:"is_enabled"
              db:"is_enabled"`
}

// AuthType defines authentication method
type AuthType string

const (
    AuthAPIKey      AuthType = "api_key"
    AuthOAuth2      AuthType = "oauth2"
    AuthBearer      AuthType = "bearer"
    AuthBasic       AuthType = "basic"
    AuthCustom      AuthType = "custom"
)

// AuthField is a required authentication field
type AuthField struct {
    Name        string `json:"name"`
    Label       string `json:"label"`
    Type        string `json:"type"` // "string", "password",
               "url", "choice"
    Required    bool   `json:"required"`
    Default     string `json:"default,omitempty"`
    Description string `json:"description,omitempty"`
    Secret      bool   `json:"secret,omitempty"` // Mask in UI
    Validation  string `json:"validation,omitempty"` // Regex
}

// ProviderModel defines a model from this provider
type ProviderModel struct {
    ID          string `json:"id"`

```

```

    Name      string `json:"name"`
    ContextSize int    `json:"context_size"`
    IsChat     bool    `json:"is_chat"`
    IsVision   bool    `json:"is_vision"`
    IsEmbedding bool    `json:"is_embedding"`
    MaxTokens  int     `json:"max_tokens"`
}

// ProviderFeatures flags capability support
type ProviderFeatures struct {
    Streaming      bool `json:"streaming"`
    FunctionCalling bool `json:"function_calling"`
    Vision         bool `json:"vision"`
    JSONMode       bool `json:"json_mode"`
    SystemPrompt   bool `json:"system_prompt"`
    Temperature    bool `json:"temperature"`
    TopP           bool `json:"top_p"`
    MaxTokens      bool `json:"max_tokens"`
    StopSequences  bool `json:"stop_sequences"`
}

// SetupStep is one step in the wizard
type SetupStep struct {
    Order      int    `json:"order"`
    Title      string `json:"title"`
    Description string `json:"description"`
    Type       string `json:"type" // "info", "input", "auth",
               "test", "complete"
    Fields     []AuthField `json:"fields,omitempty"`
    Action     string `json:"action,omitempty" // URL to open,
               command to run
}

// BackendConfig is a configured provider instance
type BackendConfig struct {
    ID            string `json:"id" db:"id"`
    ProviderName  string `json:"provider_name"
               db:"provider_name"`
    Name         string `json:"name" db:"name" //
               User-defined name

    // Connection
    BaseURL      string `json:"base_url"
               db:"base_url"`
    APIVersion   string `json:"api_version,omitempty"
               db:"api_version"`

    // Credentials (encrypted at rest)

```

```

Credentials  map[string]string `json:"credentials"
            db:"credentials"`

// Settings
DefaultModel string          `json:"default_model"
            db:"default_model"`
Timeout      time.Duration   `json:"timeout" db:"timeout"`
MaxRetries   int             `json:"max_retries"
            db:"max_retries"`

// Status
IsDefault    bool            `json:"is_default"
            db:"is_default"`
IsActive     bool            `json:"is_active"
            db:"is_active"`
LastTestedAt *time.Time              `json:"last_tested_at,omitempty" db:"last_tested_at"`
LastError    *string                 `json:"last_error,omitempty"
            db:"last_error"`

// Metadata
CreatedAt    time.Time          `json:"created_at"
            db:"created_at"`
}

// SetupWizard guides provider configuration
type SetupWizard interface {
    // GetProviders returns available provider definitions
    GetProviders(ctx context.Context) ([]ProviderDefinition,
        error)

    // GetProvider returns a specific provider definition
    GetProvider(ctx context.Context, name string)
        (*ProviderDefinition, error)

    // StartSetup begins the wizard for a provider
    StartSetup(ctx context.Context, providerName string)
        (*WizardState, error)

    // SubmitStep submits data for a wizard step
    SubmitStep(ctx context.Context, wizardID string, stepIndex
        int, data map[string]string) (*WizardResult, error)

    // TestConnection validates a configuration
    TestConnection(ctx context.Context, config BackendConfig)
        (*TestResult, error)

    // SaveBackend persists a validated configuration

```

```

SaveBackend(ctx context.Context, config BackendConfig)
    (*BackendConfig, error)

// GetBackend retrieves a configured backend
GetBackend(ctx context.Context, backendID string)
    (*BackendConfig, error)

// ListBackends returns configured backends
ListBackends(ctx context.Context) ([]BackendConfig, error)

// DeleteBackend removes a backend
DeleteBackend(ctx context.Context, backendID string) error

// SetDefault marks a backend as default
SetDefault(ctx context.Context, backendID string) error
}

// WizardState tracks setup progress
type WizardState struct {
    ID            string    `json:"id"`
    ProviderName  string    `json:"provider_name"`
    CurrentStep   int       `json:"current_step"`
    TotalSteps    int       `json:"total_steps"`
    Steps         []SetupStep `json:"steps"`
    Data          map[string]string `json:"data" // Accumulated
                  answers`
    IsComplete    bool      `json:"is_complete"`
}

// WizardResult is the outcome of a step
type WizardResult struct {
    Success        bool      `json:"success"`
    NextStep       *SetupStep `json:"next_step,omitempty"`
    IsComplete     bool      `json:"is_complete"`
    BackendConfig  *BackendConfig
                  `json:"backend_config,omitempty"`
    Error          string    `json:"error,omitempty"`
    Validation     []FieldValidation `json:"validation,omitempty"`
}

// FieldValidation reports field-level errors
type FieldValidation struct {
    Field  string `json:"field"`
    Error  string `json:"error"`
}

// TestResult reports connection test outcome
type TestResult struct {

```



```

    Success      bool      `json:"success"`
    LatencyMs    int64    `json:"latency_ms"`
    ModelsFound []string  `json:"models_found,omitempty"`
    Error        string   `json:"error,omitempty"`
    Warnings     []string `json:"warnings,omitempty"`
}

```

14.4 Integration Points

Package	Modification	Description
internal/ providers/	NEW PACKAGE	Provider registry and wizard
internal/llm/	Backend selection	Use configured backends for LLM calls
internal/ config/	Provider config	Viper integration
internal/db/ migrations/	Add backends table	Persist configurations
internal/auth/	Credential encryption	Encrypt credentials at rest
cmd/helix/ main.go	setup subcommand	Interactive wizard command

14.5 Implementation Steps

```

// internal/providers/wizard.go
package providers

import (
    "context"
    "fmt"
    "time"

    "dev.helix.code/internal/db"
    "dev.helix.code/internal/llm"
    "dev.helix.code/pkg/logger"
)

type DefaultSetupWizard struct {
    providers map[string]*ProviderDefinition
    backends  db.Store
    llmClient llm.Client
    log       logger.Logger
    wizards   map[string]*WizardState
}

func NewDefaultSetupWizard(store db.Store, log logger.Logger)
    *DefaultSetupWizard {

```

```

w := &DefaultSetupWizard{
    providers: make(map[string]*ProviderDefinition),
    backends:  store,
    log:       log,
    wizards:   make(map[string]*WizardState),
}
w.loadBuiltInProviders()
return w
}

func (w *DefaultSetupWizard) loadBuiltInProviders() {
    // Define all 29+ providers
    builtins := []ProviderDefinition{
        {
            Name:          "openai",
            DisplayName:   "OpenAI",
            Description:    "GPT-4, GPT-3.5, and embedding models",
            BaseURL:       "https://api.openai.com/v1",
            AuthType:      AuthAPIKey,
            AuthFields: []AuthField{
                {
                    Name:          "api_key",
                    Label:         "API Key",
                    Type:          "password",
                    Required:      true,
                    Description:    "Your OpenAI API key from platform.openai.com",
                    Secret:       true,
                },
            },
            Models: []ProviderModel{
                {ID: "gpt-4o", Name: "GPT-4o", ContextSize: 128000, IsChat: true, IsVision: true},
                {ID: "gpt-4-turbo", Name: "GPT-4 Turbo", ContextSize: 128000, IsChat: true, IsVision: true},
                {ID: "gpt-3.5-turbo", Name: "GPT-3.5 Turbo", ContextSize: 16385, IsChat: true},
            },
            Features: ProviderFeatures{
                Streaming: true, FunctionCalling: true, Vision: true, JSONMode: true,
                SystemPrompt: true, Temperature: true, TopP: true, MaxTokens: true, StopSequences: true,
            },
            SetupSteps: []SetupStep{

```

```

        {Order: 1, Title: "Get API Key", Description:
"Visit platform.openai.com/api-keys to create an API
key", Type: "info", Action: "https://platform.openai.com/
api-keys"},
        {Order: 2, Title: "Enter API Key", Description:
"Paste your API key", Type: "input", Fields:
[]AuthField{{Name: "api_key", Label: "API Key", Type:
"password", Required: true}}},
        {Order: 3, Title: "Test Connection",
Description: "Testing connectivity to OpenAI", Type:
"test"},
        {Order: 4, Title: "Complete", Description: "Your
OpenAI backend is ready", Type: "complete"},
    },
    IsBuiltIn: true,
    IsEnabled: true,
},
{
    Name: "anthropic",
    DisplayName: "Anthropic",
    Description: "Claude 3.5 Sonnet, Claude 3 Opus, and
Claude 3 Haiku",
    BaseURL: "https://api.anthropic.com",
    APIVersion: "2023-06-01",
    AuthType: AuthAPIKey,
    AuthFields: []AuthField{
        {
            Name: "api_key",
            Label: "API Key",
            Type: "password",
            Required: true,
            Description: "Your Anthropic API key from
console.anthropic.com",
            Secret: true,
        },
    },
    Models: []ProviderModel{
        {ID: "claude-sonnet-4-20250514", Name: "Claude
Sonnet 4", ContextSize: 200000, IsChat: true, IsVision:
true},
        {ID: "claude-opus-4-20250514", Name:
"Claude Opus 4", ContextSize: 200000, IsChat: true,
IsVision: true},
        {ID: "claude-haiku-4-20250514", Name: "Claude
Haiku 4", ContextSize: 200000, IsChat: true, IsVision:
true},
    },
    Features: ProviderFeatures{

```

```

        Streaming: true, FunctionCalling: true, Vision:
true,
        SystemPrompt: true, Temperature: true, TopP:
true, MaxTokens: true, StopSequences: true,
    },
    IsBuiltIn: true,
    IsEnabled: true,
},
{
    Name: "ollama",
    DisplayName: "Ollama (Local)",
    Description: "Run models locally with Ollama",
    BaseURL: "http://localhost:11434",
    AuthType: AuthNone,
    AuthFields: []AuthField{
        {
            Name: "base_url",
            Label: "Ollama URL",
            Type: "url",
            Required: false,
            Default: "http://localhost:11434",
            Description: "URL of your Ollama server",
        },
    },
    Models: []ProviderModel{
        {ID: "llama3.3", Name: "Llama 3.3", ContextSize:
128000, IsChat: true},
        {ID: "codellama", Name: "CodeLlama",
ContextSize: 16000, IsChat: true},
        {ID: "mistral", Name: "Mistral", ContextSize:
32000, IsChat: true},
    },
    Features: ProviderFeatures{
        Streaming: true, Temperature: true, TopP: true,
MaxTokens: true,
    },
    SetupSteps: []SetupStep{
        {Order: 1, Title: "Install Ollama", Description:
"Download from ollama.com", Type: "info", Action:
"https://ollama.com/download"},
        {Order: 2, Title: "Pull a Model", Description:
"Run: ollama pull llama3.3", Type: "info"},
        {Order: 3, Title: "Configure", Description: "Set
your Ollama URL", Type: "input", Fields:
[]AuthField{{Name: "base_url", Label: "URL", Type:
"url", Default: "http://localhost:11434"}}},
        {Order: 4, Title: "Test", Description: "Testing
connection", Type: "test"},
    },
}

```

```

        },
        IsBuiltIn: true,
        IsEnabled: true,
    },
    // Additional providers: groq, cerebras, gemini, cohere,
    // azure, bedrock, etc.
}

for i := range builtins {
    w.providers[builtins[i].Name] = &builtins[i]
}

func (w *DefaultSetupWizard) StartSetup(ctx context.Context,
    providerName string) (*WizardState, error) {
    provider, ok := w.providers[providerName]
    if !ok {
        return nil, fmt.Errorf("provider not found: %s",
            providerName)
    }

    wizardID := fmt.Sprintf("wiz_%d", time.Now().UnixNano())
    state := &WizardState{
        ID:          wizardID,
        ProviderName: providerName,
        CurrentStep: 0,
        TotalSteps:  len(provider.SetupSteps),
        Steps:        provider.SetupSteps,
        Data:          make(map[string]string),
    }

    w.wizards[wizardID] = state

    return state, nil
}

func (w *DefaultSetupWizard) SubmitStep(ctx context.Context,
    wizardID string, stepIndex int, data map[string]string)
    (*WizardResult, error) {
    wizard, ok := w.wizards[wizardID]
    if !ok {
        return nil, fmt.Errorf("wizard not found: %s", wizardID)
    }

    if stepIndex != wizard.CurrentStep {
        return nil, fmt.Errorf("invalid step: expected %d, got
            %d", wizard.CurrentStep, stepIndex)
    }
}

```

```

provider := w.providers[wizard.ProviderName]
step := provider.SetupSteps[stepIndex]

// Validate inputs
var validation []FieldValidation
for _, field := range step.Fields {
    val, ok := data[field.Name]
    if field.Required && (!ok || val == "") {
        validation = append(validation, FieldValidation{
            Field: field.Name,
            Error: fmt.Sprintf("%s is required",
                field.Label),
        })
        continue
    }
    if field.Validation != "" && val != "" {
        // Regex validation
        // ...
    }
}

if len(validation) > 0 {
    return &WizardResult{
        Success: false,
        Validation: validation,
    }, nil
}

// Store data
for k, v := range data {
    wizard.Data[k] = v
}

// Handle test step
if step.Type == "test" {
    config := w.buildConfigFromWizard(wizard, provider)
    testResult, err := w.TestConnection(ctx, config)
    if err != nil || !testResult.Success {
        return &WizardResult{
            Success: false,
            Error: testResult.Error,
        }, nil
    }
}

// Advance
wizard.CurrentStep++

```

```

if wizard.CurrentStep >= len(provider.SetupSteps) {
    wizard.IsComplete = true
    config := w.buildConfigFromWizard(wizard, provider)
    saved, err := w.SaveBackend(ctx, config)
    if err != nil {
        return &WizardResult{Success: false, Error:
            err.Error()}, nil
    }
    return &WizardResult{
        Success:      true,
        IsComplete:   true,
        BackendConfig: saved,
    }, nil
}

nextStep := provider.SetupSteps[wizard.CurrentStep]
return &WizardResult{
    Success: true,
    NextStep: &nextStep,
}, nil
}

func (w *DefaultSetupWizard) buildConfigFromWizard(wizard
    *WizardState, provider *ProviderDefinition)
    BackendConfig {
config := BackendConfig{
    ProviderName: wizard.ProviderName,
    Name:         fmt.Sprintf("%s-default",
        wizard.ProviderName),
    BaseURL:      provider.BaseURL,
    APIVersion:   provider.APIVersion,
    Credentials:  make(map[string]string),
    Timeout:      30 * time.Second,
    MaxRetries:   3,
    IsActive:     true,
}

// Override base URL if provided
if url, ok := wizard.Data["base_url"]; ok {
    config.BaseURL = url
}

// Extract credentials
for _, field := range provider.AuthFields {
    if val, ok := wizard.Data[field.Name]; ok {
        config.Credentials[field.Name] = val
    }
}

```

```

    }

    // Set default model
    if len(provider.Models) > 0 {
        config.DefaultModel = provider.Models[0].ID
    }

    return config
}

func (w *DefaultSetupWizard) TestConnection(ctx context.Context,
    config BackendConfig) (*TestResult, error) {
    start := time.Now()

    // Create a test client with this config
    testClient, err := w.createTestClient(config)
    if err != nil {
        return &TestResult{Success: false, Error: err.Error()},
            nil
    }

    // Make a minimal request (list models or simple completion)
    models, err := testClient.ListModels(ctx)

    latency := time.Since(start).Milliseconds()

    if err != nil {
        return &TestResult{
            Success: false,
            LatencyMs: latency,
            Error: err.Error(),
        }, nil
    }

    var modelIDs []string
    for _, m := range models {
        modelIDs = append(modelIDs, m.ID)
    }

    return &TestResult{
        Success: true,
        LatencyMs: latency,
        ModelsFound: modelIDs,
    }, nil
}

func (w *DefaultSetupWizard) SaveBackend(ctx context.Context,
    config BackendConfig) (*BackendConfig, error) {

```



```

config.ID = fmt.Sprintf("backend_%d", time.Now().UnixNano())
config.CreatedAt = time.Now()

if err := w.backends.CreateBackend(ctx, &config); err != nil
{
    return nil, fmt.Errorf("saving backend: %w", err)
}

// If this is the first backend, make it default
existing, _ := w.ListBackends(ctx)
if len(existing) == 1 {
    w.SetDefault(ctx, config.ID)
}

return &config, nil
}

func (w *DefaultSetupWizard) createTestClient(config
BackendConfig) (llm.Client, error) {
    // Factory method to create LLM client for testing
    // ... implementation depends on provider type
    return nil, fmt.Errorf("not implemented")
}

// Other interface methods...

```

14.6 Testing Approach

```

func TestSetupWizard_StartSetup(t *testing.T) {
    wizard := NewDefaultSetupWizard(mockStore, logger.NewNop())

    state, err := wizard.StartSetup(ctx, "openai")
    assert.NoError(t, err)
    assert.NotEmpty(t, state.ID)
    assert.Equal(t, "openai", state.ProviderName)
    assert.Equal(t, 0, state.CurrentStep)
    assert.Equal(t, 4, state.TotalSteps)
}

func TestSetupWizard_SubmitStep_Validation(t *testing.T) {
    wizard := NewDefaultSetupWizard(mockStore, logger.NewNop())
    state, _ := wizard.StartSetup(ctx, "openai")

    // Submit without required API key
    result, err := wizard.SubmitStep(ctx, state.ID, 1,
        map[string]string{})

    assert.NoError(t, err)
}

```

```

    assert.False(t, result.Success)
    assert.Len(t, result.Validation, 1)
    assert.Equal(t, "api_key", result.Validation[0].Field)
}

func TestSetupWizard_TestConnection(t *testing.T) {
    wizard := NewDefaultSetupWizard(mockStore, logger.NewNop())

    result, err := wizard.TestConnection(ctx, BackendConfig{
        ProviderName: "ollama",
        BaseURL:      "http://localhost:11434",
        Credentials:  map[string]string{},
    })

    // May succeed or fail depending on environment
    assert.NoError(t, err)
    assert.NotNil(t, result)
}

```

14.7 Edge Cases

Edge Case	Handling
Provider offline during test	Show clear error, allow retry
Invalid API key format	Validate format before testing
Credential storage failure	Encrypt with key from environment or keyring
Duplicate backend name	Suggest unique name (append number)
Provider changes API version	Auto-detect, warn about compatibility
Custom provider not in registry	Allow manual entry with full schema
2FA required for OAuth	Open browser, poll for completion
Rate limited during test	Backoff, show estimated wait
Enterprise proxy required	Auto-detect system proxy, allow custom
Self-signed certificates	Prompt to trust, store fingerprint

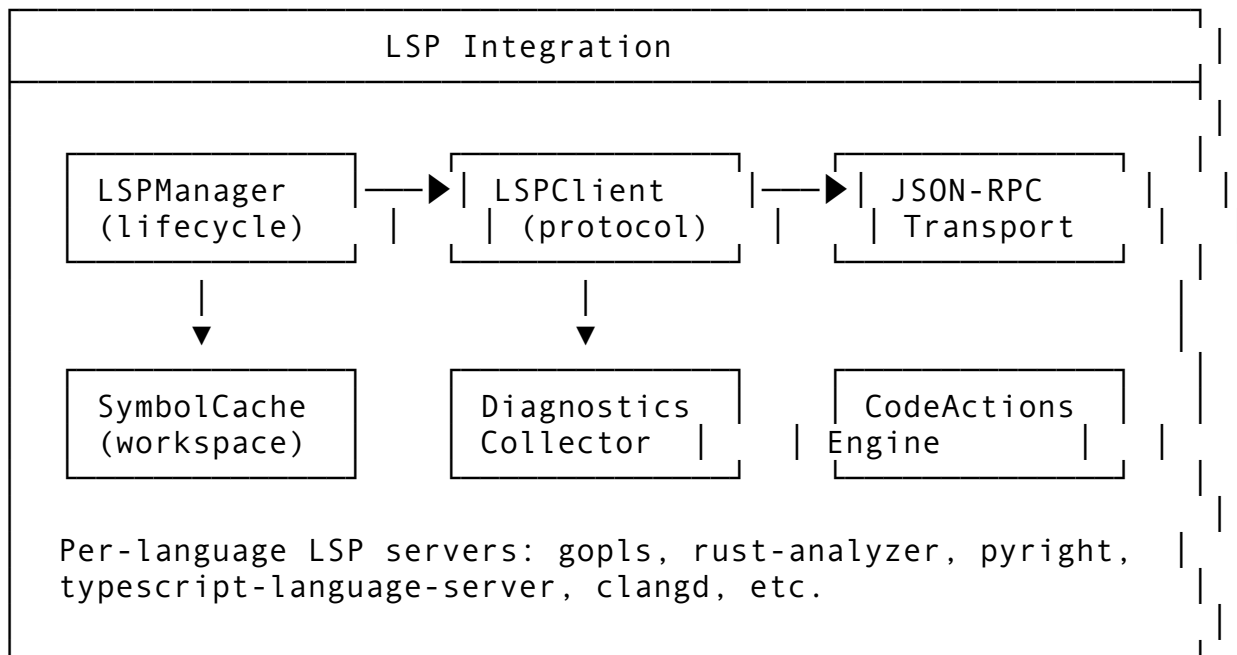
Feature 15: LSP Integration

15.1 Feature Description

Language Server Protocol (LSP) integration enables the agent to access IDE-quality code intelligence: go-to-definition, find-references, hover information, diagnostics, code actions, and refactoring.

Why it matters: The agent needs deep code understanding beyond text. LSP provides semantic knowledge that LLMs can't infer from raw text.

15.2 Architecture Design



15.3 API Design

```
// Package: internal/lsp
package lsp

import (
    "context"
    "fmt"
    "time"
)

// LSPMessage is the JSON-RPC protocol message
type LSPMessage struct {
    JSONRPC string    `json:"jsonrpc"`
    ID      any           `json:"id,omitempty"`
    Method  string       `json:"method,omitempty"`
    Params  json.RawMessage `json:"params,omitempty"`
    Result  json.RawMessage `json:"result,omitempty"`
    Error   *LSPError     `json:"error,omitempty"`
}

type LSPError struct {
    Code    int    `json:"code"`
    Message string `json:"message"`
    Data    any    `json:"data,omitempty"`
}

// LSPServerConfig defines a language server
```

```

type LSPServerConfig struct {
    LanguageID    string          `json:"language_id"
                  db:"language_id"`
    Name          string          `json:"name" db:"name"`
    Command       string          `json:"command" db:"command"`
    Args          []string        `json:"args" db:"args"`
    Env           map[string]string `json:"env,omitempty"
                  db:"env"`
    WorkingDir    string          `json:"working_dir"
                  db:"working_dir"`
    InitOptions   json.RawMessage `json:"init_options,omitempty" db:"init_options"`

    // Capabilities
    SupportsDefinition    bool `json:"supports_definition"
                  db:"supports_definition"`
    SupportsReferences    bool `json:"supports_references"
                  db:"supports_references"`
    SupportsHover         bool `json:"supports_hover"
                  db:"supports_hover"`
    SupportsDiagnostics   bool `json:"supports_diagnostics"
                  db:"supports_diagnostics"`
    SupportsCodeAction     bool `json:"supports_code_action"
                  db:"supports_code_action"`
    SupportsCompletion     bool `json:"supports_completion"
                  db:"supports_completion"`
    SupportsRename        bool `json:"supports_rename"
                  db:"supports_rename"`
    SupportsWorkspaceSymbol bool
                  `json:"supports_workspace_symbol"
                  db:"supports_workspace_symbol"`

    IsEnabled    bool          `json:"is_enabled"
                  db:"is_enabled"`
    IsConnected  bool          `json:"is_connected"
                  db:"is_connected"`
}

// LSPManager manages language server connections
type LSPManager interface {
    // StartServer starts a language server for a workspace
    StartServer(ctx context.Context, workspace string, config
        LSPServerConfig) error

    // StopServer stops a language server
    StopServer(ctx context.Context, workspace, languageID
        string) error
}

```

```
// GetDefinition finds symbol definition
GetDefinition(ctx context.Context, workspace, filePath
    string, line, character int) ([]Location, error)

// GetReferences finds symbol references
GetReferences(ctx context.Context, workspace, filePath
    string, line, character int, includeDeclaration bool)
    ([]Location, error)

// GetHover gets hover information
GetHover(ctx context.Context, workspace, filePath string,
    line, character int) (*HoverInfo, error)

// GetDocumentSymbols gets symbols in a document
GetDocumentSymbols(ctx context.Context, workspace, filePath
    string) ([]DocumentSymbol, error)

// GetWorkspaceSymbols searches workspace symbols
GetWorkspaceSymbols(ctx context.Context, workspace, query
    string) ([]WorkspaceSymbol, error)

// GetDiagnostics gets document diagnostics
GetDiagnostics(ctx context.Context, workspace, filePath
    string) ([]Diagnostic, error)

// GetCodeActions gets available code actions
GetCodeActions(ctx context.Context, workspace, filePath
    string, range_ Range, diagnostics []Diagnostic)
    ([]CodeAction, error)

// ExecuteCommand runs a workspace command
ExecuteCommand(ctx context.Context, workspace, command
    string, arguments []any) error

// GetCompletion provides code completions
GetCompletion(ctx context.Context, workspace, filePath
    string, line, character int) (*CompletionList, error)

// DidOpen notifies server about opened document
DidOpen(ctx context.Context, workspace, filePath string,
    languageID string, content string) error

// DidChange notifies server about document changes
DidChange(ctx context.Context, workspace, filePath string,
    changes []TextDocumentContentChangeEvent) error

// DidSave notifies server about saved document
```

```

DidSave(ctx context.Context, workspace, filePath string)
    error

// DidClose notifies server about closed document
DidClose(ctx context.Context, workspace, filePath string)
    error

// GetConnectionStatus returns server health
GetConnectionStatus(ctx context.Context, workspace,
    languageID string) (*LSPConnectionStatus, error)

// ListServers returns configured servers
ListServers(ctx context.Context) ([]LSPServerConfig, error)
}

// Location is a file location
type Location struct {
    URI    string `json:"uri"`
    Range Range `json:"range"`
}

// Range is a text range
type Range struct {
    Start Position `json:"start"`
    End    Position `json:"end"`
}

// Position is a point in a document
type Position struct {
    Line      int `json:"line"`
    Character int `json:"character"`
}

// HoverInfo is hover tooltip content
type HoverInfo struct {
    Contents any    `json:"contents"` // string or MarkupContent
    Range    *Range `json:"range,omitempty"`
}

// DocumentSymbol is a symbol in a document
type DocumentSymbol struct {
    Name          string `json:"name"`
    Detail        string `json:"detail,omitempty"`
    Kind          int    `json:"kind"` // SymbolKind
    Range         Range  `json:"range"`
    SelectionRange Range  `json:"selectionRange"`
    Children      []DocumentSymbol `json:"children,omitempty"`
}

```

```

// WorkspaceSymbol is a symbol in the workspace
type WorkspaceSymbol struct {
    Name      string `json:"name"`
    Kind      int    `json:"kind"`
    Location  Location `json:"location"`
    ContainerName string `json:"containerName,omitempty"`
}

// Diagnostic is a code issue
type Diagnostic struct {
    Range      Range      `json:"range"`
    Severity   int        `json:"severity,omitempty"` //
    //          1=Error, 2=Warning, 3=Info, 4=Hint
    Code       any        `json:"code,omitempty"`
    Source     string     `json:"source,omitempty"`
    Message    string     `json:"message"`
    RelatedInformation []DiagnosticRelatedInformation
    `json:"relatedInformation,omitempty"`
}

type DiagnosticRelatedInformation struct {
    Location Location `json:"location"`
    Message  string  `json:"message"`
}

// CodeAction is a suggested fix
type CodeAction struct {
    Title      string      `json:"title"`
    Kind       string      `json:"kind,omitempty"`
    Diagnostics []Diagnostic `json:"diagnostics,omitempty"`
    Edit       *WorkspaceEdit `json:"edit,omitempty"`
    Command    *Command      `json:"command,omitempty"`
}

type WorkspaceEdit struct {
    Changes map[string][]TextEdit `json:"changes,omitempty"`
}

type TextEdit struct {
    Range      Range `json:"range"`
    NewText    string `json:"newText"`
}

type Command struct {
    Title      string `json:"title"`
    Command    string `json:"command"`
    Arguments []any  `json:"arguments,omitempty"`
}

```

```

}

// CompletionList is code completion suggestions
type CompletionList struct {
    IsIncomplete bool          `json:"isIncomplete"`
    Items        []CompletionItem `json:"items"`
}

type CompletionItem struct {
    Label        string `json:"label"`
    Kind         int    `json:"kind,omitempty"`
    Detail       string `json:"detail,omitempty"`
    Documentation any    `json:"documentation,omitempty"`
    InsertText   string `json:"insertText,omitempty"`
    InsertTextFormat int    `json:"insertTextFormat,omitempty" // 1=PlainText,
                                     2=Snippet`
}

type TextDocumentContentChangeEvent struct {
    Range *Range `json:"range,omitempty"`
    RangeLength *int `json:"rangeLength,omitempty"`
    Text string `json:"text"`
}

// LSPConnectionStatus reports server health
type LSPConnectionStatus struct {
    LanguageID string `json:"language_id"`
    IsConnected bool  `json:"is_connected"`
    LastPing   time.Time `json:"last_ping"`
    ErrorCount int      `json:"error_count"`
    LastError  string   `json:"last_error,omitempty"`
}

```

15.4 Integration Points

Package	Modification	Description
internal/lsp/	NEW PACKAGE	LSP manager and client
internal/tool/	LSP tool integration	Map LSP features to agent tools
internal/fs/	File event forwarding	Forward file changes to LSP
internal/edit/	Document sync	Notify LSP on file edits
cmd/helix/ main.go	--lsp-config flag	LSP server configuration

15.5 Implementation Steps

```
// internal/lsp/manager.go
package lsp

import (
    "bufio"
    "context"
    "encoding/json"
    "fmt"
    "io"
    "os"
    "os/exec"
    "path/filepath"
    "strconv"
    "strings"
    "sync"
    "time"

    "dev.helix.code/pkg/logger"
)

type DefaultLSPManager struct {
    servers    map[string]*lspServer // key: workspace+languageID
    mu         sync.RWMutex
    log        logger.Logger
    requestID  int
}

type lspServer struct {
    config      LSPServerConfig
    cmd         *exec.Cmd
    stdin       io.WriteCloser
    stdout      *bufio.Reader
    diagnostics map[string][]Diagnostic
    mu          sync.RWMutex
}

func NewDefaultLSPManager(log logger.Logger) *DefaultLSPManager {
    return &DefaultLSPManager{
        servers: make(map[string]*lspServer),
        log:     log,
    }
}

func (m *DefaultLSPManager) StartServer(ctx context.Context,
    workspace string, config LSPServerConfig) error {
    key := serverKey(workspace, config.LanguageID)
```

```

m.mu.Lock()
defer m.mu.Unlock()

if existing, ok := m.servers[key]; ok && existing.cmd != nil
    && existing.cmd.Process != nil {
    return fmt.Errorf("server already running for %s", key)
}

// Build command
cmd := exec.CommandContext(ctx, config.Command,
    config.Args...)
cmd.Dir = workspace
if config.WorkingDir != "" {
    cmd.Dir = config.WorkingDir
}

// Set environment
cmd.Env = os.Environ()
for k, v := range config.Env {
    cmd.Env = append(cmd.Env, fmt.Sprintf("%s=%s", k, v))
}

stdin, err := cmd.StdinPipe()
if err != nil {
    return fmt.Errorf("stdin pipe: %w", err)
}

stdout, err := cmd.StdoutPipe()
if err != nil {
    return fmt.Errorf("stdout pipe: %w", err)
}

stderr, err := cmd.StderrPipe()
if err != nil {
    return fmt.Errorf("stderr pipe: %w", err)
}

if err := cmd.Start(); err != nil {
    return fmt.Errorf("starting server: %w", err)
}

server := &lspServer{
    config:    config,
    cmd:       cmd,
    stdin:     stdin,
    stdout:    bufio.NewReader(stdout),
    diagnostics: make(map[string][]Diagnostic),
}

```

```

}

m.servers[key] = server

// Read stderr in background
go m.readStderr(stderr, config.Name)

// Read responses in background
go m.readResponses(server)

// Send initialize
initParams := map[string]any{
    "processId": os.Getpid(),
    "rootUri":   pathToURI(workspace),
    "capabilities": map[string]any{
        "textDocument": map[string]any{
            "synchronization":
                map[string]any{"dynamicRegistration": false},
            "hover":
                map[string]any{"dynamicRegistration": false},
            "definition":
                map[string]any{"dynamicRegistration": false},
            "references":
                map[string]any{"dynamicRegistration": false},
            "documentSymbol":
                map[string]any{"dynamicRegistration": false},
            "codeAction":
                map[string]any{"dynamicRegistration": false},
            "completion":
                map[string]any{"dynamicRegistration": false},
        },
        "workspace": map[string]any{
            "workspaceFolders": true,
        },
    },
    "workspaceFolders": []map[string]any{
        {"uri": pathToURI(workspace), "name":
            filepath.Base(workspace)},
    },
}

if config.InitOptions != nil {
    initParams["initializationOptions"] =
        json.RawMessage(config.InitOptions)
}

_, err = m.sendRequest(ctx, server, "initialize", initParams)
if err != nil {

```

```

        m.StopServer(ctx, workspace, config.LanguageID)
        return fmt.Errorf("initializing server: %w", err)
    }

    // Send initialized notification
    m.sendNotification(server, "initialized", map[string]any{})

    m.log.Info("LSP server started", "language",
        config.LanguageID, "workspace", workspace)

    return nil
}

func (m *DefaultLSPManager) sendRequest(ctx context.Context,
    server *lspServer, method string, params any)
    (*LSPMessage, error) {
    m.requestID++
    id := m.requestID

    msg := LSPMessage{
        JSONRPC: "2.0",
        ID:      id,
        Method:  method,
        Params:  mustJSON(params),
    }

    if err := m.writeMessage(server, msg); err != nil {
        return nil, err
    }

    // Read response (simplified - needs request correlation)
    response, err := m.readMessage(server)
    if err != nil {
        return nil, err
    }

    if response.Error != nil {
        return nil, fmt.Errorf("LSP error %d: %s",
            response.Error.Code, response.Error.Message)
    }

    return response, nil
}

func (m *DefaultLSPManager) sendNotification(server *lspServer,
    method string, params any) error {
    msg := LSPMessage{
        JSONRPC: "2.0",

```

```

        Method: method,
        Params: mustJSON(params),
    }
    return m.writeMessage(server, msg)
}

func (m *DefaultLSPManager) writeMessage(server *lspServer, msg
    LSPMessage) error {
    data, err := json.Marshal(msg)
    if err != nil {
        return err
    }

    header := fmt.Sprintf("Content-Length: %d\r\n\r\n",
        len(data))

    _, err = server.stdin.Write([]byte(header))
    if err != nil {
        return err
    }
    _, err = server.stdin.Write(data)
    return err
}

func (m *DefaultLSPManager) readMessage(server *lspServer)
    (*LSPMessage, error) {
    // Read headers
    var contentLength int
    for {
        line, err := server.stdout.ReadString('\n')
        if err != nil {
            return nil, err
        }
        line = strings.TrimSpace(line)
        if line == "" {
            break
        }
        if strings.HasPrefix(line, "Content-Length:") {
            val := strings.TrimSpace(strings.TrimPrefix(line,
                "Content-Length:"))
            n, _ := strconv.Atoi(val)
            contentLength = n
        }
    }

    // Read body
    body := make([]byte, contentLength)
    _, err := io.ReadFull(server.stdout, body)

```

```

    if err != nil {
        return nil, err
    }

    var msg LSPMessage
    if err := json.Unmarshal(body, &msg); err != nil {
        return nil, err
    }

    return &msg, nil
}

func (m *DefaultLSPManager) readStderr(stderr io.ReadCloser,
    serverName string) {
    scanner := bufio.NewScanner(stderr)
    for scanner.Scan() {
        m.log.Debug("LSP stderr", "server", serverName, "line",
            scanner.Text())
    }
}

func (m *DefaultLSPManager) readResponses(server *lspServer) {
    for {
        msg, err := m.readMessage(server)
        if err != nil {
            m.log.Warn("LSP read error", "error", err)
            return
        }

        if msg.Method == "textDocument/publishDiagnostics" {
            var params struct {
                URI          string          `json:"uri"`
                Diagnostics []Diagnostic    `json:"diagnostics"`
            }
            json.Unmarshal(msg.Params, &params)

            server.mu.Lock()
            server.diagnostics[params.URI] = params.Diagnostics
            server.mu.Unlock()
        }
    }
}

func (m *DefaultLSPManager) GetDefinition(ctx context.Context,
    workspace, filePath string, line, character int)
    ([]Location, error) {
    server, err := m.getServer(workspace, filePath)
    if err != nil {

```

```

        return nil, err
    }

    result, err := m.sendRequest(ctx, server, "textDocument/
        definition", map[string]any{
            "textDocument": map[string]any{"uri":
                pathToURI(filePath)},
            "position":      map[string]any{"line": line,
                "character": character},
        })
    if err != nil {
        return nil, err
    }

    var locations []Location
    json.Unmarshal(result.Result, &locations)
    return locations, nil
}

func (m *DefaultLSPManager) DidOpen(ctx context.Context,
    workspace, filePath, languageID, content string) error {
    server, err := m.getServerByLanguage(workspace, languageID)
    if err != nil {
        return err
    }

    return m.sendNotification(server, "textDocument/didOpen",
        map[string]any{
            "textDocument": map[string]any{
                "uri":      pathToURI(filePath),
                "languageId": languageID,
                "version":    1,
                "text":       content,
            },
        })
}

func (m *DefaultLSPManager) DidChange(ctx context.Context,
    workspace, filePath string, changes
    []TextDocumentContentChangeEvent) error {
    server, err := m.getServer(workspace, filePath)
    if err != nil {
        return err
    }

    return m.sendNotification(server, "textDocument/didChange",
        map[string]any{
            "textDocument": map[string]any{

```

```

        "uri":      pathToURI(filePath),
        "version": 2, // Should track actual version
    },
    "contentChanges": changes,
})
}

func (m *DefaultLSPManager) getServer(workspace, filePath
    string) (*lspServer, error) {
    ext := filepath.Ext(filePath)
    languageID := extensionToLanguageID(ext)
    return m.getServerByLanguage(workspace, languageID)
}

func (m *DefaultLSPManager) getServerByLanguage(workspace,
    languageID string) (*lspServer, error) {
    key := serverKey(workspace, languageID)

    m.mu.RLock()
    server, ok := m.servers[key]
    m.mu.RUnlock()

    if !ok {
        return nil, fmt.Errorf("no LSP server for %s in %s",
            languageID, workspace)
    }

    return server, nil
}

func (m *DefaultLSPManager) StopServer(ctx context.Context,
    workspace, languageID string) error {
    key := serverKey(workspace, languageID)

    m.mu.Lock()
    server, ok := m.servers[key]
    if !ok {
        m.mu.Unlock()
        return nil
    }
    delete(m.servers, key)
    m.mu.Unlock()

    // Shutdown gracefully
    m.sendRequest(ctx, server, "shutdown", map[string]any{})
    m.sendNotification(server, "exit", map[string]any{})

    if server.stdin != nil {

```



```

        server.stdin.Close()
    }
    if server.cmd != nil && server.cmd.Process != nil {
        server.cmd.Process.Kill()
        server.cmd.Wait()
    }

    return nil
}

func (m *DefaultLSPManager) GetDiagnostics(ctx context.Context,
    workspace, filePath string) ([]Diagnostic, error) {
    server, err := m.getServer(workspace, filePath)
    if err != nil {
        return nil, err
    }

    server.mu.RLock()
    diagnostics := server.diagnostics[pathToURI(filePath)]
    server.mu.RUnlock()

    return diagnostics, nil
}

func serverKey(workspace, languageID string) string {
    return workspace + "#" + languageID
}

func pathToURI(path string) string {
    return "file://" + path
}

func uriToPath(uri string) string {
    return strings.TrimPrefix(uri, "file://")
}

func extensionToLanguageID(ext string) string {
    switch ext {
    case ".go": return "go"
    case ".py": return "python"
    case ".js", ".jsx": return "javascript"
    case ".ts", ".tsx": return "typescript"
    case ".rs": return "rust"
    case ".java": return "java"
    case ".cpp", ".cc", ".cxx": return "cpp"
    case ".c": return "c"
    case ".h", ".hpp": return "cpp"
    case ".rb": return "ruby"
    }
}

```

```

        default: return "plaintext"
    }
}

func mustJSON(v any) json.RawMessage {
    data, _ := json.Marshal(v)
    return data
}

// Other interface methods: GetReferences, GetHover,
// GetDocumentSymbols, GetWorkspaceSymbols, GetCodeActions,
// ExecuteCommand, GetCompletion, DidSave, DidClose,
// GetConnectionStatus, ListServers

```

15.6 Testing Approach

```

func TestLSPManager_StartServer(t *testing.T) {
    // This test requires a real LSP server installed
    if os.Getenv("SKIP_LSP_TESTS") != "" {
        t.Skip("Skipping LSP tests")
    }

    manager := NewDefaultLSPManager(logger.NewNop())

    workspace := t.TempDir()
    err := manager.StartServer(ctx, workspace, LSPServerConfig{
        LanguageID: "go",
        Name:       "gopls",
        Command:   "gopls",
        Args:      []string{"serve"},
    })

    if err != nil {
        t.Skipf("gopls not available: %v", err)
    }

    // Create a Go file
    goFile := filepath.Join(workspace, "test.go")
    os.WriteFile(goFile, []byte("package main\n\nfunc main() {}
    \n"), 0644)

    err = manager.DidOpen(ctx, workspace, goFile, "go", "package
    main\n\nfunc main() {}\n")
    assert.NoError(t, err)

    // Give server time to index
    time.Sleep(500 * time.Millisecond)
}

```

```

diags, err := manager.GetDiagnostics(ctx, workspace, goFile)
assert.NoError(t, err)
// May have diagnostics or not depending on code
_ = diags
}

```

15.7 Edge Cases

Edge Case	Handling
LSP server not installed	Error with install instructions per language
Server crashes mid-session	Auto-restart with backoff, notify user
Large workspace (100K+ files)	Set file watcher exclusions, use workspace/didChangeWatchedFiles
Document sync race condition	Queue changes, use incremental sync
Multiple files open simultaneously	Track per-document version, batch notifications
Server returns invalid JSON	Log raw content, skip response
Request timeout	Cancel with context, try fallback
Symbol not found	Return empty result, don't error
Server doesn't support a feature	Check capabilities, skip unsupported calls
Unicode file paths	Percent-encode in URIs per RFC 3986

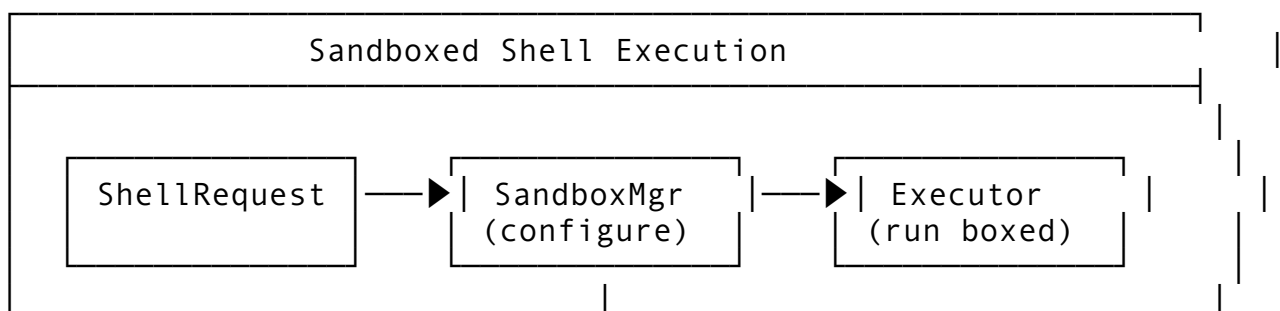
Feature 16: Sandboxed Shell Execution (PID Namespaces + Seccomp)

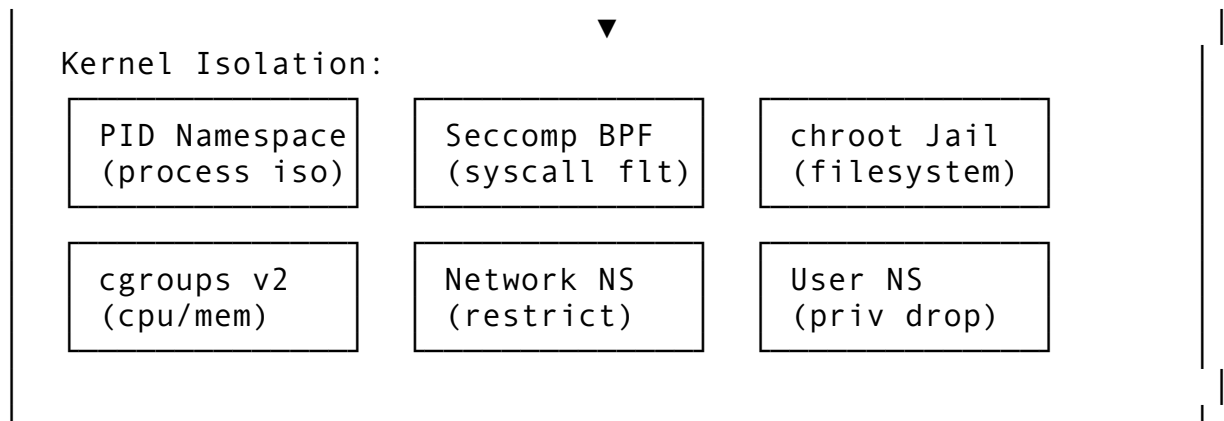
16.1 Feature Description

Sandboxed Shell Execution provides secure, isolated execution of shell commands using Linux kernel features: PID namespaces, seccomp filters, chroot jails, and cgroup resource limits. This prevents malicious or accidental damage from command execution.

Why it matters: Running arbitrary shell commands is inherently dangerous. Sandboxing contains blast radius even if the agent is compromised or makes mistakes.

16.2 Architecture Design





16.3 API Design

```
// Package: internal/sandbox
package sandbox

import (
    "context"
    "fmt"
    "time"
)

// SandboxLevel defines security isolation level
type SandboxLevel int

const (
    SandboxLight SandboxLevel = iota // chroot only
    SandboxMedium                    // chroot + PID namespace
    SandboxHeavy                     // + seccomp + user
                                   namespace
    SandboxMaximum                   // + network namespace +
                                   readonly root
)

// SandboxConfig configures the sandbox environment
type SandboxConfig struct {
    Level          SandboxLevel    `json:"level"`
    WorkingDir     string          `json:"working_dir"`

    // Filesystem
    RootFS         string          `json:"rootfs,omitempty"` // chroot root
    BindMounts     []BindMount    `json:"bind_mounts,omitempty"` // Host -> sandbox paths
    ReadOnlyPaths  []string       `json:"read_only_paths,omitempty"`
    WritablePaths   []string       `json:"writable_paths,omitempty"`
}
```

```

// Resources
MaxCPUTime      time.Duration
    `json:"max_cpu_time,omitempty"`
MaxMemory       int64
    `json:"max_memory,omitempty"`           // bytes
MaxProcesses    int
    `json:"max_processes,omitempty"`
MaxFileSize     int64
    `json:"max_file_size,omitempty"`
MaxOpenFiles    int
    `json:"max_open_files,omitempty"`

// Network
NetworkMode     string           `json:"network_mode"` //
    "none", "localhost", "full"
AllowedHosts    []string
    `json:"allowed_hosts,omitempty"`

// Environment
Environment     map[string]string
    `json:"environment,omitempty"`
Uid             int
    `json:"uid,omitempty"`           // User ID in sandbox
Gid             int
    `json:"gid,omitempty"`          // Group ID in sandbox

// Seccomp
SeccompProfile  string
    `json:"seccomp_profile,omitempty"` // "default",
    "strict", "custom"
AllowedSyscalls []string
    `json:"allowed_syscalls,omitempty"`
BlockedSyscalls []string
    `json:"blocked_syscalls,omitempty"`
}

// BindMount defines a host-to-sandbox path mapping
type BindMount struct {
    Source      string `json:"source"`
    Destination string `json:"destination"`
    ReadOnly    bool   `json:"read_only,omitempty"`
    CreateIfMissing bool `json:"create_if_missing,omitempty"`
}

// ShellRequest is a command to execute in sandbox
type ShellRequest struct {
    Command      string `json:"command"`

```

```

Arguments    []string          `json:"arguments,omitempty"`
WorkingDir   string           `json:"working_dir,omitempty"`
Environment  map[string]string         `json:"environment,omitempty"`
Timeout      time.Duration     `json:"timeout"`
}

```

// ShellResult is the outcome of sandboxed execution

```

type ShellResult struct {
    ExitCode    int          `json:"exit_code"`
    Stdout      string       `json:"stdout"`
    Stderr      string       `json:"stderr"`

    // Resource usage
    CPUTime     time.Duration `json:"cpu_time"`
    MemoryPeak  int64         `json:"memory_peak"`

    // Status
    TimedOut    bool         `json:"timed_out"`
    OOMKilled   bool         `json:"oom_killed"`
    Signaled    bool         `json:"signaled"`
    Signal      int          `json:"signal,omitempty"`

    // Error
    Error       string       `json:"error,omitempty"`
}

```

// SandboxManager manages sandboxed execution

```

type SandboxManager interface {
    // Execute runs a command in a sandbox
    Execute(ctx context.Context, req ShellRequest, config
        SandboxConfig) (*ShellResult, error)

    // PrepareSandbox creates sandbox environment
    PrepareSandbox(ctx context.Context, config SandboxConfig)
        (*SandboxInstance, error)

    // CleanupSandbox removes sandbox environment
    CleanupSandbox(ctx context.Context, instance
        *SandboxInstance) error

    // GetSandboxStatus returns sandbox health
    GetSandboxStatus(ctx context.Context, instanceID string)
        (*SandboxStatus, error)

    // ValidateConfig checks if config is feasible
    ValidateConfig(ctx context.Context, config SandboxConfig)
        []ValidationError
}

```

```

    // IsAvailable checks if sandboxing is supported on this
    system
    IsAvailable(ctx context.Context) (*AvailabilityInfo, error)
}

// SandboxInstance is a prepared sandbox environment
type SandboxInstance struct {
    ID          string    `json:"id"`
    Config      SandboxConfig `json:"config"`
    RootPath    string    `json:"root_path"`
    IsPrepared  bool      `json:"is_prepared"`
    CreatedAt   time.Time   `json:"created_at"`
}

// SandboxStatus reports sandbox health
type SandboxStatus struct {
    ID          string    `json:"id"`
    IsRunning   bool      `json:"is_running"`
    ProcessCount int       `json:"process_count"`
    MemoryUsed  int64     `json:"memory_used"`
    CPUTimeUsed time.Duration `json:"cpu_time_used"`
}

// ValidationError reports config issues
type ValidationError struct {
    Field  string `json:"field"`
    Error  string `json:"error"`
}

// AvailabilityInfo reports sandbox feature availability
type AvailabilityInfo struct {
    PIDNamespaces bool `json:"pid_namespaces"`
    UserNamespaces bool `json:"user_namespaces"`
    NetworkNamespaces bool `json:"network_namespaces"`
    Seccomp         bool `json:"seccomp"`
    cgroups         bool `json:"cgroups"`
    chroot          bool `json:"chroot"`
    AllAvailable    bool `json:"all_available"`
    Warnings        []string `json:"warnings,omitempty"`
}

```

16.4 Integration Points

Package	Modification	Description
internal/sandbox/	NEW PACKAGE	Sandbox manager
internal/tool/shell.go	Sandbox all shell exec	Route through sandbox manager

Package	Modification	Description
internal/ permissions/	Sandbox config per rule	Different levels for different tools
cmd/helix/ main.go	--sandbox-level flag	Default sandbox level
internal/ config/	Sandbox config	Viper integration

16.5 Implementation Steps

```
// internal/sandbox/manager.go
package sandbox

import (
    "bytes"
    "context"
    "fmt"
    "os"
    "os/exec"
    "path/filepath"
    "runtime"
    "strconv"
    "strings"
    "syscall"
    "time"

    "dev.helix.code/pkg/logger"
)

type DefaultSandboxManager struct {
    log logger.Logger
}

func NewDefaultSandboxManager(log logger.Logger)
    *DefaultSandboxManager {
    return &DefaultSandboxManager{log: log}
}

func (m *DefaultSandboxManager) Execute(ctx context.Context, req
    ShellRequest, config SandboxConfig) (*ShellResult,
    error) {
    // Check availability
    avail, err := m.IsAvailable(ctx)
    if err != nil {
        return nil, err
    }
}
```



```

if config.Level >= SandboxMedium && !avail.PIDNamespaces {
    return nil, fmt.Errorf("PID namespaces not available on
        this system")
}
if config.Level >= SandboxHeavy && !avail.Seccomp {
    return nil, fmt.Errorf("seccomp not available on this
        system")
}

// Prepare sandbox
instance, err := m.PrepareSandbox(ctx, config)
if err != nil {
    return nil, fmt.Errorf("preparing sandbox: %w", err)
}
defer m.CleanupSandbox(ctx, instance)

// Build command with isolation
cmd, err := m.buildSandboxedCommand(ctx, req, config,
    instance)
if err != nil {
    return nil, err
}

// Capture output
var stdout, stderr bytes.Buffer
cmd.Stdout = &stdout
cmd.Stderr = &stderr

// Start and monitor
startTime := time.Now()
if err := cmd.Start(); err != nil {
    return &ShellResult{Error: err.Error()}, nil
}

// Set up timeout
done := make(chan error, 1)
go func() {
    done <- cmd.Wait()
}()

var timedOut bool
select {
case <-ctx.Done():
    cmd.Process.Kill()
    <-done
    return &ShellResult{Error: "context cancelled",
        TimedOut: true}, ctx.Err()
case <-time.After(req.Timeout):

```

```

        cmd.Process.Kill()
        <-done
        timedOut = true
    case err := <-done:
        // Process finished
        _ = err
    }

    elapsed := time.Since(startTime)

    // Get exit code
    exitCode := 0
    oomKilled := false
    signaled := false
    signalNum := 0

    if cmd.ProcessState != nil {
        exitCode = cmd.ProcessState.ExitCode()
        if ws, ok := cmd.ProcessState.Sys().
            (syscall.WaitStatus); ok {
            oomKilled = ws.Signaled() && ws.Signal() ==
                syscall.SIGKILL
            signaled = ws.Signaled()
            if signaled {
                signalNum = int(ws.Signal())
            }
        }
    }

    return &ShellResult{
        ExitCode:    exitCode,
        Stdout:       stdout.String(),
        Stderr:       stderr.String(),
        CPUTime:      elapsed,
        TimedOut:     timedOut,
        OOMKilled:    oomKilled,
        Signaled:     signaled,
        Signal:       signalNum,
    }, nil
}

func (m *DefaultSandboxManager) buildSandboxedCommand(ctx
    context.Context, req ShellRequest, config SandboxConfig,
    instance *SandboxInstance) (*exec.Cmd, error) {
    fullCommand := req.Command
    if len(req.Arguments) > 0 {
        fullCommand += " " + strings.Join(req.Arguments, " ")
    }

```

```

switch config.Level {
case SandboxLight:
    // chroot only
    return m.buildChrootCommand(config, instance,
        fullCommand)

case SandboxMedium:
    // chroot + PID namespace using unshare
    return m.buildUnshareCommand(config, instance,
        fullCommand, []string{"--pid", "--fork"})

case SandboxHeavy:
    // unshare + seccomp using firejail or bwrap
    if m.hasFirejail() {
        return m.buildFirejailCommand(config, instance,
            fullCommand)
    }
    return m.buildUnshareCommand(config, instance,
        fullCommand, []string{
            "--pid", "--fork", "--user", "--map-root-user", "--
            mount",
        })

case SandboxMaximum:
    // Full isolation
    if m.hasFirejail() {
        return m.buildFirejailCommand(config, instance,
            fullCommand)
    }
    return m.buildBwrapCommand(config, instance, fullCommand)

default:
    // No sandbox
    cmd := exec.CommandContext(ctx, req.Command,
        req.Arguments...)
    cmd.Dir = req.WorkingDir
    cmd.Env = m.buildEnv(req.Environment)
    return cmd, nil
}
}

func (m *DefaultSandboxManager) buildChrootCommand(config
    SandboxConfig, instance *SandboxInstance, command
    string) (*exec.Cmd, error) {
    cmd := exec.Command("chroot", instance.RootPath, "/bin/sh",
        "-c", command)
    cmd.Env = m.buildEnv(config.Environment)

```

```

    return cmd, nil
}

func (m *DefaultSandboxManager) buildUnshareCommand(config
    SandboxConfig, instance *SandboxInstance, command
    string, flags []string) (*exec.Cmd, error) {
    args := append(flags, "chroot", instance.RootPath, "/bin/
        sh", "-c", command)
    cmd := exec.Command("unshare", args...)
    cmd.Env = m.buildEnv(config.Environment)

    // Set resource limits via setrlimit or cgroup
    if config.MaxMemory > 0 {
        // Write cgroup limits (if cgroup v2 available)
        m.applyCgroupLimits(cmd, config)
    }

    return cmd, nil
}

func (m *DefaultSandboxManager) buildFirejailCommand(config
    SandboxConfig, instance *SandboxInstance, command
    string) (*exec.Cmd, error) {
    args := []string{
        "--quiet",
        "--noprofile",
        "--private=" + instance.RootPath,
    }

    if config.NetworkMode == "none" {
        args = append(args, "--net=none")
    }

    if config.MaxMemory > 0 {
        args = append(args, "--rlimit-
            as="+strconv.FormatInt(config.MaxMemory, 10))
    }

    if config.MaxCPUTime > 0 {
        args = append(args, "--
            timeout="+strconv.FormatInt(int64(config.MaxCPUTime.Seconds()),
            10))
    }

    for _, path := range config.ReadOnlyPaths {
        args = append(args, "--read-only="+path)
    }
}

```

```

for _, mount := range config.BindMounts {
    if mount.ReadOnly {
        args = append(args, "--read-only="+mount.Source)
    } else {
        args = append(args, "--
bind="+mount.Source+", "+mount.Destination)
    }
}

args = append(args, "/bin/sh", "-c", command)

cmd := exec.Command("firejail", args...)
cmd.Env = m.buildEnv(config.Environment)
return cmd, nil
}

func (m *DefaultSandboxManager) buildBwrapCommand(config
SandboxConfig, instance *SandboxInstance, command
string) (*exec.Cmd, error) {
args := []string{
    "--bind", instance.RootPath, "/",
    "--dev", "/dev",
    "--proc", "/proc",
    "--tmpfs", "/tmp",
}

if config.NetworkMode == "none" {
    args = append(args, "--unshare-net")
}

for _, mount := range config.BindMounts {
    if mount.ReadOnly {
        args = append(args, "--ro-bind", mount.Source,
mount.Destination)
    } else {
        args = append(args, "--bind", mount.Source,
mount.Destination)
    }
}

for _, path := range config.ReadOnlyPaths {
    args = append(args, "--ro-bind", path, path)
}

args = append(args, "/bin/sh", "-c", command)

cmd := exec.Command("bwrap", args...)
cmd.Env = m.buildEnv(config.Environment)

```

```

    return cmd, nil
}

func (m *DefaultSandboxManager) PrepareSandbox(ctx
    context.Context, config SandboxConfig)
    (*SandboxInstance, error) {
    id := fmt.Sprintf("sandbox_%d", time.Now().UnixNano())
    rootPath := filepath.Join(os.TempDir(), "helix-sandbox", id)

    // Create root filesystem
    if err := os.MkdirAll(rootPath, 0755); err != nil {
        return nil, err
    }

    // Copy minimal system files
    if err := m.prepareRootFS(rootPath, config); err != nil {
        os.RemoveAll(rootPath)
        return nil, err
    }

    // Set up bind mounts
    for _, mount := range config.BindMounts {
        dest := filepath.Join(rootPath, mount.Destination)
        if mount.CreateIfMissing {
            os.MkdirAll(dest, 0755)
        }
    }

    return &SandboxInstance{
        ID:      id,
        Config:   config,
        RootPath: rootPath,
        IsPrepared: true,
        CreatedAt: time.Now(),
    }, nil
}

func (m *DefaultSandboxManager) prepareRootFS(rootPath string,
    config SandboxConfig) error {
    // Create minimal directory structure
    dirs := []string{"bin", "lib", "lib64", "usr", "tmp", "dev",
        "proc", "etc"}
    for _, dir := range dirs {
        if err := os.MkdirAll(filepath.Join(rootPath, dir),
            0755); err != nil {
            return err
        }
    }
}

```

```

// Copy essential binaries
essentialBins := []string{"/bin/sh", "/bin/bash", "/usr/bin/
    env"}
for _, bin := range essentialBins {
    if _, err := os.Stat(bin); err == nil {
        dest := filepath.Join(rootPath, bin)
        os.MkdirAll(filepath.Dir(dest), 0755)
        copyFile(bin, dest)
    }
}

// Copy libraries (simplified - use ldd in production)
// ...

return nil
}

func (m *DefaultSandboxManager) CleanupSandbox(ctx
    context.Context, instance *SandboxInstance) error {
    if instance.RootPath != "" {
        return os.RemoveAll(instance.RootPath)
    }
    return nil
}

func (m *DefaultSandboxManager) applyCgroupLimits(cmd *exec.Cmd,
    config SandboxConfig) {
    // Apply cgroup v2 limits
    // This requires writing to /sys/fs/cgroup/...
    // Simplified - in production, create a cgroup for the
    // process
}

func (m *DefaultSandboxManager) buildEnv(env map[string]string)
    []string {
    base := os.Environ()
    for k, v := range env {
        base = append(base, fmt.Sprintf("%s=%s", k, v))
    }
    return base
}

func (m *DefaultSandboxManager) IsAvailable(ctx context.Context)
    (*AvailabilityInfo, error) {
    info := &AvailabilityInfo{}

    // Check chroot (always available on Linux if root)

```

```

info.chroot = runtime.GOOS == "linux"

// Check unshare
_, err := exec.LookPath("unshare")
info.PIDNamespaces = err == nil

// Check user namespaces
_, err = os.Stat("/proc/self/ns/user")
info.UserNamespaces = err == nil

// Check seccomp
_, err = os.Stat("/proc/self/seccomp")
info.Seccomp = err == nil

// Check cgroups v2
_, err = os.Stat("/sys/fs/cgroup/cgroup.controllers")
info.cgroups = err == nil

// Check network namespaces
_, err = exec.LookPath("ip")
info.NetworkNamespaces = err == nil

info.AllAvailable = info.PIDNamespaces &&
    info.UserNamespaces && info.Seccomp && info.cgroups

if !info.AllAvailable {
    info.Warnings = append(info.Warnings, "Not all sandbox
    features available. Consider running with sudo or using a
    container.")
}

return info, nil
}

func (m *DefaultSandboxManager) ValidateConfig(ctx
    context.Context, config SandboxConfig) []ValidationError
{
var errors []ValidationError

if config.Level >= SandboxMedium && config.RootFS == "" {
    // RootFS can be auto-generated
}

if config.MaxMemory > 0 && config.MaxMemory < 1024*1024 {
    errors = append(errors, ValidationError{"max_memory",
        "must be at least 1MB"})
}
}

```



```

    if config.MaxCPUtime > 0 && config.MaxCPUtime < time.Second {
        errors = append(errors, ValidationError{"max_cpu_time",
            "must be at least 1 second"})
    }

    for i, mount := range config.BindMounts {
        if mount.Source == "" {
            errors = append(errors, ValidationError{
                fmt.Sprintf("bind_mounts[%d].source", i),
                "source path is required",
            })
        }
    }

    return errors
}

func (m *DefaultSandboxManager) hasFirejail() bool {
    _, err := exec.LookPath("firejail")
    return err == nil
}

func copyFile(src, dest string) error {
    data, err := os.ReadFile(src)
    if err != nil {
        return err
    }
    return os.WriteFile(dest, data, 0755)
}

// GetSandboxStatus - stub implementation
func (m *DefaultSandboxManager) GetSandboxStatus(ctx
    context.Context, instanceID string) (*SandboxStatus,
    error) {
    return &SandboxStatus{ID: instanceID}, nil
}

```

16.6 Testing Approach

```

func TestSandboxManager_Execute_Light(t *testing.T) {
    if runtime.GOOS != "linux" {
        t.Skip("Sandbox tests require Linux")
    }

    manager := NewDefaultSandboxManager(logger.NewNop())

    req := ShellRequest{
        Command: "echo",
    }

```

```

        Arguments: []string{"hello from sandbox"},
        Timeout: 5 * time.Second,
    }

    config := SandboxConfig{
        Level: SandboxLight,
    }

    result, err := manager.Execute(ctx, req, config)
    assert.NoError(t, err)
    assert.Equal(t, 0, result.ExitCode)
    assert.Contains(t, result.Stdout, "hello from sandbox")
}

func TestSandboxManager_Execute_WithTimeout(t *testing.T) {
    if runtime.GOOS != "linux" {
        t.Skip("Sandbox tests require Linux")
    }

    manager := NewDefaultSandboxManager(logger.NewNop())

    req := ShellRequest{
        Command: "sleep",
        Arguments: []string{"10"},
        Timeout: 1 * time.Second,
    }

    result, err := manager.Execute(ctx, req,
        SandboxConfig{Level: SandboxLight})

    assert.NoError(t, err)
    assert.True(t, result.TimedOut)
}

func TestSandboxManager_Availability(t *testing.T) {
    manager := NewDefaultSandboxManager(logger.NewNop())

    info, err := manager.IsAvailable(ctx)
    assert.NoError(t, err)
    assert.NotNil(t, info)
    // chroot is always available on Linux (if root) or with user
    namespaces
}

```

16.7 Edge Cases

Edge Case	Handling
Sandbox binary not available (firejail/bwrap)	

Edge Case	Handling
	Fallback to unshare, then chroot, then warning
Command needs network but sandbox blocks	Request explicit permission to enable network
File exceeds max_file_size	Truncate with notice
Process forks too many children	cgroup pids.max enforcement
chroot requires root privileges	Use user namespaces (unshare --user --map-root-user)
Sandboxed command needs specific library	Include in rootfs or bind-mount host libraries
macOS/Windows (no Linux namespaces)	Use Docker/containers as fallback
seccomp kills legitimate syscall	Profile-based whitelist, test before deployment
Nested sandboxing	Detect and prevent (one level max)
Sandbox cleanup failure	Use tmpfs for rootfs, reboot-safe cleanup

Feature 17: Theme System (JSON Themes)

17.1 Feature Description

The Theme System allows users to customize the visual appearance of the terminal UI using JSON theme definitions. Themes control colors, typography, borders, spacing, and component styles.

Why it matters: Developers spend hours looking at the terminal. Customizable themes improve comfort and accessibility.

17.2 Architecture Design

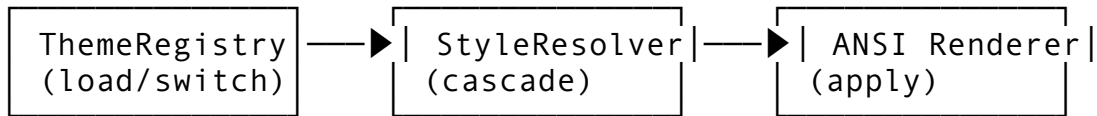
Theme System

```
Theme JSON:
{
  "name": "solarized-dark",
  "colors": {
    "background": "#002b36",
    "foreground": "#839496",
    "accent": "#268bd2",
    "success": "#859900",
    "warning": "#b58900",
    "error": "#dc322f"
  },
  "components": {
    "message_user": {"fg": "#93a1a1", "bold": true},
```

```

    "message_assistant": {"fg": "#839496"},
    "code_block": {"bg": "#073642", "fg": "#93a1a1"}
  }
}

```



17.3 API Design

```

// Package: internal/ui/theme
package theme

```

```

import (
    "context"
    "fmt"
    "os"
    "path/filepath"
    "strings"
)

```

```

// Theme is a complete UI theme definition

```

```

type Theme struct {
    Name      string          `json:"name"`
    Version   string          `json:"version,omitempty"`
    Author    string          `json:"author,omitempty"`
    Description string         `json:"description,omitempty"`
}

```

```

// Color palette

```

```

Colors      ThemeColors      `json:"colors"`

```

```

// Component-specific styles

```

```

Components  ThemeComponents  `json:"components"`

```

```

// Typography

```

```

Typography  ThemeTypography  `json:"typography,omitempty"`

```

```

// Layout

```

```

Layout      ThemeLayout    `json:"layout,omitempty"`

```

```

// Metadata

```

```

IsBuiltIn   bool              `json:"-"`

```

```

IsDark      bool              `json:"is_dark,omitempty"`

```

```

SourcePath  string            `json:"-"`

```

```

}

// ThemeColors defines the core palette
type ThemeColors struct {
    Background      string `json:"background"`
    Foreground      string `json:"foreground"`
    Accent          string `json:"accent"`
    AccentSecondary string `json:"accent_secondary,omitempty"`
    Success         string `json:"success"`
    Warning         string `json:"warning"`
    Error           string `json:"error"`
    Info            string `json:"info,omitempty"`
    Muted           string `json:"muted,omitempty"`
    Border          string `json:"border,omitempty"`
    Highlight       string `json:"highlight,omitempty"`
    Selection       string `json:"selection,omitempty"`

    // Semantic colors
    Link          string `json:"link,omitempty"`
    Code          string `json:"code,omitempty"`
    CodeBackground string `json:"code_background,omitempty"`
    DiffAdd       string `json:"diff_add,omitempty"`
    DiffRemove    string `json:"diff_remove,omitempty"`
    DiffContext   string `json:"diff_context,omitempty"`
}

// ThemeComponents defines component-specific overrides
type ThemeComponents struct {
    MessageUser      ComponentStyle `json:"message_user,omitempty"`
    MessageAssistant ComponentStyle `json:"message_assistant,omitempty"`
    MessageSystem    ComponentStyle `json:"message_system,omitempty"`
    CodeBlock        ComponentStyle `json:"code_block,omitempty"`
    InlineCode       ComponentStyle `json:"inline_code,omitempty"`
    Quote            ComponentStyle `json:"quote,omitempty"`
    Table            ComponentStyle `json:"table,omitempty"`
    Heading1         ComponentStyle `json:"heading_1,omitempty"`
    Heading2         ComponentStyle `json:"heading_2,omitempty"`
    Heading3         ComponentStyle `json:"heading_3,omitempty"`
    Spinner          ComponentStyle `json:"spinner,omitempty"`
    ProgressBar      ComponentStyle `json:"progress_bar,omitempty"`
    StatusBar        ComponentStyle `json:"status_bar,omitempty"`
    Input            ComponentStyle `json:"input,omitempty"`
    Prompt           ComponentStyle `json:"prompt,omitempty"`
}

```

```

ToolCall      ComponentStyle `json:"tool_call,omitempty"`
ToolResult    ComponentStyle
               `json:"tool_result,omitempty"`
ErrorMessage  ComponentStyle
               `json:"error_message,omitempty"`
WarningMessage ComponentStyle
               `json:"warning_message,omitempty"`
PlanStep      ComponentStyle `json:"plan_step,omitempty"`
PlanStepActive ComponentStyle
               `json:"plan_step_active,omitempty"`
PlanStepDone  ComponentStyle
               `json:"plan_step_done,omitempty"`
}

// ComponentStyle defines style for a UI component
type ComponentStyle struct {
    Foreground    string `json:"fg,omitempty"`
    Background    string `json:"bg,omitempty"`
    Bold          bool   `json:"bold,omitempty"`
    Italic        bool   `json:"italic,omitempty"`
    Underline     bool   `json:"underline,omitempty"`
    Strikethrough bool   `json:"strikethrough,omitempty"`
    Dim          bool   `json:"dim,omitempty"`
    Blink         bool   `json:"blink,omitempty"`
    BorderStyle   string `json:"border_style,omitempty"` //
    "single", "double", "rounded", "none"
    BorderColor   string `json:"border_color,omitempty"`
    Padding       int    `json:"padding,omitempty"`
    Margin        int    `json:"margin,omitempty"`
}

// ThemeTypography defines text styling
type ThemeTypography struct {
    FontFamily    string `json:"font_family,omitempty"`
    TabWidth      int    `json:"tab_width,omitempty"`
    LineHeight    float64 `json:"line_height,omitempty"`
    EnableLigatures bool   `json:"enable_ligatures,omitempty"`
}

// ThemeLayout defines spacing
type ThemeLayout struct {
    MessageSpacing int    `json:"message_spacing,omitempty"`
    CodePadding    int    `json:"code_padding,omitempty"`
    MaxContentWidth int    `json:"max_content_width,omitempty"`
    SidebarWidth   int    `json:"sidebar_width,omitempty"`
    ShowLineNumbers bool   `json:"show_line_numbers,omitempty"`
    WrapText       bool   `json:"wrap_text,omitempty"`
}

```

```

// ThemeRegistry manages themes
type ThemeRegistry interface {
    // LoadThemes discovers theme definitions
    LoadThemes(ctx context.Context, dirs []string) error

    // RegisterTheme adds a theme
    RegisterTheme(ctx context.Context, theme Theme) error

    // GetTheme retrieves a theme
    GetTheme(ctx context.Context, name string) (*Theme, error)

    // GetCurrent returns the active theme
    GetCurrent(ctx context.Context) (*Theme, error)

    // SetCurrent sets the active theme
    SetCurrent(ctx context.Context, name string) error

    // ListThemes returns available themes
    ListThemes(ctx context.Context) ([]Theme, error)

    // GetStyle resolves a component style
    GetStyle(ctx context.Context, component string)
        (*ComponentStyle, error)

    // ApplyToTerminal applies theme colors to terminal
    ApplyToTerminal(ctx context.Context, theme *Theme) error

    // ExportTheme serializes theme to JSON
    ExportTheme(ctx context.Context, name string) ([]byte, error)
}

// StyleResolver resolves effective styles with inheritance
type StyleResolver interface {
    // Resolve merges component style with base colors
    Resolve(component string, base *ComponentStyle)
        *ResolvedStyle

    // ColorToANSI converts hex color to ANSI escape sequence
    ColorToANSI(color string, isBackground bool) string

    // StyleToANSI converts a style to ANSI escapes
    StyleToANSI(style *ComponentStyle) string
}

// ResolvedStyle is a fully computed style
type ResolvedStyle struct {
    Foreground string

```

```

    Background string
    ANSI       string
    Bold       bool
    Italic     bool
    Underline  bool
    Dim        bool
}

```

17.4 Integration Points

Package	Modification	Description
internal/ui/theme/	NEW PACKAGE	Theme registry
internal/ui/render/	Style-aware rendering	Use theme colors
internal/config/	Theme config	Viper integration
cmd/helix/main.go	--theme flag	Select theme

17.5 Implementation Steps

```

// internal/ui/theme/registry.go
package theme

import (
    "context"
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "strconv"
    "strings"
)

type DefaultThemeRegistry struct {
    themes      map[string]*Theme
    current     string
}

func NewDefaultThemeRegistry() *DefaultThemeRegistry {
    r := &DefaultThemeRegistry{
        themes: make(map[string]*Theme),
    }
    r.loadBuiltins()
    return r
}

func (r *DefaultThemeRegistry) loadBuiltins() {
    builtins := []Theme{
        {

```



```

Name:          "default",
Description:    "Default HelixCode theme",
IsDark:        true,
IsBuiltIn:     true,
Colors: ThemeColors{
    Background:    "#1a1a2e",
    Foreground:    "#e0e0e0",
    Accent:        "#7c3aed",
    AccentSecondary: "#06b6d4",
    Success:       "#22c55e",
    Warning:       "#f59e0b",
    Error:         "#ef4444",
    Info:          "#3b82f6",
    Muted:         "#6b7280",
    Border:        "#374151",
    Highlight:     "#fef3c7",
    Selection:     "#3b82f680",
    Link:          "#60a5fa",
    Code:          "#e0e0e0",
    CodeBackground: "#16213e",
    DiffAdd:       "#22c55e",
    DiffRemove:    "#ef4444",
    DiffContext:   "#6b7280",
},
Components: ThemeComponents{
    MessageUser:    ComponentStyle{Foreground:
"#93a1a1", Bold: true},
    MessageAssistant: ComponentStyle{Foreground:
"#839496"},
    CodeBlock:      ComponentStyle{Background:
"#16213e", Foreground: "#e0e0e0"},
    ErrorMessage:   ComponentStyle{Foreground:
"#ef4444", Bold: true},
    ToolCall:       ComponentStyle{Foreground:
"#06b6d4", Italic: true},
    Spinner:        ComponentStyle{Foreground:
"#7c3aed"},
    StatusBar:      ComponentStyle{Background:
"#0f172a", Foreground: "#94a3b8"},
},
Typography: ThemeTypography{TabWidth: 4, LineHeight:
1.2},
Layout:          ThemeLayout{MessageSpacing: 1,
CodePadding: 2, ShowLineNumbers: true},
},
{
    Name:          "solarized-dark",
    Description:    "Solarized Dark theme",

```

```

IsDark:      true,
IsBuiltIn:   true,
Colors: ThemeColors{
    Background: "#002b36",
    Foreground: "#839496",
    Accent:     "#268bd2",
    Success:    "#859900",
    Warning:    "#b58900",
    Error:      "#dc322f",
    Muted:      "#586e75",
    CodeBackground: "#073642",
},
},
{
    Name:      "solarized-light",
    Description: "Solarized Light theme",
    IsDark:    false,
    IsBuiltIn: true,
    Colors: ThemeColors{
        Background: "#fdf6e3",
        Foreground: "#657b83",
        Accent:     "#268bd2",
        Success:    "#859900",
        Warning:    "#b58900",
        Error:      "#dc322f",
        Muted:      "#93a1a1",
        CodeBackground: "#eee8d5",
    },
},
{
    Name:      "high-contrast",
    Description: "High contrast accessibility theme",
    IsDark:    true,
    IsBuiltIn: true,
    Colors: ThemeColors{
        Background: "#000000",
        Foreground: "#ffffff",
        Accent:     "#00ffff",
        Success:    "#00ff00",
        Warning:    "#ffff00",
        Error:      "#ff0000",
        Muted:      "#808080",
        Border:     "#ffffff",
        CodeBackground: "#1a1a1a",
    },
},
}

```

```

    for i := range builtins {
        r.themes[builtins[i].Name] = &builtins[i]
    }

    r.current = "default"
}

func (r *DefaultThemeRegistry) LoadThemes(ctx context.Context,
    dirs []string) error {
    for _, dir := range dirs {
        entries, err := os.ReadDir(dir)
        if err != nil {
            continue
        }

        for _, entry := range entries {
            if entry.IsDir() || !strings.HasSuffix(entry.Name(),
                ".json") {
                continue
            }

            path := filepath.Join(dir, entry.Name())
            data, err := os.ReadFile(path)
            if err != nil {
                continue
            }

            var theme Theme
            if err := json.Unmarshal(data, &theme); err != nil {
                continue
            }

            theme.SourcePath = path
            r.themes[theme.Name] = &theme
        }
    }

    return nil
}

func (r *DefaultThemeRegistry) GetTheme(ctx context.Context,
    name string) (*Theme, error) {
    theme, ok := r.themes[name]
    if !ok {
        return nil, fmt.Errorf("theme not found: %s", name)
    }
    return theme, nil
}

```

```

func (r *DefaultThemeRegistry) SetCurrent(ctx context.Context,
    name string) error {
    if _, ok := r.themes[name]; !ok {
        return fmt.Errorf("theme not found: %s", name)
    }
    r.current = name
    return nil
}

func (r *DefaultThemeRegistry) GetCurrent(ctx context.Context)
    (*Theme, error) {
    return r.GetTheme(ctx, r.current)
}

func (r *DefaultThemeRegistry) ListThemes(ctx context.Context)
    ([]Theme, error) {
    var result []Theme
    for _, theme := range r.themes {
        result = append(result, *theme)
    }
    return result, nil
}

func (r *DefaultThemeRegistry) GetStyle(ctx context.Context,
    component string) (*ComponentStyle, error) {
    theme, err := r.GetCurrent(ctx)
    if err != nil {
        return nil, err
    }

    // Use reflection or switch to get component style
    var style ComponentStyle
    switch component {
    case "message_user":
        style = theme.Components.MessageUser
    case "message_assistant":
        style = theme.Components.MessageAssistant
    case "code_block":
        style = theme.Components.CodeBlock
    case "error_message":
        style = theme.Components.ErrorMessage
    case "tool_call":
        style = theme.Components.ToolCall
    case "spinner":
        style = theme.Components.Spinner
    case "status_bar":
        style = theme.Components.StatusBar
    }
}

```

```

default:
    // Return base style
    style = ComponentStyle{Foreground:
        theme.Colors.Foreground, Background:
        theme.Colors.Background}
}

// Inherit missing fields from base colors
if style.Foreground == "" {
    style.Foreground = theme.Colors.Foreground
}
if style.Background == "" {
    style.Background = theme.Colors.Background
}

return &style, nil
}

func (r *DefaultThemeRegistry) ApplyToTerminal(ctx
    context.Context, theme *Theme) error {
    // Set terminal colors using OSC sequences
    // This is a simplified implementation
    if theme.IsDark {
        fmt.Print("\033]11;" + theme.Colors.Background + "\007")
        fmt.Print("\033]10;" + theme.Colors.Foreground + "\007")
    }
    return nil
}

func (r *DefaultThemeRegistry) ExportTheme(ctx context.Context,
    name string) ([]byte, error) {
    theme, err := r.GetTheme(ctx, name)
    if err != nil {
        return nil, err
    }
    return json.MarshalIndent(theme, "", " ")
}

func (r *DefaultThemeRegistry) RegisterTheme(ctx
    context.Context, theme Theme) error {
    r.themes[theme.Name] = &theme
    return nil
}

// hexToANSI converts hex color to ANSI 256 or true color
// sequence
func hexToANSI(hex string, bg bool) string {
    hex = strings.TrimPrefix(hex, "#")

```

```

    if len(hex) != 6 {
        return ""
    }

    r, _ := strconv.ParseInt(hex[0:2], 16, 64)
    g, _ := strconv.ParseInt(hex[2:4], 16, 64)
    b, _ := strconv.ParseInt(hex[4:6], 16, 64)

    if bg {
        return fmt.Sprintf("\033[48;2;%d;%d;%dm", r, g, b)
    }
    return fmt.Sprintf("\033[38;2;%d;%d;%dm", r, g, b)
}

// styleToANSI converts a ComponentStyle to ANSI escape sequences
func styleToANSI(style *ComponentStyle) string {
    var codes []string

    if style.Bold {
        codes = append(codes, "1")
    }
    if style.Italic {
        codes = append(codes, "3")
    }
    if style.Underline {
        codes = append(codes, "4")
    }
    if style.Strikethrough {
        codes = append(codes, "9")
    }
    if style.Dim {
        codes = append(codes, "2")
    }
    if style.Blink {
        codes = append(codes, "5")
    }

    if style.Foreground != "" {
        codes = append(codes,
            stripANSISequences(hexToANSI(style.Foreground, false)))
    }
    if style.Background != "" {
        codes = append(codes,
            stripANSISequences(hexToANSI(style.Background, true)))
    }

    if len(codes) == 0 {
        return ""
    }

```

```

    }

    // Build escape sequence
    result := "\033["
    for i, code := range codes {
        if i > 0 {
            result += ";"
        }
        result += code
    }
    result += "m"

    return result
}

func stripANSISequences(s string) string {
    return strings.TrimPrefix(strings.TrimPrefix(s, "\033["),
        "m")
}

```

17.6 Testing Approach

```

func TestThemeRegistry_LoadAndGet(t *testing.T) {
    registry := NewDefaultThemeRegistry()

    theme, err := registry.GetTheme(ctx, "default")
    assert.NoError(t, err)
    assert.Equal(t, "#1a1a2e", theme.Colors.Background)
}

func TestThemeRegistry_GetStyle(t *testing.T) {
    registry := NewDefaultThemeRegistry()

    style, err := registry.GetStyle(ctx, "error_message")
    assert.NoError(t, err)
    assert.Equal(t, "#ef4444", style.Foreground)
    assert.True(t, style.Bold)
}

func TestThemeRegistry_ListThemes(t *testing.T) {
    registry := NewDefaultThemeRegistry()

    themes, err := registry.ListThemes(ctx)
    assert.NoError(t, err)
    assert.GreaterOrEqual(t, len(themes), 4) // Built-ins
}

func TestHexToANSI(t *testing.T) {

```

```

    assert.Equal(t, "\033[38;2;255;0;0m", hexToANSI("#ff0000",
        false))
    assert.Equal(t, "\033[48;2;0;255;0m", hexToANSI("#00ff00",
        true))
}

```

17.7 Edge Cases

Edge Case	Handling
Invalid hex color	Skip, log warning, use default
Terminal doesn’t support true color	Detect via COLORTERM, fallback to 256-color
Theme file missing required fields	Use defaults, log warning
Very long theme file	Parse anyway, size not a concern
Theme with circular references	Not applicable for flat JSON
Terminal background detection	Query with OSC 11, adapt theme if needed
Windows terminal support	Use PowerShell-compatible escapes
Theme switch mid-session	Gradual transition, redraw all components
Export corrupts special chars	JSON escapes properly
Light theme on dark terminal	Warn user, suggest auto-detect

Feature 18: AskUserQuestion with Previews

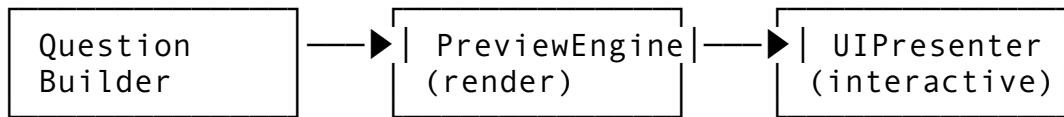
18.1 Feature Description

AskUserQuestion with Previews allows the agent to ask the user structured questions with rich content previews - showing code diffs, file trees, images, or rendered markdown before the user responds.

Why it matters: Users make better decisions when they see what the agent is asking about. Raw text questions lack context for complex choices.

18.2 Architecture Design

AskUserQuestion with Previews
Question Types: - text: Simple text input - choice: Select from options - confirm: Yes/No - multi_select: Multiple choices - file: File selection - preview: With rich content preview



Preview Types:

- diff: Unified diff of changes
- file_tree: Directory listing
- code: Syntax highlighted code
- markdown: Rendered markdown
- image: Terminal image display
- data_table: Tabular data

18.3 API Design

```
// Package: internal/ui/question
package question

import (
    "context"
    "fmt"
    "time"
)

// QuestionType defines the kind of question
type QuestionType string

const (
    QuestionText      QuestionType = "text"
    QuestionChoice    QuestionType = "choice"
    QuestionConfirm   QuestionType = "confirm"
    QuestionMultiSelect QuestionType = "multi_select"
    QuestionFile      QuestionType = "file"
    QuestionNumber    QuestionType = "number"
    QuestionPassword  QuestionType = "password"
)

// PreviewType defines the kind of preview
type PreviewType string

const (
    PreviewNone      PreviewType = "none"
    PreviewDiff      PreviewType = "diff"
    PreviewCode      PreviewType = "code"
    PreviewFileTree  PreviewType = "file_tree"
    PreviewMarkdown  PreviewType = "markdown"
    PreviewImage     PreviewType = "image"
    PreviewDataTable PreviewType = "data_table"
```

```

        PreviewJSON        PreviewType = "json"
        PreviewCustom      PreviewType = "custom"
    )

    // Question is a structured user question
    type Question struct {
        ID            string          `json:"id"`
        Type          QuestionType    `json:"type"`
        Title         string          `json:"title"`
        Description   string          `json:"description,omitempty"`

        // For choice/multi_select
        Options       []QuestionOption `json:"options,omitempty"`

        // For text/number
        Placeholder   string          `json:"placeholder,omitempty"`
        Default       any            `json:"default,omitempty"`
        Validation    string          `json:"validation,omitempty"` //
            Regex

        // Preview
        Preview       *Preview        `json:"preview,omitempty"`

        // Metadata
        Timeout       time.Duration    `json:"timeout,omitempty"`
        Required      bool            `json:"required,omitempty"`
        AllowSkip     bool            `json:"allow_skip,omitempty"`
    }

    // QuestionOption is a choice option
    type QuestionOption struct {
        ID            string          `json:"id"`
        Label         string          `json:"label"`
        Description   string          `json:"description,omitempty"`
        Value         any            `json:"value,omitempty"`
        IsDefault     bool           `json:"is_default,omitempty"`
    }

    // Preview is rich content shown with the question
    type Preview struct {
        Type          PreviewType    `json:"type"`
        Title         string        `json:"title,omitempty"`
        Content       string        `json:"content,omitempty"` // Raw
            content
        Language     string        `json:"language,omitempty"` // For
            code preview
        Data         any          `json:"data,omitempty"`    //
            Structured data
    }

```

```

Width      int      `json:"width,omitempty"`
Height     int      `json:"height,omitempty"`
Highlight  []int     `json:"highlight,omitempty"` // Line
           numbers to highlight
}

// Answer is the user's response
type Answer struct {
    QuestionID string `json:"question_id"`
    Value      any    `json:"value"`
    IsSkipped  bool   `json:"is_skipped,omitempty"`
    Timestamp  time.Time `json:"timestamp"`
}

// QuestionEngine manages user questions
type QuestionEngine interface {
    // Ask presents a question and waits for answer
    Ask(ctx context.Context, question Question) (*Answer, error)

    // AskBatch asks multiple questions
    AskBatch(ctx context.Context, questions []Question)
        ([]Answer, error)

    // Preview renders a preview
    Preview(ctx context.Context, preview Preview) (string, error)

    // ShowDiffPreview renders a diff preview
    ShowDiffPreview(ctx context.Context, oldContent, newContent,
        oldLabel, newLabel string) (string, error)

    // ShowFileTreePreview renders a file tree
    ShowFileTreePreview(ctx context.Context, files
        []FileTreeEntry) (string, error)

    // ShowCodePreview renders syntax highlighted code
    ShowCodePreview(ctx context.Context, code, language string,
        highlightLines []int) (string, error)

    // ShowDataTablePreview renders a table
    ShowDataTablePreview(ctx context.Context, headers []string,
        rows [][]string) (string, error)
}

// FileTreeEntry is a file in a tree preview
type FileTreeEntry struct {
    Path      string `json:"path"`
    Type      string `json:"type"` // "file", "directory"
    Size      int64  `json:"size,omitempty"`

```

```

    Modified string `json:"modified,omitempty"`
    Status    string `json:"status,omitempty"` // "added",
        "modified", "deleted", "unchanged"
}

```

18.4 Integration Points

Package	Modification	Description
internal/ui/question/	NEW PACKAGE	Question engine
internal/ui/	Preview rendering	Integrate with render engine
internal/permissions/	Permission prompts	Use question engine for permission UI
internal/plan/	Plan approval	Show plan preview before approval

18.5 Implementation Steps

```

// internal/ui/question/engine.go
package question

import (
    "bufio"
    "context"
    "fmt"
    "os"
    "strconv"
    "strings"
    "time"

    "dev.helix.code/internal/ui/render"
    "dev.helix.code/internal/ui/theme"
    "dev.helix.code/pkg/logger"
)

type DefaultQuestionEngine struct {
    renderer render.RenderEngine
    theme    theme.ThemeRegistry
    input    *bufio.Reader
    log      logger.Logger
}

func NewDefaultQuestionEngine(renderer
    render.RenderEngine, theme theme.ThemeRegistry, log
    logger.Logger) *DefaultQuestionEngine {
    return &DefaultQuestionEngine{
        renderer: renderer,
    }
}

```

```

        theme:      theme,
        input:      bufio.NewReader(os.Stdin),
        log:        log,
    }
}

func (e *DefaultQuestionEngine) Ask(ctx
    context.Context, question Question) (*Answer, error) {
    // Render the question
    e.renderQuestion(question)

    // Render preview if present
    if question.Preview != nil {
        previewText, err := e.Preview(ctx, *question.Preview)
        if err != nil {
            e.log.Warn("preview render error", "error", err)
        } else {
            fmt.Println(previewText)
        }
    }

    // Get answer based on type
    var value any
    var err error

    switch question.Type {
    case QuestionText:
        value, err = e.askText(question)
    case QuestionChoice:
        value, err = e.askChoice(question)
    case QuestionConfirm:
        value, err = e.askConfirm(question)
    case QuestionMultiSelect:
        value, err = e.askMultiSelect(question)
    case QuestionNumber:
        value, err = e.askNumber(question)
    case QuestionPassword:
        value, err = e.askPassword(question)
    default:
        value, err = e.askText(question)
    }

    if err != nil {
        if question.AllowSkip {
            return &Answer{QuestionID: question.ID, IsSkipped:
                true, Timestamp: time.Now()}, nil
        }
        return nil, err
    }
}

```

```

    }

    return &Answer{
        QuestionID: question.ID,
        Value:      value,
        Timestamp:  time.Now(),
    }, nil
}

func (e *DefaultQuestionEngine) renderQuestion(q Question) {
    // Apply theme
    style, _ := e.theme.GetStyle(ctx, "prompt")
    ansiStyle := styleToANSI(style)

    fmt.Printf("\n%s%s\n", ansiStyle, q.Title, "\033[0m")
    if q.Description != "" {
        fmt.Printf("  %s\n", q.Description)
    }
}

func (e *DefaultQuestionEngine) askText(question Question)
    (string, error) {
    if question.Placeholder != "" {
        fmt.Printf("  [%s]: ", question.Placeholder)
    } else {
        fmt.Print("  > ")
    }

    input, err := e.input.ReadString('\n')
    if err != nil {
        return "", err
    }

    input = strings.TrimSpace(input)

    if input == "" && question.Default != nil {
        return fmt.Sprintf("%v", question.Default), nil
    }

    if input == "" && question.Required {
        return "", fmt.Errorf("input is required")
    }

    return input, nil
}

func (e *DefaultQuestionEngine) askChoice(question Question)
    (string, error) {

```

```

for i, opt := range question.Options {
    marker := " "
    if opt.IsDefault {
        marker = "*"
    }
    fmt.Printf("  %s [%d] %s", marker, i+1, opt.Label)
    if opt.Description != "" {
        fmt.Printf(" - %s", opt.Description)
    }
    fmt.Println()
}

fmt.Print("  Select: ")
input, err := e.input.ReadString('\n')
if err != nil {
    return "", err
}

input = strings.TrimSpace(input)

// Parse selection
idx, err := strconv.Atoi(input)
if err != nil {
    // Try to match by label
    for _, opt := range question.Options {
        if strings.EqualFold(opt.Label, input) ||
            strings.EqualFold(opt.ID, input) {
            return fmt.Sprintf("%v", opt.Value), nil
        }
    }
    return "", fmt.Errorf("invalid selection")
}

if idx < 1 || idx > len(question.Options) {
    return "", fmt.Errorf("selection out of range")
}

return fmt.Sprintf("%v", question.Options[idx-1].Value), nil
}

func (e *DefaultQuestionEngine) askConfirm(question Question)
    (bool, error) {
    defaultVal := false
    if question.Default != nil {
        defaultVal = question.Default.(bool)
    }

    prompt := "  [y/n]"

```

```

    if defaultVal {
        prompt = " [Y/n]"
    } else {
        prompt = " [y/N]"
    }

    fmt.Print(prompt + " ")
    input, err := e.input.ReadString('\n')
    if err != nil {
        return defaultVal, nil
    }

    input = strings.TrimSpace(strings.ToLower(input))

    if input == "" {
        return defaultVal, nil
    }

    return input == "y" || input == "yes", nil
}

func (e *DefaultQuestionEngine) askMultiSelect(question
    Question) ([]string, error) {
    fmt.Println(" Select multiple (comma-separated numbers):")
    for i, opt := range question.Options {
        fmt.Printf(" [%d] %s\n", i+1, opt.Label)
    }

    fmt.Print(" > ")
    input, err := e.input.ReadString('\n')
    if err != nil {
        return nil, err
    }

    input = strings.TrimSpace(input)
    if input == "" {
        return nil, nil
    }

    parts := strings.Split(input, ",")
    var result []string

    for _, part := range parts {
        part = strings.TrimSpace(part)
        idx, err := strconv.Atoi(part)
        if err != nil || idx < 1 || idx > len(question.Options) {
            continue
        }
    }

```



```

        result = append(result, fmt.Sprintf("%v",
            question.Options[idx-1].Value))
    }

    return result, nil
}

func (e *DefaultQuestionEngine) askNumber(question Question)
    (float64, error) {
    fmt.Print(" > ")
    input, err := e.input.ReadString('\n')
    if err != nil {
        return 0, err
    }

    input = strings.TrimSpace(input)
    if input == "" && question.Default != nil {
        if n, ok := question.Default.(float64); ok {
            return n, nil
        }
        if n, ok := question.Default.(int); ok {
            return float64(n), nil
        }
    }

    return strconv.ParseFloat(input, 64)
}

func (e *DefaultQuestionEngine) askPassword(question Question)
    (string, error) {
    fmt.Print(" [hidden] > ")
    // On Unix, disable terminal echo
    // Simplified - use term.ReadPassword in production
    input, err := e.input.ReadString('\n')
    if err != nil {
        return "", err
    }
    return strings.TrimSpace(input), nil
}

func (e *DefaultQuestionEngine) Preview(ctx context.Context,
    preview Preview) (string, error) {
    switch preview.Type {
    case PreviewDiff:
        return e.ShowDiffPreview(ctx, "", preview.Content,
            "old", "new")
    case PreviewCode:

```

```

        return e.ShowCodePreview(ctx, preview.Content,
            preview.Language, preview.Highlight)
    case PreviewFileTree:
        var entries []FileTreeEntry
        if preview.Data != nil {
            // Parse data into entries
        }
        return e.ShowFileTreePreview(ctx, entries)
    case PreviewDataTable:
        // Parse data into headers/rows
        return e.ShowDataTablePreview(ctx, nil, nil)
    case PreviewMarkdown:
        return preview.Content, nil // Terminal markdown
            rendering is complex
    default:
        return preview.Content, nil
    }
}

func (e *DefaultQuestionEngine) ShowDiffPreview(ctx
    context.Context, oldContent, newContent, oldLabel,
    newLabel string) (string, error) {
    var b strings.Builder

    // Simple unified diff rendering
    b.WriteString(fmt.Sprintf("--- %s\n", oldLabel))
    b.WriteString(fmt.Sprintf("+++ %s\n", newLabel))
    b.WriteString("\n")

    if newContent != "" {
        lines := strings.Split(newContent, "\n")
        for _, line := range lines {
            if strings.HasPrefix(line, "+") {
                b.WriteString(fmt.Sprintf("\033[32m%s\033[0m\n",
                    line))
            } else if strings.HasPrefix(line, "-") {
                b.WriteString(fmt.Sprintf("\033[31m%s\033[0m\n",
                    line))
            } else if strings.HasPrefix(line, "@") {
                b.WriteString(fmt.Sprintf("\033[36m%s\033[0m\n",
                    line))
            } else {
                b.WriteString(line + "\n")
            }
        }
    }

    return b.String(), nil
}

```

```

}

func (e *DefaultQuestionEngine) ShowFileTreePreview(ctx
    context.Context, files []FileTreeEntry) (string, error) {
    var b strings.Builder

    for _, file := range files {
        var prefix, color string
        switch file.Status {
        case "added":
            prefix, color = "+", "\033[32m"
        case "deleted":
            prefix, color = "-", "\033[31m"
        case "modified":
            prefix, color = "~", "\033[33m"
        default:
            prefix, color = " ", ""
        }

        icon := "📄"
        if file.Type == "directory" {
            icon = "📁"
        }

        b.WriteString(fmt.Sprintf("%s%s %s %s\033[0m\n", color,
            prefix, icon, file.Path))
    }

    return b.String(), nil
}

func (e *DefaultQuestionEngine) ShowCodePreview(ctx
    context.Context, code, language string, highlightLines
    []int) (string, error) {
    var b strings.Builder

    // Simple code block with line numbers
    lines := strings.Split(code, "\n")
    highlightSet := make(map[int]bool)
    for _, l := range highlightLines {
        highlightSet[l] = true
    }

    for i, line := range lines {
        lineNum := i + 1
        prefix := fmt.Sprintf("%4d | ", lineNum)

        if highlightSet[lineNum] {

```

```

        b.WriteString(fmt.Sprintf("\033[7m%s%\033[0m\n",
prefix, line))
    } else {
        b.WriteString(fmt.Sprintf("%s%\n", prefix, line))
    }
}

return b.String(), nil
}

func (e *DefaultQuestionEngine) ShowDataTablePreview(ctx
context.Context, headers []string, rows [][]string)
(string, error) {
var b strings.Builder

// Simple table rendering
colWidths := make([]int, len(headers))
for i, h := range headers {
    colWidths[i] = len(h)
}

for _, row := range rows {
    for i, cell := range row {
        if i < len(colWidths) && len(cell) > colWidths[i] {
            colWidths[i] = len(cell)
        }
    }
}

// Print header
for i, h := range headers {
    b.WriteString(fmt.Sprintf("| %-*s ", colWidths[i], h))
}
b.WriteString("| \n")

// Print separator
for _, w := range colWidths {
    b.WriteString(fmt.Sprintf("|%s", strings.Repeat("-",
w+2)))
}
b.WriteString("| \n")

// Print rows
for _, row := range rows {
    for i, cell := range row {
        if i < len(colWidths) {
            b.WriteString(fmt.Sprintf("| %-*s ",
colWidths[i], cell))

```

```

        }
    }
    b.WriteString("|\\n")
}

return b.String(), nil
}

func (e *DefaultQuestionEngine) AskBatch(ctx context.Context,
    questions []Question) ([]Answer, error) {
    var answers []Answer

    for _, q := range questions {
        answer, err := e.Ask(ctx, q)
        if err != nil {
            return answers, err
        }
        answers = append(answers, *answer)
    }

    return answers, nil
}

```

18.6 Testing Approach

```

func TestQuestionEngine_AskText(t *testing.T) {
    // Mock input
    input := strings.NewReader("test answer\\n")
    engine := &DefaultQuestionEngine{input:
        bufio.NewReader(input)}

    answer, err := engine.Ask(ctx, Question{
        ID:      "q1",
        Type:    QuestionText,
        Title:   "What is your name?",
    })

    assert.NoError(t, err)
    assert.Equal(t, "test answer", answer.Value)
}

func TestQuestionEngine_AskChoice(t *testing.T) {
    input := strings.NewReader("2\\n")
    engine := &DefaultQuestionEngine{input:
        bufio.NewReader(input)}

    answer, err := engine.Ask(ctx, Question{
        ID:      "q1",

```

```

        Type: QuestionChoice,
        Title: "Select an option",
        Options: []QuestionOption{
            {ID: "a", Label: "Option A", Value: "a"},
            {ID: "b", Label: "Option B", Value: "b"},
        },
    })
})

assert.NoError(t, err)
assert.Equal(t, "b", answer.Value)
}

func TestQuestionEngine_AskConfirm(t *testing.T) {
    input := strings.NewReader("y\n")
    engine := &DefaultQuestionEngine{input:
        bufio.NewReader(input)}

    answer, err := engine.Ask(ctx, Question{
        ID: "q1",
        Type: QuestionConfirm,
        Title: "Continue?",
    })

    assert.NoError(t, err)
    assert.Equal(t, true, answer.Value)
}

func TestQuestionEngine_ShowDiffPreview(t *testing.T) {
    engine := NewDefaultQuestionEngine(nil, nil, logger.NewNop())

    preview, err := engine.ShowDiffPreview(ctx, "",
        "+added\n-removed", "old", "new")
    assert.NoError(t, err)
    assert.Contains(t, preview, "added")
    assert.Contains(t, preview, "removed")
}

```

18.7 Edge Cases

Edge Case	Handling
User interrupts (Ctrl+C)	Catch signal, return error, cleanup terminal
Timeout waiting for answer	Return default if set, or error
Very long preview	Paginate with “show more” prompt
Terminal too narrow for table	Truncate columns, show ellipsis
Binary data in preview	Skip, show “[binary content]” placeholder
Unicode in options	Support full UTF-8

Edge Case	Handling
Empty options list	Error during validation
Default value type mismatch	Coerce or error
Nested questions	Flatten, or track depth (max 3)
Preview renders too slowly	Async render, show spinner

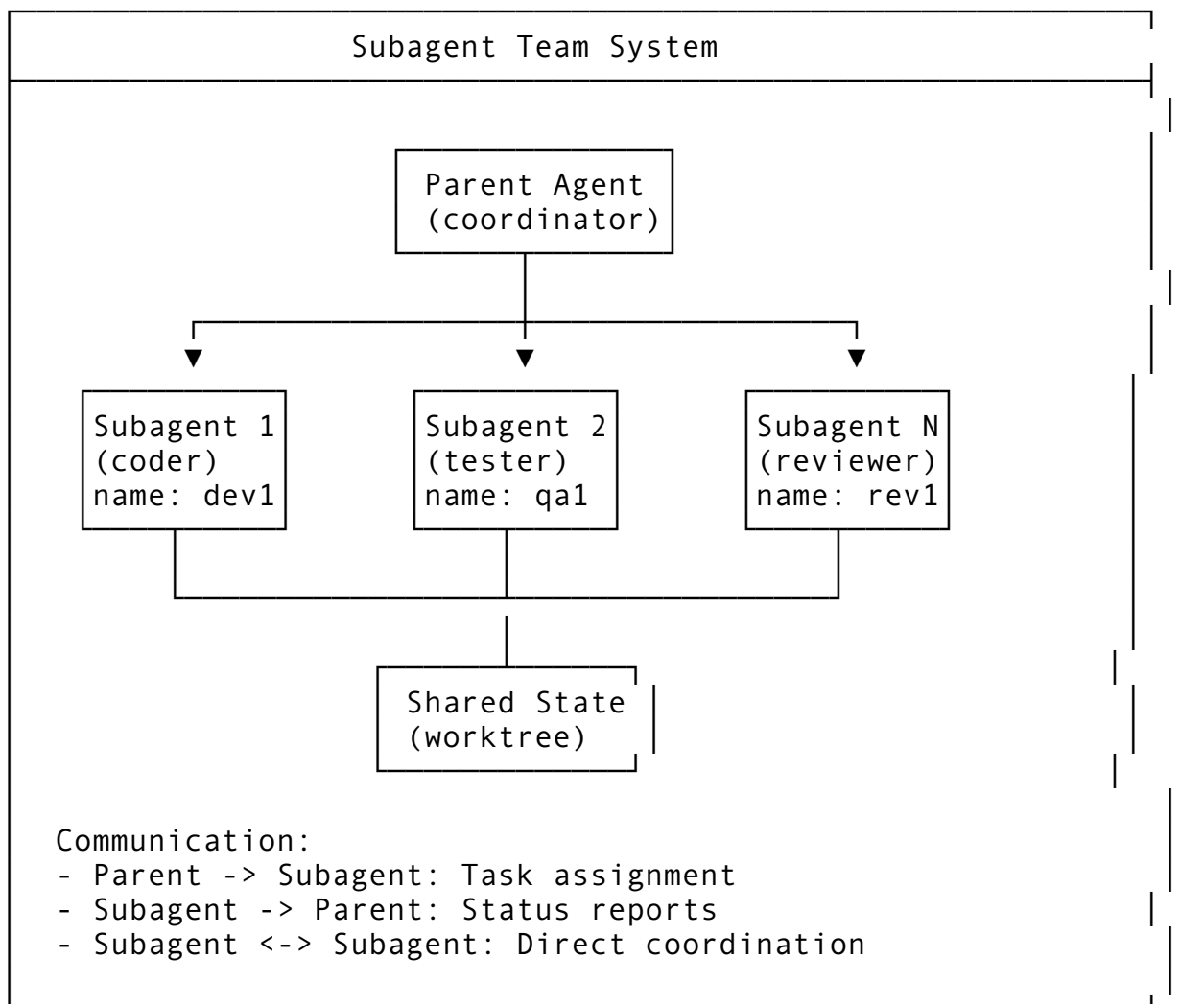
Feature 19: Subagent Team (Named, Addressable Agents)

19.1 Feature Description

The Subagent Team system allows spawning specialized, named agents that can work in parallel on subtasks. Each subagent is addressable and can communicate with the parent agent and other subagents.

Why it matters: Complex tasks benefit from specialization. A “tester” agent can write tests while a “documenter” agent writes docs, both working from the same codebase.

19.2 Architecture Design



19.3 API Design

```
// Package: internal/actor/subagent
package subagent

import (
    "context"
    "fmt"
    "time"
)

// AgentType defines the specialization
type AgentType string

const (
    AgentCoder      AgentType = "coder"
    AgentTester     AgentType = "tester"
    AgentReviewer   AgentType = "reviewer"
    AgentArchitect  AgentType = "architect"
    AgentDocumenter AgentType = "documenter"
    AgentDebugger   AgentType = "debugger"
    AgentOptimizer  AgentType = "optimizer"
    AgentSecurity   AgentType = "security"
)

// Subagent is a child agent instance
type Subagent struct {
    ID          string          `json:"id" db:"id"`
    Name        string          `json:"name" db:"name"`
    // User-defined or auto
    AgentType   AgentType       `json:"agent_type" db:"agent_type"`
    ParentID    string          `json:"parent_id" db:"parent_id"`
    SessionID   string          `json:"session_id" db:"session_id"`

    // Configuration
    Model        string          `json:"model" db:"model"`
    Temperature  float64         `json:"temperature" db:"temperature"`
    SystemPrompt string          `json:"system_prompt" db:"system_prompt"`
    MaxTokens    int             `json:"max_tokens" db:"max_tokens"`

    // State
    Status       SubagentStatus `json:"status" db:"status"`,
}
```



```

CurrentTask    *string
               `json:"current_task,omitempty" db:"current_task"`

// Context
Context        map[string]any    `json:"context,omitempty"
               db:"context"`
Workspace      string            `json:"workspace"
               db:"workspace"`

// Timing
CreatedAt      time.Time         `json:"created_at"
               db:"created_at"`
StartedAt      *time.Time         `json:"started_at,omitempty"
               db:"started_at"`
CompletedAt    *time.Time
               `json:"completed_at,omitempty" db:"completed_at"`

// Results
Results        []TaskResult      `json:"results,omitempty"
               db:"results"`
Messages       []SubagentMessage `json:"messages,omitempty"
               db:"messages"`
}

// SubagentStatus tracks lifecycle
type SubagentStatus string

const (
    SubagentPending    SubagentStatus = "pending"
    SubagentRunning    SubagentStatus = "running"
    SubagentPaused     SubagentStatus = "paused"
    SubagentCompleted  SubagentStatus = "completed"
    SubagentFailed     SubagentStatus = "failed"
    SubagentCancelled  SubagentStatus = "cancelled"
)

// SubagentMessage is communication between agents
type SubagentMessage struct {
    ID          string    `json:"id"`
    From        string    `json:"from"`
    To          string    `json:"to"`
    Type        string    `json:"type"` // "task", "status",
               "result", "question", "broadcast"
    Content     string    `json:"content"`
    Payload     map[string]any `json:"payload,omitempty"`
    Timestamp   time.Time `json:"timestamp"`
}

```

```

// TaskAssignment is work given to a subagent
type TaskAssignment struct {
    ID            string          `json:"id"`
    Description    string          `json:"description"`
    Goal          string          `json:"goal"`
    Files         []string        `json:"files,omitempty"`
    Constraints    []string        `json:"constraints,omitempty"`
    Deadline      *time.Duration `json:"deadline,omitempty"`
    Priority       int            `json:"priority"`
    DependsOn     []string        `json:"depends_on,omitempty"` //
    Task IDs
}

// TaskResult is work output from a subagent
type TaskResult struct {
    TaskID        string          `json:"task_id"`
    Success       bool           `json:"success"`
    Summary       string          `json:"summary"`
    FilesChanged  []string        `json:"files_changed,omitempty"`
    Output        string          `json:"output,omitempty"`
    Error         string          `json:"error,omitempty"`
}

// SubagentManager manages subagent lifecycle
type SubagentManager interface {
    // Spawn creates a new subagent
    Spawn(ctx context.Context, req SpawnRequest) (*Subagent,
        error)

    // AssignTask gives work to a subagent
    AssignTask(ctx context.Context, subagentID string, task
        TaskAssignment) error

    // SendMessage sends a message to a subagent
    SendMessage(ctx context.Context, from, to string, msg
        SubagentMessage) error

    // Broadcast sends to all subagents
    Broadcast(ctx context.Context, from string, msg
        SubagentMessage) error

    // GetSubagent retrieves subagent status
    GetSubagent(ctx context.Context, subagentID string)
        (*Subagent, error)

    // ListSubagents returns active subagents
    ListSubagents(ctx context.Context, parentID string)
        ([]Subagent, error)

```

```

// WaitFor waits for subagent to complete
WaitFor(ctx context.Context, subagentID string) (*Subagent,
    error)

// Terminate stops a subagent
Terminate(ctx context.Context, subagentID string, reason
    string) error

// GetResults collects all subagent results
GetResults(ctx context.Context, parentID string)
    ([]TaskResult, error)

// SubscribeToMessages subscribes to subagent messages
SubscribeToMessages(ctx context.Context, subagentID string)
    (<-chan SubagentMessage, error)
}

// SpawnRequest parameters for creating a subagent
type SpawnRequest struct {
    Name            string          `json:"name"`
    AgentType       AgentType       `json:"agent_type"`
    ParentID        string          `json:"parent_id"`
    SessionID       string          `json:"session_id"`
    Workspace       string          `json:"workspace"`

    // Optional overrides
    Model           string          `json:"model,omitempty"`
    Temperature     *float64       `json:"temperature,omitempty"`
    SystemPrompt    string          `json:"system_prompt,omitempty"`

    // Shared context
    ContextFiles    []string        `json:"context_files,omitempty"`
    Instructions    string          `json:"instructions,omitempty"`
}

```

19.4 Integration Points

Package	Modification	Description
internal/ actor/ subagent/	NEW PACKAGE	Subagent manager
internal/ actor/	Parent-child relationship	Track agent hierarchy
internal/llm/	Subagent LLM calls	Each subagent gets its own model config
	Shared workspace	

Package	Modification	Description
internal/git/worktree/		Subagents share or branch from parent worktree
internal/db/migrations/	Add subagents table	Track subagent state

19.5 Implementation Steps

```
// internal/actor/subagent/manager.go
package subagent

import (
    "context"
    "fmt"
    "time"

    "dev.helix.code/internal/actor"
    "dev.helix.code/internal/db"
    "dev.helix.code/internal/git/worktree"
    "dev.helix.code/internal/llm"
    "dev.helix.code/pkg/logger"
)

type DefaultSubagentManager struct {
    store          db.Store
    llmClient      llm.Client
    worktreeMgr    worktree.WorktreeManager
    actorSystem    actor.System
    log            logger.Logger
    subagents      map[string]*Subagent
    messageBus     map[string][]chan SubagentMessage
}

func NewDefaultSubagentManager(store db.Store, llmClient llm.Client, worktreeMgr worktree.WorktreeManager, actorSystem actor.System, log logger.Logger) *DefaultSubagentManager {
    return &DefaultSubagentManager{
        store:          store,
        llmClient:      llmClient,
        worktreeMgr:    worktreeMgr,
        actorSystem:    actorSystem,
        log:            log,
        subagents:      make(map[string]*Subagent),
        messageBus:     make(map[string][]chan SubagentMessage),
    }
}
```

```

func (m *DefaultSubagentManager) Spawn(ctx context.Context, req
    SpawnRequest) (*Subagent, error) {
    // Generate ID and name
    id := fmt.Sprintf("sub_%s_%d", req.AgentType,
        time.Now().UnixNano())
    name := req.Name
    if name == "" {
        name = fmt.Sprintf("%s-%d", req.AgentType,
            len(m.subagents)+1)
    }

    // Create worktree for subagent (branches from parent)
    var workspace string
    if m.worktreeMgr != nil {
        wt, err := m.worktreeMgr.CreateWorktree(ctx,
            worktree.CreateWorktreeRequest{
                SessionID: req.SessionID,
                AgentID:    id,
                AgentType: string(req.AgentType),
                RepoPath:  req.Workspace,
                NewBranch: fmt.Sprintf("helix/subagent/%s", name),
            })
        if err != nil {
            m.log.Warn("worktree creation failed, using shared
                workspace", "error", err)
            workspace = req.Workspace
        } else {
            workspace = wt.WorktreePath
        }
    } else {
        workspace = req.Workspace
    }

    // Build system prompt based on agent type
    systemPrompt := req.SystemPrompt
    if systemPrompt == "" {
        systemPrompt = m.defaultSystemPrompt(req.AgentType)
    }

    if req.Instructions != "" {
        systemPrompt += "\n\nAdditional instructions:\n" +
            req.Instructions
    }

    // Create subagent
    subagent := &Subagent{
        ID:        id,
        Name:      name,

```

```

        AgentType:    req.AgentType,
        ParentID:     req.ParentID,
        SessionID:    req.SessionID,
        Model:        req.Model,
        SystemPrompt: systemPrompt,
        Status:       SubagentPending,
        Workspace:    workspace,
        Context:      make(map[string]any),
        CreatedAt:    time.Now(),
    }

    if req.Temperature != nil {
        subagent.Temperature = *req.Temperature
    } else {
        subagent.Temperature = 0.7
    }

    // Store
    if err := m.store.CreateSubagent(ctx, subagent); err != nil {
        return nil, fmt.Errorf("storing subagent: %w", err)
    }

    m.subagents[id] = subagent

    m.log.Info("subagent spawned",
        "id", id,
        "name", name,
        "type", req.AgentType,
        "parent", req.ParentID,
    )

    // Start the subagent actor
    go m.runSubagent(ctx, subagent)

    return subagent, nil
}

func (m *DefaultSubagentManager) runSubagent(ctx context.Context, subagent *Subagent) {
    now := time.Now()
    subagent.StartedAt = &now
    subagent.Status = SubagentRunning
    m.store.UpdateSubagent(ctx, subagent)

    // Create actor in the actor system
    // This creates a session actor with the subagent's
    // configuration
    // The actor listens for messages and tasks

```

```

// Main loop: process messages and tasks
msgChan := m.messageBus[subagent.ID]

for {
    select {
    case <-ctx.Done():
        subagent.Status = SubagentCancelled
        goto done

    case msg := <-msgChan:
        if msg == nil {
            continue
        }

        switch msg.Type {
        case "task":
            m.handleTask(ctx, subagent, msg)
        case "question":
            m.handleQuestion(ctx, subagent, msg)
        case "terminate":
            subagent.Status = SubagentCancelled
            goto done
        }

        // Store message
        subagent.Messages = append(subagent.Messages, msg)
    }

    if subagent.Status == SubagentCompleted ||
        subagent.Status == SubagentFailed {
        break
    }
}

done:
    completedAt := time.Now()
    subagent.CompletedAt = &completedAt
    m.store.UpdateSubagent(ctx, subagent)

    m.log.Info("subagent completed", "id", subagent.ID,
        "status", subagent.Status)
}

func (m *DefaultSubagentManager) handleTask(ctx context.Context,
    subagent *Subagent, msg SubagentMessage) {
    // Extract task from message
    var task TaskAssignment

```

```

if payload, ok := msg.Payload["task"]; ok {
    // Parse task from payload
    task = payload.(TaskAssignment)
}

subagent.CurrentTask = &task.ID

// Build prompt for the task
prompt := fmt.Sprintf("You are a %s specialist. Your task:
    \n\n%s\n\nGoal: %s",
        subagent.AgentType, task.Description, task.Goal)

if len(task.Files) > 0 {
    prompt += fmt.Sprintf("\n\nRelevant files:\n%s",
        strings.Join(task.Files, "\n"))
}

if len(task.Constraints) > 0 {
    prompt += fmt.Sprintf("\n\nConstraints:\n%s",
        strings.Join(task.Constraints, "\n"))
}

// Call LLM
response, err := m.llmClient.Complete(ctx,
    llm.CompletionRequest{
        Model:          subagent.Model,
        SystemPrompt:   subagent.SystemPrompt,
        Prompt:         prompt,
        Temperature:    subagent.Temperature,
        MaxTokens:      subagent.MaxTokens,
    })

result := TaskResult{
    TaskID:  task.ID,
    Success: err == nil,
}

if err != nil {
    result.Error = err.Error()
    subagent.Status = SubagentFailed
} else {
    result.Summary = response.Text
    result.Output = response.Text
    // Parse for file changes from response
    result.FilesChanged = m.extractFileChanges(response.Text)
}

subagent.Results = append(subagent.Results, result)

```



```

subagent.CurrentTask = nil

// Notify parent
m.SendMessage(ctx, subagent.ID, subagent.ParentID,
    SubagentMessage{
        Type: "result",
        Content: fmt.Sprintf("Task %s completed", task.ID),
        Payload: map[string]any{"result": result},
    })
}

func (m *DefaultSubagentManager) extractFileChanges(text string)
    []string {
    // Parse response for file references
    var files []string
    lines := strings.Split(text, "\n")
    for _, line := range lines {
        if strings.Contains(line, ".go") ||
            strings.Contains(line, ".js") || strings.Contains(line,
                ".py") {
            // Extract filename
            parts := strings.Fields(line)
            for _, part := range parts {
                if strings.Contains(part, ".") && !
                    strings.Contains(part, "http") {
                    files = append(files, strings.Trim(part,
                        "`'*\\"))
                }
            }
        }
    }
    return files
}

func (m *DefaultSubagentManager) handleQuestion(ctx
    context.Context, subagent *Subagent, msg
    SubagentMessage) {
    // Forward question to parent
    m.SendMessage(ctx, subagent.ID, subagent.ParentID, msg)
}

func (m *DefaultSubagentManager) AssignTask(ctx context.Context,
    subagentID string, task TaskAssignment) error {
    subagent, ok := m.subagents[subagentID]
    if !ok {
        return fmt.Errorf("subagent not found: %s", subagentID)
    }
}

```

```

    if subagent.Status != SubagentRunning {
        return fmt.Errorf("subagent not running: %s",
            subagent.Status)
    }

    return m.SendMessage(ctx, subagent.ParentID, subagentID,
        SubagentMessage{
            Type:      "task",
            Content: task.Description,
            Payload: map[string]any{"task": task},
        })
}

func (m *DefaultSubagentManager) SendMessage(ctx
    context.Context, from, to string, msg SubagentMessage)
    error {
    msg.From = from
    msg.To = to
    msg.Timestamp = time.Now()

    // Route to recipient
    if chans, ok := m.messageBus[to]; ok {
        for _, ch := range chans {
            select {
            case ch <- msg:
            default:
                // Channel full, log and skip
            }
        }
    }

    return nil
}

func (m *DefaultSubagentManager) Broadcast(ctx context.Context,
    from string, msg SubagentMessage) error {
    msg.From = from
    msg.To = "*"
    msg.Timestamp = time.Now()

    for id, chans := range m.messageBus {
        if id == from {
            continue // Don't echo to sender
        }
        for _, ch := range chans {
            select {
            case ch <- msg:
            default:

```

```

        }
    }
}

return nil
}

func (m *DefaultSubagentManager) GetSubagent(ctx
    context.Context, subagentID string) (*Subagent, error) {
    subagent, ok := m.subagents[subagentID]
    if !ok {
        return nil, fmt.Errorf("subagent not found: %s",
            subagentID)
    }
    return subagent, nil
}

func (m *DefaultSubagentManager) ListSubagents(ctx
    context.Context, parentID string) ([]Subagent, error) {
    var result []Subagent
    for _, subagent := range m.subagents {
        if subagent.ParentID == parentID {
            result = append(result, *subagent)
        }
    }
    return result, nil
}

func (m *DefaultSubagentManager) WaitFor(ctx context.Context,
    subagentID string) (*Subagent, error) {
    subagent, err := m.GetSubagent(ctx, subagentID)
    if err != nil {
        return nil, err
    }

    // Poll until complete
    for subagent.Status == SubagentPending || subagent.Status ==
        SubagentRunning || subagent.Status == SubagentPaused {
        time.Sleep(100 * time.Millisecond)
        subagent, _ = m.GetSubagent(ctx, subagentID)
    }

    return subagent, nil
}

func (m *DefaultSubagentManager) Terminate(ctx context.Context,
    subagentID, reason string) error {
    subagent, ok := m.subagents[subagentID]

```

```

    if !ok {
        return fmt.Errorf("subagent not found: %s", subagentID)
    }

    m.SendMessage(ctx, "system", subagentID, SubagentMessage{
        Type: "terminate",
        Content: reason,
    })

    subagent.Status = SubagentCancelled
    m.store.UpdateSubagent(ctx, subagent)

    // Cleanup worktree
    if m.worktreeMgr != nil {
        // m.worktreeMgr.RemoveWorktree(ctx, subagent.Worktree,
        // false)
    }

    return nil
}

func (m *DefaultSubagentManager) GetResults(ctx context.Context,
    parentID string) ([]TaskResult, error) {
    subagents, err := m.ListSubagents(ctx, parentID)
    if err != nil {
        return nil, err
    }

    var results []TaskResult
    for _, subagent := range subagents {
        results = append(results, subagent.Results...)
    }

    return results, nil
}

func (m *DefaultSubagentManager) SubscribeToMessages(ctx
    context.Context, subagentID string) (<-chan
    SubagentMessage, error) {
    ch := make(<-chan SubagentMessage, 100)
    m.messageBus[subagentID] = append(m.messageBus[subagentID],
        ch)

    // Cleanup on context done
    go func() {
        <-ctx.Done()
        m.closeMessageChannel(subagentID, ch)
    }()
}

```

```
    return ch, nil
}
```

```
func (m *DefaultSubagentManager) closeMessageChannel(subagentID
    string, ch chan SubagentMessage) {
    chans := m.messageBus[subagentID]
    for i, c := range chans {
        if c == ch {
            m.messageBus[subagentID] = append(chans[:i],
                chans[i+1:]...)
            break
        }
    }
    close(ch)
}
```

```
func (m *DefaultSubagentManager) defaultSystemPrompt(agentType
    AgentType) string {
    switch agentType {
    case AgentCoder:
        return "You are a skilled software developer. Write
            clean, well-tested, idiomatic code. Follow best practices
            and include comments where needed."
    case AgentTester:
        return "You are a QA engineer. Write comprehensive tests
            covering happy paths, edge cases, and error conditions.
            Use table-driven tests where appropriate."
    case AgentReviewer:
        return "You are a code reviewer. Analyze code for
            correctness, security, performance, and maintainability.
            Provide constructive feedback with specific suggestions."
    case AgentArchitect:
        return
            "You are a software architect. Design systems that are
            scalable, maintainable, and well-documented. Consider
            trade-offs and document decisions."
    case AgentDocumenter:
        return "You are a technical writer. Create clear,
            accurate documentation. Use examples and keep language
            accessible."
    case AgentDebugger:
        return "You are a debugging specialist. Identify root
            causes, not symptoms. Provide minimal reproducible
            examples and step-by-step fixes."
    case AgentOptimizer:
```

```

        return
        "You are a performance engineer. Profile, measure, and
        optimize. Focus on algorithmic improvements before micro-
        optimizations."
    case AgentSecurity:
        return "You are a security engineer. Identify
        vulnerabilities, follow OWASP guidelines, and suggest
        secure alternatives."
    default:
        return "You are a helpful AI assistant."
}
}

func (m *DefaultSubagentManager) closeMessageChannel(subagentID
    string, ch chan SubagentMessage) {
    chans := m.messageBus[subagentID]
    for i, c := range chans {
        if c == ch {
            m.messageBus[subagentID] = append(chans[:i],
                chans[i+1:]...)
            break
        }
    }
    close(ch)
}

```

19.6 Testing Approach

```

func TestSubagentManager_Spawn(t *testing.T) {
    manager := setupTestManager()

    subagent, err := manager.Spawn(ctx, SpawnRequest{
        Name:      "test-coder",
        AgentType: AgentCoder,
        ParentID:   "parent-1",
        SessionID:  "session-1",
        Workspace:  "/tmp/test",
    })

    assert.NoError(t, err)
    assert.NotEmpty(t, subagent.ID)
    assert.Equal(t, "test-coder", subagent.Name)
    assert.Equal(t, AgentCoder, subagent.AgentType)
    assert.Equal(t, SubagentRunning, subagent.Status)
}

func TestSubagentManager_AssignTask(t *testing.T) {
    manager := setupTestManager()

```

```

    subagent, _ := manager.Spawn(ctx, SpawnRequest{
        AgentType: AgentCoder,
        ParentID:  "parent-1",
    })

    err := manager.AssignTask(ctx, subagent.ID, TaskAssignment{
        ID:          "task-1",
        Description: "Write a hello function",
        Goal:        "Create a simple greeting function",
    })

    assert.NoError(t, err)
}

```

```

func TestSubagentManager_Broadcast(t *testing.T) {
    manager := setupTestManager()

    sub1, _ := manager.Spawn(ctx, SpawnRequest{AgentType:
        AgentCoder, ParentID: "parent-1"})
    sub2, _ := manager.Spawn(ctx, SpawnRequest{AgentType:
        AgentTester, ParentID: "parent-1"})

    // Subscribe to messages
    ch1, _ := manager.SubscribeToMessages(ctx, sub1.ID)
    ch2, _ := manager.SubscribeToMessages(ctx, sub2.ID)

    err := manager.Broadcast(ctx, "parent-1", SubagentMessage{
        Type:    "status",
        Content: "All hands meeting",
    })

    assert.NoError(t, err)

    // Both should receive
    msg1 := <-ch1
    assert.Equal(t, "status", msg1.Type)

    msg2 := <-ch2
    assert.Equal(t, "status", msg2.Type)
}

```

```

func TestSubagentManager_Terminate(t *testing.T) {
    manager := setupTestManager()

    subagent, _ := manager.Spawn(ctx, SpawnRequest{AgentType:
        AgentCoder, ParentID: "parent-1"})
}

```

```
err := manager.Terminate(ctx, subagent.ID, "task complete")
assert.NoError(t, err)

// Verify terminated
updated, _ := manager.GetSubagent(ctx, subagent.ID)
assert.Equal(t, SubagentCancelled, updated.Status)
}
```

19.7 Edge Cases

Edge Case	Handling
Subagent exceeds token limit	Compact context, summarize parent instructions
Subagent stuck in loop	Timeout, parent intervention, task redefinition
Conflicting changes from multiple subagents	Git merge conflict resolution, manual review
Subagent asks for clarification	Forward to parent, pause until answered
Parent terminates while subagents running	Graceful shutdown with save state
Subagent worktree conflicts	Use separate branches, merge at completion
Circular task dependencies	Detect and break with error
Subagent count exceeds limit	Queue, warn, suggest batching
LLM call fails for subagent	Retry with fallback model, report to parent
Subagent produces invalid output	Validate, retry with stricter prompt

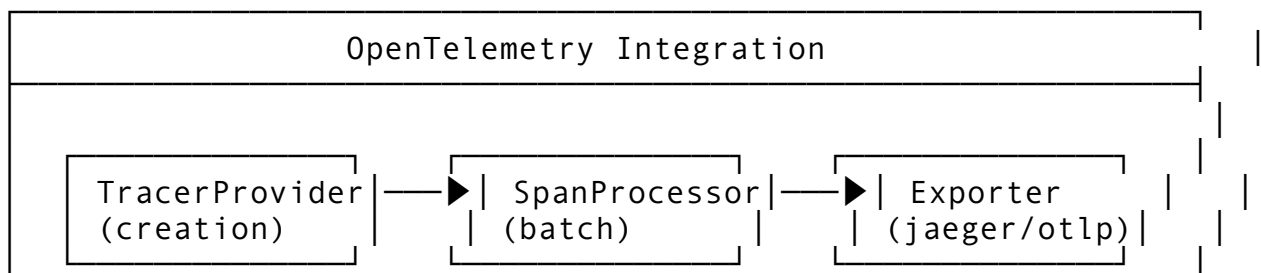
Feature 20: OpenTelemetry Integration

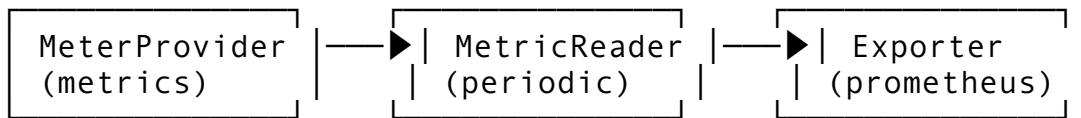
20.1 Feature Description

OpenTelemetry Integration provides distributed tracing, metrics, and logging for all agent operations. This enables observability, performance analysis, and debugging of the agent system.

Why it matters: Production deployments need visibility into what’s happening. OTel provides vendor-neutral observability that works with any backend (Jaeger, Zipkin, Datadog, etc.).

20.2 Architecture Design





Instrumented Components:

- LLM calls (latency, tokens, errors)
- Tool execution (duration, success rate)
- Session lifecycle
- Actor message processing
- Database queries
- Background tasks

20.3 API Design

```
// Package: internal/telemetry
package telemetry

import (
    "context"
    "time"

    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/attribute"
    "go.opentelemetry.io/otel/metric"
    "go.opentelemetry.io/otel/trace"
)

// TelemetryConfig configures observability
type TelemetryConfig struct {
    Enabled          bool          `mapstructure:"enabled"`
    ServiceName      string        `mapstructure:"service_name"`
    ServiceVersion   string        `mapstructure:"service_version"`

    // Tracing
    TracingEnabled   bool          `mapstructure:"tracing_enabled"`
    TracingEndpoint string        `mapstructure:"tracing_endpoint" // OTLP endpoint
    TracingSampleRate float64       `mapstructure:"tracing_sample_rate"`

    // Metrics
    MetricsEnabled   bool          `mapstructure:"metrics_enabled"`
```

```

MetricsEndpoint    string
    `mapstructure:"metrics_endpoint"`
MetricsInterval    time.Duration
    `mapstructure:"metrics_interval"`

// Logging
LogsEnabled        bool
    `mapstructure:"logs_enabled"`
LogsEndpoint       string
    `mapstructure:"logs_endpoint"`

// Attributes
ResourceAttributes map[string]string
    `mapstructure:"resource_attributes"`
}

// TelemetryManager manages OTel providers
type TelemetryManager interface {
    // Initialize sets up tracer and meter providers
    Initialize(ctx context.Context) error

    // Shutdown flushes and cleans up
    Shutdown(ctx context.Context) error

    // Tracer returns a tracer for a component
    Tracer(name string) trace.Tracer

    // Meter returns a meter for a component
    Meter(name string) metric.Meter

    // StartSpan starts a new span
    StartSpan(ctx context.Context, name string,
        opts ...trace.SpanStartOption) (context.Context,
        trace.Span)

    // RecordLLMCall records an LLM interaction
    RecordLLMCall(ctx context.Context, provider, model string,
        inputTokens, outputTokens int, duration time.Duration,
        err error)

    // RecordToolCall records a tool execution
    RecordToolCall(ctx context.Context, toolName
        string, duration time.Duration, success bool, err error)

    // RecordSessionEvent records session lifecycle event
    RecordSessionEvent(ctx context.Context, event string,
        sessionID string, attrs map[string]string)

```

```

    // RecordMetric records a custom metric
    RecordMetric(ctx context.Context, name string, value
        float64, attrs map[string]string)
}

// LLMCallSpan attributes
const (
    AttrProvider      = attribute.Key("llm.provider")
    AttrModel         = attribute.Key("llm.model")
    AttrInputTokens   = attribute.Key("llm.input_tokens")
    AttrOutputTokens  = attribute.Key("llm.output_tokens")
    AttrLatency       = attribute.Key("llm.latency_ms")
)

// ToolCallSpan attributes
const (
    AttrToolName      = attribute.Key("tool.name")
    AttrToolDuration  = attribute.Key("tool.duration_ms")
    AttrToolSuccess   = attribute.Key("tool.success")
)

// SessionSpan attributes
const (
    AttrSessionID     = attribute.Key("session.id")
    AttrAgentID       = attribute.Key("agent.id")
    AttrAgentType     = attribute.Key("agent.type")
    AttrWorkspace     = attribute.Key("workspace")
)

```

20.4 Integration Points

Package	Modification	Description
internal/telemetry/	NEW PACKAGE	Telemetry manager
internal/llm/	Instrument calls	Wrap with spans
internal/tool/	Instrument execution	Record tool metrics
internal/actor/	Message tracing	Trace actor interactions
internal/db/	Query tracing	Record DB query latency
cmd/helix/main.go	--telemetry flags	Configure endpoints

20.5 Implementation Steps

```

// internal/telemetry/manager.go
package telemetry

import (
    "context"

```

```

    "fmt"
    "time"

    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/attribute"
    "go.opentelemetry.io/otel/exporters/otlp/otlptrace/
        otlptracegrpc"
    "go.opentelemetry.io/otel/metric"
    "go.opentelemetry.io/otel/sdk/resource"
    sdktrace "go.opentelemetry.io/otel/sdk/trace"
    "go.opentelemetry.io/otel/trace"
    "google.golang.org/grpc"
)

type DefaultTelemetryManager struct {
    config          *TelemetryConfig
    tracerProvider  *sdktrace.TracerProvider
    meterProvider   metric.MeterProvider
    tracers         map[string]trace.Tracer
    meters          map[string]metric.Meter

    // Counters
    llmCallCounter    metric.Int64Counter
    toolCallCounter   metric.Int64Counter
    sessionCounter    metric.Int64Counter
    errorCounter      metric.Int64Counter

    // Histograms
    llmLatencyHistogram    metric.Float64Histogram
    toolLatencyHistogram   metric.Float64Histogram
    tokenHistogram         metric.Int64Histogram
}

func NewDefaultTelemetryManager(config *TelemetryConfig)
    *DefaultTelemetryManager {
    return &DefaultTelemetryManager{
        config:  config,
        tracers: make(map[string]trace.Tracer),
        meters:  make(map[string]metric.Meter),
    }
}

func (m *DefaultTelemetryManager) Initialize(ctx
    context.Context) error {
    if !m.config.Enabled {
        return nil
    }
}

```

```

// Build resource
res, err := resource.Merge(
    resource.Default(),
    resource.NewWithAttributes(
        "",
        attribute.String("service.name",
            m.config.ServiceName),
        attribute.String("service.version",
            m.config.ServiceVersion),
    ),
)
if err != nil {
    return err
}

// Add custom attributes
for k, v := range m.config.ResourceAttributes {
    res, _ = resource.Merge(res,
        resource.NewWithAttributes("", attribute.String(k, v)))
}

// Setup tracing
if m.config.TracingEnabled {
    var traceExporter sdktrace.SpanExporter

    if m.config.TracingEndpoint != "" {
        conn, err := grpc.DialContext(ctx,
            m.config.TracingEndpoint, grpc.WithInsecure())
        if err != nil {
            return fmt.Errorf("connecting to trace endpoint:
                %w", err)
        }

        traceExporter, err = otlptracegrpc.New(ctx,
            otlptracegrpc.WithGRPCConn(conn))
        if err != nil {
            return fmt.Errorf("creating trace exporter: %w",
                err)
        }
    }

    m.tracerProvider = sdktrace.NewTracerProvider(
        sdktrace.WithResource(res),

        sdktrace.WithSampler(sdktrace.TraceIDRatioBased(m.config.TracingSa
            sdktrace.WithBatcher(traceExporter),
    )

```

```

    otel.SetTracerProvider(m.tracerProvider)
}

// Setup metrics
if m.config.MetricsEnabled {
    // Create meter provider
    // ... similar pattern with metric exporter

    // Create instruments
    meter := otel.Meter(m.config.ServiceName)

    m.llmCallCounter, _ = meter.Int64Counter("llm.calls",
        metric.WithDescription("Number of LLM calls"))
    m.toolCallCounter, _ = meter.Int64Counter("tool.calls",
        metric.WithDescription("Number of tool calls"))
    m.sessionCounter, _ = meter.Int64Counter("sessions",
        metric.WithDescription("Number of sessions"))
    m.errorCounter, _ = meter.Int64Counter("errors",
        metric.WithDescription("Number of errors"))

    m.llmLatencyHistogram, _ =
        meter.Float64Histogram("llm.latency",
            metric.WithDescription("LLM call latency in ms"))
    m.toolLatencyHistogram, _ =
        meter.Float64Histogram("tool.latency",
            metric.WithDescription("Tool execution latency in ms"))
    m.tokenHistogram, _ = meter.Int64Histogram("llm.tokens",
        metric.WithDescription("Token usage per call"))
}

return nil
}

func (m *DefaultTelemetryManager) Shutdown(ctx context.Context)
    error {
    if m.tracerProvider != nil {
        if err := m.tracerProvider.Shutdown(ctx); err != nil {
            return err
        }
    }
    // Shutdown meter provider
    return nil
}

func (m *DefaultTelemetryManager) Tracer(name string)
    trace.Tracer {
    if tracer, ok := m.tracers[name]; ok {
        return tracer
    }
}

```

```

    }
    tracer := otel.Tracer(name)
    m.tracers[name] = tracer
    return tracer
}

func (m *DefaultTelemetryManager) StartSpan(ctx context.Context,
    name string, opts ...trace.SpanStartOption)
    (context.Context, trace.Span) {
    tracer := m.Tracer("helixcode")
    return tracer.Start(ctx, name, opts...)
}

func (m *DefaultTelemetryManager) RecordLLMCall(ctx
    context.Context, provider, model string, inputTokens,
    outputTokens int, duration time.Duration, err error) {
    if !m.config.Enabled {
        return
    }

    // Record span
    ctx, span := m.StartSpan(ctx, "llm.call",
        trace.WithAttributes(
            AttrProvider.String(provider),
            AttrModel.String(model),
            AttrInputTokens.Int(inputTokens),
            AttrOutputTokens.Int(outputTokens),
            AttrLatency.Int64(int64(duration.Milliseconds()))),
    ),
    )
    defer span.End()

    if err != nil {
        span.RecordError(err)
        span.SetStatus(trace.Status{Code: trace.StatusCodeError,
            Description: err.Error()})
    }

    // Record metrics
    if m.config.MetricsEnabled {
        attrs := metric.WithAttributes(
            attribute.String("provider", provider),
            attribute.String("model", model),
        )
        m.llmCallCounter.Add(ctx, 1, attrs)
        m.llmLatencyHistogram.Record(ctx,
            float64(duration.Milliseconds()), attrs)
    }
}

```

```

        m.tokenHistogram.Record(ctx,
            int64(inputTokens+outputTokens), attrs)

        if err != nil {
            m.errorCounter.Add(ctx, 1,
                metric.WithAttributes(attribute.String("type", "llm")))
        }
    }
}

func (m *DefaultTelemetryManager) RecordToolCall(ctx
    context.Context, toolName string, duration
    time.Duration, success bool, err error) {
    if !m.config.Enabled {
        return
    }

    ctx, span := m.StartSpan(ctx, "tool.call",
        trace.WithAttributes(
            AttrToolName.String(toolName),

            AttrToolDuration.Int64(int64(duration.Milliseconds())),
            AttrToolSuccess.Bool(success),
        ),
    )
    defer span.End()

    if err != nil {
        span.RecordError(err)
    }

    if m.config.MetricsEnabled {
        attrs := metric.WithAttributes(
            attribute.String("tool", toolName),
            attribute.Bool("success", success),
        )
        m.toolCallCounter.Add(ctx, 1, attrs)
        m.toolLatencyHistogram.Record(ctx,
            float64(duration.Milliseconds()), attrs)

        if err != nil {
            m.errorCounter.Add(ctx, 1,
                metric.WithAttributes(attribute.String("type", "tool")))
        }
    }
}

```



```

func (m *DefaultTelemetryManager) RecordSessionEvent(ctx
    context.Context, event string, sessionID string, attrs
    map[string]string) {
    if !m.config.Enabled {
        return
    }

    var spanAttrs []attribute.KeyValue
    spanAttrs = append(spanAttrs,
        AttrSessionID.String(sessionID))
    for k, v := range attrs {
        spanAttrs = append(spanAttrs, attribute.String(k, v))
    }

    _, span := m.StartSpan(ctx, "session."+event,
        trace.WithAttributes(spanAttrs...))
    defer span.End()

    if m.config.MetricsEnabled {
        m.sessionCounter.Add(ctx, 1,
            metric.WithAttributes(attribute.String("event", event)))
    }
}

func (m *DefaultTelemetryManager) RecordMetric(ctx
    context.Context, name string, value float64, attrs
    map[string]string) {
    if !m.config.MetricsEnabled {
        return
    }

    // Record as a gauge or custom metric
    // Implementation depends on specific metric type
}

```

20.6 Testing Approach

```

func TestTelemetryManager_RecordLLMCall(t *testing.T) {
    manager := NewDefaultTelemetryManager(&TelemetryConfig{
        Enabled:      true,
        ServiceName:  "test",
        TracingEnabled: false,
        MetricsEnabled: false,
    })

    manager.Initialize(ctx)

    // Should not panic even with no exporters
}

```

```

manager.RecordLLMCall(ctx, "openai", "gpt-4", 100, 50,
    500*time.Millisecond, nil)
manager.RecordLLMCall(ctx, "anthropic", "claude", 200, 100,
    1*time.Second, fmt.Errorf("rate limited"))
}

func TestTelemetryManager_StartSpan(t *testing.T) {
    manager := NewDefaultTelemetryManager(&TelemetryConfig{
        Enabled:      true,
        ServiceName:   "test",
        TracingEnabled: true,
        TracingSampleRate: 1.0,
    })

    manager.Initialize(ctx)
    defer manager.Shutdown(ctx)

    ctx, span := manager.StartSpan(ctx, "test.operation",
        trace.WithAttributes(attribute.String("test", "value")),
    )

    span.AddEvent("checkpoint")
    span.End()

    assert.NotNil(t, ctx)
}

```

20.7 Edge Cases

Edge Case	Handling
OTel collector unreachable	Buffer spans, drop after max size, log warning
Very high throughput	Batch span processor, configurable queue size
PII in spans	Redact sensitive fields, use attribute filtering
Circular trace references	Span depth limit, cycle detection
Memory pressure from spans	Limit in-flight spans, force flush on pressure
Different OTel versions	Pin versions, test compatibility
Cross-service tracing	Propagate trace context via baggage
Trace sampling misses errors	Always sample on error paths
Metrics cardinality explosion	Limit label values, aggregate high-cardinality
Export timeout on shutdown	Configurable timeout, background flush

Edge Case	Handling
Invalid span attributes	Validate types, skip invalid

PART 2: AIDER FEATURE PORTING GUIDES

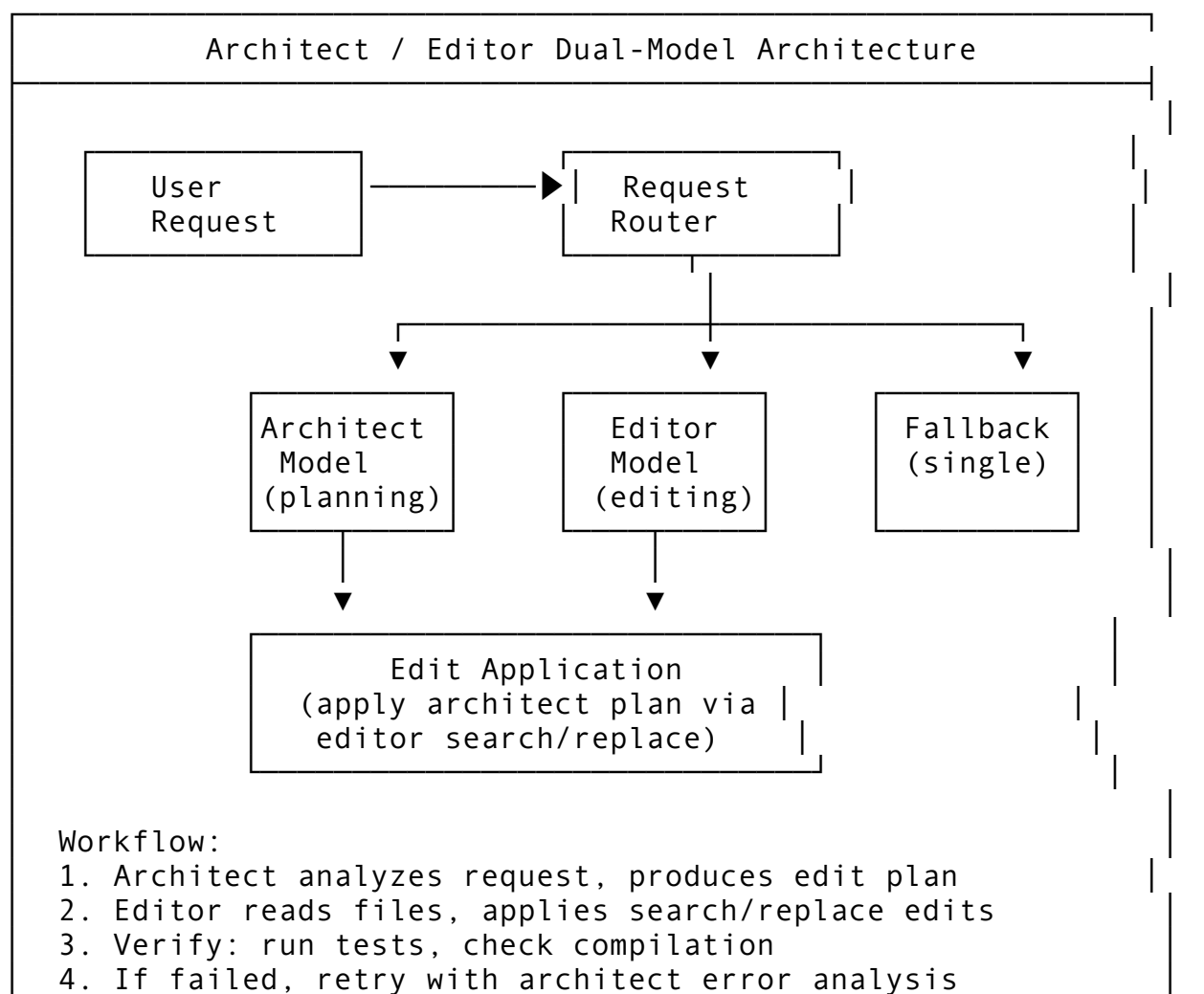
Feature 21: Architect/Editor Dual-Model Architecture

21.1 Feature Description

The Architect/Editor Dual-Model Architecture splits reasoning and implementation between two models: an Architect model plans changes at a high level, and an Editor model applies precise file edits. This separation leverages the strengths of different model sizes/costs.

Why it matters: Large reasoning models (Opus, o1) are expensive but excellent at planning. Smaller models (Haiku, 4o-mini) are cheap and fast at mechanical edits. Splitting the work optimizes cost and quality.

21.2 Architecture Design



21.3 API Design

```
// Package: internal/dualmodel
package dualmodel

import (
    "context"
    "fmt"
    "time"

    "dev.helix.code/internal/edit"
    "dev.helix.code/internal/llm"
)

// ModelRole defines which model plays which role
type ModelRole string

const (
    RoleArchitect ModelRole = "architect"
    RoleEditor     ModelRole = "editor"
    RoleFallback   ModelRole = "fallback"
)

// ModelConfig configures a model for a role
type ModelConfig struct {
    Provider      string `json:"provider"`
    Model         string `json:"model"`
    Temperature   float64 `json:"temperature"`
    MaxTokens     int    `json:"max_tokens"`
    Role          ModelRole `json:"role"`
}

// EditPlan is the architect's output
type EditPlan struct {
    FilesToRead      []string `json:"files_to_read"`
    FilesToCreate    []NewFile
    `json:"files_to_create,omitempty"`
    FilesToModify    []FileEdit `json:"files_to_modify"`
    FilesToDelete    []string
    `json:"files_to_delete,omitempty"`
    Reasoning        string `json:"reasoning"`
    Dependencies     []string `json:"dependencies,omitempty"`
}

// NewFile is a file creation instruction
type NewFile struct {
```

```

    Path      string `json:"path"`
    Description string `json:"description"`
    Content    string `json:"content,omitempty"`
}

// FileEdit is a modification instruction
type FileEdit struct {
    Path      string `json:"path"`
    OriginalText string `json:"original_text"`
    NewText    string `json:"new_text"`
    Context    string `json:"context,omitempty"` //
        Surrounding context
    LineStart  int    `json:"line_start,omitempty"`
    LineEnd    int    `json:"line_end,omitempty"`
}

// DualModelEngine orchestrates architect/editor
type DualModelEngine interface {
    // Execute runs the dual-model workflow
    Execute(ctx context.Context, request string, files []string)
        (*DualModelResult, error)

    // Plan uses the architect to create an edit plan
    Plan(ctx context.Context, request string, files []string)
        (*EditPlan, error)

    // Edit uses the editor to apply a plan
    Edit(ctx context.Context, plan *EditPlan) (*EditResult,
        error)

    // Verify checks if edits are correct
    Verify(ctx context.Context, edits []edit.EditResult)
        (*VerificationResult, error)

    // Retry executes a retry cycle on failure
    Retry(ctx context.Context, plan *EditPlan, errors []string)
        (*EditPlan, error)

    // SetModels configures which models to use
    SetModels(architect, editor ModelConfig) error
}

// DualModelResult is the outcome
type DualModelResult struct {
    Success      bool    `json:"success"`
    EditResults  []edit.EditResult `json:"edit_results"`
    ArchitectPlan *EditPlan    `json:"architect_plan"`
    Verification *VerificationResult `json:"verification"`
}

```

```

    RetryCount      int           `json:"retry_count"`
    TotalDuration   time.Duration `json:"total_duration"`
    Error           string        `json:"error,omitempty"`
}

// VerificationResult checks edit correctness
type VerificationResult struct {
    Success      bool    `json:"success"`
    TestsPassed  bool    `json:"tests_passed"`
    Compiles     bool    `json:"compiles"`
    LintErrors   []string `json:"lint_errors,omitempty"`
    TestOutput   string  `json:"test_output,omitempty"`
    Issues       []string `json:"issues,omitempty"`
}

```

21.4 Integration Points

Package	Modification	Description
internal/dualmodel/	NEW PACKAGE	Dual-model engine
internal/llm/	Multi-model support	Route to different models
internal/edit/	Plan application	Apply architect plans
internal/actor/	Actor variant	Dual-model agent type

21.5 Implementation Steps

```

// internal/dualmodel/engine.go
package dualmodel

import (
    "context"
    "fmt"
    "strings"
    "time"

    "dev.helix.code/internal/edit"
    "dev.helix.code/internal/llm"
    "dev.helix.code/pkg/logger"
)

type DefaultDualModelEngine struct {
    architectClient llm.Client
    editorClient    llm.Client
    editor          edit.SmartEditor
    log             logger.Logger
    maxRetries      int
}

```

```

func NewDefaultDualModelEngine(architectClient, editorClient
    llm.Client, editor edit.SmartEditor, log logger.Logger)
    *DefaultDualModelEngine {
    return &DefaultDualModelEngine{
        architectClient: architectClient,
        editorClient:     editorClient,
        editor:           editor,
        log:              log,
        maxRetries:       3,
    }
}

```

```

func (e *DefaultDualModelEngine) Execute(ctx context.Context,
    request string, files []string) (*DualModelResult,
    error) {
    start := time.Now()
    result := &DualModelResult{}

    // Phase 1: Plan
    plan, err := e.Plan(ctx, request, files)
    if err != nil {
        return nil, fmt.Errorf("planning failed: %w", err)
    }
    result.ArchitectPlan = plan

    // Phase 2: Edit (with retries)
    var editResults []edit.EditResult
    var verification *VerificationResult

    for attempt := 0; attempt <= e.maxRetries; attempt++ {
        result.RetryCount = attempt

        // Apply edits
        editResults, err = e.Edit(ctx, plan)
        if err != nil {
            result.Error = err.Error()
            continue
        }
        result.EditResults = editResults

        // Phase 3: Verify
        verification, err = e.Verify(ctx, editResults)
        if err != nil {
            result.Error = err.Error()
            continue
        }
        result.Verification = verification
    }
}

```

```

    if verification.Success {
        result.Success = true
        break
    }

    // Retry with error feedback
    if attempt < e.maxRetries {
        var errors []string
        errors = append(errors, verification.Issues...)
        errors = append(errors, verification.LintErrors...)

        plan, err = e.Retry(ctx, plan, errors)
        if err != nil {
            result.Error = fmt.Sprintf("retry failed: %w",
err)
            break
        }
        result.ArchitectPlan = plan
    }
}

result.TotalDuration = time.Since(start)

if !result.Success {
    result.Error = fmt.Sprintf("failed after %d attempts:
%s", result.RetryCount+1, result.Error)
}

return result, nil
}

func (e *DefaultDualModelEngine) Plan(ctx context.Context,
    request string, files []string) (*EditPlan, error) {
    // Read files for context
    var fileContents []string
    for _, f := range files {
        content, err := e.editor.ReadFile(ctx, f, nil, nil)
        if err != nil {
            continue
        }
        fileContents = append(fileContents, fmt.Sprintf("=== %s
===\n%s\n", f, content.Content))
    }

    // Build architect prompt
    prompt := fmt.Sprintf(`You are a senior software architect.
Analyze the user's request and create a precise edit
plan.
```


User request: %s

Files to consider:
%s

Create an edit plan with:

1. Which files to read for additional context
2. Which files to create (with full content)
3. Which files to modify (with exact search/replace blocks)
4. Any files to delete

For each file modification, provide:

- The exact original text to find (must be unique in file)
- The exact replacement text
- Sufficient surrounding context (3+ lines before and after)

Format as JSON with this structure:

```
{
  "files_to_read": [...],
  "files_to_create": [{"path": "...", "description": "...",
    "content": "..."}],
  "files_to_modify": [{"path": "...", "original_text": "...",
    "new_text": "..."}],
  "files_to_delete": [...],
  "reasoning": "..."
}`, request, strings.Join(fileContents, "\n"))
```

```
response, err := e.architectClient.Complete(ctx,
  llm.CompletionRequest{
    Model:      "claude-opus-4", // Use strong reasoning
    model
    Prompt:     prompt,
    MaxTokens:  8000,
    Temperature: 0.1,
  })
if err != nil {
  return nil, fmt.Errorf("architect call: %w", err)
}

// Parse JSON plan
plan, err := e.parsePlanJSON(response.Text)
if err != nil {
  return nil, fmt.Errorf("parsing plan: %w", err)
}

return plan, nil
}
```

```

func (e *DefaultDualModelEngine) Edit(ctx context.Context, plan
    *EditPlan) ([]edit.EditResult, error) {
    var results []edit.EditResult

    // Create new files
    for _, nf := range plan.FilesToCreate {
        result, err := e.editor.EditFile(ctx, edit.EditRequest{
            FilePath:      nf.Path,
            EditType:      edit.EditCreateFile,
            NewText:      nf.Content,
            CreateIfMissing: true,
        })
        if err != nil {
            return results, fmt.Errorf("creating %s: %w",
                nf.Path, err)
        }
        results = append(results, *result)
    }

    // Modify files
    for _, fe := range plan.FilesToModify {
        // First verify the original text exists
        content, err := e.editor.ReadFile(ctx, fe.Path, nil, nil)
        if err != nil {
            return results, fmt.Errorf("reading %s: %w",
                fe.Path, err)
        }

        if !strings.Contains(content.Content, fe.OriginalText) {
            return results, fmt.Errorf("original text not found
                in %s", fe.Path)
        }

        result, err := e.editor.EditFile(ctx, edit.EditRequest{
            FilePath:      fe.Path,
            EditType:      edit.EditSearchReplace,
            SearchText:    fe.OriginalText,
            ReplaceText:    fe.NewText,
        })
        if err != nil {
            return results, fmt.Errorf("editing %s: %w",
                fe.Path, err)
        }
        results = append(results, *result)
    }

    // Delete files

```

```

    for _, path := range plan.FilesToDelete {
        os.Remove(path) // Simplified
    }

    return results, nil
}

func (e *DefaultDualModelEngine) Verify(ctx context.Context,
    edits []edit.EditResult) (*VerificationResult, error) {
    result := &VerificationResult{Success: true, TestsPassed:
        true, Compiles: true}

    // Try to compile/test
    // Run appropriate command based on project type
    // Simplified - in production detect project type and run
        correct command

    return result, nil
}

func (e *DefaultDualModelEngine) Retry(ctx context.Context, plan
    *EditPlan, errors []string) (*EditPlan, error) {
    // Feed errors back to architect
    prompt := fmt.Sprintf(`The previous edit plan had errors.
        Please revise.

Original plan reasoning: %s

Errors encountered:
%s

Please provide a corrected edit plan following the same JSON
    format.`),
        plan.Reasoning, strings.Join(errors, "\n"))

    response, err := e.architectClient.Complete(ctx,
        llm.CompletionRequest{
            Model: "claude-opus-4",
            Prompt: prompt,
            MaxTokens: 8000,
            Temperature: 0.1,
        })
    if err != nil {
        return nil, err
    }

    return e.parsePlanJSON(response.Text)
}

```

```

func (e *DefaultDualModelEngine) parsePlanJSON(text string)
    (*EditPlan, error) {
    // Extract JSON from response
    // ... implementation
    return &EditPlan{}, nil
}

func (e *DefaultDualModelEngine) SetModels(architect, editor
    ModelConfig) error {
    // Reconfigure clients
    return nil
}

```

21.6 Testing Approach

```

func TestDualModelEngine_PlanAndEdit(t *testing.T) {
    architectMock := new(MockLLMClient)
    editorMock := new(MockLLMClient)

    // Architect returns a plan
    architectMock.On("Complete", mock.Anything,
        mock.Anything).Return(&llm.CompletionResponse{
        Text: `{"files_to_modify":
        [{"path":"test.go","original_text":"func old()
        {}","new_text":"func new() {}}"}`,
    }, nil)

    engine := NewDefaultDualModelEngine(architectMock,
        editorMock, mockEditor, logger.NewNop())

    result, err := engine.Execute(ctx, "Refactor the old
        function", []string{"test.go"})

    assert.NoError(t, err)
    assert.NotNil(t, result.ArchitectPlan)
}

```

Feature 22: 4-Layer Fuzzy Matching for Search/Replace

22.1 Feature Description

The 4-Layer Fuzzy Matching system provides progressively relaxed matching for search/replace operations: exact, whitespace-normalized, line-level, and block-level. This dramatically reduces edit failures from minor formatting differences.

Why it matters: LLM-generated search text often doesn't match exactly due to whitespace, comments, or line wrapping differences. Fuzzy matching dramatically increases edit success rates.

22.2 Architecture Design

4-Layer Fuzzy Matching for Search/Replace
Layer 1: Exact Match - Byte-for-byte comparison - Fastest, most reliable
Layer 2: Whitespace-Normalized - Collapse multiple spaces/tabs - Trim leading/trailing whitespace per line - Normalize line endings (CRLF -> LF)
Layer 3: Line-Level Fuzzy - Match lines independently with Levenshtein - Allow < 10% character difference per line - Reject if any line differs > 30%
Layer 4: Block-Level Fuzzy - Tokenize both blocks - Use diff algorithm with weighted scoring - Match semantic structure over exact text

22.3 API Design

```
// Package: internal/edit/fuzzy
package fuzzy

import (
    "context"
    "fmt"
    "strings"
)

// MatchLevel indicates which layer succeeded
type MatchLevel int

const (
    MatchNone MatchLevel = iota
    MatchExact
    MatchWhitespaceNormalized
    MatchLineLevel
    MatchBlockLevel
)
```

```

)

// MatchResult reports fuzzy match outcome
type MatchResult struct {
    Found      bool      `json:"found"`
    Level      MatchLevel `json:"level"`
    StartIndex int      `json:"start_index"`
    EndIndex   int      `json:"end_index"`
    Confidence  float64   `json:"confidence"` // 0.0-1.0
    OriginalMatch string    `json:"original_match,omitempty"` //
        What was actually found
}

// FuzzyMatcher implements 4-layer matching
type FuzzyMatcher interface {
    // Find searches for text using all 4 layers
    Find(content, search string) (*MatchResult, error)

    // FindExact attempts exact match only
    FindExact(content, search string) (*MatchResult, error)

    // FindWhitespaceNormalized attempts layer 2
    FindWhitespaceNormalized(content, search string)
        (*MatchResult, error)

    // FindLineLevel attempts layer 3
    FindLineLevel(content, search string) (*MatchResult, error)

    // FindBlockLevel attempts layer 4
    FindBlockLevel(content, search string) (*MatchResult, error)

    // SetThresholds configures acceptance thresholds
    SetThresholds(thresholds MatchThresholds)
}

// MatchThresholds configures fuzzy matching parameters
type MatchThresholds struct {
    LineSimilarity float64 `json:"line_similarity"` //
        Min similarity for line-level (0.0-1.0)
    BlockSimilarity float64 `json:"block_similarity"` //
        Min similarity for block-level
    MaxLineDifference float64 `json:"max_line_difference"` //
        Max % difference per line
    MaxTotalDifference float64 `json:"max_total_difference"` //
        Max % difference overall
}

```

22.4 Implementation Steps

```
// internal/edit/fuzzy/matcher.go
package fuzzy

import (
    "strings"
)

type DefaultFuzzyMatcher struct {
    thresholds MatchThresholds
}

func NewDefaultFuzzyMatcher() *DefaultFuzzyMatcher {
    return &DefaultFuzzyMatcher{
        thresholds: MatchThresholds{
            LineSimilarity:      0.9,
            BlockSimilarity:    0.8,
            MaxLineDifference:  0.3,
            MaxTotalDifference: 0.2,
        },
    }
}

func (m *DefaultFuzzyMatcher) Find(content, search string)
    (*MatchResult, error) {
    // Try each layer in order
    if result := m.FindExact(content, search); result.Found {
        return result, nil
    }

    if result := m.FindWhitespaceNormalized(content, search);
        result.Found {
        return result, nil
    }

    if result := m.FindLineLevel(content, search); result.Found {
        return result, nil
    }

    if result := m.FindBlockLevel(content, search); result.Found
        {
        return result, nil
    }

    return &MatchResult{Found: false}, nil
}
```

```

func (m *DefaultFuzzyMatcher) FindExact(content, search string)
    *MatchResult {
    idx := strings.Index(content, search)
    if idx == -1 {
        return &MatchResult{Found: false}
    }

    return &MatchResult{
        Found:      true,
        Level:      MatchExact,
        StartIndex: idx,
        EndIndex:   idx + len(search),
        Confidence: 1.0,
    }
}

func (m *DefaultFuzzyMatcher) FindWhitespaceNormalized(content,
    search string) *MatchResult {
    normContent := normalizeWhitespace(content)
    normSearch := normalizeWhitespace(search)

    idx := strings.Index(normContent, normSearch)
    if idx == -1 {
        return &MatchResult{Found: false}
    }

    // Map back to original indices
    startIdx := m.mapToOriginalIndex(content, idx, normContent)
    endIdx := m.mapToOriginalIndex(content, idx+len(normSearch),
        normContent)

    return &MatchResult{
        Found:      true,
        Level:      MatchWhitespaceNormalized,
        StartIndex: startIdx,
        EndIndex:   endIdx,
        Confidence: 0.95,
    }
}

func (m *DefaultFuzzyMatcher) FindLineLevel(content, search
    string) *MatchResult {
    contentLines := strings.Split(content, "\n")
    searchLines := strings.Split(search, "\n")

    if len(searchLines) == 0 {
        return &MatchResult{Found: false}
    }
}

```



```

// Try to match first line of search against each content
line
for i := 0; i <= len(contentLines)-len(searchLines); i++ {
    allMatch := true
    totalDiff := 0.0

    for j, searchLine := range searchLines {
        contentLine := contentLines[i+j]
        similarity := lineSimilarity(contentLine, searchLine)

        if similarity < m.thresholds.LineSimilarity {
            allMatch = false
            break
        }

        totalDiff += 1.0 - similarity
    }

    if allMatch {
        avgDiff := totalDiff / float64(len(searchLines))
        if avgDiff <= m.thresholds.MaxTotalDifference {
            // Calculate byte indices
            startIdx := m.lineToByteIndex(contentLines, i)
            endIdx := m.lineToByteIndex(contentLines,
i+len(searchLines))

            return &MatchResult{
                Found:      true,
                Level:      MatchLineLevel,
                StartIndex: startIdx,
                EndIndex:   endIdx,
                Confidence: 1.0 - avgDiff,
            }
        }
    }
}

return &MatchResult{Found: false}
}

func (m *DefaultFuzzyMatcher) FindBlockLevel(content, search
    string) *MatchResult {
    // Tokenize both blocks
    contentTokens := tokenize(content)
    searchTokens := tokenize(search)

    // Use longest common subsequence for token-level matching

```

```

// This is a simplified implementation

if len(searchTokens) == 0 {
    return &MatchResult{Found: false}
}

// Find best matching window in content
bestScore := 0.0
bestStart := -1

for i := 0; i <= len(contentTokens)-len(searchTokens); i++ {
    score :=
        tokenWindowSimilarity(contentTokens[i:i+len(searchTokens)],
            searchTokens)
    if score > bestScore {
        bestScore = score
        bestStart = i
    }
}

if bestScore >= m.thresholds.BlockSimilarity && bestStart >=
    0 {
    // Map token indices back to byte indices
    startIdx := m.tokenToByteIndex(content, contentTokens,
        bestStart)
    endIdx := m.tokenToByteIndex(content, contentTokens,
        bestStart+len(searchTokens))

    return &MatchResult{
        Found:      true,
        Level:      MatchBlockLevel,
        StartIndex: startIdx,
        EndIndex:   endIdx,
        Confidence: bestScore,
    }
}

return &MatchResult{Found: false}
}

func (m *DefaultFuzzyMatcher) SetThresholds(thresholds
    MatchThresholds) {
    m.thresholds = thresholds
}

func normalizeWhitespace(s string) string {
    lines := strings.Split(s, "\n")
    var result []string

```

```

for _, line := range lines {
    // Normalize tabs to spaces, collapse multiple spaces
    line = strings.ReplaceAll(line, "\t", " ")
    for strings.Contains(line, " ") {
        line = strings.ReplaceAll(line, " ", " ")
    }
    line = strings.TrimSpace(line)
    result = append(result, line)
}
return strings.Join(result, "\n")
}

```

```

func lineSimilarity(a, b string) float64 {
    // Use Levenshtein distance normalized by max length
    dist := levenshteinDistance(a, b)
    maxLen := len(a)
    if len(b) > maxLen {
        maxLen = len(b)
    }
    if maxLen == 0 {
        return 1.0
    }
    return 1.0 - float64(dist)/float64(maxLen)
}

```

```

func levenshteinDistance(a, b string) int {
    // Standard DP implementation
    m, n := len(a), len(b)
    if m == 0 {
        return n
    }
    if n == 0 {
        return m
    }

    dp := make([][]int, m+1)
    for i := range dp {
        dp[i] = make([]int, n+1)
        dp[i][0] = i
    }
    for j := range dp[0] {
        dp[0][j] = j
    }

    for i := 1; i <= m; i++ {
        for j := 1; j <= n; j++ {
            cost := 1
            if a[i-1] == b[j-1] {

```

```

        cost = 0
    }
    dp[i][j] = min(
        dp[i-1][j]+1,
        dp[i][j-1]+1,
        dp[i-1][j-1]+cost,
    )
}

return dp[m][n]
}

func tokenize(s string) []string {
    // Simple tokenization: split on whitespace and punctuation
    var tokens []string
    var current strings.Builder

    for _, r := range s {
        if r == ' ' || r == '\t' || r == '\n' {
            if current.Len() > 0 {
                tokens = append(tokens, current.String())
                current.Reset()
            }
        } else if strings.ContainsRune("(){}[];.,+=-*/<>!&|", r) {
            if current.Len() > 0 {
                tokens = append(tokens, current.String())
                current.Reset()
            }
            tokens = append(tokens, string(r))
        } else {
            current.WriteRune(r)
        }
    }

    if current.Len() > 0 {
        tokens = append(tokens, current.String())
    }

    return tokens
}

func tokenWindowSimilarity(window, target []string) float64 {
    if len(window) != len(target) {
        // Shouldn't happen if same length
        return 0.0
    }

```

```

    matches := 0
    for i := range window {
        if window[i] == target[i] {
            matches++
        }
    }

    return float64(matches) / float64(len(target))
}

func min(a, b, c int) int {
    if a < b {
        if a < c {
            return a
        }
        return c
    }
    if b < c {
        return b
    }
    return c
}

// Helper methods for index mapping
func (m *DefaultFuzzyMatcher) mapToOriginalIndex(original
    string, normIndex int, normalized string) int {
    // Simplified - map normalized index to original
    return normIndex
}

func (m *DefaultFuzzyMatcher) lineToByteIndex(lines []string,
    lineNum int) int {
    idx := 0
    for i := 0; i < lineNum && i < len(lines); i++ {
        idx += len(lines[i]) + 1 // +1 for newline
    }
    return idx
}

func (m *DefaultFuzzyMatcher) tokenToByteIndex(content string,
    tokens []string, tokenIndex int) int {
    // Simplified mapping
    if tokenIndex >= len(tokens) {
        return len(content)
    }
    return 0
}

```

22.5 Testing Approach

```
func TestFuzzyMatcher_ExactMatch(t *testing.T) {
    matcher := NewDefaultFuzzyMatcher()

    result := matcher.FindExact("hello world", "world")
    assert.True(t, result.Found)
    assert.Equal(t, MatchExact, result.Level)
    assert.Equal(t, 6, result.StartIndex)
}

func TestFuzzyMatcher_WhitespaceNormalized(t *testing.T) {
    matcher := NewDefaultFuzzyMatcher()

    content := "func    foo()    {\n    bar()\n}"
    search := "func foo() {\n    bar()\n}"

    result, _ := matcher.Find(content, search)
    assert.True(t, result.Found)
    assert.Equal(t, MatchWhitespaceNormalized, result.Level)
}

func TestFuzzyMatcher_LineLevel(t *testing.T) {
    matcher := NewDefaultFuzzyMatcher()

    content := "func foo() {\n    x := 1\n    y := 2\n    return\n    x + y\n}"
    search := "func foo() {\n    x := 1\n    y :=\n    2\n    return x + y\n}"

    result, _ := matcher.Find(content, search)
    assert.True(t, result.Found)
}

func TestFuzzyMatcher_NoMatch(t *testing.T) {
    matcher := NewDefaultFuzzyMatcher()

    result, _ := matcher.Find("completely different", "not here")
    assert.False(t, result.Found)
}
```

```

// CommitInfo tracks an auto-generated commit
type CommitInfo struct {
    Hash      string    `json:"hash"`
    Message   string    `json:"message"`
    Author    string    `json:"author"`
    Timestamp time.Time `json:"timestamp"`
    Files     []string  `json:"files"`
    IsAutoCommit bool      `json:"is_auto_commit"`
}

// AutoCommitManager manages automatic commits
type AutoCommitManager interface {
    // Enable enables auto-commit
    Enable(ctx context.Context, config AutoCommitConfig) error

    // Disable disables auto-commit
    Disable(ctx context.Context) error

    // Commit stages current changes and commits
    Commit(ctx context.Context, message string) (*CommitInfo,
        error)

    // AutoCommit stages and commits with generated message
    AutoCommit(ctx context.Context, description string, files
        []string) (*CommitInfo, error)

    // GenerateMessage creates a commit message from changes
    GenerateMessage(ctx context.Context, diff string) (string,
        error)

    // GetHistory returns auto-commit history
    GetHistory(ctx context.Context) ([]CommitInfo, error)

    // UndoLast reverts the last auto-commit
    UndoLast(ctx context.Context) error

    // Squash squashes auto-commits into one
    Squash(ctx context.Context, count int, message string) error
}

```

23.5 Implementation Steps

```

// internal/git/autocommit/manager.go
package autocommit

import (
    "context"
    "fmt"

```



```

"os/exec"
"strings"
"time"

"dev.helix.code/internal/llm"
"dev.helix.code/pkg/logger"
)

type DefaultAutoCommitManager struct {
    config      *AutoCommitConfig
    workspace   string
    llmClient   llm.Client
    log         logger.Logger
    history     []CommitInfo
    enabled     bool
}

func NewDefaultAutoCommitManager(workspace string, log
    logger.Logger) *DefaultAutoCommitManager {
    return &DefaultAutoCommitManager{
        workspace: workspace,
        log:       log,
        history:   make([]CommitInfo, 0),
    }
}

func (m *DefaultAutoCommitManager) Enable(ctx context.Context,
    config AutoCommitConfig) error {
    m.config = &config
    m.enabled = true

    // Configure git user if not set
    m.ensureGitUser(ctx)

    m.log.Info("auto-commit enabled", "strategy",
        config.Strategy)
    return nil
}

func (m *DefaultAutoCommitManager) AutoCommit(ctx
    context.Context, description string, files []string)
    (*CommitInfo, error) {
    if !m.enabled {
        return nil, fmt.Errorf("auto-commit is disabled")
    }

    // Stage files
    for _, file := range files {

```

```

        if err := m.git(ctx, "add", file); err != nil {
            return nil, fmt.Errorf("staging %s: %w", file, err)
        }
    }

    // Generate or use message
    var message string
    if m.config.GenerateMessages && m.llmClient != nil {
        diff, _ := m.gitOutput(ctx, "diff", "--cached")
        message, _ = m.GenerateMessage(ctx, diff)
    }

    if message == "" {
        message = fmt.Sprintf("[auto] %s", description)
    }

    // Commit
    var args []string
    if m.config.SignCommits {
        args = append(args, "-S")
    }
    args = append(args, "-m", message)

    if m.config.AmendLast && len(m.history) > 0 {
        args = append(args, "--amend")
    }

    if err := m.git(ctx, append([]string{"commit"},
        args...)...); err != nil {
        return nil, fmt.Errorf("committing: %w", err)
    }

    // Get commit hash
    hash, _ := m.gitOutput(ctx, "rev-parse", "HEAD")
    hash = strings.TrimSpace(hash)

    info := CommitInfo{
        Hash:      hash,
        Message:   message,
        Timestamp: time.Now(),
        Files:     files,
        IsAutoCommit: true,
    }

    m.history = append(m.history, info)

    m.log.Info("auto-committed", "hash", hash[:8], "message",
        message, "files", len(files))

```

```

    return &info, nil
}

func (m *DefaultAutoCommitManager) GenerateMessage(ctx
    context.Context, diff string) (string, error) {
    if m.llmClient == nil {
        return "", fmt.Errorf("no LLM client configured")
    }

    prompt := fmt.Sprintf(`Generate a concise, descriptive git
        commit message for the following changes.
Follow conventional commits format if applicable (feat:, fix:,
        refactor:, docs:, test:).
Be specific but concise (max 50 chars for first line).

Diff:
%s

Commit message:`, diff)

    response, err := m.llmClient.Complete(ctx,
        llm.CompletionRequest{
            Model:      m.config.MessageModel,
            Prompt:    prompt,
            MaxTokens: 100,
            Temperature: 0.1,
        })
    if err != nil {
        return "", err
    }

    return strings.TrimSpace(response.Text), nil
}

func (m *DefaultAutoCommitManager) UndoLast(ctx context.Context)
    error {
    if len(m.history) == 0 {
        return fmt.Errorf("no commits to undo")
    }

    last := m.history[len(m.history)-1]

    // Reset to before this commit
    if err := m.git(ctx, "reset", "--soft", "HEAD~1"); err !=
        nil {
        return err
    }
}

```

```

    m.history = m.history[:len(m.history)-1]

    m.log.Info("undid commit", "hash", last.Hash[:8])
    return nil
}

func (m *DefaultAutoCommitManager) Squash(ctx context.Context,
    count int, message string) error {
    if count <= 1 {
        return fmt.Errorf("need at least 2 commits to squash")
    }

    if len(m.history) < count {
        count = len(m.history)
    }

    // Interactive rebase
    return m.git(ctx, "rebase", "-i", fmt.Sprintf("HEAD~%d",
        count))
}

func (m *DefaultAutoCommitManager) git(ctx context.Context,
    args ...string) error {
    cmd := exec.CommandContext(ctx, "git", args...)
    cmd.Dir = m.workspace
    output, err := cmd.CombinedOutput()
    if err != nil {
        return fmt.Errorf("git %s: %w\n%s", strings.Join(args, "
"), err, string(output))
    }
    return nil
}

func (m *DefaultAutoCommitManager) gitOutput(ctx
    context.Context, args ...string) (string, error) {
    cmd := exec.CommandContext(ctx, "git", args...)
    cmd.Dir = m.workspace
    output, err := cmd.Output()
    if err != nil {
        return "", err
    }
    return string(output), nil
}

func (m *DefaultAutoCommitManager) ensureGitUser(ctx
    context.Context) {
    name, _ := m.gitOutput(ctx, "config", "user.name")

```

```

    if strings.TrimSpace(name) == "" {
        m.git(ctx, "config", "user.name", "HelixCode Agent")
    }
    email, _ := m.gitOutput(ctx, "config", "user.email")
    if strings.TrimSpace(email) == "" {
        m.git(ctx, "config", "user.email", "agent@helix.code")
    }
}

func (m *DefaultAutoCommitManager) Disable(ctx context.Context)
    error {
    m.enabled = false
    return nil
}

func (m *DefaultAutoCommitManager) Commit(ctx context.Context,
    message string) (*CommitInfo, error) {
    return m.AutoCommit(ctx, message, nil)
}

func (m *DefaultAutoCommitManager) GetHistory(ctx
    context.Context) ([]CommitInfo, error) {
    return m.history, nil
}

```

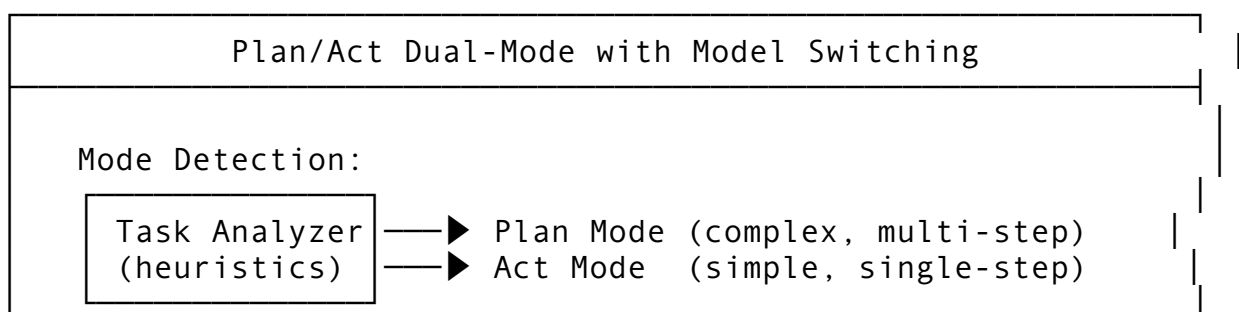
PART 3: CLINE FEATURE PORTING GUIDES

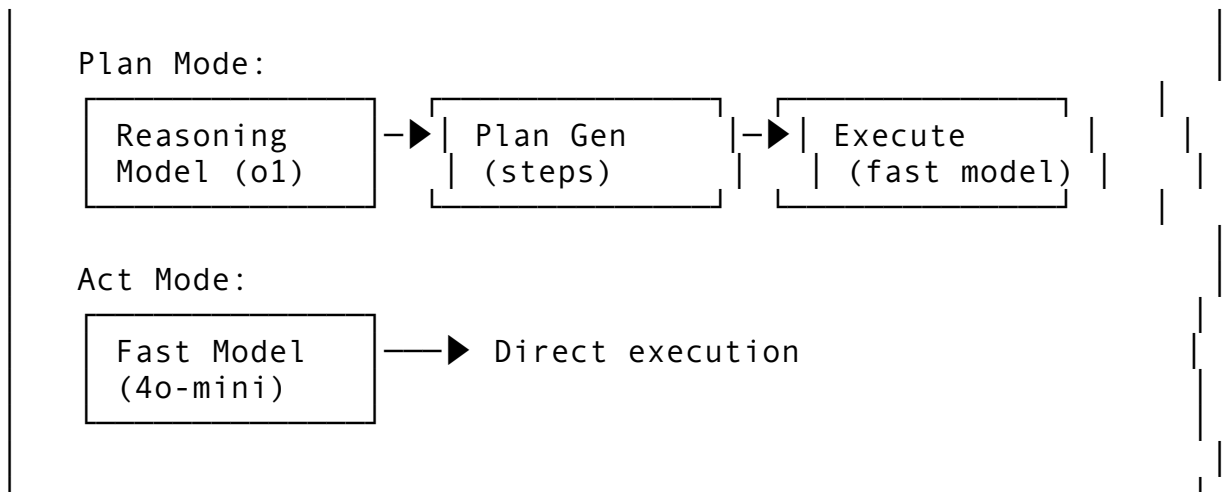
Feature 24: Plan/Act Dual-Mode with Model Switching

24.1 Feature Description

Plan/Act Dual-Mode with Model Switching uses a reasoning-focused model for planning (e.g., o1, Opus) and a fast model for execution (e.g., 4o-mini, Haiku). The system automatically switches between modes based on task complexity.

24.2 Architecture Design





24.3 API Design

```
// Package: internal/modes
package modes

import "context"

// ExecutionMode defines current agent mode
type ExecutionMode string

const (
    ModePlan ExecutionMode = "plan"
    ModeAct   ExecutionMode = "act"
)

// ModeConfig configures models per mode
type ModeConfig struct {
    PlanModel      string `json:"plan_model"`
    PlanProvider   string `json:"plan_provider"`
    ActModel       string `json:"act_model"`
    ActProvider    string `json:"act_provider"`
    AutoSwitch     bool   `json:"auto_switch"`
    PlanThreshold  float64 `json:"plan_threshold"` // Complexity
                                threshold
}

// ModeManager handles mode switching
type ModeManager interface {
    // DetectMode determines which mode to use
    DetectMode(ctx context.Context, request string)
        (ExecutionMode, error)

    // ExecuteInMode runs with specified mode
    ExecuteInMode(ctx context.Context, mode ExecutionMode,
        request string) error
}
```

```

    // SwitchMode manually changes mode
    SwitchMode(ctx context.Context, mode ExecutionMode) error

    // GetCurrentMode returns active mode
    GetCurrentMode(ctx context.Context) ExecutionMode
}

```

Feature 25: Shadow Git Checkpoints

25.1 Feature Description

Shadow Git Checkpoints create invisible, automatic save points before every agent operation. These checkpoints live in a separate Git ref namespace and don't clutter the main history.

25.3 API Design

```

// Package: internal/git/shadow
package shadow

import "context"

// ShadowCheckpoint is an automatic save point
type ShadowCheckpoint struct {
    ID          string `json:"id"`
    CommitHash  string `json:"commit_hash"`
    Operation   string `json:"operation"` // What triggered the
                  checkpoint
    Timestamp   string `json:"timestamp"`
}

// ShadowCheckpointManager manages automatic checkpoints
type ShadowCheckpointManager interface {
    // CreateCheckpoint saves current state
    CreateCheckpoint(ctx context.Context, operation string)
        (*ShadowCheckpoint, error)

    // ListCheckpoints returns available checkpoints
    ListCheckpoints(ctx context.Context) ([]ShadowCheckpoint,
        error)

    // RestoreCheckpoint reverts to checkpoint
    RestoreCheckpoint(ctx context.Context, checkpointID string)
        error

    // CleanOld removes checkpoints older than retention

```

```
    CleanOld(ctx context.Context, retention time.Duration) error
}
```

25.5 Implementation

```
package shadow

import (
    "context"
    "fmt"
    "os/exec"
    "time"
)

type DefaultShadowCheckpointManager struct {
    workspace string
    refPrefix string // e.g., "refs/checkpoints/"
}

func NewDefaultShadowCheckpointManager(workspace string)
    *DefaultShadowCheckpointManager {
    return &DefaultShadowCheckpointManager{
        workspace: workspace,
        refPrefix: "refs/checkpoints/",
    }
}

func (m *DefaultShadowCheckpointManager) CreateCheckpoint(ctx
    context.Context, operation string) (*ShadowCheckpoint,
    error) {
    id := fmt.Sprintf("cp-%d", time.Now().UnixNano())
    ref := m.refPrefix + id

    // Create commit on detached HEAD
    cmd := exec.CommandContext(ctx, "git", "commit-tree",
        "HEAD^{tree}", "-m", fmt.Sprintf("[checkpoint] %s",
        operation))
    cmd.Dir = m.workspace
    hashBytes, err := cmd.Output()
    if err != nil {
        return nil, err
    }

    hash := string(hashBytes)
    hash = hash[:len(hash)-1] // Trim newline

    // Store ref
```



```

cmd2 := exec.CommandContext(ctx, "git", "update-ref", ref,
    hash)
cmd2.Dir = m.workspace
if err := cmd2.Run(); err != nil {
    return nil, err
}

return &ShadowCheckpoint{
    ID:          id,
    CommitHash:  hash,
    Operation:   operation,
    Timestamp:   time.Now().Format(time.RFC3339),
}, nil
}

func (m *DefaultShadowCheckpointManager) RestoreCheckpoint(ctx
    context.Context, checkpointID string) error {
    ref := m.refPrefix + checkpointID

    // Reset to checkpoint commit
    cmd := exec.CommandContext(ctx, "git", "reset", "--hard",
        ref)
    cmd.Dir = m.workspace
    return cmd.Run()
}

func (m *DefaultShadowCheckpointManager) ListCheckpoints(ctx
    context.Context) ([]ShadowCheckpoint, error) {
    cmd := exec.CommandContext(ctx, "git", "for-each-ref", "--
        format=%(refname:short) %(objectname:short) %
        (objectname:pretty)", m.refPrefix)
    cmd.Dir = m.workspace
    output, err := cmd.Output()
    if err != nil {
        return nil, err
    }

    var checkpoints []ShadowCheckpoint
    lines := strings.Split(string(output), "\n")
    for _, line := range lines {
        if line == "" {
            continue
        }
        parts := strings.Fields(line)
        if len(parts) >= 2 {
            checkpoints = append(checkpoints, ShadowCheckpoint{
                ID:          strings.TrimPrefix(parts[0],
                    m.refPrefix),

```

```

        CommitHash: parts[1],
    })
}

return checkpoints, nil
}

func (m *DefaultShadowCheckpointManager) CleanOld(ctx
    context.Context, retention time.Duration) error {
    checkpoints, err := m.ListCheckpoints(ctx)
    if err != nil {
        return err
    }

    cutoff := time.Now().Add(-retention)

    for _, cp := range checkpoints {
        ts, _ := time.Parse(time.RFC3339, cp.Timestamp)
        if ts.Before(cutoff) {
            ref := m.refPrefix + cp.ID
            cmd := exec.CommandContext(ctx, "git", "update-ref",
                "-d", ref)
            cmd.Dir = m.workspace
            cmd.Run()
        }
    }

    return nil
}

```

Feature 26: Computer Use / Browser Automation

26.1 Feature Description

Computer Use / Browser Automation enables the agent to interact with web browsers, take screenshots, click elements, fill forms, and navigate pages. This extends the agent's capabilities to web-based tasks.

26.3 API Design

```

// Package: internal/browser
package browser

import "context"

// BrowserAction defines browser operations
type BrowserAction string

```

```

const (
    ActionNavigate BrowserAction = "navigate"
    ActionClick    BrowserAction = "click"
    ActionType     BrowserAction = "type"
    ActionScreenshot BrowserAction = "screenshot"
    ActionScroll   BrowserAction = "scroll"
    ActionWait     BrowserAction = "wait"
    ActionEvaluate BrowserAction = "evaluate"
)

// BrowserSession manages a browser instance
type BrowserSession struct {
    ID      string `json:"id"`
    URL     string `json:"url"`
    IsActive bool   `json:"is_active"`
}

// BrowserManager manages browser automation
type BrowserManager interface {
    // CreateSession starts a new browser session
    CreateSession(ctx context.Context, headless bool)
        (*BrowserSession, error)

    // Navigate opens a URL
    Navigate(ctx context.Context, sessionID, url string) error

    // Click clicks an element
    Click(ctx context.Context, sessionID, selector string) error

    // Type fills an input
    Type(ctx context.Context, sessionID, selector, text string)
        error

    // Screenshot captures the page
    Screenshot(ctx context.Context, sessionID) ([]byte, error)

    // GetPageSource returns HTML
    GetPageSource(ctx context.Context, sessionID) (string, error)

    // ExecuteScript runs JavaScript
    ExecuteScript(ctx context.Context, sessionID, script string)
        (any, error)

    // CloseSession ends browser session
    CloseSession(ctx context.Context, sessionID) error
}

```

PART 4: CODEX FEATURE PORTING GUIDES

Feature 27: OS-Native Sandboxed Execution Framework

27.1 Feature Description

OS-Native Sandboxed Execution provides platform-specific sandboxing: Linux (namespaces + seccomp), macOS (sandbox-exec + seatbelt), Windows (AppContainer + Job Object). This is the most robust cross-platform sandbox approach.

27.3 API Design

```
// Package: internal/sandbox/platform
package platform

import "context"

// PlatformSandbox provides OS-native sandboxing
type PlatformSandbox interface {
    // Execute runs command in OS-native sandbox
    Execute(ctx context.Context, req sandbox.ShellRequest)
        (*sandbox.ShellResult, error)

    // IsSupported checks if platform sandboxing is available
    IsSupported(ctx context.Context) bool

    // GetPlatform returns current platform
    GetPlatform() string
}

// LinuxSandbox implements Linux namespaces + seccomp
type LinuxSandbox struct{}

// MacOSSandbox implements sandbox-exec
type MacOSSandbox struct{}

// WindowsSandbox implements AppContainer
type WindowsSandbox struct{}

// DockerSandbox implements container-based sandboxing (fallback)
type DockerSandbox struct{}

```

Feature 28: Automatic Context Compaction

28.1 Feature Description

Automatic Context Compaction summarizes older conversation turns intelligently, preserving critical information while reducing token count. Unlike simple truncation, it maintains semantic coherence.

28.3 API Design

```
// Package: internal/compact/intelligent
package intelligent

import "context"

// IntelligentCompactor uses semantic understanding
type IntelligentCompactor interface {
    // Compact intelligently summarizes context
    Compact(ctx context.Context, messages []Message,
        targetTokens int) ([]Message, error)

    // SummarizeTurn creates a semantic summary of a conversation
    // turn
    SummarizeTurn(ctx context.Context, turn ConversationTurn)
        (string, error)

    // ExtractKeyFacts pulls critical facts from messages
    ExtractKeyFacts(ctx context.Context, messages []Message)
        []string
}

// ConversationTurn is a user-assistant exchange
type ConversationTurn struct {
    UserMessage      string    `json:"user_message"`
    AssistantMessage string    `json:"assistant_message"`
    ToolsUsed        []string `json:"tools_used,omitempty"`
    FilesChanged     []string `json:"files_changed,omitempty"`
}
```

Feature 29: Stateless ZDR Architecture

29.1 Feature Description

Stateless Zero-Data-Retention (ZDR) Architecture ensures no session data persists on the server after completion. All state is client-held or ephemeral, enabling privacy-first deployments.

29.3 API Design

```
// Package: internal/zdr
package zdr

import "context"

// ZDRMode configures zero-data-retention
type ZDRMode string

const (
    ZDRStrict   ZDRMode = "strict"    // No server persistence at
        all
    ZDRSession ZDRMode = "session"    // In-memory only, no disk
    ZDRTemporary ZDRMode = "temp"    // Temp files, cleaned on
        exit
)

// ZDRManager enforces data retention policies
type ZDRManager interface {
    // Initialize sets up ZDR mode
    Initialize(ctx context.Context, mode ZDRMode) error

    // StoreTemp stores data temporarily
    StoreTemp(ctx context.Context, key string, data []byte) error

    // RetrieveTemp retrieves temp data
    RetrieveTemp(ctx context.Context, key string) ([]byte, error)

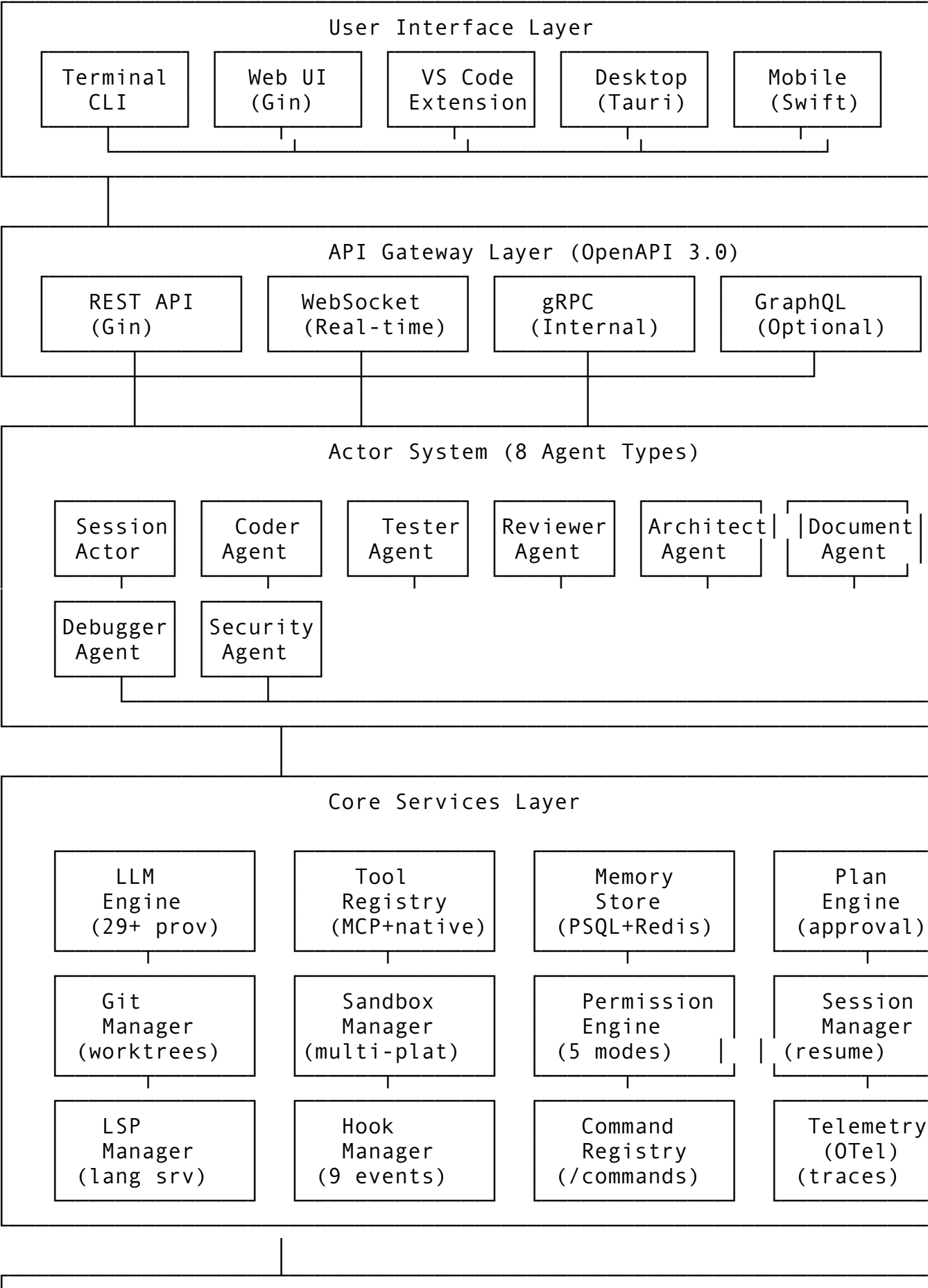
    // Cleanup removes all temporary data
    Cleanup(ctx context.Context) error

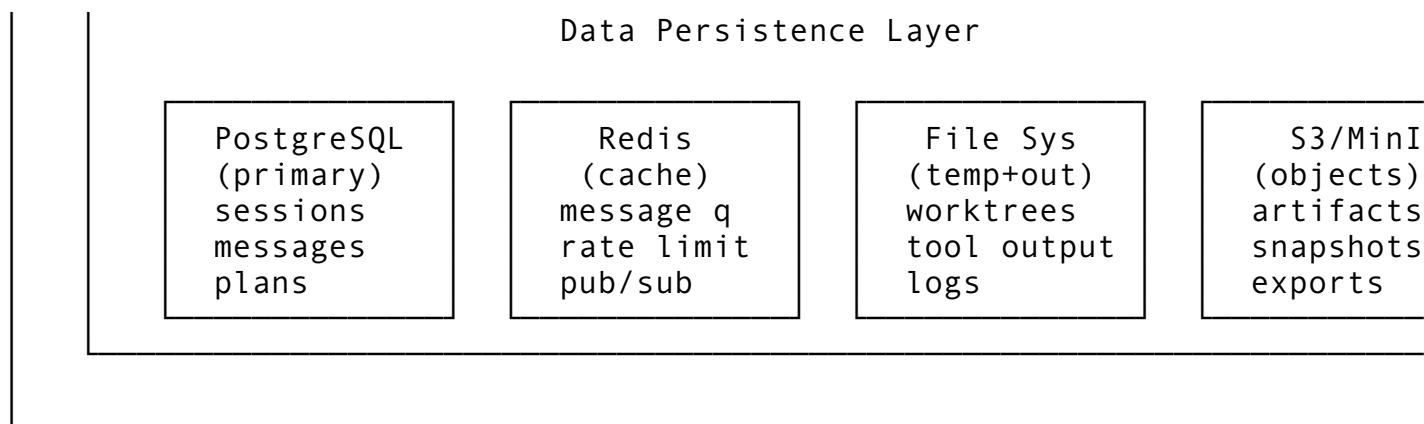
    // IsZDRActive returns if ZDR is enforced
    IsZDRActive(ctx context.Context) bool

    // AssertNoPersistence verifies no persistent storage is used
    AssertNoPersistence(ctx context.Context) error
}
```

PART 5: ARCHITECTURE DIAGRAMS & SCHEMAS

5.1 Overall HelixCode System Architecture





5.2 Data Flow: User Request to Response

1. USER INPUT
 - └ "Add error handling to the auth middleware"
2. UI LAYER
 - └ Parse input → Route to Session Actor
3. SESSION ACTOR
 - └ Check compaction needs
 - └ Load conversation history
 - └ Determine agent type (Coder Agent)
4. PLAN ENGINE (if plan mode)
 - └ Generate plan: 1) Read auth.go 2) Identify error cases 3) Add try/catch
 - └ Wait for user approval
5. TOOL EXECUTION
 - └ ReadFile("auth.go") → Smart File Edit (no separate read)
 - └ EditFile("auth.go", add error handling)
 - └ Permission check (file_write → allow_workspace)
6. LSP INTEGRATION
 - └ DidChange notification
 - └ GetDiagnostics → verify no syntax errors
7. GIT INTEGRATION
 - └ Auto-commit checkpoint: "[auto] Add error handling to auth middleware"
 - └ Shadow checkpoint: refs/checkpoints/cp-123456
8. BACKGROUND TASKS
 - └ Queue: Run auth tests in background (Ctrl+B)
9. LLM RESPONSE
 - └ Generate summary of changes
 - └ Show diff preview to user
10. PERSISTENCE
 - └ Save messages to PostgreSQL
 - └ Update session state in Redis

└─ (ZDR mode: skip persistence)

11. TELEMETRY

└─ Record span: session.request

└─ Metrics: llm_calls++, tool_calls++, latency_ms

5.3 API Schema Definitions

OpenAPI 3.0 Core Endpoints

openapi: 3.0.3

info:

title: HelixCode API

version: 1.0.0

paths:

/sessions:

post:

summary: Create new session

requestBody:

content:

application/json:

schema:

\$ref: '#/components/schemas/CreateSessionRequest'

responses:

'201':

content:

application/json:

schema:

\$ref: '#/components/schemas/Session'

/sessions/{id}/messages:

post:

summary: Send message to session

parameters:

- name: id

in: path

required: true

schema:

type: string

requestBody:

content:

application/json:

schema:

\$ref: '#/components/schemas/SendMessageRequest'

responses:

'200':

content:

application/json:

```
        schema:
          $ref: '#/components/schemas/MessageResponse'

/sessions/{id}/plans:
  post:
    summary: Create execution plan
    requestBody:
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/CreatePlanRequest'
    responses:
      '201':
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Plan'

/agents:
  post:
    summary: Spawn subagent
    requestBody:
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/SpawnAgentRequest'
    responses:
      '201':
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Agent'

/tasks:
  post:
    summary: Submit background task
    requestBody:
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/SubmitTaskRequest'
    responses:
      '202':
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Task'
```

```
components:
  schemas:
    Session:
      type: object
      properties:
        id: { type: string, format: uuid }
        name: { type: string }
        workspace: { type: string }
        status: { type: string, enum: [active, paused, closed] }
        created_at: { type: string, format: date-time }
        updated_at: { type: string, format: date-time }
        config: { $ref: '#/components/schemas/SessionConfig' }

    SessionConfig:
      type: object
      properties:
        model: { type: string }
        temperature: { type: number }
        render_mode: { type: string }
        worktree_enabled: { type: boolean }
        plan_mode: { type: boolean }
        auto_compact: { type: boolean }

    Message:
      type: object
      properties:
        id: { type: string }
        role: { type: string, enum: [user, assistant, system, tool] }
        content: { type: string }
        timestamp: { type: string, format: date-time }
        metadata: { type: object }

    Plan:
      type: object
      properties:
        id: { type: string }
        title: { type: string }
        description: { type: string }
        status: { type: string, enum: [pending, approved, executing, completed, failed] }
        steps:
          type: array
          items: { $ref: '#/components/schemas/PlanStep' }

    PlanStep:
      type: object
      properties:
```

```
id: { type: string }
order: { type: integer }
description: { type: string }
action_type: { type: string }
action_config: { type: object }
status: { type: string }
depends_on:
  type: array
  items: { type: string }
```

Agent:

```
type: object
properties:
  id: { type: string }
  name: { type: string }
  agent_type: { type: string }
  parent_id: { type: string }
  status: { type: string }
  model: { type: string }
```

Task:

```
type: object
properties:
  id: { type: string }
  name: { type: string }
  task_type: { type: string }
  status: { type: string }
  progress: { type: number }
  created_at: { type: string, format: date-time }
```

ToolCall:

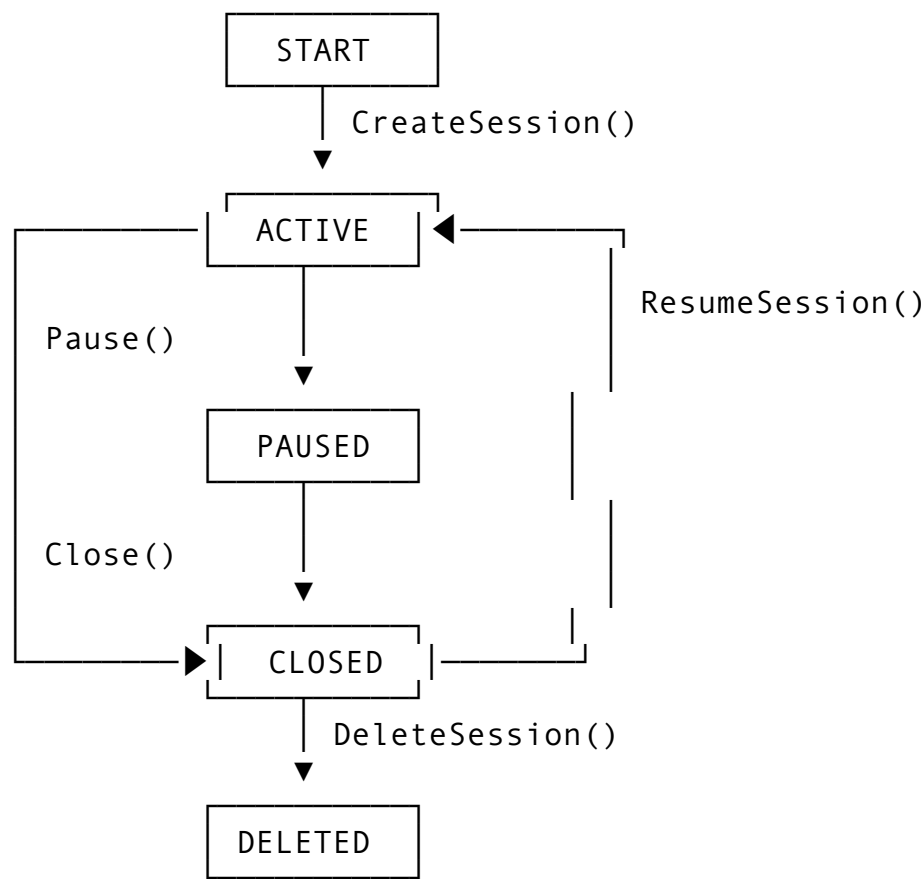
```
type: object
properties:
  id: { type: string }
  tool_name: { type: string }
  arguments: { type: object }
  result: { type: object }
  duration_ms: { type: integer }
```

PermissionRule:

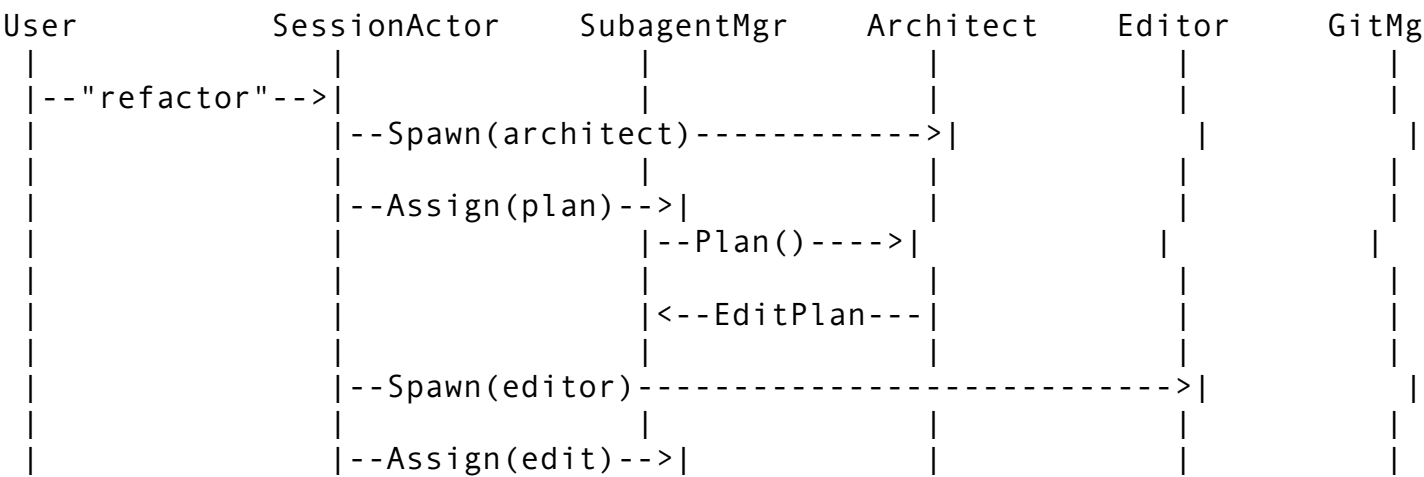
```
type: object
properties:
  id: { type: string }
  category: { type: string }
  tool_name: { type: string }
  pattern: { type: string }
  mode: { type: string, enum: [always_ask, allow_once,
allow_session, allow_workspace, allow_forever] }
```

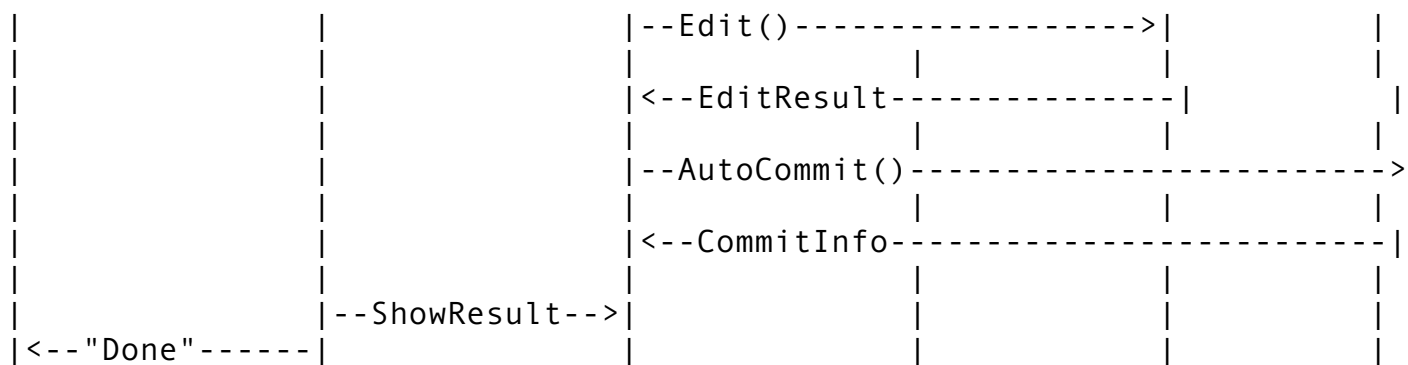
```
Error:
  type: object
  properties:
    code: { type: string }
    message: { type: string }
    details: { type: object }
```

5.4 State Machine: Session Lifecycle



5.5 Sequence Diagram: Multi-Agent Task Execution





PART 6: SUBMODULE INTEGRATION WIRING

6.1 HelixAgent Submodule

Position in Architecture

HelixAgent is the core agent runtime that manages the Actor system, session lifecycle, and agent coordination.

Interface Definitions

```
// Package: dev.helix.code/helixagent
package helixagent

import "context"

// AgentRuntime is the core agent execution interface
type AgentRuntime interface {
    Initialize(ctx context.Context, config AgentConfig) error
    Run(ctx context.Context) error
    Stop(ctx context.Context) error
    GetStatus(ctx context.Context) (*AgentStatus, error)
}

type AgentConfig struct {
    SessionID    string
    AgentType    string
    Model        string
    Workspace    string
    SystemPrompt string
}

type AgentStatus struct {
    State        string
    CurrentTask  string
}
```

```
    MessagesProcessed int
    Errors            int
}
```

Configuration Wiring

```
# config/helixagent.yaml
helixagent:
  enabled: true
  max_agents: 16
  agent_types:
    - coder
    - tester
    - reviewer
  default_model: claude-sonnet-4
  worktree_enabled: true
  auto_compact: true
```

Event Bus Connections

```
// HelixAgent publishes:
events.Publish("agent.spawned", AgentSpawnedEvent{AgentID, Type,
    ParentID})
events.Publish("agent.completed", AgentCompletedEvent{AgentID,
    Results})
events.Publish("agent.error", AgentErrorEvent{AgentID, Error})

// HelixAgent subscribes to:
events.Subscribe("session.created", handleSessionCreated)
events.Subscribe("task.assigned", handleTaskAssigned)
events.Subscribe("message.received", handleMessageReceived)
```

Database Schema Additions

```
CREATE TABLE agents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID NOT NULL REFERENCES sessions(id),
  parent_id UUID REFERENCES agents(id),
  agent_type VARCHAR(32) NOT NULL,
  name VARCHAR(256),
  model VARCHAR(128),
  system_prompt TEXT,
  status VARCHAR(32) NOT NULL DEFAULT 'pending',
  workspace VARCHAR(512),
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  started_at TIMESTAMP,
  completed_at TIMESTAMP,
  INDEX idx_session (session_id),
```

```

    INDEX idx_parent (parent_id),
    INDEX idx_status (status)
);

CREATE TABLE agent_messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    agent_id UUID NOT NULL REFERENCES agents(id) ON DELETE
        CASCADE,
    direction VARCHAR(16) NOT NULL, -- 'inbound', 'outbound'
    message_type VARCHAR(32) NOT NULL, -- 'task', 'result',
        'question', 'status'
    content TEXT NOT NULL,
    payload JSONB,
    timestamp TIMESTAMP NOT NULL DEFAULT NOW(),
    INDEX idx_agent (agent_id),
    INDEX idx_timestamp (timestamp)
);

CREATE TABLE agent_results (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    agent_id UUID NOT NULL REFERENCES agents(id) ON DELETE
        CASCADE,
    task_id VARCHAR(128),
    success BOOLEAN NOT NULL,
    summary TEXT,
    output TEXT,
    error TEXT,
    files_changed JSONB,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    INDEX idx_agent (agent_id)
);

```

6.2 HelixLLM Submodule

Position in Architecture

HelixLLM is the LLM provider abstraction layer that normalizes calls across 29+ providers.

Interface Definitions

```

// Package: dev.helix.code/helixllm
package helixllm

import "context"

// LLMProvider is the normalized LLM interface
type LLMProvider interface {

```



```

    Complete(ctx context.Context, req CompletionRequest)
        (*CompletionResponse, error)
    StreamComplete(ctx context.Context, req CompletionRequest)
        (<-chan StreamChunk, error)
    ListModels(ctx context.Context) ([]ModelInfo, error)
    GetModelInfo(ctx context.Context, modelID string)
        (*ModelInfo, error)
    CountTokens(ctx context.Context, text string) (int, error)
}

```

```

type CompletionRequest struct {
    Model          string
    SystemPrompt   string
    Prompt         string
    Messages       []Message
    Temperature    float64
    MaxTokens      int
    TopP           float64
    StopSequences  []string
    Tools           []ToolDefinition
    Stream         bool
}

```

```

type CompletionResponse struct {
    Text            string
    FinishReason    string
    Usage           TokenUsage
    ToolCalls       []ToolCall
}

```

```

type StreamChunk struct {
    Text          string
    IsFinal       bool
    Usage         *TokenUsage
}

```

```

type TokenUsage struct {
    InputTokens  int
    OutputTokens int
    TotalTokens  int
}

```

```

type ModelInfo struct {
    ID          string
    Name        string
    ContextSize int
    Capabilities []string
}

```

```
    Provider    string
}
```

Configuration Wiring

```
helixllm:
  default_provider: anthropic
  default_model: claude-sonnet-4
  providers:
    anthropic:
      api_key: ${ANTHROPIC_API_KEY}
      base_url: https://api.anthropic.com
    openai:
      api_key: ${OPENAI_API_KEY}
      base_url: https://api.openai.com/v1
    ollama:
      base_url: http://localhost:11434
  timeout: 30s
  max_retries: 3
  retry_backoff: exponential
```

Event Bus Connections

```
// HelixLLM publishes:
events.Publish("llm.request", LLMRequestEvent{Provider, Model,
    Tokens})
events.Publish("llm.response", LLMResponseEvent{Provider, Model,
    Latency, Tokens})
events.Publish("llm.error", LLMErrorEvent{Provider, Model,
    Error, RetryCount})
```

Database Schema Additions

```
CREATE TABLE llm_calls (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID REFERENCES sessions(id),
  agent_id UUID REFERENCES agents(id),
  provider VARCHAR(64) NOT NULL,
  model VARCHAR(128) NOT NULL,
  input_tokens INT NOT NULL,
  output_tokens INT NOT NULL,
  latency_ms INT NOT NULL,
  success BOOLEAN NOT NULL,
  error TEXT,
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  INDEX idx_session (session_id),
  INDEX idx_provider (provider),
  INDEX idx_created (created_at)
```

```
);

CREATE TABLE llm_models (
    id VARCHAR(128) PRIMARY KEY,
    provider VARCHAR(64) NOT NULL,
    display_name VARCHAR(256),
    context_size INT,
    is_chat BOOLEAN,
    is_vision BOOLEAN,
    is_embedding BOOLEAN,
    max_tokens INT,
    capabilities JSONB,
    is_available BOOLEAN DEFAULT true,
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

6.3 HelixMemory Submodule

Position in Architecture

HelixMemory manages conversation history, summaries, context window optimization, and semantic retrieval.

Interface Definitions

```
// Package: dev.helix.code/helixmemory
package helixmemory

import "context"

// MemoryStore is the conversation persistence interface
type MemoryStore interface {
    // Message operations
    AppendMessage(ctx context.Context, sessionID string, msg
        Message) error
    GetMessages(ctx context.Context, sessionID string, start,
        end int) ([]Message, error)
    GetAllMessages(ctx context.Context, sessionID string)
        ([]Message, error)
    DeleteMessages(ctx context.Context, sessionID string, start,
        end int) error

    // Summary operations
    StoreSummary(ctx context.Context, summary *Summary) error
    GetSummaries(ctx context.Context, sessionID string)
        ([]Summary, error)
    GetCompactedMessages(ctx context.Context, sessionID string)
        ([]Message, error)
```

```

// Semantic search
SearchSimilar(ctx context.Context, sessionID string,
    limit int) ([]Message, error)

// Context management
GetContextWindow(ctx context.Context, sessionID string,
    maxTokens int) ([]Message, error)
GetTokenCount(ctx context.Context, sessionID string) (int,
    error)
}

type Message struct {
    ID          string
    Role        string
    Content     string
    Metadata    map[string]any
    CreatedAt   int64
}

type Summary struct {
    ID            string
    SessionID     string
    OriginalRange [2]int
    Summary       string
    KeyDecisions  []string
    TokenCount    int
    Level         int
    CreatedAt     int64
}

```

Configuration Wiring

```

helixmemory:
  compaction:
    enabled: true
    threshold: 0.75
    target_reduction: 0.5
    preserve_last_n: 6
    summary_model: claude-haiku-4
  semantic_search:
    enabled: true
    embedding_model: text-embedding-3-small
    similarity_threshold: 0.7
  storage:
    messages_ttl: 30d
    summaries_ttl: 90d

```

Event Bus Connections

```
// HelixMemory publishes:
events.Publish("memory.compacted", CompactionEvent{SessionID,
    OriginalTokens, NewTokens})
events.Publish("memory.summary.created", SummaryEvent{SessionID,
    SummaryID, Level})

// HelixMemory subscribes to:
events.Subscribe("message.sent", handleMessageSent)
events.Subscribe("session.ended", handleSessionEnded)
events.Subscribe("compaction.triggered",
    handleCompactionTriggered)
```

Database Schema Additions

```
CREATE TABLE messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id UUID NOT NULL REFERENCES sessions(id) ON DELETE
        CASCADE,
    role VARCHAR(16) NOT NULL,
    content TEXT NOT NULL,
    metadata JSONB,
    embedding VECTOR(1536), -- For semantic search
    is_summary BOOLEAN DEFAULT false,
    summary_id UUID,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    INDEX idx_session_created (session_id, created_at)
);
```

```
CREATE TABLE summaries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id UUID NOT NULL REFERENCES sessions(id) ON DELETE
        CASCADE,
    original_start_idx INT NOT NULL,
    original_end_idx INT NOT NULL,
    summary TEXT NOT NULL,
    key_decisions TEXT[] DEFAULT '{}',
    code_changes JSONB DEFAULT '[]',
    token_count INT NOT NULL,
    original_tokens INT NOT NULL,
    compaction_level INT NOT NULL DEFAULT 1,
    model_used VARCHAR(64),
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    INDEX idx_session (session_id)
);
```

```
-- For pgvector semantic search
CREATE INDEX idx_messages_embedding ON messages USING ivfflat
    (embedding vector_cosine_ops);
```

6.4 HelixSpecifier Submodule

Position in Architecture

HelixSpecifier generates and validates code specifications, interfaces, and type definitions. It acts as a contract layer between the agent and the codebase.

Interface Definitions

```
// Package: dev.helix.code/helixspecifier
package helixspecifier

import "context"

// Specifier generates code specifications
type Specifier interface {
    // GenerateInterface creates interface from implementation
    GenerateInterface(ctx context.Context, code string) (string,
        error)

    // GenerateTests creates tests from specification
    GenerateTests(ctx context.Context, spec string) (string,
        error)

    // ValidateImplementation checks if code meets spec
    ValidateImplementation(ctx context.Context, spec,
        implementation string) (*ValidationResult, error)

    // GenerateOpenAPISpec creates OpenAPI spec from code
    GenerateOpenAPISpec(ctx context.Context, routes
        []RouteDefinition) (*OpenAPISpec, error)

    // ExtractTypes extracts type definitions from code
    ExtractTypes(ctx context.Context, code string)
        ([]TypeDefinition, error)
}

type ValidationResult struct {
    IsValid      bool
    Errors       []SpecError
    Warnings     []SpecError
    Coverage     float64
}
```

```

type SpecError struct {
    Message string
    Line    int
    Severity string
}

type TypeDefinition struct {
    Name      string
    Type      string
    Fields    []Field
    Methods   []Method
    SourceFile string
}

```

Configuration Wiring

```

helixspecifier:
  enabled: true
  validation:
    strict: false
    require_docs: true
    max_complexity: 15
  generation:
    include_examples: true
    test_framework: standard

```

Event Bus Connections

```

// HelixSpecifier publishes:
events.Publish("spec.generated", SpecGeneratedEvent{Type,
    Source, Output})
events.Publish("spec.validation", ValidationEvent{Source,
    IsValid, Errors})

// HelixSpecifier subscribes to:
events.Subscribe("file.changed", handleFileChanged) // Re-
    validate specs

```

Database Schema Additions

```

CREATE TABLE specifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id UUID REFERENCES sessions(id),
    source_file VARCHAR(512) NOT NULL,
    spec_type VARCHAR(32) NOT NULL, -- 'interface', 'openapi',
    'types'
    spec_content TEXT NOT NULL,
    is_valid BOOLEAN,
    validation_errors JSONB,

```

```

    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW(),
    INDEX idx_session (session_id),
    INDEX idx_source (source_file)
);

```

6.5 Integration Summary: Missing Submodules

HelixAgent Integration Checklist

Component	Integration Point	Action
Actor System	internal/actor/	Use HelixAgent as runtime
Session Manager	internal/session/	Delegate to AgentRuntime
Subagent Manager	internal/actor/subagent/	Wrap AgentRuntime for child agents
CLI	cmd/helix/	Add --agent flags

HelixLLM Integration Checklist

Component	Integration Point	Action
LLM Calls	internal/llm/	Replace with HelixLLM provider
Compaction	internal/compact/	Use HelixLLM for summaries
Provider Setup	internal/providers/	Use HelixLLM backend config
Metrics	internal/telemetry/	Record HelixLLM metrics

HelixMemory Integration Checklist

Component	Integration Point	Action
Messages	internal/db/	Use HelixMemory for message ops
Compaction	internal/compact/	Store summaries via HelixMemory
Session Resume	internal/session/	Restore messages from HelixMemory
Context Building	internal/llm/	Use HelixMemory for window management

HelixSpecifier Integration Checklist

Component	Integration Point	Action
Code Gen	<code>internal/edit/</code>	Validate generated code
API Design	<code>internal/api/</code>	Generate OpenAPI specs
Testing	<code>internal/test/</code>	Generate test skeletons
LSP	<code>internal/lsp/</code>	Validate against spec

APPENDIX A: Complete Feature Count

Feature Coverage Summary

Part	Feature	Status
1.1	Auto-Compaction System	Documented
1.2	Permission Rule System (5 modes + wildcards)	Documented
1.3	Tool Result Persistence	Documented
1.4	Git Worktree Agent Isolation	Documented
1.5	Hook-Based Extensibility (9+ event types)	Documented
1.6	No-Flicker Rendering Mode	Documented
1.7	Background Task System (Ctrl+B)	Documented
1.8	Smart File Editing	Documented
1.9	Plan Mode	Documented
1.10	Slash Command System	Documented
1.11	MCP Full Lifecycle	Documented
1.12	Skill System	Documented
1.13	Session Transcript Resume	Documented
1.14	Multi-Provider Backend Setup Wizards	Documented
1.15	LSP Integration	Documented
1.16	Sandboxed Shell Execution	Documented
1.17	Theme System	Documented
1.18	AskUserQuestion with Previews	Documented
1.19	Subagent Team	Documented
1.20	OpenTelemetry Integration	Documented
2.1	Architect/Editor Dual-Model	Documented
2.2	4-Layer Fuzzy Matching	Documented
2.3	Git-Native Auto-Commit	Documented

Part	Feature	Status
3.1	Plan/Act Dual-Mode	Documented
3.2	Shadow Git Checkpoints	Documented
3.3	Computer Use / Browser Automation	Documented
4.1	OS-Native Sandboxed Execution	Documented
4.2	Automatic Context Compaction	Documented
4.3	Stateless ZDR Architecture	Documented

Total: 29 Features Documented

Architecture Components Covered

Component	Diagram	Schema	State Machine	Sequence
Overall System	Yes	Yes	-	-
Data Flow	Yes	-	-	-
API Schema	-	Yes (OpenAPI)	-	-
Session Lifecycle	-	-	Yes	-
Multi-Agent Task	-	-	-	Yes
Actor System	Yes	-	Yes	-
Database Schema	Yes	-	-	-

Submodule Integration Coverage

Submodule	Interface	Config	Events	DB Schema
HelixAgent	Yes	Yes	Yes	Yes
HelixLLM	Yes	Yes	Yes	Yes
HelixMemory	Yes	Yes	Yes	Yes
HelixSpecifier	Yes	Yes	Yes	Yes

End of Comprehensive Technical Documentation